[강의] 나를 바꾸는 C++2

작성자: 오희성[Heesung Oh]

2부. 클래스와 객체지향 프로그래밍 **77**
03. 클래스와 객체 **77**
3.1 소프트웨어 모델링 77
3.1.1 모델링의 개념 77
3.1.2 모델링 산출물 77
3.1.3 절차적 모델링 78
3.1.4 순서도와 DFD 78
3.1.5 데이터와 함수 78
3.1.6 절차적 구조의 특징과 한계 79
3.2 객체지향 모델링 79
3.2.1 객체의 식별과 추상화 79
3.2.2 객체의 상태와 책임 79
3.2.3 객체의 관계와 프로그램 구조 80
3.2.4 객체의 협력과 실행 흐름 81
3.2.5 절차적 구조와 객체지향 구조의 비교 82
3.2.6 UML(Unified Modeling Language)의 역할 82
3.3 객체 모델에서 C++ 클래스로 83
3.3.1 클래스의 정의 83
3.3.2 멤버 변수와 멤버 함수 83
3.3.3 객체의 생성 84
3.3.4 객체와 인스턴스 84
3.4 객체지향 프로그래밍의 주요 특징 84
3.4.1 추상화 84
3.4.2 캡슐화 85
3.4.3 상속 85
3.4.4 다형성 85
3.5 class와 struct의 사용 기준 85
3.5.1 기본 접근 수준(Default Access Level) 85
3.5.2 class와 struct의 공통점 86
3.5.3 단순한 데이터 표현 87
3.5.4 값 의미를 가지는 struct 88
3.5.5 객체 설계에 사용하는 class 88
3.5.6 상태와 규칙이 있는 객체 89
3.5.7 프로젝트에서의 선택 기준 90
3.6 접근 제어(Access Control)와 캡슐화(Encapsulation) 91
3.6.1 public 92
3.6.2 protected 92
3.6.3 private 93
3.6.4 정보 은닉(Information Hiding) 94
3.6.5 객체의 유효한 상태(Valid State) 95
3.6.6 접근자 함수(Accessor Function) 사용 시 주의점 97
3.7 멤버 함수(Member Function)와 this 100
3.7.1 멤버 함수의 호출(Member Function Call) 100
3.7.2 this 포인터(this Pointer) 101
3.7.3 객체 자신을 반환하기(Returning the Current Object) 102
3.7.4 메서드 체이닝(Method Chaining) 104
3.7.5 상수 멤버 함수(Const Member Function) 105
3.8 정적 멤버(Static Member) 105
3.8.1 정적 데이터 멤버(Static Data Member) 105
3.8.2 정적 멤버 함수(Static Member Function) 107
3.8.3 인라인 정적 멤버(Inline Static Member) 109
3.8.4 객체 상태(Object State)와 클래스 상태(Class State) 110
3.8.5 전역 변수(Global Variable)와 정적 멤버의 차이 112
3.9 클래스 내부의 선언(Declarations Inside a Class) 113
3.9.1 중첩 클래스(Nested Class) 114
3.9.2 중첩 열거형(Nested Enumeration) 115
3.9.3 타입 별칭(Type Alias) 117
3.9.4 friend 함수(Friend Function) 118
3.9.5 friend 클래스(Friend Class) 119
3.9.6 friend 사용 시 주의점(Considerations for Using friend) 120

3.10 클래스 선언과 구현의 분리
(Separation of Class Declaration and Implementation) 121
3.10.1 헤더 파일(Header File)과 소스 파일(Source File) 121
3.10.2 클래스 선언(Class Declaration) 122
3.10.3 멤버 함수 정의(Member Function Definition) 124
3.10.4 인터페이스(Interface)와 구현(Implementation) 126
3.10.5 전방 선언(Forward Declaration) 128
3.11 UML 다이어그램과 C++ 코드 131
3.11.1 클래스 다이어그램(Class Diagram) 131
3.11.2 클래스 사이의 관계(Relationships Between Classes) 133
3.11.3 시퀀스 다이어그램(Sequence Diagram) 138
3.11.4 UML에서 C++ 코드로(From UML to C++ Code) 140
3.12 학습 정리 143
3.13 학습 과제 143
04. 생성자와 소멸자, 초기화 **145**
4.1 생성자 (constructor) 145
4.1.1 생성자의 역할 145
4.1.2 기본 생성자 (default constructor) 146
4.1.3 암시적으로 선언되는 기본 생성자
(Implicitly-declared Default Constructor) 147
4.1.4 매개변수가 있는 생성자 (Parameterized Constructor) 148
4.1.5 생성자 오버로딩 (Constructor Overloading) 149
4.1.6 변환 생성자 (Converting Constructor) 150
4.1.7 객체 생성과 초기화 152
4.2 소멸자 (Destructor) 152
4.2.1 소멸자의 역할 152
4.2.2 객체의 소멸 시점 153
4.2.3 지역 객체의 소멸 154
4.2.4 동적 객체의 소멸 154
4.2.5 생성과 소멸 순서 155
4.2.6 소멸자 작성 시 주의점 156
4.3 초기화 방식 156
4.3.1 기본 초기화 (Default Initialization) 157
4.3.2 값 초기화 (Value Initialization) 157
4.3.3 직접 초기화 (Direct Initialization) 158
4.3.4 복사 초기화 (Copy Initialization) 158
4.3.5 중괄호 초기화 (List Initialization) 158
4.3.6 집합 초기화 (Aggregate Initialization) 159
4.4 멤버 초기화 리스트 (Member Initializer List) 160
4.4.1 대입과 초기화의 차이 160
4.4.2 멤버 초기화 순서 161
4.4.3 객체 타입 멤버 초기화 162
4.4.4 상수 멤버 초기화 163
4.4.5 참조 멤버 초기화 163
4.4.6 기반 클래스 초기화 164
4.5 생성자 기능의 확장 164
4.5.1 기본 멤버 초기값 (Default Member Initializer) 164
4.5.2 위임 생성자 (Delegating Constructor) 165
4.5.3 explicit 166
4.5.4 = default 166
4.5.5 = delete 168
4.5.6 std::initializer_list 168
4.6 const 정확성과 클래스 인터페이스(Const Correctness) 169
4.6.1 상수 멤버 함수와 this(Const Member Function and this) 170
4.6.2 상수 객체(Const Object) 171
4.6.3 const 오버로딩(Const Overloading) 172
4.6.4 상수 참조 매개변수와 반환값
(Const Reference Parameters and Return Values) 173
4.6.5 논리적 상수성과 mutable(Logical Constness and mutable) 174
4.6.6 상수성을 반영한 인터페이스 176
4.7 constexpr와 컴파일 타임 초기화 177
4.7.1 상수 표현식 (Constant Expression) 177
4.7.2 constexpr 변수 178
4.7.3 constexpr 함수 178
4.7.4 constexpr 생성자 179
4.7.5 리터럴 타입 (Literal Type) 179

4.7.6 컴파일 시점과 실행 시점 평가 180
4.7.7 즉시 함수 (Immediate Function) 와 consteval 180
4.7.8 정적 초기화 (Static Initialization) 와 constinit 180
4.8 학습 정리 181
4.9 학습 과제 182
05. 복사**,** 이동과 값 의미 **183**
5.1 값 의미와 객체 정체성 183
5.1.1 값 동일성 183
5.1.2 객체 정체성 184
5.1.3 값 의미와 참조 의미 184
5.1.4 값 의미를 가지는 타입 184
5.1.5 자원을 소유하는 객체 185
5.2 기본 복사와 복사 정책 185
5.2.1 멤버 단위 복사 185
5.2.2 포인터 멤버와 얕은 복사 186
5.2.3 깊은 복사 187
5.2.4 공유, 비소유와 복사 금지 정책 187
5.2.5 자명한 복사와 바이트 단위 복사 188
5.2.6 이중 해제와 메모리 오류 188
5.3 복사 생성자 189
5.3.1 복사 생성자의 선언 189
5.3.2 암시적으로 선언되는 복사 생성자 189
5.3.3 복사 생성자의 호출 시점 190
5.3.4 사용자 정의 복사 191
5.3.5 복사 생성자의 매개변수 191
5.4 복사 대입 연산자 192
5.4.1 복사 대입 192
5.4.2 기존 자원의 교체 192
5.4.3 자기 대입 193
5.4.4 새 자원을 먼저 준비하는 대입 193
5.4.5 복사-교환 기법 194
5.5 Rule of Three 194
5.5.1 Rule of Three의 의미 194
5.5.2 세 특별 멤버 함수의 관계 195
5.5.3 직접 자원 관리의 부담 195
5.6 값 범주와 우측값 참조 195
5.6.1 값 범주의 관계 195
5.6.2 lvalue (좌측값) 196
5.6.3 prvalue (pure rvalue: 순수우측값) 196
5.6.4 xvalue (eXpiring value: 소멸예정 값) 196
5.6.5 rvalue (우측값) 197
5.6.6 rvalue reference(우측값 참조) 197
5.6.7 이름이 있는 우측값 참조 197
5.6.8 임시 객체와 수명 198
5.7 이동 생성자와 이동 대입 연산자 198
5.7.1 이동 생성자 198
5.7.2 이동 대입 연산자 199
5.7.3 자원 소유권 이전 200
5.7.4 이동 후 객체의 상태 200
5.7.5 복사와 이동의 선택 200
5.7.6 상수 객체와 이동 201
5.8 std::move와 noexcept 201
5.8.1 std::move의 의미 201
5.8.2 std::move가 이동을 수행하지 않는 이유 202
5.8.3 noexcept 이동 202
5.8.4 std::move_if_noexcept 203
5.8.5 컨테이너 재배치와 이동 203
5.8.6 반환값에 std::move를 사용할 때의 문제 203
5.9 특별 멤버 함수의 설계 204
5.9.1 특별 멤버 함수의 암시적 선언과 삭제 204
5.9.2 Rule of Five 205
5.9.3 Rule of Zero 206
5.9.4 복사 금지 객체 206
5.9.5 이동 전용 객체 207
5.9.6 = default와 = delete의 적용 208
5.10 복사 생략과 값 반환 211

5.10.1 반환값 최적화 211
5.10.2 이름 있는 반환값 최적화 212
5.10.3 보장된 복사 생략 212
5.10.4 지역 객체 반환과 암시적 이동 213
5.10.5 값 반환의 비용 213
5.10.6 불필요한 임시 객체 줄이기 213
5.11 학습 정리 214
06. 연산자 오버로딩과 사용자 정의 타입 **218**
6.1 연산자 오버로딩의 목적 218
6.1.1 사용자 정의 타입과 연산 218
6.1.2 기존 연산자 의미 유지 219
6.1.3 오버로딩 규칙과 오버로딩할 수 없는 연산자 220
6.2 멤버 연산자와 비 멤버 연산자 221
6.2.1 멤버 함수로 구현하는 연산자 221
6.2.2 비 멤버 함수로 구현하는 연산자 222
6.2.3 멤버 함수와 비 멤버 함수의 선택 222
6.3 복합 대입 연산자와 산술 연산자 223
6.3.1 복합 대입 연산자 223
6.3.2 이항 산술 연산자 224
6.3.3 단항 연산자 225
6.3.4 증가와 감소 연산자 225
6.4 비교 연산자 226
6.4.1 동등 비교 226
6.4.2 순서 비교 227
6.4.3 삼방향 비교 연산자 228
6.4.4 기본 비교 연산자 생성 229
6.5 첨자 연산자 (subscript operator) 230
6.5.1 일반 객체의 원소 접근 230
6.5.2 상수 객체의 원소 접근 230
6.5.3 범위 검사와 at 형태의 함수 231
6.6 역참조와 멤버 접근 연산자 232
6.6.1 역참조 연산자 232
6.6.2 멤버 접근 연산자 233
6.7 형 변환 연산자 235
6.7.1 사용자 정의 형 변환 235
6.7.2 explicit 형 변환 연산자 236
6.7.3 explicit operator bool 236
6.7.4 변환 생성자와 형 변환 연산자의 선택 237
6.8 스트림 연산자 238
6.8.1 출력 연산자 238
6.8.2 입력 연산자 239
6.8.3 비 멤버 함수와 friend 사용 239
6.9 연산자 오버로딩의 설계 기준 240
6.9.1 자연스러운 의미와 일관성 240
6.9.2 예상하지 못한 부작용 피하기 241
6.9.3 일반 함수가 더 적합한 경우 241
6.10 종합 예제: Vector3 242
6.11 학습 정리 246
6.12 학습 과제 246
07. 객체 수명**, RAII**와 소유권 **249**
7.1 저장 기간과 객체 수명 249
7.1.1 자동 저장 기간(Automatic Storage Duration) 249
7.1.2 정적 저장 기간(Static Storage Duration) 250
7.1.3 스레드 저장 기간(Thread Storage Duration) 251
7.1.4 동적 저장 기간(Dynamic Storage Duration) 251
7.1.5 저장 기간과 객체 수명의 차이 252
7.2 동적 메모리와 자원 253
7.2.1 new와 delete 253
7.2.2 new[]와 delete[] 254
7.2.3 할당 실패와 std::bad_alloc 255
7.2.4 메모리 누수(Memory Leak) 255
7.2.5 이중 해제와 댕글링 포인터 256
7.2.6 메모리 이외의 자원 257
7.3 RAII 258
7.3.1 자원 획득과 객체 초기화 258
7.3.2 결정적 소멸(Deterministic Destruction) 259

7.3.3 범위와 자원 해제 259
7.3.4 자원 래퍼 클래스(Resource Wrapper Class) 260
7.3.5 Rule of Zero와 RAII 262
7.4 예외 처리 262
7.4.1 try, throw와 catch 263
7.4.2 표준 예외(Standard Exceptions) 264
7.4.3 사용자 정의 예외(User-defined Exception) 265
7.4.4 스택 되감기(Stack Unwinding) 266
7.4.5 생성자와 예외 267
7.4.6 소멸자와 예외 268
7.5 예외 안전성(Exception Safety) 268
7.5.1 예외 발생 후 객체 상태 268
7.5.2 기본 보장(Basic Guarantee) 269
7.5.3 강한 보장(Strong Guarantee) 270
7.5.4 무 예외 보장(No-throw Guarantee) 270
7.5.5 복사-교환과 강한 보장 271
7.5.6 noexcept의 설계 기준 272
7.6 std::unique_ptr 272
7.6.1 고유 소유권(Unique Ownership) 273
7.6.2 std::make_unique 273
7.6.3 이동 전용 스마트 포인터 274
7.6.4 배열 소유 275
7.6.5 get, reset과 release 275
7.6.6 사용자 정의 삭제자(Custom Deleter) 276
7.6.7 함수 매개변수와 반환값 277
7.7 std::shared_ptr 278
7.7.1 공유 소유권(Shared Ownership) 279
7.7.2 참조 횟수와 제어 블록 279
7.7.3 std::make_shared 280
7.7.4 공유 소유권의 비용 280
7.7.5 잘못된 shared_ptr 생성 281
7.7.6 enable_shared_from_this 개요 281
7.8 std::weak_ptr 282
7.8.1 비소유 참조(Non-owning Reference) 282
7.8.2 만료된 객체 확인 282
7.8.3 lock을 이용한 접근 283
7.8.4 순환 참조(Cyclic Reference) 284
7.9 소유권 설계 285
7.9.1 고유 소유권 285
7.9.2 공유 소유권 286
7.9.3 비소유 포인터와 참조 286
7.9.4 값, 참조와 포인터 선택 287
7.9.5 소유권을 전달하는 함수 인터페이스 288
7.9.6 UML로 표현하는 소유 관계 289
7.9.7 스마트 포인터 선택 기준 290
7.10 종합 예제: 게임 장면과 텍스처 소유권 291
7.10.1 클래스 구조와 UML 291
7.10.2 Texture와 TextureCache 291
7.10.3 Sprite와 공유 텍스처 293
7.10.4 GameScene의 고유 소유권 294
7.10.5 실행 흐름과 객체 소멸 295
7.11 학습 정리 299
7.12 학습 과제 299
08. 상속과 다형성 **303**
8.1 일반화와 상속 303
8.1.1 기반 클래스와 파생 클래스 304
8.1.2 is-a 관계와 대체 가능성 306
8.1.3 상속을 사용하기 위한 조건 307
8.1.4 UML로 표현하는 일반화 308
8.2 상속의 기본 구조 309
8.2.1 public 상속 309
8.2.2 protected 상속 310
8.2.3 private 상속 310
8.2.4 protected 멤버 311
8.2.5 상속 접근 지정자 선택 312
8.3 기반 클래스와 파생 클래스의 초기화 313

8.3.1 기반 클래스 생성자 호출 313
8.3.2 멤버 초기화 리스트를 이용한 기반 클래스 초기화 314
8.3.3 기반 클래스와 데이터 멤버의 초기화 순서 315
8.3.4 생성과 소멸 순서 316
8.4 함수 재정의와 이름 숨김 317
8.4.1 오버로딩과 오버라이딩 317
8.4.2 이름 숨김 318
8.4.3 using 선언 319
8.4.4 기반 클래스 함수 호출 319
8.4.5 재정의 시그니처와 override 320
8.5 가상 함수와 동적 바인딩 321
8.5.1 정적 바인딩과 동적 바인딩 321
8.5.2 가상 함수 322
8.5.3 override와 final 324
8.5.4 가상 함수 테이블의 개념적 모델 324
8.5.5 공변 반환형 (covariant return type) 325
8.5.6 생성자와 소멸자에서의 가상 함수 호출 326
8.5.7 가상 함수와 기본 인수 327
8.6 추상 클래스와 인터페이스 327
8.6.1 순수 가상 함수 327
8.6.2 추상 클래스 328
8.6.3 인터페이스 역할의 클래스 329
8.6.4 구현 상속과 인터페이스 상속 330
8.6.5 최소 인터페이스와 계약 330
8.6.6 기본 구현과 비가상 인터페이스 331
8.7 다형 객체 (Polymorphic object) 의 수명 332
8.7.1 가상 소멸자 332
8.7.2 기반 클래스 포인터를 통한 삭제 332
8.7.3 객체 잘림 (Object slicing) 333
8.7.4 다형 객체의 복사와 Clone 334
8.7.5 스마트 포인터와 다형 객체 335
8.8 형 변환과 RTTI 335
8.8.1 업캐스팅 335
8.8.2 다운캐스팅 336
8.8.3 dynamic_cast 336
8.8.4 typeid 337
8.8.5 다운캐스팅을 줄이는 설계 337
8.9 다중 상속 338
8.9.1 다중 상속의 구조 338
8.9.2 이름 충돌 338
8.9.3 다이아몬드 상속 339
8.9.4 가상 상속 340
8.9.5 가상 기반 클래스 초기화 341
8.9.6 인터페이스 다중 상속 342
8.10 상속과 구성 342
8.10.1 has-a 관계와 구성 342
8.10.2 위임 344
8.10.3 상속의 결합도 344
8.10.4 상속보다 구성을 우선하는 이유 345
8.10.5 상속이 필요한 경우 346
8.10.6 UML로 비교하는 상속과 구성 347
8.11 종합 예제: 게임 객체 계층과 구성 347
8.11.1 클래스 구조와 UML 347
8.11.2 GameSystem, HealthComponent와 GameObject 348
8.11.3 Character, PlayerBase와 Player 350
8.11.4 MobBase, Monster와 BossMonster 352
8.11.5 GameScene의 다형 객체 소유 354
8.11.6 실행 흐름과 다형 복제 355
8.12 학습 정리 355
8.13 학습 과제 356
09. 객체지향 설계 원칙 **358**
9.1 책임과 협력 358
9.1.1 객체의 책임 358
9.1.2 객체의 역할 359
9.1.3 데이터와 행위의 배치 360
9.1.4 책임이 많은 객체의 문제 361

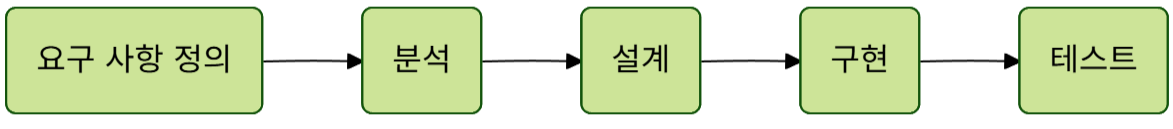
9.2 응집도와 결합도 362
9.2.1 응집도 363
9.2.2 결합도 363
9.2.3 높은 응집도와 낮은 결합도 364
9.2.4 변경의 영향 범위 365
9.3 인터페이스 설계 366
9.3.1 최소 인터페이스 366
9.3.2 정보 은닉과 객체 불변식 367
9.3.3 명령과 질의 368
9.3.4 오류와 실패의 표현 368
9.3.5 소유권과 수명의 표현 369
9.4 단일 책임 원칙 (SRP) 370
9.4.1 하나의 변경 이유 370
9.4.2 책임 분리 371
9.4.3 단일 책임의 범위 373
9.5 개방-폐쇄 원칙 (OCP) 373
9.5.1 확장에는 열려 있고 변경에는 닫혀 있는 구조 374
9.5.2 조건문 증가의 문제 374
9.5.3 추상화와 확장 지점 375
9.5.4 상속과 구성을 통한 확장 377
9.6 리스코프 치환 원칙 (LSP) 378
9.6.1 기반 타입과 파생 타입의 치환 378
9.6.2 계약 유지 379
9.6.3 사전 조건과 사후 조건 379
9.6.4 잘못된 상속 관계 380
9.7 인터페이스 분리 원칙 (ISP) 382
9.7.1 큰 인터페이스의 문제 382
9.7.2 역할별 인터페이스 382
9.7.3 작은 인터페이스의 조합 384
9.8 의존 역전 원칙 (DIP) 384
9.8.1 구체 클래스에 대한 의존 384
9.8.2 추상 인터페이스에 대한 의존 385
9.8.3 의존성 방향 385
9.8.4 의존성 주입 386
9.8.5 테스트 가능한 구조 386
9.9 객체지향 설계의 적용 387
9.9.1 설계 문제 분석 388
9.9.2 책임 분리 389
9.9.3 인터페이스 추출 390
9.9.4 상속을 구성으로 변경 390
9.9.5 전역 상태 줄이기 392
9.9.6 과도한 추상화 피하기 392
9.10 학습 정리 399
9.11 학습 과제 399

2부. 클래스와 객체지향 프로그래밍

03. 클래스와 객체

소프트웨어 개발은 사용자가 필요로 하는 기능과 서비스를 프로그램으로 구현하는 작업이다. 프로그램의 규모가 작다면 필요한 기능을 파악한 뒤 곧바로 코드를 작성할 수도 있다. 그러나 여러 개발자가 참여하고 많은 기능과 데이터가 서로 연결되는 소프트웨어는 코드를 작성하는 것만으로 완성하기 어렵다.

일반적으로 소프트웨어는 여러 단계를 거쳐 개발된다.



요구 사항 정의 단계에서는 사용자가 필요로 하는 기능과 시스템이 만족해야 할 조건을 정리한다. 분석 단계에서는 요구 사항을 처리하기 위해 필요한 데이터와 기능, 프로그램에서 다루어야 할 대상을 파악한다. 설계 단계에서는 분석한 내용을 어떤 구조로 구성하고 각 구성 요소가 어떻게 협력하도록 만들 것인지 결정한다. 구현 단계에서는 설계된 구조를 프로그래밍 언어로 작성하고, 테스트 단계에서는 구현된 프로그램이 요구 사항에 맞게 동작하는지 확인한다.

이러한 개발 절차는 개발에 필요한 기간과 인력을 계획하고, 구현 과정에서 발생할 수 있는 변경과 재작업을 줄이기 위해 필요하다. 요구 사항과 프로그램의 구조를 충분히 검토하지 않고 구현을 시작하면 기능이 추가되거나 요구 사항이 변경될 때 이미 작성한 코드를 크게 수정해야 할 수 있다. 개발 전에 필요한 기능과 구조를 분석하고 설계하면 어느 정도의 기간과 인력이 필요한지 예측할 수 있으며, 구현 중 발생할 수 있는 구조적인 문제도 미리 검토할 수 있다.

실제 소프트웨어 개발에서는 각 단계를 한 번씩만 수행하지 않는다. 구현 중 새로운 문제가 발견되면 분석이나 설계 단계로 돌아가 구조를 수정할 수 있고, 테스트 결과에 따라 요구 사항을 다시 검토할 수도 있다. 개발 모델에 따라 단계의 순서와 반복 방식은 달라질 수 있지만, 요구 사항을 분석하고 구현할 구조를 설계한 뒤 코드로 옮긴다는 기본 과정은 공통적이다.

분석과 설계에서는 프로그램을 어떤 관점에서 나누고 구성할지 결정한다. 요구 사항을 분석하여 프로그램으로 구현할 구조를 표현하는 과정을 모델링이라고 한다. 대표적인 방식으로는 프로그램의 작업과 처리 흐름을 중심으로 표현하는 절차적 모델링과, 객체의 상태와 책임, 객체 사이의 관계와 협력을 중심으로 표현하는 객체지향 모델링이 있다. 프로그램의 특성과 설계 목적에 따라 적합한 모델링 방식을 선택한다.

절차적 모델링에서는 처리 과정과 함수의 실행 순서를 중심으로 프로그램을 구성한다. 객체지향 모델링에서는 프로그램에 필요한 대상을 객체로 식별하고, 각 객체의 상태와 책임, 객체 사이의 관계와 협력을 중심으로 프로그램을 구성한다.

C++는 절차적 프로그래밍과 객체지향 프로그래밍을 모두 지원한다. 단순한 계산이나 처리 순서가 중요한 기능은 함수 중심으로 구성할 수 있다. 반면 여러 상태와 변경 규칙을 함께 관리해야 하거나 여러 대상이 서로 협력하는 프로그램은 객체 중심으로 구성할 수 있다.

이 장에서는 소프트웨어 모델링의 기본 개념을 살펴보고, 절차적 구조와 객체지향 구조를 차례로 비교한다. UML을 이용해 객체와 클래스의 구조 및 협력을 표현하고, 설계한 모델을 C++ 클래스로 구현하는 과정을 학습한다. 이후 클래스와 객체, 접근 제어와 캡슐화, 멤버 함수와 this, 정적 멤버, 클래스 내부 선언, 클래스 선언과 구현의 분리를 살펴본다.

3.1 소프트웨어 모델링

3.1.1 모델링의 개념

소프트웨어 개발의 모델링은 요구 사항을 분석하여 프로그램으로 구현할 구조와 동작을 표현하는 과정이다. 복잡한 시스템을 쉽게 이해하고 개발자 사이에서 공유할 수 있도록 필요한 요소를 추상화하여 일정한 형태로 나타낸다. 실제 시스템의 모든 내용을 그대로 옮기는 것이 아니라, 프로그램을 개발하는 데 필요한 데이터와 기능, 구성 요소와 관계를 선택하여 단순화한다.

프로그램의 구조를 설계하려면 요구 사항에서 필요한 데이터와 기능을 찾아내고, 이를 어떤 단위로 나누어 구성할지 결정해야 한다. 프로그램의 작업과 처리 흐름을 중심으로 표현하는 절차적 모델링과, 객체의 상태와 책임, 객체 사이의 관계와 협력을 중심으로 표현하는 객체지향 모델링 등이 있다. 프로그램의 특성과 설계 목적에 따라 적합한 모델링 방식을 선택한다.

3.1.2 모델링 산출물

모델링의 결과는 문서나 다이어그램과 같은 산출물로 표현된다. 프로그램을 바라보는 관점과 설계 목적에 따라 요구 사항 명세서, 순서도, DFD(Data Flow Diagram), UML 다이어그램 등을 작성할 수 있다.

<모델링 산출물>

```
- 요구 사항 명세서
- 순서도
```

```
- DFD
- UML 다이어그램
```

순서도는 작업의 처리 순서와 제어 흐름을 표현하고, DFD는 데이터가 어디에서 들어와 어떤 처리 과정을 거쳐 어디로 이동하는지를 표현한다. UML 다이어그램은 객체지향 프로그램의 구조와 동작, 객체 사이의 관계와 협력을 표현한다.

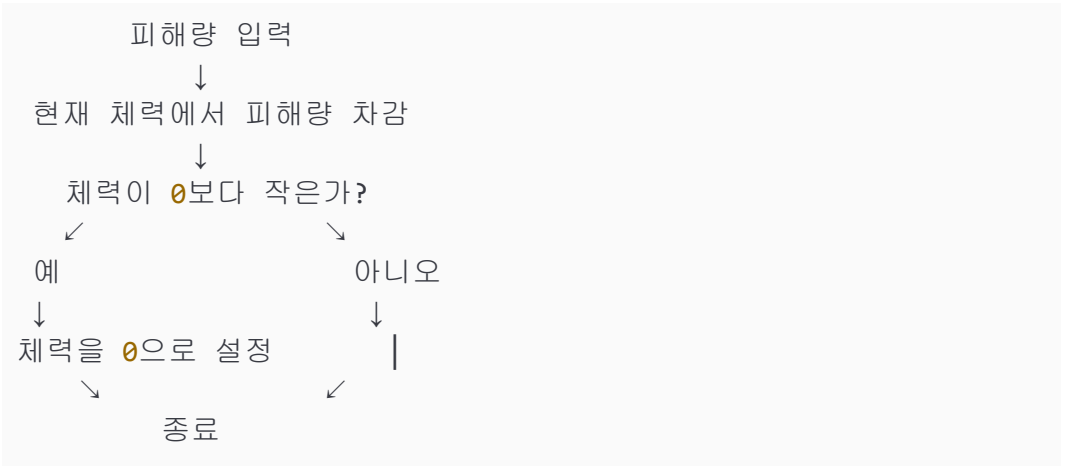
3.1.3 절차적 모델링

C에서 주로 사용한 절차적 프로그래밍은 프로그램이 수행할 작업을 함수로 나누고 처리 순서를 중심으로 구성한다. 절차적 모델링에서는 프로그램이 수행할 작업과 데이터의 처리 과정을 중심으로 구조를 정리한다.

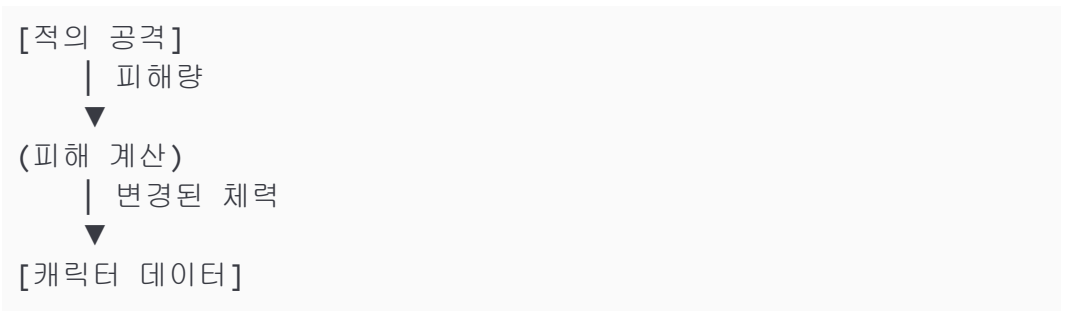
- 어떤 작업을 수행하는가
- 어떤 순서로 처리하는가
- 어떤 함수가 데이터를 처리하는가
- 데이터가 어떤 처리 과정을 거치는가

3.1.4 순서도와 DFD

절차적 모델링에서는 작업의 처리 순서와 제어 흐름을 순서도로 표현할 수 있다.



또한 DFD(Data Flow Diagram)는 시스템에서 데이터가 어디에서 들어오고 어떤 처리 과정을 거쳐 어디로 이동하는지를 표현한다.



3.1.5 데이터와 함수

순서도와 DFD로 표현한 처리 과정은 데이터와 함수로 구현할 수 있다. 절차적 프로그래밍에서는 프로그램이 수행할 작업을 함수 단위로 나누고 처리 순서에 따라 구성한다.

절차적 프로그래밍은 입력된 데이터를 어떤 함수가 처리하고, 처리된 결과가 다른 함수로 어떻게 전달되는지가 중요하다.

앞에서 표현한 처리 과정을 코드로 옮기면 체력 데이터를 별도로 저장하고, 이를 처리하는 함수에 전달하는 구조로 작성될 수 있다.

```c
struct Character
{
    int hp = 100;
};

void TakeDamage(Character* character, int damage)
{
    if (character->hp -= damage; character->hp < 0)
    {
        character->hp = 0;
    }
}
```

Character 구조체는 데이터를 저장하고, TakeDamage() 함수는 전달받은 구조체의 데이터를 변경한다.

처리 흐름이 단순하거나, 프로그램의 규모가 작고 처리 규칙이 단순하다면 이 구조도 충분히 명확하다.

또한 캐릭터와 관련된 기능이 증가하면 여러 함수가 같은 구조체를 전달받아 처리하는 구조로 확장할 수 있다.

```
void Move(Character* character);
void Attack(Character* character);
void TakeDamage(Character* character, int damage);
void Recover(Character* character, int amount);
void SaveCharacter(const Character* character);
```

3.1.6 절차적 구조의 특징과 한계

절차적 구조는 프로그램이 수행할 작업을 기능 단위로 나누고, 데이터가 각 함수를 거쳐 처리되는 흐름을 중심으로 구성한다. 처리 순서가 분명하고 다루는 데이터가 단순한 프로그램에서는 각 함수의 역할과 실행 과정을 쉽게 파악할 수 있다.

프로그램의 규모가 커지면 여러 함수가 같은 데이터를 공유하고 변경하게 된다. 피해 처리, 회복, 상태 초기화, 데이터 불러오기와 같은 여러 함수가 Character의 체력을 직접 변경한다면 각 함수가 체력의 최소값과 최대값, 사망 조건과 같은 규칙을 알아야 한다. 규칙이 변경되면 관련된 함수를 모두 확인해야 할 수 있다.

문제는 함수의 수가 많다는 사실보다 상태와 변경 규칙을 관리하는 책임이 한곳에 모이지 않는다는 데 있다. 상태와 규칙이 많아지고 여러 기능이 같은 데이터를 함께 다루면 데이터와 기능의 관계를 일관되게 유지하기 어려워진다.

상태와 변경 규칙을 하나의 책임 단위로 묶고, 각 단위의 관계와 협력을 중심으로 프로그램을 표현하는 방식이 객체지향 모델링이다.

3.2 객체지향 모델링

3.2.1 객체의 식별과 추상화

객체지향 모델링에서는 프로그램에 필요한 대상을 객체로 나누고, 각 객체가 자신의 상태와 책임을 관리하면서 다른 객체와 협력하도록 구조를 표현한다.

객체지향 분석에서는 요구 사항에 나타나는 대상과 개념을 살펴보고, 프로그램에서 독립적인 역할을 담당할 객체를 식별한다. 객체는 반드시 현실 세계에 물리적으로 존재하는 사물만을 의미하지 않는다. 사람, 자동차, 건물과 같은 실제 대상을 객체로 표현할 수도 있지만, 계좌, 주문, 인벤토리, 게임 상태, 입력 관리자와 같이 프로그램의 필요에 따라 정의한 논리적인 대상도 객체가 될 수 있다.

현실 세계에서 식별한 대상에는 사람, 자동차, 건물, 상품 등이 있다. 프로그램을 위해 정의한 논리적인 대상에는 계좌, 주문, 인벤토리, 게임 상태, 입력 관리자, 충돌 검사기 등이 있다.

객체를 식별할 때는 대상의 모든 특징을 프로그램에 포함하지 않는다. 프로그램의 목적에 필요한 특징만 선택하여 상태와 동작으로 표현한다. 실제 대상이나 요구 사항에서 프로그램에 필요한 특징을 선택하고 단순화하여 표현한 결과를 모델이라고 한다.

```
요구 사항과 대상
        ↓
필요한 특징 선택
        ↓
객체 식별
        ↓
객체의 상태와 책임 결정
        ↓
객체 사이의 관계와 협력 설계
        ↓
C++ 코드로 구현
```

3.2.2 객체의 상태와 책임

게임에서 플레이어 캐릭터를 구현한다고 가정해 보자. 현실의 사람은 키, 몸무게, 나이와 성격 등 수많은 특징을 가진다. 그러나 게임 프로그램에서는 이름, 체력, 위치와 공격력 등의 상태와 이동, 공격, 피해 처리 등의 동작만 필요할 수 있다.

```
게임 캐릭터
├─ 상태
│   ├─ 이름
│   ├─ 체력
│   ├─ 위치
│   └─ 공격력
└─ 동작
    ├─ 이동
```

```
├─ 공격
└─ 피해 처리
```

객체지향에서 중요한 것은 데이터를 어느 객체가 관리하고, 데이터의 변경 규칙을 어느 객체가 책임질 것인지 결정하는 것이다. 객체가 자신의 상태를 관리하고 다른 객체는 필요한 기능만 요청하도록 구성하면 상태 변경 규칙이 여러 위치에 흩어지는 것을 줄일 수 있다.

객체지향 모델링에서는 객체를 찾은 뒤 상태와 기능을 함께 정리한다.

- 어떤 객체가 필요한가
- 객체가 어떤 상태를 가지는가
- 객체가 어떤 책임을 담당하는가
- 객체들이 어떤 관계를 맺는가
- 객체들이 어떻게 협력하는가

3.2.3 객체의 관계와 프로그램 구조

객체지향 프로그램에서는 객체를 독립된 부품처럼 구성할 수 있다. 각 객체는 자신이 담당하는 상태와 기능을 관리하고, 다른 객체의 공개된 기능을 사용하여 더 큰 프로그램을 구성한다.

독립적으로 완성된 작은 객체를 연결하고 조합하여 더 큰 객체와 프로그램으로 확장하는 방식을 상향식 설계라고 한다. 객체 사이의 관계를 형성하는 과정에서 새로운 상태와 기능이 필요하면 이를 담당하는 객체를 추가할 수 있다.

학생과 과목을 관리하는 프로그램을 생각해 보자. 학생은 학번과 이름을 관리하고, 과목은 과목 코드, 과목명과 이수 학점을 관리한다.

```
• 학생
├─ 학번
└─ 이름
```

```
• 과목
├─ 과목 코드
├─ 과목명
└─ 이수 학점
```

학생과 과목은 각각 독립적으로 완성된 대상이지만, 두 대상만으로는 실제로 개설된 수업을 표현할 수 없다. 특정 학기에 어떤 과목이 개설되었는지, 어떤 학생들이 그 수업을 듣는지, 담당 교수와 강의실 및 수업 시간을 어떻게 관리할지도 나타내야 한다.

학생과 과목의 관계를 형성하면 개설된 수업을 나타내는 강좌를 구성할 수 있다.

```
학생 <──── 강좌 ────> 과목
```

강좌는 개설된 과목과 수강하는 학생들을 연결하고 수업 운영에 필요한 정보를 관리한다.

```
강좌
├─ 과목
├─ 수강 학생 목록
├─ 담당 교수
├─ 강의실
├─ 수업 시간
└─ 개설 학기
```

강좌는 학생과 과목의 정보를 다시 정의하지 않는다. 학생의 학번과 이름은 학생이 관리하고, 과목 코드와 과목명 및 이수 학점은 과목이 관리한다. 강좌는 학생과 과목의 공개된 기능을 사용하면서 두 대상의 관계와 수업 운영에 필요한 상태를 관리한다.

처음에는 학생과 과목이라는 독립된 대상만 있었지만, 두 대상의 관계를 형성하면서 강좌라는 더 큰 대상이 만들어졌다. 여러 강좌를 모으면 학기별 강좌 목록을 구성할 수 있고, 이를 기반으로 수강 관리 프로그램의 전체 구조를 확장할 수 있다.

```
학생과 과목
      ↓ 관계 설정
강좌
      ↓ 여러 강좌 관리
학기별 강좌 목록
      ↓
수강 관리 프로그램
```

객체지향 설계는 서로 관련된 상태와 기능을 하나의 객체에 모아 응집도를 높이고, 객체의 내부 구현을 감춘 채 공개된 기능을 통해 협력하도록 하여 객체 사이의 결합도를 낮추기 좋은 구조를 제공한다. 각 객체의 책임이 명확하고 다른 객체의 내부 상태에 직접 의존하지 않으면, 객체를 독립적으로 개발하고 수정하며 재사용하기 쉬워진다.

학생과 과목도 각각 자신의 상태와 기능을 관리하도록 독립적으로 구성할 수 있다. 강좌는 학생과 과목의 내부 구현을 다시 작성하지 않고 두 대상의 공개된 기능을 사용하여 관계를 형성한다. 요구 사항이 변경되더라도 학생과 과목의 역할이 유지된다면 기존 설계를 그대로 활용하고, 강좌의 책임이나 객체 사이의 관계를 중심으로 수정할 수 있다.

관련된 상태와 기능을 책임 단위로 모으고, 독립된 단위를 관계로 연결하여 더 큰 프로그램 구조로 확장하는 것은 객체지향 설계가 프로그램의 유연성과 안정성을 높이는 중요한 이유이다.

3.2.4 객체의 협력과 실행 흐름

객체지향 모델링에서는 객체 사이의 관계를 이용하여 프로그램의 구조를 구성할 뿐만 아니라, 객체들이 어떤 순서로 기능을 요청하며 협력할 것인지도 설계한다.

게임 프로그램에서 플레이어가 무기를 사용하여 적을 공격하는 기능을 생각해 보자. 이 기능은 Player, Weapon, Enemy 객체로 나누어 구성할 수 있다.

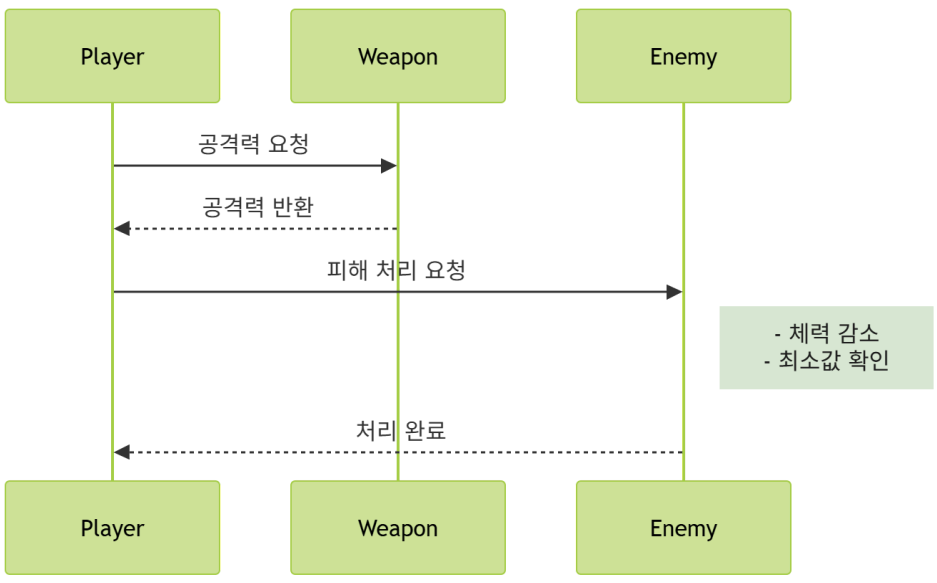
```
• Player
  - 공격을 시작한다.
  - 사용할 무기를 선택한다.
• Weapon
  - 공격력을 관리한다.
  - 공격에 필요한 공격력을 제공한다.
• Enemy
  - 자신의 체력을 관리한다.
  - 전달받은 피해를 처리한다.
  - 체력이 0보다 작아지지 않도록 유지한다.
```

Player가 무기의 공격력과 적의 체력을 모두 직접 다루도록 만들 수도 있다. 그러나 이렇게 구성하면 Player가 공격을 시작하는 기능뿐만 아니라 무기의 공격력과 적의 체력 변경 규칙까지 관리해야 한다.

각 객체가 자신의 상태와 기능을 담당하도록 구성하면 Player는 Weapon에 공격력을 요청하고, 전달받은 공격력으로 Enemy에 피해 처리를 요청한다. Enemy는 전달받은 피해량에 따라 자신의 체력을 변경한다.

```
Player ──공격력 요청──▶ Weapon
Weapon ──공격력 반환──▶ Player
Player ──피해 처리 요청──▶ Enemy
Enemy  ──체력 변경──▶ 자신의 상태
```

호출 과정을 시간의 흐름에 따라 표현할 수 있다.



Player는 Weapon의 공격력을 직접 변경하지 않고 필요한 값을 요청한다. `Player`는 `Enemy`의 체력을 직접 감소시키지 않고 피해 처리를 요청한다. Weapon은 자신의 공격력을 관리하고, Enemy는 자신의 체력과 체력 변경 규칙을 관리한다.

수강 관리 예제는 학생과 과목의 정적인 관계를 바탕으로 강좌라는 새로운 객체를 구성하는 과정을 보여 준다. 게임 프로그램 예제는 Player, Weapon, Enemy가 서로 기능을 요청하며 협력하는 과정과 시간 순서에 따른 메시지의 흐름을 보여 준다.

```
수강 관리 프로그램
학생과 과목의 관계
        ↓
강좌 객체 구성
        ↓
```

```
객체 사이의 정적인 구조
```

```
게임 프로그램
Player, Weapon, Enemy의 관계
        ↓
객체 사이의 기능 요청
        ↓
시간 순서에 따른 동적인 협력
```

이 두 예제는 객체지향 프로그램을 객체의 구성과 관계뿐만 아니라 객체 사이의 상호작용과 실행 흐름까지 다양한 관점에서 표현할 수 있음을 보여 준다.

객체지향 모델링은 요구 사항에 등장하는 명사를 기계적으로 모두 객체로 만드는 작업이 아니다. 프로그램에 필요한 객체를 찾고, 각 객체가 담당할 상태와 기능을 정하며, 객체들이 어떤 관계를 맺고 어떻게 협력할 것인지 설계하는 작업이다.

객체 모델을 정리할 때는 주요 항목을 점검한다.

- 프로그램에 어떤 객체가 필요한가?
- 각 객체는 어떤 상태와 기능을 가져야 하는가?
- 상태와 변경 규칙은 어느 객체가 관리해야 하는가?
- 객체 사이에는 어떤 관계가 필요한가?
- 관계에서 새로운 객체가 필요한가?
- 객체들은 어떤 기능을 요청하며 협력하는가?

3.2.5 절차적 구조와 객체지향 구조의 비교

절차적 구조와 객체지향 구조의 차이는 함수가 클래스 안에 들어갔다는 사실만으로 설명되지 않는다. 핵심은 데이터와 변경 규칙을 관리하는 책임을 어디에 두는가이다.

- 절차적 구조
  데이터 → 함수 → 함수 → 함수

- 객체지향 구조
  객체 ──메시지──▶ 객체 ──메시지──▶ 객체

객체지향 구조에서도 멤버 함수 내부는 조건문, 반복문과 함수 호출을 사용하는 절차적 코드로 구현한다. 객체지향 프로그래밍은 절차적 프로그래밍을 없애는 방식이 아니라, 데이터와 기능을 책임을 가진 객체 단위로 조직하는 방식이다.

두 방식 중 하나가 항상 더 우수한 것은 아니다. 단순한 계산이나 처리 순서가 중요한 기능은 절차적으로 작성하는 편이 명확할 수 있다. 반면 상태와 규칙이 많고 여러 기능이 함께 변경되는 대상은 객체로 구성하는 편이 관리하기 쉽다.

C++는 절차적 프로그래밍과 객체지향 프로그래밍을 모두 지원한다. 모든 기능을 클래스로 만들 필요는 없으며, 상태와 책임을 함께 관리할 필요가 있는 부분을 객체로 구성하면 된다.

3.2.6 UML(Unified Modeling Language)의 역할

객체의 상태와 기능, 객체 사이의 관계와 실행 흐름을 문장만으로 표현하면 프로그램의 규모가 커질수록 전체 구조를 파악하기 어렵다. UML은 객체지향 설계 내용을 다이어그램으로 표현하기 위한 표준화된 모델링 언어이다.

UML은 Unified Modeling Language의 약자이다. UML은 프로그램을 실행하는 언어가 아니며 프로그램을 자동으로 완성해 주는 설계 방법도 아니다. 분석하고 설계한 클래스의 구조와 객체의 협력을 일정한 표기법으로 나타내어, 구현 전에 프로그램의 구조를 검토하고 개발자 사이에서 공유할 수 있도록 돕는 표현 수단이다.

본 교재에서는 여러 UML 다이어그램 중 클래스 다이어그램과 시퀀스 다이어그램을 사용한다.

```
객체의 상태와 기능, 관계 → 클래스 다이어그램
객체의 요청과 처리 순서  → 시퀀스 다이어그램
```

클래스 다이어그램은 클래스가 가지는 상태와 기능, 클래스 사이의 관계를 표현한다. 시퀀스 다이어그램은 객체들이 시간의 흐름에 따라 기능을 요청하고 처리하는 순서를 표현한다.

UML을 사용하면 실제 코드를 작성하기 전에 클래스의 구조와 객체의 협력 과정을 검토하고, 이를 C++ 클래스와 멤버 함수로 어떻게 구현할 것인지 예측할 수 있다.

모든 C++ 프로그램을 작성할 때 UML을 반드시 그려야 하는 것은 아니다. 클래스가 몇 개뿐인 단순한 프로그램이나 문법을 확인하기 위한 예제에서는 코드를 직접 작성하는 편이 빠를 수 있다. 그러나 클래스가 많아지고 객체 사이의 관계가 복잡해지면 UML을 통해 전체 구조와 변경의 영향을 먼저 확인할 수 있다.

UML은 코드를 대신하는 문서가 아니다. 구현 전에는 코드의 구조를 예측하고, 구현 후에는 복잡한 코드에서 클래스와 객체의 관계를 파악하기 위한 설계 도구이다.

3.3 객체 모델에서 C++ 클래스로

프로그램에 필요한 객체를 찾고, 각 객체의 상태와 기능, 객체 사이의 관계와 협력을 설계한 내용은 아직 프로그램에서 직접 사용할 수 있는 코드가 아니다. C++에서는 객체의 상태와 기능을 클래스로 정의하고, 클래스에 따라 실제 객체를 생성하여 사용한다.

3.3.1 클래스의 정의

객체지향 모델링에서 찾은 객체는 프로그램에서 구현해야 할 논리적인 대상이다. 객체가 가져야 할 상태는 데이터로, 객체가 수행할 기능은 함수로 표현할 수 있다.

C++에서는 데이터와 기능을 하나의 사용자 정의 타입으로 묶기 위해 클래스를 사용한다. 클래스는 객체를 만들어 내기 위한 설계도이자 객체를 정의하는 타입이다.

```
<Character 객체 모델>
• 상태
  - 체력
  - 위치
  - 공격력
• 기능
- 이동
- 공격
- 피해 처리
```

Character 객체 모델은 C++에서 클래스로 표현할 수 있다.

```
class Character
{
public:
    void Move(int x, int y);
    void Attack();
    void TakeDamage(int damage);

private:
    int hp = 100;
    int x = 0, y = 0;
    int attackPower = 10;
};
```

C++에서 클래스는 객체가 가질 상태와 기능을 정의한 사용자 정의 타입이다. 객체 모델에서 정리한 체력, 위치와 공격력은 멤버 변수로 표현하고, 이동, 공격과 피해 처리는 멤버 함수로 표현한다.

```
객체의 상태 → 멤버 변수
객체의 기능 → 멤버 함수
객체의 상태와 기능을 정의한 모델 → 클래스
```

3.3.2 멤버 변수와 멤버 함수

클래스나 구조체 안에 선언한 변수를 데이터 멤버라고 하며, 일반적으로 멤버 변수라고도 부른다. 데이터 멤버는 객체의 상태를 저장한다. 멤버 변수는 객체의 상태를 저장한다.

```
private:
    int hp = 100;
    int x = 0, y = 0;
    int attackPower = 10;
```

클래스 안에 선언한 함수를 멤버 함수라고 한다. 멤버 함수는 객체가 수행할 기능과 객체의 상태를 처리하는 규칙을 표현한다.

```
public:
    void Move(int x, int y);
    void Attack();
    void TakeDamage(int damage);
```

멤버 함수는 같은 객체의 멤버 변수에 직접 접근할 수 있다. 외부에서는 공개된 멤버 함수를 호출하여 객체에 필요한 작업을 요청한다.

TakeDamage()는 Character가 자신의 체력과 변경 규칙을 관리하도록 구현할 수 있다.

```
void Character::TakeDamage(int damage)
{
    if (hp -= damage; hp < 0)
    {
        hp = 0;
    }
}
```

외부에서는 hp를 직접 변경하지 않고 TakeDamage()를 호출하여 상태 변경을 요청한다.

3.3.3 객체의 생성

클래스 자체는 하나의 타입 정의이다. 클래스에 정의된 구조를 사용하여 변수를 선언하면 실제 객체가 생성된다.

```
Character player;
Character enemy;
```

player와 enemy는 모두 Character 타입의 객체이다. 두 객체는 같은 클래스에 따라 생성되지만 각각 자신의 멤버 변수를 가진다.

```
Character 클래스
├─ player 객체
│   ├─ hp
│   ├─ x, y
│   └─ attackPower
└─ enemy 객체
    ├─ hp
    ├─ x, y
    └─ attackPower
```

3.3.4 객체와 인스턴스

특정 클래스에 따라 생성된 객체를 해당 클래스의 인스턴스라고 한다.

```
클래스(Class)     → 객체가 가질 상태와 기능을 정의한 C++ 사용자 정의 타입(설계도)
객체(Object)      → 클래스에 따라 생성되어 프로그램에서 실제 저장 공간을 가지는 실체
인스턴스(Instance) → 특정 클래스에 따라 생성된 객체임을 강조하는 표현
```

player는 객체이며 동시에 Character 클래스의 인스턴스이다. 객체와 인스턴스는 서로 다른 대상을 가리키는 용어가 아니라, 어떤 관점에서 설명하는지에 따라 구분하여 사용한다.

클래스를 객체를 만들기 위한 틀이나 설계도에 비유할 수 있다. 그러나 C++에서 클래스는 단순한 설계 문서가 아니라 컴파일러가 해석하는 실제 타입 정의이다. 클래스 타입으로 객체를 선언하면 객체에 필요한 저장 공간이 마련되고 각 객체는 자신의 상태를 가진다.

프로세스는 실행 중인 프로그램 전체를 의미한다. 객체는 프로세스 안에서 특정 타입에 따라 생성되어 독립적인 상태와 수명을 가지는 값 또는 실체이다.

3.4 객체지향 프로그래밍의 주요 특징

객체지향 프로그래밍의 주요 특징은 일반적으로 추상화, 캡슐화, 상속, 다형성으로 설명한다.

3.4.1 추상화

추상화는 대상의 모든 특징을 표현하지 않고 프로그램의 목적에 필요한 상태와 기능만 선택하여 모델로 표현하는 것이다.

게임 캐릭터를 구현할 때 현실 인물의 모든 특징을 표현하지 않고 체력, 위치, 공격력과 이동, 공격, 피해 처리만 선택하는 것이 추상화에 해당한다.
C++에서는 필요한 상태와 기능을 클래스로 정의하여 추상화된 타입을 만들 수 있다.

```
class Character
{
```

```
public:
    void Move();
    void Attack();
private:
    int hp = 100;
};
```

3.4.2 캡슐화

캡슐화는 객체의 상태와 그 상태를 처리하는 기능을 하나의 단위로 묶고, 내부 상태가 정해진 규칙에 따라 변경되도록 외부 접근을 제한하는 것이다. 즉, 객체의 내부 구현은 감추고, 외부에는 객체를 사용하는 데 필요한 기능만 공개하여 객체를 하나의 블랙박스처럼 구성하는 것이다.

이러한 원리를 정보 은닉이라고 한다.

```
class Character
{
public:
    void TakeDamage(int damage)
    {
        if (hp -= damage; hp < 0)
        {
            hp = 0;
        }
    }
private:
    int hp = 100;
};
```

외부에서는 hp를 직접 변경하지 않고 TakeDamage()를 호출한다. 체력이 0보다 작아지지 않도록 유지하는 책임은 Character 객체가 담당한다.

3.4.3 상속

상속은 기존 클래스의 특성을 바탕으로 새로운 클래스를 정의하는 기능이다. 하위 클래스는 상위 클래스의 상태와 기능을 이어받아 새로운 기능을 추가하거나 기존 기능을 재정의할 수 있다.

```
Character
   △
   |
 Player
```

C++에서는 파생 클래스가 기반 클래스의 상태와 기능을 바탕으로 새로운 상태와 기능을 추가할 수 있으며, 상속받은 기능의 동작을 재정의할 수도 있다. 상속은 이후 장에서 자세히 다룬다.

3.4.4 다형성

다형성은 동일한 인터페이스를 통해 서로 다른 객체가 각자의 방식으로 동작할 수 있는 특성이다.

```
Character의 Attack()
├─ Player는 검으로 공격
├─ Archer는 활로 공격
└─ Mage는 마법으로 공격
```

C++에서는 함수 오버로딩, 템플릿과 가상 함수 등 여러 방식으로 다형성을 구현할 수 있다. 객체지향 프로그래밍의 런타임 다형성은 상속과 가상 함수를 다루는 장에서 자세히 살펴본다.

클래스는 이 네 가지 특징을 자동으로 완성하는 문법이 아니다. 클래스는 객체의 상태와 기능을 타입으로 표현하는 기본 수단이며, 접근 제어, 상속과 가상 함수 등의 C++ 기능을 함께 사용하여 객체지향 구조를 구현한다.

3.5 class와 struct의 사용 기준

C++에서는 class와 struct를 사용하여 사용자 정의 타입을 만들 수 있다. 두 문법은 사용할 수 있는 기능이 거의 같지만, 기본 접근 수준과 일반적으로 표현하는 설계 의도가 다르다.

일반적으로 struct는 여러 데이터를 하나의 값으로 묶을 때 사용하고, class는 객체가 자신의 상태와 변경 규칙을 관리하도록 설계할 때 사용하는데, 이러한 구분은 C++ 문법이 강제하는 규칙이 아니라 타입의 의도를 명확하게 표현하기 위한 경험에의한 관례이다.

3.5.1 기본 접근 수준(Default Access Level)

클래스와 구조체의 멤버에는 public, protected, private 세 가지 접근 수준을 지정할 수 있다.

<접근제어-접근수준>

접근 제어	접근 수준
private	해당 클래스 내부에서만 접근
protected	해당 클래스 내부와 파생 클래스에서 접근
public	모든 클래스 외부에서도 접근

접근 수준을 생략하면 class와 struct에 서로 다른 기본값(디폴트 접근제어)이 적용된다.

<디폴트 접근제어>

접근 제어	접근 수준
class	private
struct	public

```
class ClassTypeSample
{
    int value;
};
struct StructTypeSample
{
    int value;
};
```

ClassTypeSample의 value는 private 멤버이므로 외부에서 직접 접근할 수 없다.

```
ClassTypeSample first;
// first.value = 10;    // 오류
```

StructTypeSample의 value는 public 멤버이므로 외부에서 직접 접근할 수 있다.

```
StructTypeSample second;
second.value = 10;
```

접근 수준을 직접 지정하면 class에서도 공개 멤버를 만들 수 있고, struct에서도 비공개 멤버를 만들 수 있다.

```
class PublicData
{
public:
    int value;
};
struct PrivateData
{
private:
    int value;
};
```

따라서 class의 모든 멤버가 항상 비공개이고, struct의 모든 멤버가 항상 공개되는 것은 아니다. 접근 수준을 생략했을 때 적용되는 기본값이 다를 뿐이다.
상속에서도 기본 접근 수준이 다르다. class의 기본 상속은 private 상속이고, struct의 기본 상속은 public 상속이다. 상속의 접근 수준은 이후 장에서 자세히 다룬다.

3.5.2 class와 struct의 공통점

C++의 class와 struct는 모두 클래스 타입을 정의하는 문법이며 같은 클래스 기능을 사용할 수 있다.

- 멤버 변수와 멤버 함수

- 생성자와 소멸자
- 접근 제어
- 정적 멤버
- 연산자 오버로딩
- 상속과 가상 함수
- 템플릿

C++ 구조체는 멤버 변수뿐만 아니라 멤버 함수도 정의할 수 있다.

```
struct Position
{
    int x,y;
    void Move(int offsetX, int offsetY)
    {
        x += offsetX;
        y += offsetY;
    }
};

Position position;
position.x = 10;
position.y = 20;
position.Move(5, -10);
```

구조체에 멤버 함수가 있다고 해서 반드시 class로 바꿔야 하는 것은 아니다. 멤버 함수가 단순히 구조체가 나타내는 값을 계산하거나 변경하는 기능이라면 struct를 사용할 수 있다.

C++은 C에서 사용하던 struct 문법을 계승하였다. 그러나 C++의 struct는 단순히 C 언어와의 호환을 위해서만 존재하는 문법은 아니다. C++에서는 struct도 생성자, 멤버 함수, 접근 제어와 상속 등을 사용할 수 있는 완전한 클래스 타입이다.

<class, struct의 문법적 차이>

구분	class	struct
기본 멤버 접근 수준	private	public
기본 상속 접근 수준	private	public
멤버 함수	사용 가능	사용 가능
생성자와 소멸자	사용 가능	사용 가능
상속과 가상 함수	사용 가능	사용 가능

두 문법이 제공하는 기능에는 본질적인 차이가 없다. 어떤 타입을 표현하려는지에 따라 적절한 문법을 선택한다.

3.5.3 단순한 데이터 표현

서로 관련된 여러 데이터를 하나로 묶는 것이 목적이라면 struct를 사용하는 것이 자연스럽다.
화면의 한 점은 x 좌표와 y 좌표로 표현할 수 있다.

```
struct Position
{
    int x, y;
};
```

Position은 두 좌표를 하나의 데이터로 묶은 타입이다.

```
Position playerPosition;
playerPosition.x = 100;
playerPosition.y = 200;
```

구조체는 함수의 매개변수와 반환 타입으로도 사용할 수 있다.

```
void PrintPosition(const Position& position)
{
    std::cout << position.x << ", "
```

```
              << position.y << '\n';
}
Position MovePosition( const Position& position, int offsetX, int offsetY)
{
    Position result;
    result.x = position.x + offsetX;
    result.y = position.y + offsetY;
    return result;
}
```

Position의 멤버는 좌표값 자체를 나타낸다. 외부에서 x와 y를 직접 읽고 변경해도 타입의 의미가 손상되지 않으므로 공개된 데이터로 구성할 수 있다.
일반적으로 구조체는 값 자체를 나타내며 각 멤버를 독립적으로 읽고 변경할 수 있는 데이터를 표현하는 데 적합하다.

<구조체로 표현하기 좋은 데이터>

```
좌표와 크기
색상값
날짜와 시간
정점 데이터
파일이나 네트워크에서 읽은 단순 데이터
함수의 여러 반환값
```

3.5.4 값 의미를 가지는 struct

구조체는 여러 데이터를 모아 놓은 형태뿐만 아니라 하나의 값을 나타내는 타입으로 사용할 수도 있다. 값 타입은 객체의 고유한 정체성보다 저장된 값 자체가 중요한 타입이다. 같은 값을 가진 두 객체는 같은 의미로 취급할 수 있으며, 값을 복사하면 원본과 독립된 새로운 값이 만들어진다.

앞에서 정의한 Position을 복사해 보자.

```
Position first;
first.x = 10;
first.y = 20;
Position second = first;
```

first의 멤버값이 second로 복사된다.

```
first  → (10, 20)
second → (10, 20)
```

이후 second를 변경해도 first에는 영향을 주지 않는다.

```
second.x = 50;
first  → (10, 20)
second → (50, 20)
```

두 객체는 같은 값에서 시작했지만 서로 독립적으로 존재한다.
값 타입으로 사용하는 구조체에도 값을 계산하거나 변환하는 멤버 함수를 둘 수 있다.

```
struct Position
{
    int x, y;
    void Move(int offsetX, int offsetY)
    {
        x += offsetX;
        y += offsetY;
    }
    bool IsOrigin() const
    {
        return x == 0 && y == 0;
    }
};
```

멤버 함수의 존재 여부만으로 struct와 class를 구분하지 않는다. 타입이 단순한 데이터나 값의 의미를 가지며, 멤버를 직접 공개해도 문제가 없다면 struct를 사용할 수 있다.

<값 타입 구조체 특징>

```
값 자체로 의미 표현
비교적 작은 크기
복사본과 원본의 독립적 사용
멤버의 직접 접근과 변경 허용
복잡한 상태 변경 규칙 없음
```

3.5.5 객체 설계에 사용하는 class

데이터를 단순히 묶는 것이 아니라 객체가 자신의 상태와 기능을 관리해야 한다면 **class**를 사용하는 것이 자연스럽다.

은행 계좌를 생각해 보자. 계좌는 잔액을 가지며 입금과 출금 기능을 제공한다. 잔액은 외부에서 아무 값으로나 변경하는 데이터가 아니라, 정해진 기능을 통해 관리해야 하는 상태이다.

```
class BankAccount
{
public:
    void Deposit(int amount)
    {
        if (amount > 0)
        {
            balance += amount;
        }
    }
    bool Withdraw(int amount)
    {
        if (amount <= 0 || amount > balance)
        {
            return false;
        }
        balance -= amount;
        return true;
    }
    int GetBalance() const
    {
        return balance;
    }
private:
    int balance = 0;
};
```

외부에서는 잔액을 직접 변경하지 않고 계좌가 제공하는 기능을 사용한다.

```
BankAccount account;
account.Deposit(10000);
account.Withdraw(3000);
std::cout << account.GetBalance() << '\n';
```

BankAccount 클래스에서 balance 를 private로 접근 권한을 설정해서 외부에서 잔액에 직접 접근할 수는 없다.

```
// account.balance = -10000;    // 오류
```

BankAccount는 잔액이라는 데이터만 저장하는 타입이 아니다. 입금과 출금 기능을 제공하고, 잔액을 어떻게 변경할 것인지도 직접 결정한다.

```
외부 코드
    │
    ├─ Deposit() 요청
    ├─ Withdraw() 요청
    └─ GetBalance() 요청
          ↓
BankAccount가 잔액 관리
```

이처럼 객체가 자신의 상태를 관리하고 외부에는 필요한 기능만 공개하도록 설계할 때 **class**를 사용한다.

3.5.6 상태와 규칙이 있는 객체

객체의 상태에는 반드시 지켜야 하는 규칙이 있을 수 있다. 객체가 항상 만족해야 하는 상태 조건을 불변식이라고 한다.

BankAccount의 불변식은 잔액을 유효한 상태로 유지하기 위한 규칙으로 정의할 수 있다.

```
- 입금액은 0보다 큰 값으로 제한
- 출금액은 0보다 큰 값으로 제한
- 잔액을 초과하는 출금 금지
- 잔액은 0 이상으로 유지
```

잔액을 공개된 데이터로 두면 외부 코드가 이러한 규칙을 무시할 수 있다.

```
struct AccountData
{
    int balance;
};
AccountData account;
account.balance = -10000;
```

문법적으로는 가능하지만 계좌의 상태로는 올바르지 않다. 앞의 BankAccount 클래스에서는 balance를 private으로 선언하고, Deposit()과 Withdraw()를 통해서만 값을 변경한다.

```
bool Withdraw(int amount)
{
    if (amount <= 0 || amount > balance)
    {
        return false;
    }
    balance -= amount;
    return true;
}
```

출금 조건을 만족하지 않으면 잔액을 변경하지 않는다. 따라서 외부 코드가 Withdraw()를 호출하더라도 계좌는 올바른 상태를 유지할 수 있다.

```
Withdraw() 호출
      ↓
출금액과 잔액 확인
  ↙          ↘
가능          불가능
 ↓             ↓
잔액 감소      상태 유지
```

중요한 것은 데이터를 감추는 행위 자체가 아니다. 객체가 자신의 상태와 상태 변경 규칙을 함께 관리하도록 만드는 것이 목적이다.

class는 상태의 유효성을 유지하기 위한 규칙을 관리하는 경우에 적합하다.

```
- 멤버 사이의 유효 조건 유지
- 외부의 잘못된 직접 대입 제한
- 정해진 절차를 통한 상태 변경
- 상태 변경 결과에 대한 객체의 책임 유지
```

3.5.7 프로젝트에서의 선택 기준

C++ 문법은 struct와 class의 용도를 강제로 구분하지 않는다. struct로도 비공개 멤버와 멤버 함수를 가진 객체를 만들 수 있고, class의 모든 멤버를 공개할 수도 있다. 그러나 프로젝트에서 일관된 기준을 사용하면 타입의 선언만 보고도 설계 의도를 쉽게 파악할 수 있다.

본 교재에서는 객체 설계에는 class, 단순한 데이터나 값의 표현에는 struct를 사용한다.

```
객체를 설계할 때의 기본 선택 → class
단순한 데이터나 값의 표현    → struct
```

앞에서 작성한 두 타입을 비교해 보자.

```
struct Position
{
```

```
    int x, y;
};
```

Position은 두 좌표값을 하나로 묶은 단순한 값이다. 외부에서 멤버를 직접 읽고 변경해도 문제가 없다.

```
class BankAccount
{
public:
    void Deposit(int amount);
    bool Withdraw(int amount);
    int GetBalance() const;
private:
    int balance = 0;
};
```

반면, BankAccount는 잔액과 잔액의 변경 규칙을 관리하는 객체이다. 외부에서는 공개된 기능을 통해서만 상태를 변경해야 한다.

class는 객체가 자신의 상태와 규칙을 관리해야 할 때 우선 고려한다.

```
- 객체 자신의 상태 관리
- 멤버 사이의 상태 규칙 유지
- 외부의 직접적인 상태 변경 제한
- 공개된 기능을 통한 상태 변경
- 명확한 역할과 책임 표현
```

struct는 단순한 데이터나 값의 표현이 목적일 때 우선 고려한다.

```
- 여러 데이터의 단순한 결합
- 외부에서 각 멤버의 직접 접근과 변경
- 값 자체의 의미 표현
- 복사된 값의 독립적인 사용
- 복잡한 상태 변경 규칙 없음
```

<class, struct 선택 기준>

구분	class	struct
기본 접근 수준	private	public
주된 용도	상태와 기능을 관리하는 객체	단순 데이터와 값 표현
멤버 접근	공개된 멤버 함수를 통해 접근	직접 접근하는 경우가 많음
상태 변경 규칙	객체가 직접 관리	없거나 단순함
대표 예	BankAccount, Inventory, Player	Position, Vector2, Color

이 기준은 절대적인 규칙이 아니다. 값 타입이면서 내부 상태를 보호하는 클래스도 있고, 여러 멤버 함수를 가진 구조체도 있다.
중요한 것은 class와 struct 중 어떤 문법이 더 많은 기능을 제공보다는 타입의 역할과 사용 의도가 코드에서 명확하게 드러나는가를 기준으로 선택해야 한다.

3.6 접근 제어(Access Control)와 캡슐화(Encapsulation)

접근 제어는 클래스의 멤버를 어디에서 사용할 수 있는지 제한하는 기능이다. C++에서는 public, protected, private 접근 지정자(Access Specifier)를 사용하여 멤버의 접근 범위를 정한다.

접근 지정자	클래스 내부	파생 클래스	클래스 외부
private	가능	불가능	불가능
protected	가능	가능	불가능
public	가능	가능	가능

class에서 접근 지정자를 생략하면 private가 적용되고, struct에서는 public이 적용된다.

```
class Player
```

```
{
public:
    void Move();
protected:
    int level = 1;
private:
    int hp = 100;
};
```

하나의 접근 지정자는 다음 접근 지정자가 나오거나 클래스 정의가 끝날 때까지 적용된다. 접근 지정자는 필요한 경우 여러 번 사용할 수 있으며, 선언 순서가 문법으로 정해져 있는 것은 아니다.

일반적으로 외부에서 사용할 기능은 public, 파생 클래스에서 사용할 멤버는 protected, 객체 내부에서만 관리할 상태와 기능은 private으로 선언한다.

3.6.1 public

public 멤버는 클래스 외부에서 접근할 수 있다. 외부 객체가 사용할 수 있도록 공개한 멤버 함수들은 클래스의 공개 인터페이스(Public Interface)를 구성한다.

```
class Door
{
public:
    void Open()
    {
        isOpen = true;
    }
    void Close()
    {
        isOpen = false;
    }
    bool IsOpen() const
    {
        return isOpen;
    }
private:
    bool isOpen = false;
};
```

외부에서는 공개된 멤버 함수를 통해 Door 객체를 사용한다.

```
Door door;
door.Open();
if (door.IsOpen())
{
    std::cout << "문이 열려 있습니다.\n";
}
```

public은 단순히 외부에서 접근할 수 있다는 의미만 가지는 것이 아니다. 클래스가 외부에 어떤 기능을 제공할 것인지 나타내는 역할도 한다.

```
• Door의 공개 기능
- Open()
- Close()
- IsOpen()
```

외부에서는 문이 열린 상태를 어떤 변수에 저장하는지 알 필요가 없다. 공개된 기능의 사용 방법만 알면 객체를 사용할 수 있다.
멤버 변수를 public으로 선언할 수도 있다.

```
struct Position
{
    int x, y;
};
```

```
Position position;
position.x = 100;
position.y = 200;
```

Position처럼 멤버를 직접 읽고 변경하는 것이 타입의 목적에 맞는 단순한 데이터에서는 공개 멤버를 사용할 수 있다. 그러나 상태 변경 규칙이 있는 객체에서는 멤버 변수를 직접 공개하기보다 필요한 멤버 함수를 제공하는 편이 좋다.

3.6.2 protected

protected 멤버는 클래스 외부에서는 접근할 수 없지만, 해당 클래스를 상속한 파생 클래스(Derived Class)에서는 접근할 수 있다.

```
class Character
{
protected:
    int hp = 100;
};
// 파생클래스
class Player : public Character
{
public:
    void Recover()
    {
        hp += 10;
    }
};
```

Player는 Character를 상속하므로 기반 클래스의 protected 멤버인 hp에 접근할 수 있다. 그러나 일반 코드에서는 직접 접근할 수 없다.

```
Player player;
// player.hp = 200;    // 오류
```

protected는 기반 클래스가 파생 클래스에 내부 상태나 기능을 제공할 때 사용한다.

접근 수준	외부 코드	파생 클래스
public	접근 가능	접근 가능
protected	접근 불가능	접근 가능
private	접근 불가능	직접 접근 불가능

모든 멤버를 파생 클래스에 공개할 필요는 없다. 파생 클래스가 기반 클래스의 내부 상태에 지나치게 의존하면 기반 클래스의 구현을 변경하기 어려워질 수 있다.
따라서 특별한 이유가 없다면 상태를 private으로 두고, 파생 클래스에도 필요한 기능을 멤버 함수로 제공하는 방법을 우선 고려할 수 있다.
protected의 자세한 사용 기준은 상속을 다루는 장에서 다시 살펴본다.

3.6.3 private

private 멤버는 클래스 내부의 멤버 함수에서만 직접 접근할 수 있다. 객체의 내부 상태와 외부에 공개할 필요가 없는 구현을 표현할 때 사용한다.

```
class Counter
{
public:
    void Increase()
    {
        ++count;
    }
    int GetCount() const
    {
        return count;
    }
private:
    int count = 0;
};
```

Counter 객체의 count는 외부에서 직접 변경할 수 없다.

```
Counter counter;
counter.Increase();
```

```
std::cout << counter.GetCount() << '\n';
// counter.count = 100;    // 오류
```

외부에서는 Increase()와 GetCount()만 사용할 수 있다. 값을 어떻게 저장하고 증가시키는지는 Counter가 관리한다.
private을 사용하면 클래스의 내부 구현과 외부에서 사용하는 기능을 분리할 수 있다.

구분	내용
공개 인터페이스	외부에서 사용할 수 있는 public 멤버
내부 구현	외부에서 직접 접근할 수 없는 private 멤버
상태 변경	클래스가 제공하는 멤버 함수를 통해 수행

private 멤버가 있다고 해서 자동으로 좋은 객체 설계가 되는 것은 아니다. 중요한 것은 객체의 상태를 어떤 기능을 통해 변경하게 할 것인지 함께 설계하는 것이다.

3.6.4 정보 은닉(Information Hiding)

정보 은닉은 객체를 사용하는 데 필요하지 않은 내부 구현을 외부에 노출하지 않는 것이다. 외부에서는 객체가 제공하는 기능만 사용하고, 내부에서 데이터를 어떤 방식으로 저장하고 처리하는지는 알 필요가 없다.

객체를 하나의 블랙박스처럼 생각할 수 있다.

```
외부 코드
   │
   ├─ AddItem() 요청
   ├─ RemoveItem() 요청
   └─ GetItemCount() 요청
            ↓
        ┌──────────────────┐
        │  Inventory       │
        │                  │
        │ 내부 저장 방식   │
        │ 내부 변경 규칙   │
        └──────────────────┘
```

현재 Inventory 클래스는 아이템 저장 방법을 외부에 공개하지 않는다.

```
class Inventory
{
public:
    bool AddItem(int itemId);
    bool RemoveItem(int itemId);
    int GetItemCount() const;

private:
    int itemIds[100];
    int itemCount = 0;
};
```

외부에서는 itemIds 배열이나 itemCount를 직접 다루지 않는다.

```
Inventory inventory;
inventory.AddItem(1001);
inventory.AddItem(1002);
inventory.RemoveItem(1001);
std::cout << inventory.GetItemCount() << '\n';
```

나중에 아이템을 저장하는 방식을 배열에서 다른 컨테이너로 변경하더라도 AddItem(), RemoveItem(), GetItemCount()의 사용 방법을 그대로 유지할 수 있다.

```
[기존 내부 구현]
int itemIds[100];
      |
```

```
      ↓ 변경
      |
새로운 내부 구현
[다른 컨테이너 사용]
```

공개 인터페이스가 유지되면 외부 코드는 내부 구현의 변경에 영향을 적게 받는다.
캡슐화는 객체의 상태와 기능을 하나의 단위로 묶고 외부 접근을 제어하는 것이며, 정보 은닉은 그 과정에서 객체의 내부 구현을 외부에 노출하지 않는 원리이다.

개념	의미
캡슐화	상태와 기능을 객체라는 하나의 단위로 묶음
접근 제어	멤버에 접근할 수 있는 범위를 제한함
정보 은닉	객체 사용에 필요하지 않은 내부 구현을 감춤

3.6.5 객체의 유효한 상태(Valid State)

객체의 상태는 아무 값이나 가질 수 있는 것이 아니다. 프로그램의 규칙에 맞는 올바른 상태를 유지해야 한다.

객체가 항상 만족해야 하는 상태 조건을 불변식(Invariant)이라고 한다. 여기서 불변식은 값이 변하지 않는다는 뜻이 아니라, 값이 변경된 뒤에도 반드시 유지되어야 하는 규칙을 의미한다.

게임 캐릭터의 체력은 몇 가지 규칙을 만족해야 한다.

```
최대 체력은 0보다 커야 한다.
현재 체력은 0보다 작을 수 없다.
현재 체력은 최대 체력보다 클 수 없다.
```

체력값을 공개하면 외부에서 잘못된 상태를 만들 수 있다.

```
struct HealthData
{
    int current, maximum;
};

HealthData health;

health.current = -100;
health.maximum = 0;
```

문법적으로는 가능하지만 체력 데이터로는 올바르지 않은 상태이다.
상태와 변경 규칙을 객체가 관리하도록 클래스로 구성할 수 있다.

```
class Health
{
public:
    void TakeDamage(int damage)
    {
        if (damage <= 0)
        {
            return;
        }
        if (current -= damage; current < 0)
        {
            current = 0;
        }
    }
    void Recover(int amount)
    {
        if (amount <= 0)
        {
            return;
        }
```

```
        if (current += amount; current > maximum)
        {
            current = maximum;
        }
    }
    int GetCurrent() const
    {
        return current;
    }
    int GetMaximum() const
    {
        return maximum;
    }
private:
    int current = 100;
    int maximum = 100;
};
```

외부에서는 current와 maximum을 직접 변경할 수 없다.

```
Health health;
health.TakeDamage(30);
health.Recover(10);
std::cout << health.GetCurrent() << '\n';
```

상태는 공개된 멤버 함수를 통해서만 변경되므로 객체가 정한 규칙을 항상 적용할 수 있다.

```
TakeDamage()
      ↓
피해량 검사
      ↓
현재 체력 감소
      ↓
0보다 작으면 0으로 조정
```

```
Recover()
      ↓
회복량 검사
      ↓
현재 체력 증가
      ↓
최대 체력보다 크면 최대 체력으로 조정
```

객체의 유효한 상태를 유지하려면 초기 상태, 입력값 검사, 변경 규칙, 경계값과 외부 접근을 함께 고려해야 한다.

확인 항목	내용
초기 상태	객체가 생성된 직후 올바른 상태인가
입력값 검사	잘못된 값이 전달되었을 때 처리할 수 있는가
변경 규칙	상태를 변경할 때 필요한 조건을 검사하는가
경계값	최소값과 최대값을 벗어나지 않는가
외부 접근	외부에서 규칙을 우회하여 상태를 변경할 수 있는가

객체의 내부 데이터를 숨기는 목적은 단순히 외부 접근을 막는 데 있지 않다. 객체가 자신의 상태와 상태 변경 규칙을 함께 관리하여 항상 올바른 상태를 유지하도록 만드는 것이 중요하다.

메모리 직접 접근과 불변식
private은 객체의 메모리를 물리적으로 보호하는 기능이 아니라, 외부 코드의 직접적인 멤버 접근을 제한하는 기능이다. 포인터와 형 변환을 사용하여 객체의 내부 메모리를 강제로 조작하면 클래스가 제공하는 상태 변경 규칙을 우회하여 불변식을 파괴할 수 있다.

```
#include <iostream>
```

```cpp
using namespace std;

class PlayerStats
{
private:
    // 외부의 직접 변경 제한. 클래스의 변경 규칙을 통한 불변식(Invariant) 유지.
    int hp{10}, mp{20}, level{30};
public:
    PlayerStats() = default;
    int GetHp() const { return hp; }
    int GetMp() const { return mp; }
    int GetLevel() const { return level; }
};

int main()
{
    PlayerStats playerStat;
    for(int i = 0; i < 3; ++i)
    {
        // 위험: 접근 제어와 캡슐화 우회
        // 객체 시작 주소를 int*로 강제 변환. 포인터 연산으로
        // 객체 내부 메모리에 직접 접근.
        int* data = (int*)&playerStat + i;

        // 위험: 객체의 불변식 파괴. private 접근 제어 우회
        // 상태 변경 규칙을 거치지 않고 값 변경.
        // 객체를 int 배열처럼 가정한 접근
        // Undefined Behavior 발생 가능.
        *data = 100 + i * i + i * 500;
    }

    cout << "invalid_private" << '\n';
    // 강제 메모리 변경 이후의 객체 상태 확인
    cout << "HP   : " << playerStat.GetHp() << '\n';
    cout << "MP   : " << playerStat.GetMp() << '\n';
    cout << "Level: " << playerStat.GetLevel() << '\n';
}
```

이 코드는 객체의 멤버가 int 배열처럼 연속되어 있다고 가정하여 접근한다. 그러나 C++은 이러한 포인터 연산을 보장하지 않으며, 객체의 메모리 배치와 패딩 등을 무시한 접근은 미정의 동작(Undefined Behavior)을 일으킬 수 있다.

```
공개 멤버 함수 → 변경 규칙 적용 → 불변식 유지
메모리 직접 접근 → 변경 규칙 우회 → 불변식 파괴 가능
```

3.6.6 접근자 함수(Accessor Function) 사용 시 주의점

비공개 멤버의 값을 읽거나 변경하기 위해 제공하는 함수를 접근자 함수라고 한다. 값을 반환하는 함수를 게터(Getter), 값을 설정하는 함수를 세터(Setter)라고 부르기도 한다.

```cpp
class Player
{
public:
    int GetHp() const
    {
        return hp;
    }
    void SetHp(int value)
    {
        hp = value;
    }
private:
    int hp = 100;
};
```

hp는 private이지만 SetHp()를 사용하면 외부에서 어떤 값이든 대입할 수 있다.

```cpp
Player player;
player.SetHp(-1000);
```

멤버 변수를 private으로 선언하고 단순한 게터와 세터를 추가했다고 해서 자동으로 캡슐화가 이루어지는 것은 아니다. 모든 값을 그대로 읽고 변경할 수 있게 하면 공개 멤버 변수와 큰 차이가 없을 수 있다.
체력을 관리하는 객체라면 임의의 값을 설정하는 SetHp()보다 체력의 의미에 맞는 기능을 제공하는 것이 좋다.

```
class Player
{
public:
    void TakeDamage(int damage)
    {
        if (damage <= 0)
        {
            return;
        }
        if (hp -= damage; hp < 0)
        {
            hp = 0;
        }
    }
    void Recover(int amount)
    {
        if (amount <= 0)
        {
            return;
        }
        if (hp += amount; hp > maxHp)
        {
            hp = maxHp;
        }
    }
    int GetHp() const
    {
        return hp;
    }

private:
    int hp = 100;
    int maxHp = 100;
};
```

```
SetHp(-1000) → 외부에서 객체의 상태를 임의로 결정
TakeDamage(30) → 객체가 피해 처리 규칙에 따라 상태 변경
```

게터도 반드시 필요한 경우에만 제공해야 한다. 외부 코드가 내부 상태를 알 필요가 없다면 값을 공개하지 않는 편이 좋다.
예를 들어 외부에서 체력이 0인지 확인하는 것만 필요하다면 GetHp() 대신 목적이 분명한 함수를 제공할 수 있다.

```
bool IsDead() const
{
    return hp == 0;
}
```

접근자 함수를 설계할 때는 외부 읽기와 변경의 필요성, 값 변경 규칙, 반환을 통한 내부 상태 노출 여부를 확인한다.

질문	권장 방법
외부에서 값을 읽어야 하는가	필요한 경우에만 게터 제공
외부에서 임의의 값을 대입해도 되는가	가능할 때만 세터 제공
값 변경에 규칙이 있는가	의미가 분명한 멤버 함수 제공
외부에서 필요한 것이 값 자체인가	목적을 나타내는 질의 함수 고려
반환값을 통해 내부 상태가 변경될 수 있는가	값이나 const 참조로 제한

비공개 멤버의 비상수 참조를 반환하면 외부 코드가 반환된 참조를 통해 객체의 내부 상태를 직접 변경할 수 있다.

```
#include <string>
```

```cpp
class Profile
{
public:
    std::string& GetName()
    {
        return name;    // 비공개 멤버 name의 비상수 참조 반환.
    }

private:
    std::string name;
};

int main()
{
    Profile profile;

    // 반환된 참조를 통해 비공개 멤버 name을 직접 변경.
    profile.GetName() = "Player";

    return 0;
}
```

name은 private 멤버이므로 외부에서 직접 접근할 수 없다. 그러나 GetName()이 비상수 참조를 반환하므로 외부 코드는 반환된 참조를 통해 name을 직접 변경할 수 있다. 이는 접근 제어를 우회하여 내부 상태를 공개하는 것과 비슷한 결과를 만들 수 있으므로 주의해야 한다.

```cpp
const std::string& GetName() const
{
    return name;
}
```

이름 변경에 별도의 규칙이 있다면 SetName()보다 의미가 분명한 Rename()과 같은 함수를 제공할 수도 있다.

```cpp
bool Rename(const std::string& newName)
{
    if (newName.empty())
    {
        return false;
    }

    name = newName;
    return true;
}
```

접근자 함수는 비공개 멤버마다 기계적으로 만드는 함수가 아니다. 객체 외부에서 실제로 필요한 기능을 기준으로 제공해야 한다.

```
멤버 변수
    ↓ 기계적으로 게터와 세터 추가
좋은 객체 설계가 보장되지 않음

객체의 역할과 규칙 확인
    ↓
필요한 기능만 공개
    ↓
상태와 불변식 유지
```

접근 제어와 캡슐화의 목적은 데이터를 무조건 숨기는 것이 아니다. 객체가 자신의 상태와 규칙을 책임지고, 외부에는 객체를 올바르게 사용하는 데 필요한 기능만 제공하도록 만드는 것이다.

```cpp
class Health
{
public:
    void TakeDamage(int damage)
    {
        if (damage <= 0)
        {
            return;
        }
```

```
        current -= damage;
        if (current < 0)
        {
            current = 0;
        }
    }
    void Recover(int amount)
    {
        if (amount <= 0)
        {
            return;
        }
        current += amount;
        if (current > maximum)
        {
            current = maximum;
        }
    }
    int GetCurrent() const
    {
        return current;
    }
    int GetMaximum() const
    {
        return maximum;
    }
private:
    int current = 100;
    int maximum = 100;
};
```

3.7 멤버 함수(Member Function)와 this

멤버 함수는 클래스에 정의되어 객체의 상태를 처리하거나 객체가 제공할 기능을 구현하는 함수이다. 일반 함수는 필요한 데이터를 매개변수로 전달받지만, 멤버 함수는 자신을 호출한 객체의 멤버에 직접 접근할 수 있다.

```
struct Vector2
{
    float x, y;
};
class Player
{
public:
    void Move(float offsetX, float offsetY)
    {
        position.x += offsetX;
        position.y += offsetY;
    }
private:
    Vector2 position = { 0.0f, 0.0f };
};
```

Move()는 매개변수로 Player 객체를 전달받지 않지만, 자신을 호출한 객체의 position을 변경한다. 멤버 함수 안에서는 호출한 객체를 가리키는 this 포인터를 사용할 수 있다.

3.7.1 멤버 함수의 호출(Member Function Call)

멤버 함수는 객체와 멤버 접근 연산자 .을 사용하여 호출한다.

```
Player first;
Player second;
first.Move(10.0f, 20.0f);
second.Move(-5.0f, 30.0f);
```

두 객체는 같은 Move()를 호출하지만 서로 다른 position을 변경한다.

```
first.Move(10.0f, 20.0f)
        ↓
first의 position 변경

second.Move(-5.0f, 30.0f)
        ↓
second의 position 변경
```

클래스에 정의된 멤버 함수의 코드는 객체마다 별도로 만들어지지 않는다. 여러 객체가 같은 멤버 함수 코드를 공유하고, 호출할 때 어떤 객체를 대상으로 실행할 것인지 결정된다.

호출 표현	멤버 함수가 처리하는 객체
first.Move(10.0f, 20.0f)	first
second.Move(-5.0f, 30.0f)	second

객체의 포인터로 멤버 함수를 호출할 때는 화살표 연산자 ->를 사용한다.

```
Player player;
Player* selectedPlayer = &player;
selectedPlayer->Move(10.0f, 20.0f);
```

화살표 연산자를 사용한 호출과 포인터를 역참조한 뒤 점 연산자를 사용한 호출은 같은 의미이다.

```
selectedPlayer->Move(10.0f, 20.0f);
(*selectedPlayer).Move(10.0f, 20.0f);
```

대상	멤버 접근	예
객체	.	player.Move()
객체의 포인터	->	playerPointer->Move()

멤버 함수가 호출되면 호출에 사용한 객체의 주소가 함수에 암시적으로 전달된다. 멤버 함수는 이 주소를 이용하여 어느 객체의 멤버를 처리할 것인지 구분한다.

```
객체.멤버함수()
        ↓
호출한 객체의 주소가 암시적으로 전달
        ↓
해당 객체의 멤버 처리
```

이때 호출한 객체의 주소를 가리키는 포인터가 this이다.

3.7.2 this 포인터(this Pointer)

this는 현재 멤버 함수를 호출한 객체를 가리키는 포인터이다. 정적 멤버 함수가 아닌 모든 멤버 함수 안에서 사용할 수 있다.

제시된 Window 클래스는 창의 크기를 관리한다.

```
class Window
{
public:
    void SetSize(int width, int height)
    {
        this->width = width;
        this->height = height;
    }
private:
    int width = 800, height = 600;
};
```

SetSize()의 매개변수 이름과 멤버 변수 이름이 모두 width, height로 같다. 함수 안에서 이름만 사용하면 가까운 범위에 선언된 매개변수를 의미한다.

```
width = width;
height = height;
```

위 코드는 매개변수에 자기 자신의 값을 대입하므로 멤버 변수는 변경되지 않는다.

this->를 사용하면 현재 객체의 멤버를 명확하게 지정할 수 있다.

```
this->width = width;
this->height = height;
```

표현	의미
this	현재 객체의 주소
*this	현재 객체
this->width	현재 객체의 멤버 width
(*this).width	현재 객체의 멤버 width

샘프로 코드에서 first.SetSize()가 호출될 때 this는 first를 가리킨다.

```
Window first;
Window second;

first.SetSize(1280, 720);
second.SetSize(1920, 1080);

first.SetSize(1280, 720)
this → first

second.SetSize(1920, 1080)
this → second
```

멤버 이름과 매개변수 이름이 다르면 this->를 생략할 수 있다.

```
void SetSize(int newWidth, int newHeight)
{
    width = newWidth;
    height = newHeight;
}
```

따라서 모든 멤버 접근에 this->를 반드시 붙여야 하는 것은 아니며 상황에 맞게 사용한다.

<this 사용 상황>

사용 상황	목적
매개변수와 멤버의 이름이 같을 때	두 이름을 구분
현재 객체를 다른 함수에 전달할 때	객체의 주소 사용
현재 객체 자신을 반환할 때	*this 사용
템플릿의 기반 클래스 멤버를 참조할 때	멤버 탐색을 명확하게 지정

마지막 경우는 템플릿 상속을 다룰 때 살펴본다.

this는 정적 멤버 함수(Static Member Function)에서는 사용할 수 없다. 정적 멤버 함수는 특정 객체를 통해서만 실행되는 함수가 아니므로 현재 객체를 가리키는 this가 존재하지 않는다.

멤버 함수 종류	this 사용
일반 멤버 함수	가능

상수 멤버 함수	가능
정적 멤버 함수	불가능

정적 멤버 함수는 다음 절에서 자세히 다룬다.

3.7.3 객체 자신을 반환하기(Returning the Current Object)

멤버 함수는 처리 결과로 현재 객체를 반환할 수 있다. 현재 객체를 가리키는 포인터는 this이며, *this는 this가 가리키는 현재 객체를 의미한다.

화면 출력 설정을 관리하는 RenderOption 클래스에서 각 설정 함수는 상태를 변경한 뒤 현재 객체를 참조로 반환한다.

```
class RenderOption
{
public:
    RenderOption& SetResolution(int width, int height)
    {
        if (width > 0 && height > 0)
        {
            this->width = width;
            this->height = height;
        }
        return *this;
    }
    RenderOption& SetFullScreen(bool enabled)
    {
        fullScreen = enabled;
        return *this;
    }
    RenderOption& SetVSync(bool enabled)
    {
        vSync = enabled;
        return *this;
    }
private:
    int width = 1280, height = 720;
    bool fullScreen = false;
    bool vSync = true;
};
```

this는 현재 객체를 가리키는 포인터이므로 *this는 현재 객체 자체를 의미한다.

```
this  → 현재 객체의 주소
*this → 현재 객체
```

반환 타입이 레퍼런스 타입 "RenderOption&" 이면 참조로 반환하여 호출한 객체 자체가 반환된다.

```
RenderOption& SetFullScreen(bool enabled)
{
    fullScreen = enabled;
    return *this;
}
```

현재 객체의 반환 방식은 반환 타입에 따라 달라진다.

반환 타입	반환 표현	의미
RenderOption	return *this;	현재 객체의 복사본을 값으로 반환
RenderOption&	return *this;	현재 객체를 가리키는 참조를 반환
RenderOption*	return this;	현재 객체의 주소를 반환

현재 객체와 같은 객체를 계속 사용하려면 일반적으로 참조 타입을 반환한다.

```
RenderOption option;
RenderOption& result = option.SetResolution(1920, 1080);
```

result는 새로운 객체가 아니라 option을 참조한다.

```
option ──────┐
             ├─ 같은 객체
result ──────┘
```

result를 통해 상태를 변경하면 option의 상태가 변경된다.

```
result.SetFullScreen(true);
```

반환 타입을 값 타입으로 선언하면 return *this;는 현재 객체의 복사본을 반환한다.

```
RenderOption SetFullScreen(bool enabled)
{
    fullScreen = enabled;
    return *this;
}
RenderOption option;
RenderOption result = option.SetFullScreen(true);
```

이 경우 option과 result는 서로 다른 객체이다.

```
option ──────────── 원본 객체
result ──────────── 반환된 복사본
```

컴파일러가 실제 복사 과정을 최적화할 수는 있지만, 프로그램에서 관찰되는 의미는 호출한 객체와 독립된 값을 반환하는 것이다. 호출한 객체와 같은 객체를 계속 사용하려는 목적이라면 RenderOption& 반환이 적합하다.

객체 자신을 가리키는 참조를 반환하면 같은 객체에 여러 멤버 함수를 연속해서 호출할 수 있다.

3.7.4 메서드 체이닝(Method Chaining)

객체 자신을 참조로 반환하면 반환된 참조를 대상으로 다른 멤버 함수를 계속 호출할 수 있다. 이러한 호출 방식을 메서드 체이닝이라고 한다. RenderOption의 설정 함수는 모두 RenderOption&를 반환하므로 점 연산자(.)를 사용해 호출을 연속해서 연결할 수 있다.

```
RenderOption option;
option.SetResolution(1920, 1080)
      .SetFullScreen(true)
      .SetVSync(false);
```

첫 번째 호출인 SetResolution()은 option의 해상도를 변경하고 option을 가리키는 참조를 반환한다. 두 번째 호출은 그 참조를 대상으로 실행되며, 마지막 호출까지 같은 객체를 계속 사용한다.

```
option.SetResolution(1920, 1080)
      │
      └─ option의 참조 반환
                │
                ▼
       .SetFullScreen(true)
                │
                └─ option의 참조 반환
                          │
                          ▼
                 .SetVSync(false)
```

각 호출을 분리해서 작성해도 결과는 같다.

```
option.SetResolution(1920, 1080);
option.SetFullScreen(true);
option.SetVSync(false);
```

<호출 과정 상태와 반환값>

호출 순서	실행되는 함수	변경되는 상태	반환값

1	SetResolution()	해상도	option의 참조
2	SetFullScreen()	전체 화면 여부	option의 참조
3	SetVSync()	수직 동기화 여부	option의 참조

메서드 체이닝은 하나의 객체를 여러 단계로 설정하는 코드를 간결하게 만든다.

```
RenderOption option;
option.SetResolution(2560, 1440)
      .SetFullScreen(false)
      .SetVSync(true);
```

모든 멤버 함수가 객체 자신을 반환해야 하는 것은 아니다. 함수의 반환 타입은 함수가 호출자에게 전달해야 하는 결과를 기준으로 결정해야 한다.

아이템 추가 기능이 작업의 성공 여부를 알려야 한다면 bool을 반환하는 편이 자연스럽다.

```
bool AddItem(int itemId);
```

메서드 체이닝을 지원하기 위해 이 함수를 억지로 객체 참조 반환으로 변경하면 호출자는 아이템 추가의 성공 여부를 직접 확인하기 어려워진다.

함수의 목적	적합한 반환값
객체의 설정을 변경하고 계속 사용	객체의 참조
작업 성공 여부 전달	bool
계산 결과 전달	계산 결과의 타입
새로운 객체 생성	새로운 객체의 타입
반환할 결과가 없음	void

메서드 체이닝은 호출 관계를 간결하고 읽기 쉽게 만들 때 사용한다. 호출이 지나치게 길어지거나 각 단계에서 오류를 확인해야 한다면 멤버 함수 호출을 분리하는 편이 명확하다.

3.7.5 상수 멤버 함수(Const Member Function)

객체의 상태를 변경하지 않는 멤버 함수는 매개변수 목록 뒤에 const를 붙여 상수 멤버 함수로 선언할 수 있다.

```
class RenderOption
{
public:
    int GetWidth() const
    {
        return width;
    }
private:
    int width = 1280;
};
```

상수 멤버 함수에서는 일반 멤버 변수를 변경할 수 없으며, 상수 객체는 상수 멤버 함수만 호출할 수 있다. const 오버로딩, 상수 참조 반환과 논리적 상수성은 4장에서 자세히 다룬다.

3.8 정적 멤버(Static Member)

클래스의 멤버는 객체마다 별도로 존재하는 멤버와 클래스 전체가 공유하는 멤버로 구분할 수 있다.
일반 멤버 변수는 객체마다 별도의 저장 공간을 가지므로 인스턴스 변수(Instance Variable)라고도 한다. C++에서는 이를 비정적 데이터 멤버(Non-static Data Member)라고 한다.

반면 static으로 선언한 멤버 변수는 클래스 전체에서 하나만 존재하며 모든 객체가 공유한다. 이러한 변수를 클래스 변수(Class Variable)라고도 하며, C++에서는 정적 데이터 멤버(Static Data Member)라고 한다.

<인스턴스 변수와 클래스 변수 비교>

구분	비정적 데이터 멤버	정적 데이터 멤버
소속	각각의 객체	클래스
저장 공간	객체마다 별도로 존재	클래스 전체에서 하나만 존재
값	객체마다 다를 수 있음	모든 객체가 공유
객체 생성 필요	객체가 생성되어야 존재	객체가 없어도 사용 가능
접근 방법	객체를 통해 접근	클래스 범위로 접근
this와의 관계	this가 가리키는 객체의 멤버	특정 객체에 속하지 않음

static의 기본적인 의미는 C++이라고 달라지지 않는다. 저장 공간은 한 번만 마련되고 프로그램이 실행되는 동안 유지된다. 다만 클래스 안에서 사용하면 이름과 기능이 클래스에 소속된다는 점이 추가된다.

3.8.1 정적 데이터 멤버(Static Data Member)

변수는 이름, 형식, 값, 저장 위치, 범위와 수명의 여섯 가지 속성을 가진다.

<변수의 속성>

속성	의미
이름(Name)	변수를 구분하는 식별자
형식(Type)	값의 크기와 해석 방법
값(Value)	변수에 저장된 데이터
저장 위치(Storage Location)	값이 저장되는 메모리 위치
범위(Scope)	이름을 사용할 수 있는 영역
수명(Lifetime)	저장 공간이 유지되는 기간

이 여섯 가지 속성을 기준으로 일반 멤버 변수와 정적 데이터 멤버를 비교해 보자.

```
class Player
{
public:
    int hp = 100;
    static int maxPlayerCount;
};
```

hp는 객체마다 존재하는 비정적 데이터 멤버이고, maxPlayerCount는 클래스 전체에서 하나만 존재하는 정적 데이터 멤버이다.

속성	hp	maxPlayerCount
이름	Player 클래스의 멤버 이름	Player 클래스의 멤버 이름
형식	int	int
값	객체마다 별도의 값	모든 객체가 하나의 값 공유
저장 위치	각 객체의 저장 공간 안	객체와 별도의 저장 공간
범위	Player 클래스 범위	Player 클래스 범위
수명	객체의 수명과 같음	프로그램 실행 동안 유지

정적 데이터 멤버는 클래스 안에서 선언하지만, 일반적으로 클래스 밖에서 별도로 정의해야 한다.

```
int Player::maxPlayerCount = 4;
```

클래스 밖에서 정의할 때는 static을 다시 쓰지 않는다.
static int Player::maxPlayerCount = 4; // 오류

<정적 변수의 클래스 안에서 선언과 클래스 밖에서 정의>

코드	역할
static int maxPlayerCount;	정적 데이터 멤버 선언
int Player::maxPlayerCount = 4;	저장 공간 정의와 초기화

정적 데이터 멤버는 클래스 범위 연산자 ::를 사용하여 접근할 수 있다.

```
std::cout << Player::maxPlayerCount << '\n';
Player::maxPlayerCount = 8;
```

객체를 통해 접근하는 것도 문법적으로 허용된다.

```
Player first;
Player second;
first.maxPlayerCount = 10;
std::cout << second.maxPlayerCount << '\n';
```

first를 통해 값을 변경했지만 second를 통해 읽어도 같은 값이 출력된다. maxPlayerCount는 first나 second에 속한 값이 아니라 Player 클래스가 공유하는 하나의 값이기 때문이다.

```
Player 클래스
└─ maxPlayerCount = 10

first 객체
└─ hp = 100

second 객체
└─ hp = 100
```

<정적 멤버 변수 접근 방법 비교>

접근 표현	사용 가능 여부	의미 전달
Player::maxPlayerCount	가능	클래스에 속한 값 으로 명확
first.maxPlayerCount	가능	객체의 멤버처럼 보일 수 있음
second.maxPlayerCount	가능	객체의 멤버처럼 보일 수 있음

정적 데이터 멤버는 객체보다 클래스에 속하므로 클래스 범위 연산자를 사용하여 접근하는 것이 좋다.

```
Player::maxPlayerCount = 8;
```

정적 데이터 멤버는 객체의 저장 공간에 포함되지 않는다. 따라서 정적 데이터 멤버를 추가해도 각 객체의 크기는 증가하지 않는다.

```
class Sample
{
public:
    int value;

    static int sharedValue;
};
```

sharedValue는 Sample 객체마다 들어 있는 멤버가 아니므로 sizeof(Sample)에는 포함되지 않는다. 정적 데이터 멤버도 public, protected, private 접근 제어를 적용받는다.

```
class Player
{
public:
```

```
    static int GetMaxPlayerCount()
    {
        return maxPlayerCount;
    }
private:
    static int maxPlayerCount;
};

int Player::maxPlayerCount = 4;
```

maxPlayerCount는 private이므로 클래스 외부에서 직접 접근할 수 없다.

```
std::cout << Player::GetMaxPlayerCount() << '\n';
// Player::maxPlayerCount = 10;    // 오류
```

static은 멤버를 자동으로 공개하지 않는다. 정적 멤버의 접근 가능 범위는 접근 지정자에 따라 결정된다.

3.8.2 정적 멤버 함수(Static Member Function)

정적 멤버 함수는 특정 객체가 아니라 클래스에 속한 함수이다. 함수 선언 앞에 static을 붙여 정의한다.

```
class Player
{
public:
    static void SetMaxPlayerCount(int count)
    {
        if (count > 0)
        {
            maxPlayerCount = count;
        }
    }
    static int GetMaxPlayerCount()
    {
        return maxPlayerCount;
    }
private:
    static int maxPlayerCount;
};

int Player::maxPlayerCount = 4;
```

정적 멤버 함수는 객체를 생성하지 않고 클래스 범위로 호출할 수 있다.

```
Player::SetMaxPlayerCount(8);
std::cout << Player::GetMaxPlayerCount() << '\n';
```

객체를 통해 호출하는 것도 문법적으로 가능하다.

```
Player player;
player.SetMaxPlayerCount(10);
```

그러나 정적 멤버 함수는 특정 객체에 속하지 않으므로 클래스 범위로 호출하는 것이 함수의 성격을 더 명확하게 나타낸다.
Player::SetMaxPlayerCount(10);

호출 표현	사용 가능 여부	권장 여부
Player::SetMaxPlayerCount(8)	가능	권장
player.SetMaxPlayerCount(8)	가능	일반적으로 권장하지 않음

정적 멤버 함수는 특정 객체를 통해 호출되지 않으므로 this 포인터를 가지지 않는다.

```
class Player
{
public:
```

```
    static void Print()
    {
        // std::cout << this << '\n';    // 오류
    }
};
```

정적 멤버 함수에서는 비정적 데이터 멤버에 직접 접근할 수 없다.

```
class Player
{
public:
    static void Reset()
    {
        // hp = 100;    // 오류
    }
private:
    int hp = 100;
};
```

hp는 각각의 객체에 존재하지만, 정적 멤버 함수에는 어떤 객체의 hp를 사용해야 하는지 판단할 수 있는 this가 없다.
반면 정적 데이터 멤버에는 직접 접근할 수 있다.

```
class Player
{
public:
    static void SetMaxPlayerCount(int count)
    {
        maxPlayerCount = count;
    }
private:
    static int maxPlayerCount;
};
```

<정적 멤버 함수와 일반 멤버 함수의 차이>

구분	일반 멤버 함수	정적 멤버 함수
소속	객체를 통해 실행	클래스에 속함
this 포인터	있음	없음
비정적 멤버 직접 접근	가능	불가능
정적 멤버 직접 접근	가능	가능
객체 없이 호출	불가능	가능
권장 호출 방식	object.Function()	Class::Function()

정적 멤버 함수에서 특정 객체의 멤버를 처리해야 한다면 객체를 매개변수로 전달받아야 한다.

```
class Player
{
public:
    static void Recover(Player& player, int amount)
    {
        if (amount > 0)
        {
            player.hp += amount;
        }
    }
private:
    int hp = 100;
};

Player player;
Player::Recover(player, 10);
```

이 경우 Recover()는 전달받은 player 객체의 hp를 변경한다. 정적 멤버 함수라고 해서 모든 객체의 비정적 멤버에 접근할 수 없는 것은 아니며, 특정 객체가 명시적으로 제공되어야 한다.

3.8.3 인라인 정적 멤버(Inline Static Member)

C++17부터 정적 데이터 멤버에 inline을 사용하면 클래스 안에서 정의와 초기화를 함께 할 수 있다.

```
class Player
{
public:
    inline static int maxPlayerCount = 4;
};
```

이 경우 클래스 밖에서 별도의 정의를 작성하지 않아도 된다.

```
// 별도의 정의가 필요하지 않음
// int Player::maxPlayerCount = 4;
```

일반 정적 멤버와 인라인 정적 멤버의 가장 큰 차이는 일반 정적 멤버는 클래스 외부에서 정의가 필수이다.

<inline 정적 멤버 변수 비교>

방식	클래스 내부	클래스 외부 정의
일반 정적 데이터 멤버	static int value;	필요
인라인 정적 데이터 멤버	inline static int value = 0;	필요 없음

일반 정적 멤버는 선언과 정의를 나누어 작성한다.

```
class GameConfig
{
public:
    static int maxStage;
};

// 반드시 .cpp 파일에서 구현.
int GameConfig::maxStage = 10;
```

인라인 정적 멤버를 사용하면 하나의 선언으로 작성할 수 있다.

```
class GameConfig
{
public:
    inline static int maxStage = 10;
};
```

inline은 정적 데이터 멤버의 저장 공간을 객체마다 만들라는 뜻이 아니다. 여러 소스 파일에서 같은 클래스 정의가 보이더라도 프로그램 전체에서 하나의 정적 데이터 멤버로 사용될 수 있도록 한다.
인라인 정적 멤버에도 const와 constexpr을 사용할 수 있다.

```
class GameConfig
{
public:
    inline static int maxPlayerCount = 4;
    inline static const int minimumPlayerCount = 1;
    inline static constexpr int maximumStageCount = 100;
};
```

<inline 멤버 변수 종류와 특징>

선언	변경 가능 여부	컴파일 타임 상수
inline static int	가능	아님
inline static const int	불가능	초기화 방식에 따라 다름

inline static constexpr int	불가능	가능

constexpr 정적 데이터 멤버는 C++17부터 암시적으로 inline이다.

```
class GameConfig
{
public:
    static constexpr int maximumStageCount = 100;
};
```

inline을 함께 작성한 형태도 문법적으로 가능하지만 필요하지는 않다.

```
inline static constexpr int maximumStageCount = 100;
```

문법적으로는 가능하지만 constexpr만으로 충분하다.

3.8.4 객체 상태(Object State)와 클래스 상태(Class State)

객체마다 서로 다른 값을 가져야 하는 상태는 비정적 데이터 멤버로 표현하고, 모든 객체가 공유해야 하는 상태는 정적 데이터 멤버로 표현한다.

```
struct Vector2
{
    float x, y;
};
class Player
{
public:
    void Move(float offsetX, float offsetY)
    {
        position.x += offsetX;
        position.y += offsetY;
    }
    static void SetMovementLimit(float value)
    {
        if (value > 0.0f)
        {
            movementLimit = value;
        }
    }
private:
    Vector2 position = { 0.0f, 0.0f };
    int hp = 100, level = 1;

    inline static float movementLimit = 1000.0f;
};
```

position, hp, level은 플레이어마다 서로 다른 값을 가져야 한다. 반면 movementLimit은 모든 플레이어에게 공통으로 적용되는 값이다.

<정적 멤버 변수가 포함된 Player 객체의 상태>

멤버	상태의 종류	저장 개수
position	객체 상태	객체마다 하나
hp	객체 상태	객체마다 하나
level	객체 상태	객체마다 하나
movementLimit	클래스 상태	클래스 전체에서 하나

객체가 두 개 생성되면 각각의 객체마다 상태가 설정이 된다..

```
Player 클래스
└─ movementLimit = 1000.0f
```

```
first 객체
├─ position
├─ hp
└─ level

second 객체
├─ position
├─ hp
└─ level
```

클래스 상태를 변경하면 모든 객체가 변경된 값을 함께 사용한다.

```
Player first;
Player second;
Player::SetMovementLimit(2000.0f);
```

movementLimit은 first와 second에 각각 저장된 값이 아니다. 따라서 한 번 변경하면 모든 Player 객체에 공통으로 적용된다.
어떤 값을 정적 멤버로 만들 것인지는 값의 의미를 기준으로 판단해야 한다.

<정적 멤버 설정 고려사항>

판단할 내용	비정적 데이터 멤버	정적 데이터 멤버
객체마다 값이 달라야 하는가	적합	부적합
모든 객체가 같은 값을 공유하는가	부적합	적합
객체가 없어도 의미가 있는가	일반적으로 부적합	적합
특정 객체의 상태인가	적합	부적합
클래스 전체의 정책이나 통계인가	부적합	적합

캐릭터의 현재 체력, 플레이어의 위치, 아이템의 개수, 계좌의 잔액과 창의 현재 크기는 일반적으로 객체 상태로 표현한다.

- 캐릭터의 현재 체력
- 플레이어의 위치
- 아이템의 개수
- 계좌의 잔액
- 창의 현재 크기

생성된 객체의 수, 클래스 전체가 사용하는 최대값, 공통 설정과 기본값, 클래스 단위 통계는 클래스 상태로 표현할 수 있다.

- 생성된 객체의 수
- 클래스 전체가 사용하는 최대값
- 모든 객체에 공통으로 적용되는 설정
- 클래스가 공유하는 기본값
- 클래스 단위의 통계값

그러나 여러 객체가 공유한다는 이유만으로 모든 값을 정적 멤버로 만들면 객체 사이의 의존성이 커질 수 있다. 정적 데이터 멤버를 변경하면 그 값을 사용하는 모든 객체의 동작에 영향을 줄 수 있으므로 클래스 전체에 실제로 공통인 상태에만 사용해야 한다.

3.8.5 전역 변수(Global Variable)와 정적 멤버의 차이

전역 변수와 정적 데이터 멤버는 모두 프로그램이 실행되는 동안 하나의 저장 공간을 유지하며 여러 코드에서 공유될 수 있다.

```
int maxPlayerCount = 4;
```

위 전역 변수는 특정 클래스에 속하지 않는다. 프로그램의 여러 부분에서 사용할 수 있지만, 어떤 기능과 관련된 값인지 선언만으로 명확하지 않을 수 있다.
같은 값을 정적 데이터 멤버로 표현하면 값의 소속을 나타낼 수 있다.

```
class Player
{
public:
    static int GetMaxPlayerCount()
    {
        return maxPlayerCount;
    }
```

```
    static void SetMaxPlayerCount(int count)
    {
        if (count > 0)
        {
            maxPlayerCount = count;
        }
    }
private:
    inline static int maxPlayerCount = 4;
};

Player::SetMaxPlayerCount(8);

std::cout << Player::GetMaxPlayerCount() << '\n';
```

<전역 변수와 정적 데이터 멤버>

구분	전역 변수	정적 데이터 멤버
소속	특정 클래스에 속하지 않음	클래스에 속함
이름의 범위	네임스페이스 범위	클래스 범위
접근 제어	public, private 사용 불가	접근 지정자 사용 가능
접근 표현	maxPlayerCount	Player::maxPlayerCount
관련 기능 묶기	별도의 함수와 분리될 수 있음	정적 멤버 함수와 함께 구성 가능
의미 전달	소속이 불분명할 수 있음	어떤 타입의 상태인지 명확함

정적 데이터 멤버를 사용하면 관련된 상태와 기능을 클래스 안에 묶을 수 있다.

```
// 전역 변수
maxPlayerCount
// 정적 멤버
Player::maxPlayerCount
```

Player::maxPlayerCount는 이름만 보아도 Player 클래스와 관련된 값이라는 사실을 알 수 있다.
또한 정적 데이터 멤버를 private으로 선언하면 외부에서 직접 값을 변경하지 못하게 하고 정적 멤버 함수를 통해 변경 규칙을 적용할 수 있다.

```
class Player
{
public:
    static bool SetMaxPlayerCount(int count)
    {
        if (count < 1 || count > 100)
        {
            return false;
        }
        maxPlayerCount = count;
        return true;
    }
private:
    inline static int maxPlayerCount = 4;
};
```

전역 변수를 사용하더라도 함수나 네임스페이스를 이용하여 구조화할 수 있다. 따라서 전역 변수는 항상 잘못되고 정적 멤버는 항상 올바른 것은 아니다.
정적 데이터 멤버도 프로그램 전체에서 공유되는 상태이므로 사용시 주의가 필요하다.

<전역 변수, 정적 멤버 변수 고려 사항>

문제	설명
숨은 의존성	여러 함수와 객체가 공유 상태에 의존함
변경 범위 확대	한 곳의 변경이 여러 객체에 영향을 줌

테스트 어려움	이전 테스트에서 변경한 값이 남을 수 있음
실행 순서 의존	값을 변경한 순서에 따라 결과가 달라질 수 있음
동시 접근 문제	여러 스레드가 동시에 변경할 수 있음

정적 데이터 멤버는 값이 특정 클래스에 속하고 모든 객체가 실제로 공유해야 하며, 객체가 없어도 의미가 성립할 때 사용하는 것이 좋다.

```
- 특정 클래스에 명확하게 소속되는 값
- 모든 객체가 실제로 공유해야 하는 하나의 값
- 객체가 없어도 성립하는 값의 의미
- 클래스가 책임질 수 있는 값의 변경 규칙
- 지나치게 크지 않은 공유 상태의 의존성
```

정적 멤버는 전역 상태를 단순히 클래스 안으로 옮기는 문법이 아니다. 클래스 전체가 공유해야 하는 상태와 기능을 타입의 책임으로 표현하는 수단이다.

3.9 클래스 내부의 선언(Declarations Inside a Class)

클래스 내부에는 멤버 변수와 멤버 함수뿐만 아니라 클래스, 열거형, 타입 별칭과 같은 새로운 타입을 선언할 수 있다. 클래스 내부에 선언된 이름은 해당 클래스의 범위(Class Scope)에 속한다.

```
class Inventory
{
public:
    class Item;
    enum class State;
    using ItemId = int;
};
```

외부에서는 클래스 범위 연산자 ::를 사용하여 내부에 선언된 이름에 접근한다.

```
Inventory::Item
Inventory::State
Inventory::ItemId
```

<클래스 내부에 선언한 타입의 장점>

장점	설명
이름의 소속 표현	해당 타입이 어떤 클래스와 관련되는지 명확해짐
이름 충돌 방지	클래스 범위 안에 이름을 제한할 수 있음
관련 선언 구성	함께 사용되는 타입을 하나의 클래스 안에 모을 수 있음
접근 제어 적용	내부 타입에도 public, protected, private 적용 가능

클래스 내부의 선언은 관련된 타입을 정리하는 데 유용하지만, 모든 타입을 무조건 클래스 안에 넣어야 하는 것은 아니다. 여러 클래스에서 독립적으로 사용하는 타입이라면 클래스 밖에 선언하는 편이 더 자연스러울 수 있다.

3.9.1 중첩 클래스(Nested Class)

클래스 내부에 선언된 클래스를 중첩 클래스(클래스 안에 포함된 의미의 내포 클래스 라고 번역하기도 함)라 한다. 중첩 클래스를 감싸고 있는 원래의 클래스를 바깥 클래스(Enclosing Class)'라고 한다.

중첩 클래스는 외부 접근 보다는 클래스 내부에만 사용하거나 클래스의 범위를 명시적으로 한정하는 경우에 자주 사용된다.
이에 대한 예로 인벤토리 클래스 안에 아이템 클래스를 정의해 보자.

```
class Inventory
{
public:
```

```
    class Item
    {
    public:
        void Set(int itemId, int itemCount)
        {
            id = itemId;
            count = itemCount;
        }
        int GetId() const
        {
            return id;
        }
        int GetCount() const
        {
            return count;
        }
    private:
        int id = 0, count = 0;
    };
private:
    Item items[100];
    int itemCount = 0;
};
```

Item은 Inventory 클래스 내부에 선언되었으므로 전체 이름은 Inventory::Item 이다.

```
Inventory::Item item;
item.Set(1001, 5);
```

중첩 클래스의 접근 수준에 따라 외부에서 타입 이름을 사용할 수 있는지가 결정된다.

```
class Inventory
{
public:
    class PublicItem
    {
    };
private:
    class PrivateNode
    {
    };
};

Inventory::PublicItem item;
// Inventory::PrivateNode node;    // 오류
```

바깥 클래스 외부에서 중첩 클래스 타입에 접근하는 규칙은 일반 멤버 변수에 적용되는 접근 권한과 완벽히 일치한다.

<바깥 클래스 외부에서 중첩 클래스로의 접근>

중첩 클래스의 위치	바깥 클래스에서 타입 사용
public	가능
protected	파생 클래스에서만 가능 (일반 외부 코드에서는 불가능)
private	불가능

중첩 클래스는 바깥 클래스의 멤버이지만, 바깥 클래스 객체의 일부가 자동으로 되는 것은 아니다.
중첩 클래스에서 바깥 클래스는 중첩 클래스 이름이 속하는 범위를 제공할 뿐이다.

```
Inventory::Item item;
```

이 코드는 Inventory 객체를 생성하지 않고 Item 객체만 생성한다. 중첩은 이름의 소속을 나타낼 뿐, 객체 사이의 포함 관계를 자동으로 만들지 않는다. 중첩 클래스는 바깥 클래스의 private 멤버에 접근할 수 있지만, 바깥 클래스의 비정적 멤버를 사용하려면 대상 객체를 별도로 지정해야 한다.

<바깥 클래스와 중첩 클래스 관계>

내용	결과
중첩 클래스는 바깥 클래스의 범위에 속함	Outer::Nested로 접근
바깥 클래스 객체가 없어도 중첩 클래스 객체 생성 가능	중첩은 자동 포함 관계가 아님
중첩 클래스는 바깥 클래스의 비공개 멤버에 접근 가능	접근할 바깥 객체는 별도로 필요
바깥 클래스가 중첩 클래스의 비공개 멤버에 자동 접근	불가능
중첩 클래스에도 접근 지정자 적용	가능

중첩 클래스는 클래스 내부에서만 사용하는 타입을 구성할 때 유용하므로, 필요한 경우 적극적으로 활용한다.

```
- 특정 클래스에서만 의미가 있는 보조 타입
- 클래스 내부 자료 구조의 노드
- 반복자와 같은 보조 객체
- 클래스 전용 결과 타입
- 외부에 공개할 필요가 없는 구현 타입
```

여러 클래스에서 공통으로 사용하는 타입이라면 클래스 밖에 독립된 타입으로 선언하는 편이 좋다.

3.9.2 중첩 열거형(Nested Enumeration)

클래스가 사용하는 상태나 종류를 클래스 내부의 열거형으로 선언할 수 있다.

```
class Character
{
public:
    enum class State
    {
        Idle,
        Move,
        Attack,
        Dead
    };
    void SetState(State newState)
    {
        state = newState;
    }
    State GetState() const
    {
        return state;
    }

private:
    State state = State::Idle;
};
```

외부에서는 클래스 이름과 열거형 이름을 함께 사용한다.

```
Character character;
character.SetState(Character::State::Attack);
```

Character::State::Attack은 클래스 이름, 열거형 이름과 열거자의 순서로 구성된다.

```
Character::State::Attack
    |         |      |
    |         |      └─ 열거자
    |         └──────── 열거형
    └────────────────── 클래스
```

클래스 내부에 열거형을 선언하면 열거형이 어떤 클래스와 관련된 것인지 명확하게 나타낼 수 있다.

클래스 밖에 선언하면 이름의 관계가 약하게 표현된다.

```
enum class CharacterState
{
    Idle,
    Move,
    Attack,
    Dead
};
```

클래스 내부에 선언하면 타입의 소속이 직접 드러난다.

```
Character::State
```

<열거형 선언 위치 비교>

선언 위치	적합한 경우
클래스 내부	특정 클래스에서만 사용하는 상태나 종류
클래스 외부	여러 클래스가 공통으로 사용하는 상태나 종류

중첩 열거형에도 접근 제어를 적용할 수 있다.

```
class NetworkConnection
{
public:
    bool IsConnected() const;
private:
    enum class State
    {
        Disconnected,
        Connecting,
        Connected
    };
    State state = State::Disconnected;
};
```

State가 private이면 외부에서는 NetworkConnection::State 타입을 직접 사용할 수 없다. 외부에서는 IsConnected()와 같은 공개된 기능만 사용한다.

```
NetworkConnection connection;
if (connection.IsConnected())
{
    std::cout << "연결됨\n";
}
```

내부 열거형을 비공개로 두면 객체의 내부 상태 표현 방법을 외부에 노출하지 않을 수 있다.

열거형의 접근 수준	의미
public	외부에서도 열거형 값을 선택하거나 확인할 수 있음
private	객체 내부의 구현 상태로만 사용

외부에서 상태값을 직접 선택해야 한다면 public으로 선언하고, 내부에서만 사용하는 구현 상태라면 private으로 선언할 수 있다.

3.9.3 타입 별칭(Type Alias)

타입 별칭은 기존 타입에 새로운 이름을 붙이는 기능이다. C++에서는 using을 사용하여 타입 별칭을 선언한다.

```
using ItemId = int;
```

클래스 내부에 타입 별칭을 선언하면 해당 클래스가 사용하는 타입의 의미를 표현할 수 있다.

```
class Inventory
{
public:
    using ItemId = int;
    using ItemCount = int;

    bool AddItem(ItemId id, ItemCount count);

private:
    ItemCount totalItemCount = 0;
};
```

외부에서는 클래스 범위 연산자를 사용한다.

```
Inventory::ItemId swordId = 1001;
Inventory::ItemCount swordCount = 5;
```

ItemId와 ItemCount의 실제 타입은 모두 int이지만 서로 다른 의미를 나타낸다.

타입 표현	의미
int	일반적인 정수
Inventory::ItemId	인벤토리에서 사용하는 아이템 식별자
Inventory::ItemCount	인벤토리에서 사용하는 아이템 수량

타입 별칭은 새로운 타입을 만드는 것이 아니다.

```
Inventory::ItemId id = 1001;
int value = id;
```

ItemId는 int의 다른 이름이므로 두 타입 사이에 별도의 형변환이 필요하지 않다.
클래스 내부의 타입 별칭은 컨테이너나 템플릿 클래스에서 자주 사용된다.

```
class ItemList
{
public:
    using ValueType = int;
    using SizeType = std::size_t;
    SizeType GetSize() const
    {
        return size;
    }
private:
    ValueType values[100];
    SizeType size = 0;
};
```

이름을 통해 타입의 역할을 표현할 수 있다.

별칭	역할
ValueType	클래스가 저장하는 값의 타입
SizeType	크기와 개수를 나타내는 타입
Iterator	요소를 순회하는 타입
Pointer	클래스가 사용하는 포인터 타입

접근 수준에 따라 외부에서 타입 별칭을 사용할 수 있는지도 결정된다.

```
class Sample
```

```
{
public:
    using PublicType = int;
private:
    using PrivateType = int;
};
```

```
Sample::PublicType first = 10;
// Sample::PrivateType second = 20;    // 오류
```

외부 인터페이스에서 사용할 타입이면 public에 선언하고, 클래스 구현에서만 사용할 타입이면 private에 선언한다.

3.9.4 friend 함수(Friend Function)

friend 함수는 클래스의 멤버 함수는 아니지만, 해당 클래스의 private과 protected 멤버에 접근할 수 있도록 허용된 함수이다.

일반적으로 클래스의 멤버에는 접근 제한을 적용하지만, 수학 벡터로 사용하는 Vector2는 좌표값에 자주 접근하므로 x, y를 public으로 둔다.

```
#include <ostream>
class Vector2
{
public:
    Vector2(float x, float y) : x(x), y(y) { }
    friend std::ostream& operator<<(std::ostream& output, const Vector2& value);
public:
    float x, y;
};
```

operator<<는 Vector2의 멤버 함수가 아니다. 그러나 클래스 안에서 friend로 선언했으므로 x, y에 접근할 수 있다.

```
std::ostream& operator<<(std::ostream& output, const Vector2& value)
{
    output << '('
           << value.x << ", "
           << value.y << ')';
    return output;
}

Vector2 position(10.0f, 20.0f);
std::cout << position << '\n';
```

<일반 멤버 함수, friend 함수 비교>

구분	멤버 함수	friend 함수
클래스의 멤버	맞음	아님
this 포인터	있음	없음
private 멤버 접근	가능	허용된 클래스에 한해 가능
호출 방법	object.Function()	일반 함수 방식
클래스 밖에서 정의	범위 연산자 필요	일반 함수로 정의

friend 함수는 클래스의 공개 멤버가 아니다.

```
position.operator<<();    // 이러한 멤버 함수가 아님
```

operator<<는 일반 함수이므로 왼쪽 피연산자인 std::ostream과 오른쪽 피연산자인 Vector2를 매개변수로 전달받는다.

```
std::cout << position
    |            |
    |            └─ 두 번째 매개변수
    |
```

```
    └──────────────────── 첫 번째 매개변수
```

이처럼 왼쪽 피연산자가 클래스 객체가 아닌 연산자를 정의할 때 **friend** 함수를 사용할 수 있다.
friend 선언은 해당 함수에 접근 권한을 부여한다. 함수가 클래스의 소유가 되거나 멤버 함수로 바뀌는 것은 아니다.

3.9.5 friend 클래스(Friend Class)

특정 클래스의 모든 멤버 함수에 비공개 멤버 접근을 허용하려면 **friend** 클래스로 선언할 수 있다.

```
class GameData
{
    friend class SaveSystem;

public:
    int GetStage() const
    {
        return stage;
    }
private:
    int score = 0, stage = 1;
};
```

SaveSystem은 GameData의 private 멤버에 접근할 수 있다.

```
class SaveSystem
{
public:
    static void Save(const GameData& data)
    {
        std::cout << "Score: " << data.score << '\n';
        std::cout << "Stage: " << data.stage << '\n';
    }
};

GameData gameData;
SaveSystem::Save(gameData);
```

friend 클래스는 일반적으로 한 클래스가 다른 클래스의 내부 표현을 직접 처리해야 할 때 사용한다.

- 저장과 불러오기
- 디버깅 도구
- 밀접하게 연결된 자료 구조
- 반복자와 컨테이너
- 생성 전용 보조 클래스

친구 관계(Friendship)는 선언한 방향으로만 적용된다.

```
GameData가 SaveSystem을 friend로 선언
                ↓
SaveSystem은 GameData의 비공개 멤버 접근 가능
```

반대로 **GameData**가 **SaveSystem**의 비공개 멤버에 접근할 수 있는 것은 아니다.

친구 관계의 특성	결과
단방향	반대 방향으로 자동 적용되지 않음
비대칭	서로 친구가 되려면 각각 선언해야 함
비 상속	파생 클래스가 친구 권한을 이어받지 않음
비 전이	친구의 친구에게 권한이 전달되지 않음

friend 관계는 전이되지 않으므로 **A**가 **B**를 **friend**로 선언하고 **B**가 **C**를 **friend**로 선언해도 **A**와 **C** 사이에 친구 관계가 생기지 않는다.

```
A가 B를 friend로 선언
B가 C를 friend로 선언
A와 C가 자동으로 friend 관계가 되는 것은 아님
```

friend 권한은 비공개 멤버를 가진 클래스가 직접 부여한다.

3.9.6 friend 사용 시 주의점(Considerations for Using friend)

friend는 접근 제어를 우회하는 기능이므로 필요한 경우에 제한적으로 사용해야 한다. friend 함수나 클래스가 많아지면 클래스 외부의 여러 코드가 내부 표현에 직접 의존하게 된다.

```
클래스 내부 표현 변경
        ↓
여러 friend 함수와 클래스도 함께 수정
```

이는 캡슐화와 정보 은닉을 약화할 수 있다.

문제	설명
내부 결합 증가	외부 함수나 클래스가 비공개 멤버 구조에 의존
변경 범위 확대	내부 표현 변경 시 관련 friend 코드도 수정
책임 불분명	객체 상태를 누가 변경하는지 파악하기 어려움
접근 권한 확대	필요한 범위보다 많은 멤버에 접근 가능

특히 friend 클래스는 해당 클래스의 모든 멤버 함수에 접근 권한을 제공한다. 하나의 함수만 접근하면 된다면 클래스 전체보다 friend 함수만 허용하는 편이 접근 범위를 줄일 수 있다.

필요한 접근	권장 방법
하나의 함수만 비공개 멤버 접근	해당 함수만 friend로 선언
밀접한 보조 클래스 전체가 접근	friend 클래스 검토
값을 읽는 기능만 필요	공개된 멤버 함수 제공
상태 변경 규칙을 적용해야 함	클래스가 의미 있는 멤버 함수 제공
단순한 편의를 위한 접근	friend 사용 재검토

public GetScore()처럼 값을 반환하는 공개 함수로 충분하다면 friend를 사용할 필요가 없다.

```
class GameData
{
public:
    int GetScore() const
    {
        return score;
    }
private:
    int score = 0;
};
```

반면 출력 연산자처럼 비멤버 함수로 구현하는 것이 자연스럽고 내부 값에 접근해야 하는 경우에는 friend가 적절할 수 있다.

```
friend std::ostream& operator<<(std::ostream& output, const Vector2& value);
```

friend는 캡슐화를 무조건 파괴하는 기능이 아니다. 클래스가 특정 함수나 클래스에 의도적으로 접근 권한을 부여하는 것이다. 다만 권한을 부여하는 대상과 이유가 명확해야 한다.

3.10 클래스 선언과 구현의 분리 (Separation of Class Declaration and Implementation)

작은 예제에서는 클래스 정의와 멤버 함수의 본문을 하나의 파일에 작성해도 큰 문제가 없다.

```
class Player
{
public:
    void Move(float offsetX, float offsetY)
    {
        position.x += offsetX;
        position.y += offsetY;
    }
private:
    Vector2 position;
};
```

그러나 클래스의 기능이 많아지면 클래스의 전체 구조를 파악하기 어려워지고, 하나의 파일이 지나치게 길어진다. 실제 프로젝트에서는 일반적으로 클래스의 선언과 멤버 함수의 구현을 헤더 파일과 소스 파일로 분리한다.

```
Player.h
└─ Player 클래스의 구성과 멤버 함수 선언

Player.cpp
└─ Player 멤버 함수의 실제 처리 내용
```

이를 통해 헤더 파일에서는 클래스가 제공하는 기능을 확인하고, 소스 파일에서는 각 기능이 어떻게 처리되는지 구현할 수 있다.

3.10.1 헤더 파일(Header File)과 소스 파일(Source File)

C++ 프로젝트에서는 코드를 주로 헤더 파일과 소스 파일로 나누어 작성한다.

구분	일반적인 확장자	주요 내용
헤더 파일	.h, .hpp	클래스와 구조체 정의, 함수 선언, 열거형, 상수
소스 파일	.cpp	함수와 멤버 함수의 정의
프로그램 시작 파일	.cpp	main() 함수와 객체 사용 코드

게임의 플레이어를 나타내는 클래스를 구성한다고 생각해 보자.
각각의 헤더 파일은 클래스의 선언 으로, 소스 파일은 클래스의 구현을 목적으로 역할을 정할 수 있다.

파일	역할
Vector2.h	위치를 나타내는 Vector2 정의
Weapon.h	무기를 나타내는 Weapon 정의
Player.h	Player 클래스 정의
Player.cpp	Player 멤버 함수 정의
main.cpp	객체 생성과 기능 호출

```
// Vector2.h
#pragma once

struct Vector2
{
    float x = 0.0f, y = 0.0f;
};
```

헤더 파일이 여러 번 포함되면 같은 타입이 중복으로 정의될 수 있다. #pragma once는 하나의 소스 파일을 컴파일하는 과정에서 해당 헤더가 한 번만 포함되도록 한다.

전통적인 헤더 가드(Header Guard) 는 매크로의 정의 여부를 검사하여 중복 포함을 막는다.

```
#ifndef VECTOR2_H
#define VECTOR2_H

struct Vector2
{
    float x = 0.0f, y = 0.0f;
};

#endif
```

#pragma once와 헤더 가드는 중복 포함을 막는 목적은 같지만 작성 방식이 다르다.

방법	특징
`#pragma once`	간결하며 Visual C++, GCC, Clang 등에서 지원
헤더 가드	표준 전처리기 문법으로 작성

이 책에서는 코드를 간결하게 작성하기 위해 #pragma once를 사용한다.
헤더 파일에서는 다른 파일의 선언이 필요할 수 있다. 표준 라이브러리 헤더는 꺾쇠괄호로, 프로젝트에서 작성한 헤더는 큰따옴표로 포함하는 것이 일반적이다.

```
#include <string>
#include "Vector2.h"
```

포함 방법	일반적인 용도
`#include <string>`	표준 라이브러리나 외부 라이브러리의 헤더
`#include "Vector2.h"`	현재 프로젝트에서 작성한 헤더

소스 파일에서는 자신이 구현할 클래스의 헤더를 먼저 포함하는 것이 좋다.

```
#include "Player.h"
```

이렇게 하면 Player.h가 다른 헤더의 우연한 포함에 의존하지 않고, 그 자체로 필요한 선언을 포함하고 있는지 확인하기 쉽다.

3.10.2 클래스 선언(Class Declaration)

헤더 파일에 클래스의 이름과 멤버 구성을 헤더 파일에 작성한다.

```
// Player.h
#pragma once

#include <string>
#include "Vector2.h"

class Weapon;    // 전방 선언.
class Player
{
public:
    Player(const std::string& name);
    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);
    void EquipWeapon(Weapon* newWeapon);
    int GetHp() const;
    int GetAttackPower() const;
    const std::string& GetName() const;
    const Vector2& GetPosition() const;
```

```
private:
    std::string name;
    Vector2 position;
    Weapon* weapon = nullptr;
    int hp = 100;
};
```

클래스 안에는 멤버 함수의 이름과 매개변수, 반환 타입만 작성하고 함수 본문은 생략하였다.

```
void Move(float offsetX, float offsetY);
int GetHp() const;
```

이러한 코드는 멤버 함수 선언(Member Function Declaration)이다. 컴파일러는 선언을 통해 함수를 어떻게 호출해야 하는지 알 수 있다.
클래스 전체에 대해서는 선언과 정의를 구분할 필요가 있다.

```
class Player;
```

위 코드는 Player라는 클래스가 존재한다는 사실만 알리는 클래스 선언이다. 클래스의 멤버와 크기는 알 수 없다.

```
class Player
{
public:
    void Move(float offsetX, float offsetY);

private:
    Vector2 position;
};
```

멤버 구성을 포함한 위 코드는 클래스 정의(Class Definition)이다.

문법	의미
`class Player;`	클래스 이름만 알리는 선언
`class Player { ... };`	클래스의 멤버 구성을 나타내는 정의

일반적으로 헤더에 클래스를 작성하는 일을 클래스 선언이라고 표현하지만, 멤버 목록을 포함한 클래스 본문은 문법적으로 클래스 정의에 해당한다.
클래스 정의가 끝난 뒤에는 세미콜론을 붙인다.

```
class Player
{
};
```

함수 정의의 닫는 중괄호 뒤에는 세미콜론을 붙이지 않지만, 클래스 정의 뒤에는 세미콜론이 필요하다.
헤더 파일에는 public 멤버뿐만 아니라 private 멤버도 작성해야 한다. 컴파일러가 객체의 크기와 메모리 배치를 계산하려면 모든 비정적 데이터 멤버의 형식을 알아야 하기 때문이다.

클래스 구성 요소	헤더에 필요한 이유
public 멤버 함수	외부 코드가 호출 방법을 알아야 함
private 멤버 함수	클래스의 전체 멤버 선언이 필요함
비정적 데이터 멤버	객체의 크기와 메모리 배치 계산
중첩 타입	클래스 내부의 타입 구성 확인
정적 멤버 선언	클래스에 속한 이름과 형식 확인

헤더 파일에는 가능한 한 필요한 선언만 포함하는 것이 좋다. 불필요한 헤더를 많이 포함하면 해당 헤더를 사용하는 여러 소스 파일의 컴파일 시간이 증가할 수 있다.

3.10.3 멤버 함수 정의(Member Function Definition)

클래스 밖에서 멤버 함수를 정의할 때는 클래스 이름과 범위 연산자 ::를 사용한다.

```cpp
// Player.cpp
#include "Player.h"
#include "Weapon.h"

Player::Player(const std::string& name) : name(name)
{
}

void Player::Move(float offsetX, float offsetY)
{
    position.x += offsetX;
    position.y += offsetY;
}

void Player::TakeDamage(int damage)
{
    if (damage <= 0)
    {
        return;
    }
    hp -= damage;
    if (hp < 0)
    {
        hp = 0;
    }
}

void Player::EquipWeapon(Weapon* newWeapon)
{
    weapon = newWeapon;
}

int Player::GetHp() const
{
    return hp;
}

int Player::GetAttackPower() const
{
    if (weapon == nullptr)
    {
        return 1;
    }
    return weapon->GetAttackPower();
}

const std::string& Player::GetName() const
{
    return name;
}

const Vector2& Player::GetPosition() const
{
    return position;
}
```

Player::Move() 함수의 정의를 구성 요소별로 살펴보자.

구성	의미
`void`	반환 타입
`Player::`	함수가 속한 클래스
`Move`	멤버 함수 이름

(float offsetX, float offsetY)	매개변수
함수 본문	실제 처리 내용

클래스 밖에서는 Move라는 이름만으로 이 함수가 어느 클래스의 멤버인지 알 수 없다. 따라서 Player::Move와 같이 함수의 소속을 지정한다.
헤더의 선언과 소스 파일의 정의는 서로 일치해야 한다.

```
// Player.h
int GetHp() const;

// Player.cpp
int Player::GetHp() const
{
    return hp;
}
```

만약 헤더에서 선언한 함수의 형태와 일치하지 않은 함수를 정의한 경우를 보자.

```
// Player.cpp: 헤더와 일치하지 않은 함수 정의
int Player::GetHp()
{
    return hp;
}
```

헤더에는 const가 있지만 정의에는 없으므로 서로 다른 멤버 함수로 처리된다. 따라서 선언과 정의를 분리해서 함수를 작성할 경우에 함수의 이름과 시그니처가 일치하는 지 확인한다.

<함수의 선언과 정의 분리시 일치 여부 확인>

항목	일치 여부
멤버 함수 이름	완전 일치
반환 타입	완전 일치
매개변수의 형식과 순서	완전 일치
멤버 함수의 const	완전 일치
참조 한정자	완전 일치
예외 명세	완전 일치
매개변수 이름	무관하나 동일하게 유지 권장

예를 들어, 선언과 정의에서 매개변수 이름이 달라도 타입과 순서가 같으면 같은 함수이다.

```
// Player.h
void Move(float x, float y);

// Player.cpp
// 매개 변수 이름이 선언과 다르나 타입이 같고, 매개변수의 개수가
같아 같은 함수.
void Player::Move(float offsetX, float offsetY)
{
    position.x += offsetX;
    position.y += offsetY;
}
```

함수 호출 방법은 매개변수의 이름이 아니라 형식과 순서에 의해 결정된다. 그러나 선언과 정의의 이름을 같게 작성하면 코드를 비교하기 쉽다.
생성자(Constructor)도 같은 방법으로 클래스 밖에서 정의한다.

```
// Player.h
Player(const std::string& name);
```

```cpp
// Player.cpp
Player::Player(const std::string& name) : name(name)
{
}
```

생성자에는 반환 타입을 작성하지 않으며, 클래스 이름 앞에 Player::를 붙여 소속을 나타낸다.

클래스 내부에 정의한 멤버 함수
짧은 멤버 함수는 클래스 안에서 선언과 정의를 함께 작성할 수 있다.

```cpp
class Weapon
{
public:
    int GetAttackPower() const
    {
        return attackPower;
    }

private:
    int attackPower = 10;
};
```

클래스 내부에 정의한 멤버 함수는 암시적으로 인라인 함수(Inline Function)가 된다. 여기서 inline은 컴파일러가 반드시 함수 호출을 본문으로 치환한다는 뜻이 아니다. 같은 함수 정의가 여러 번역 단위(Translation Unit)에 나타날 수 있도록 허용하는 언어 규칙과 관련된다.

클래스 내부 정의에 적합	소스 파일 분리에 적합
짧은 접근자 함수	처리 과정이 긴 함수
단순한 상태 확인 함수	조건과 반복이 많은 함수
간단한 계산 함수	자주 변경되는 구현
템플릿 멤버 함수	다른 헤더 의존성이 큰 함수

모든 짧은 함수를 반드시 클래스 내부에 정의할 필요는 없다. 프로젝트의 작성 규칙에 따라 간단한 함수까지 소스 파일로 분리할 수도 있다.

3.10.4 인터페이스(Interface)와 구현(Implementation)

인터페이스는 클래스 외부에서 사용할 수 있도록 공개한 기능의 집합이다. 구현은 그 기능이 내부에서 실제로 처리되는 방법이다.

Player 클래스는 생성, 이동, 피해 처리, 무기 장착과 상태 확인 기능을 공개 인터페이스로 제공한다.

```cpp
class Player
{
public:
    Player(const std::string& name);

    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);
    void EquipWeapon(Weapon* newWeapon);

    int GetHp() const;
    int GetAttackPower() const;
    const std::string& GetName() const;
    const Vector2& GetPosition() const;
};
```

외부 코드는 공개된 함수의 이름, 매개변수와 반환 타입을 보고 클래스를 사용한다.

```cpp
Player player("Warrior");
player.Move(10.0f, 20.0f);
player.TakeDamage(30);
std::cout << player.GetHp() << '\n';
```

Move()가 위치를 어떤 방식으로 계산하고, TakeDamage()가 체력을 어떻게 제한하는지는 구현에 해당한다.

구분	내용
인터페이스	외부에 공개된 함수와 사용 방법
구현	함수 본문과 내부 처리 과정
내부 상태	private 데이터 멤버
내부 규칙	입력 검사와 상태 변경 조건

헤더 파일은 Player가 제공하는 기능을 나타내고, 소스 파일은 각 기능의 처리 방법을 구현한다.

```
Player.h
└ Player가 무엇을 제공하는가

Player.cpp
└ Player가 기능을 어떻게 처리하는가
```

구현을 변경하더라도 인터페이스가 유지되면 외부 코드의 호출 방법은 바뀌지 않는다.
예를 들어 처음에는 이동값을 그대로 더할 수 있다.

```
void Player::Move(float offsetX, float offsetY)
{
    position.x += offsetX;
    position.y += offsetY;
}
```

나중에 이동할 수 있는 범위를 제한하도록 구현을 변경할 수 있다.

```
void Player::Move(float offsetX, float offsetY)
{
    position.x += offsetX;
    position.y += offsetY;

    if (position.x < 0.0f)
    {
        position.x = 0.0f;
    }

    if (position.y < 0.0f)
    {
        position.y = 0.0f;
    }
}
```

외부의 호출 방법은 그대로 유지된다.

```
player.Move(10.0f, 20.0f);
```

<인터페이스와 구현의 분리 시 장점>

장점	설명
사용 방법 파악	헤더에서 공개 기능을 빠르게 확인할 수 있음
코드 관리	긴 함수 본문을 소스 파일에서 관리할 수 있음
변경 범위 감소	공개 인터페이스를 유지하며 구현을 변경할 수 있음
의존성 관리	구현에서만 필요한 헤더를 소스 파일에 포함할 수 있음
협업	클래스 사용 코드와 구현 코드를 구분하여 작업할 수 있음

다만 헤더와 소스 파일을 분리했다고 해서 클래스의 모든 내부 구조가 완전히 감춰지는 것은 아니다.

```
private:
    std::string name;
    Vector2 position;
    Weapon* weapon = nullptr;
    int hp = 100;
```

컴파일러가 Player 객체의 크기를 계산해야 하므로 비정적 데이터 멤버는 헤더의 클래스 정의에 포함되어야 한다. 따라서 데이터 멤버를 추가하거나 형식을 변경하면 Player.h를 포함하는 소스 파일도 다시 컴파일해야 할 수 있다.

변경 내용	외부 코드에 미치는 영향
소스 파일의 함수 본문 변경	호출 방법에는 영향 없음
공개 함수의 매개변수 변경	호출 코드 수정 필요
공개 함수의 반환 타입 변경	호출 코드 수정 가능성 있음
데이터 멤버 변경	객체 크기와 배치 변경
비공개 함수 본문 변경	일반적으로 외부 호출 방법에는 영향 없음

클래스 선언과 구현의 분리는 클래스의 사용 방법과 처리 방법을 구분하는 기본적인 방법이다.

3.10.5 전방 선언(Forward Declaration)

전방 선언은 타입의 전체 정의를 작성하기 전에 해당 타입의 이름만 컴파일러에 알리는 것이다.

```
class Weapon;
```

이 선언을 본 컴파일러는 Weapon이라는 클래스가 있다는 사실은 알지만, 어떤 멤버를 가지며 객체의 크기가 얼마인지는 알 수 없다. 전체 정의가 알려지지 않은 타입을 불완전 타입(Incomplete Type)이라고 한다.
Player.h에서는 Weapon 객체 자체가 아니라 포인터만 저장한다.

```
class Weapon;  // 전방 선언.
class Player
{
public:
    void EquipWeapon(Weapon* newWeapon);
private:
    Weapon* weapon = nullptr;
};
```

포인터의 크기는 가리키는 객체의 형식과 관계없이 결정된다. 따라서 Weapon의 전체 정의가 없어도 Weapon* 멤버를 선언할 수 있다.
반면 Vector2는 객체 자체를 멤버로 저장한다.

```
Vector2 position;
```

컴파일러가 Player의 크기를 계산하려면 Vector2의 크기를 알아야 하므로 Vector2.h를 포함해야 한다.

```
#include "Vector2.h"

class Weapon;  // 전방 선언.

class Player
{
private:
    Vector2 position;
    Weapon* weapon = nullptr;
};
```

<전방 선언 타입을 사용하는 멤버의 비교>

멤버	저장 방식	헤더에서 필요한 처리

Vector2 position	객체를 값으로 저장	Vector2.h 포함
Weapon* weapon	객체의 주소를 저장	class Weapon; 전방 선언 가능

Player.cpp에서 Weapon의 멤버 함수를 호출하려면 전체 클래스 정의가 필요하다.

```
#include "Player.h"
#include "Weapon.h"

int Player::GetAttackPower() const
{
    if (weapon == nullptr)
    {
        return 1;
    }
    return weapon->GetAttackPower();
}
```

전방 선언만으로는 Weapon에 GetAttackPower()가 있는지 알 수 없으므로 소스 파일에서는 Weapon.h를 포함한다.

<전방 선언으로 가능한 경우>

```
class Weapon;
Weapon* FindWeapon();
void EquipWeapon(Weapon* weapon);
void PrintWeapon(const Weapon& weapon);
```

포인터와 참조를 선언하는 데에는 객체의 전체 크기가 필요하지 않다.

<전체 정의가 필요한 경우>

```
class Weapon;
class Player
{
private:
    Weapon weapon;    // 오류
};
```

값 타입 멤버를 선언하려면 Weapon 객체의 크기를 알아야 한다.

```
#include "Weapon.h"
class Player
{
private:
    Weapon weapon;
};
```

<전방 선언으로 가능한 작업, 전체 정의가 필요한 작업>

작업	전방 선언만으로 가능
포인터 선언	가능
참조 선언	가능
포인터 또는 참조를 사용하는 함수 선언	가능
값 타입 데이터 멤버 선언	불가능
객체 생성	불가능
멤버 함수 호출	불가능
sizeof 적용	불가능
기반 클래스로 상속	불가능
값 타입에 접근하거나 복사	불가능

전방 선언을 사용하면 헤더 사이의 포함 관계를 줄일 수 있다.

```
Player.h
├─ Vector2.h 포함
└─ Weapon 전방 선언

Player.cpp
├─ Player.h 포함
└─ Weapon.h 포함
```

Player.h를 포함하는 모든 파일이 Weapon.h까지 반드시 읽을 필요가 없으므로 헤더 의존성을 줄일 수 있다.
전방 선언은 순환 포함(Circular Inclusion)을 줄이는 데에도 도움이 된다. 두 클래스가 서로의 포인터를 저장한다고 생각해 보자.

```cpp
// Player.h
#pragma once

class Party;
class Player
{
private:
    Party* party = nullptr;
};

// Party.h
#pragma once

class Player;
class Party
{
private:
    Player* leader = nullptr;
};
```

두 헤더가 서로를 직접 포함하지 않고 상대 클래스의 이름만 전방 선언하므로 불필요한 포함 관계를 만들지 않는다.
그러나 전방 선언을 무조건 사용하는 것이 좋은 것은 아니다. 타입의 전체 정의가 필요하거나 해당 타입을 값으로 포함한다면 헤더를 직접 포함해야 한다.

다른 타입의 사용 방법	선택
객체를 값으로 멤버에 저장	헤더 포함
기반 클래스로 상속	헤더 포함
포인터나 참조만 저장	전방 선언 검토
소스 파일에서 멤버 함수 호출	소스 파일에서 헤더 포함
타입의 크기 확인	헤더 포함

<클래스 선언, 구현 분리 시 파일 구성 기준>

내용	일반적인 작성 위치
클래스 정의	헤더 파일
공개 멤버 함수 선언	헤더 파일
비공개 멤버 선언	헤더 파일
일반 멤버 함수 정의	소스 파일
짧은 인라인 멤버 함수	헤더 파일 가능
다른 타입의 포인터, 참조	전방 선언 검토
다른 타입의 값 멤버	해당 타입의 헤더 포함

헤더 파일은 클래스가 제공하는 구조와 기능을 나타내고, 소스 파일은 각 기능의 처리 과정을 구현한다. 전방 선언은 타입의 이름만으로 충분한 경우에 헤더 사이의 의존성을 줄이는 방법이다. 선언, 정의와 구현을 적절히 구분하면 클래스의 역할이 분명해지고 프로젝트의 변경과 확장을 관리하기 쉬워진다.

3.11 UML 다이어그램과 C++ 코드

앞에서 UML 이 객체지향 프로그램의 구조와 동작을 표현하는 역할을 살펴보았다. 이제 클래스의 기본 문법을 학습했으므로 UML 표기와 C++ 코드가 어떻게 대응하는지 구체적으로 살펴본다.

UML은 C++ 문법을 그대로 옮겨 적는 표기법이 아니다. 프로그램을 구성하는 클래스, 클래스가 가지는 상태와 기능, 클래스 사이의 관계와 객체의 동작 순서를 코드보다 간결하게 표현하는 모델링 언어이다.

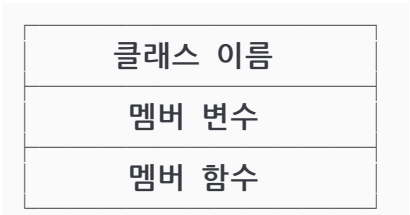
이 절에서는 클래스 다이어그램과 시퀀스 다이어그램을 사용한다.

다이어그램	표현하는 내용
클래스 다이어그램(Class Diagram)	클래스의 데이터와 기능, 클래스 사이의 정적인 관계
시퀀스 다이어그램(Sequence Diagram)	객체가 시간의 흐름에 따라 주고받는 메시지

클래스 다이어그램은 프로그램이 어떤 클래스로 구성되는가를 보여 주고, 시퀀스 다이어그램은 객체들이 어떤 순서로 협력하는가를 보여 준다.

3.11.1 클래스 다이어그램(Class Diagram)

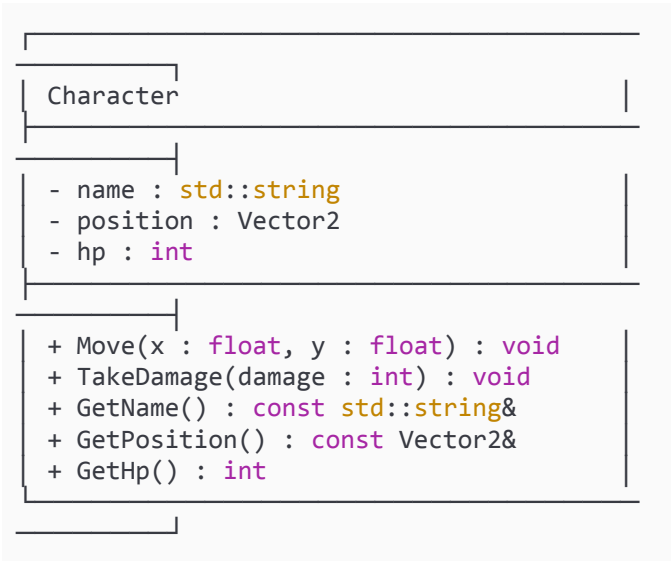
클래스 다이어그램은 클래스가 가지는 데이터와 기능, 클래스 사이의 관계를 표현한다. 프로그램을 구성하는 클래스의 정적인 구조를 이해하고 설계하는 데 사용한다.
클래스는 일반적으로 세 영역으로 나누어 표현한다.



UML에서는 멤버 변수를 속성(Attribute), 멤버 함수를 연산(Operation)이라고 부르기도 한다.

영역	표현하는 내용
클래스 이름	타입의 이름
멤버 변수	객체가 가지는 상태
멤버 함수	객체가 제공하는 기능

Character 클래스는 캐릭터의 이름, 위치와 체력을 관리한다.



접근 수준 표기**(Visibility)**
클래스 다이어그램에서는 멤버의 접근 수준을 기호로 표시한다.

기호	접근 수준	의미
+	**public**	클래스 외부에서 접근 가능
-	**private**	클래스 내부에서만 접근 가능

#	protected	클래스 내부와 파생 클래스에서 접근 가능

멤버 변수의 표기 형식은 접근수준 이름 : 타입이다.

```
접근수준 이름 : 타입
```

C++에서는 타입을 먼저 작성한다.

```
- hp : int
- position : Vector2
```

이를 C++ 코드로 간단히 옮길 수 있다.

```
private:
    Vector2 position;
    int hp;
```

초기값도 속성 표기에 포함할 수 있다.

```
- hp : int = 100
private:
    int hp = 100;
```

멤버 함수 표기**(Operation)**
멤버 함수의 표기 형식은 접근수준 함수이름 : 반환타입이다.

```
접근수준 함수이름(매개변수이름 : 타입) : 반환타입
```

```
+ Move(offsetX : float, offsetY : float) : void
+ TakeDamage(damage : int) : void
+ GetHp() : int
```

이에 대응하는 C++ 선언은 아래 코드와 같다.

```
public:
    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);
    int GetHp() const;
```

UML 표기와 C++ 코드에서는 이름과 타입을 작성하는 순서가 다르다.

구분	UML	C++
멤버 변수	name : std::string	std::string name
매개변수	damage : int	int damage
반환 타입	GetHp() : int	int GetHp()

UML에서는 이름을 먼저 쓰고 타입을 뒤에 표시하지만, C++에서는 타입을 먼저 쓰고 이름을 뒤에 표시한다.

생성자 표기**(Constructor)**
생성자는 클래스와 같은 이름으로 표시하며 반환 타입은 작성하지 않는다.

```
+ Character(name : const std::string&)
public:
    Character(const std::string& name);
```

C++ 세부 문법의 표현
클래스 다이어그램은 설계의 핵심 구조를 표현하므로 C++의 모든 문법 요소를 반드시 표시하지는 않는다.
상수 멤버 함수는 C++에서 매개변수 목록 뒤에 const를 붙인다.

```
int GetHp() const;
```

기본 UML 표기는 반환 타입까지만 표시할 수 있다.

```
+ GetHp() : int
```

const 여부가 설계에서 중요하다면 프로젝트의 표기 규칙에 따라 제약 조건(Constraint)을 추가할 수 있다.

```
+ GetHp() : int {const}
```

UML 표준이 C++의 const 문법을 직접 정의하는 것은 아니므로, 이러한 추가 표기를 사용할 때는 문서 전체에서 같은 규칙을 유지해야 한다.
정적 멤버(Static Member)는 UML에서 이름에 밑줄을 그어 표현한다. 텍스트 환경에서는 {static}을 붙여 구분할 수 있다.

```
- playerCount : int {static}
+ GetPlayerCount() : int {static}

private:
    inline static int playerCount = 0;
public:
    static int GetPlayerCount();
```

UML 도구나 프로젝트의 작성 규칙에 따라 세부 표기는 달라질 수 있다. 중요한 것은 한 문서 안에서 표기의 의미를 일관되게 유지하는 것이다.

클래스 다이어그램과 **C++** 선언
앞의 Character 클래스 다이어그램은 C++ 헤더 파일로 옮길 수 있다.

```
// Character.h
#pragma once

#include <string>
#include "Vector2.h"

class Character
{
public:
    Character(const std::string& name);

    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);

    const std::string& GetName() const;
    const Vector2& GetPosition() const;
    int GetHp() const;

private:
    std::string name;
    Vector2 position;
    int hp = 100;
};
```

다이어그램의 속성은 멤버 변수에 대응하고, 연산은 멤버 함수 선언에 대응한다.

3.11.2 클래스 사이의 관계(Relationships Between Classes)

객체지향 프로그램은 여러 클래스의 협력으로 구성된다. 클래스는 공통 특성을 공유하거나, 다른 객체의 기능을 사용하고, 다른 객체와 연결되거나, 객체를 자신의 구성 요소로 포함한다.
클래스 다이어그램은 이러한 관계를 선, 화살표와 마름모로 나타낸다.

관계	의미	C++에서 주로 나타나는 형태
일반화(Generalization)	상위 타입의 특성을 하위 타입이 이어받음	상속
의존(Dependency)	다른 객체를 일시적으로 사용	매개변수, 반환값, 지역 변수
연관(Association)	다른 객체와 지속적으로 연결	포인터나 참조 멤버

집합(Aggregation)	독립적으로 존재하는 객체를 모아 구성	외부 객체의 포인터나 참조 저장
합성(Composition)	다른 객체를 자신의 구성 요소로 소유	값 타입 멤버 또는 독점 소유

UML 관계가 특정 C++ 문법 하나와 항상 정확히 일치하는 것은 아니다. 포인터 멤버는 연관, 집합 또는 합성을 모두 표현할 수 있다. 관계의 정확한 의미는 객체의 소유권과 수명 관리 방법까지 함께 살펴보아야 한다.

일반화와 상속**(Generalization and Inheritance)**
여러 클래스의 공통 특성을 상위 클래스에 정의하고 하위 클래스가 이를 이어받는 관계를 일반화라고 한다. C++에서는 상속을 사용하여 일반화 관계를 구현한다.

Player와 Enemy가 이름, 위치, 체력과 이동 기능을 공통으로 가진다면 해당 내용을 Character에 정의할 수 있다.

```
Player ──────────▷ Character
Enemy  ──────────▷ Character
```

일반화 관계는 실선과 속이 빈 삼각형으로 표현한다. 삼각형은 더 일반적인 타입인 상위 클래스를 향한다.

```
Player ──────────▷ Character
Enemy  ──────────▷ Character
```

C++의 상속 문법은 클래스 이름 뒤에 기반 클래스를 지정한다.

```
class Character
{
public:
    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);
    int GetHp() const;

protected:
    Vector2 position;
    int hp = 100;
};
class Player : public Character
{
public:
    void LevelUp();

private:
    int level = 1;
};
class Enemy : public Character
{
public:
    void Update();

private:
    int reward = 100;
};
```

<UML과 C++의 용어 비교>

UML	C++
상위 클래스	기반 클래스(Base Class)
하위 클래스	파생 클래스(Derived Class)
일반화 관계	상속 관계
빈 삼각형 화살표	class Player : public Character

Player와 Enemy는 모두 Character의 한 종류이다.

```
Player는 Character이다.
```

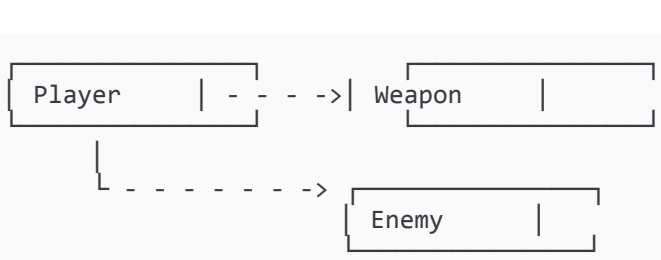
```
Enemy는 Character이다.
```

이러한 관계를 is-a 관계라고도 한다.

관계	문장	상속 관계로 적합
Player와 Character	Player는 Character이다	적합
Enemy와 Character	Enemy는 Character이다	적합
Player와 Weapon	Player는 Weapon이다	부적합

Player가 Weapon을 사용하거나 장착하는 관계는 일반화가 아니라 의존 또는 연관으로 표현해야 한다.
상속의 접근 수준, 생성자와 소멸자 호출 순서, 함수 재정의와 가상 함수는 상속을 다루는 장에서 자세히 설명한다.

의존 관계**(Dependency)**
한 클래스가 함수의 매개변수, 반환값이나 지역 변수로 다른 클래스를 일시적으로 사용하면 의존 관계가 형성된다.
Player는 공격 기능을 처리하는 동안 Weapon과 Enemy 객체를 사용한다.



의존 관계는 점선 화살표로 표현하며, 화살표는 사용하는 클래스에서 사용되는 클래스를 향한다.

```
class Player
{
public:
    void Attack(Weapon& weapon, Enemy& enemy);
};
void Player::Attack(Weapon& weapon, Enemy& enemy)
{
    int power = weapon.GetPower();
    enemy.TakeDamage(power);
}
```

Weapon과 Enemy를 멤버로 저장하지 않으므로 함수 호출이 끝나면 관계도 종료된다.

항목	내용
관계의 시작	함수가 호출될 때
관계의 유지	함수가 실행되는 동안
객체 저장	하지 않음
대표 코드	매개변수, 반환값, 지역 변수

연관 관계**(Association)**
한 객체가 다른 객체와 지속적인 관계를 유지하면 연관 관계가 형성된다.
플레이어가 현재 장착한 무기를 기억하려면 Weapon 객체의 주소를 멤버로 저장할 수 있다.



연관 관계는 실선으로 표현한다. 화살표를 표시하면 어느 클래스가 다른 클래스를 알고 있는지 나타낼 수 있다.

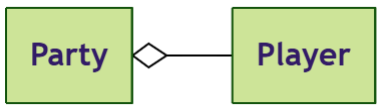
```
class Player
{
public:
    void EquipWeapon(Weapon* newWeapon);
```

```
private:
    Weapon* weapon = nullptr;
};
void Player::EquipWeapon(Weapon* newWeapon)
{
    weapon = newWeapon;
}
```

Player는 Weapon 객체의 주소를 저장하므로 EquipWeapon() 호출이 끝난 뒤에도 관계가 유지된다.
포인터 멤버만으로는 객체의 소유권을 판단할 수 없다. 무기를 생성하고 소멸시키는 책임이 다른 객체에 있다면 두 클래스는 단순한 연관 관계일 수 있다.

항목	내용
관계의 시작	객체의 주소나 참조를 저장할 때
관계의 유지	연결을 변경하거나 해제할 때까지
객체 저장	객체의 주소나 참조 저장
소유권	포인터만으로 판단할 수 없음

집합 관계**(Aggregation)**
집합 관계는 전체 객체가 여러 부분 객체를 모아 관리하지만, 부분 객체가 전체 객체와 독립적으로 존재할 수 있는 관계이다.
Party는 여러 Player를 구성원으로 관리한다. 플레이어는 파티에 참가하기 전에도 존재할 수 있고, 파티가 해체된 뒤에도 계속 존재할 수 있다.



집합 관계는 전체 객체 쪽에 속이 빈 마름모를 표시한다.

```
class Party
{
public:
    bool AddMember(Player* player);

private:
    Player* members[4] = {};
    int memberCount = 0;
};
bool Party::AddMember(Player* player)
{
    if (player == nullptr || memberCount >= 4)
    {
        return false;
    }

    members[memberCount] = player;
    ++memberCount;

    return true;
}
```

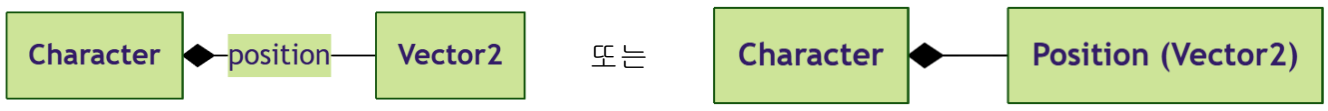
Party는 Player의 주소를 보관하지만 플레이어의 생성과 소멸을 책임지지 않는다고 가정한다.

항목	내용
전체 객체	Party
부분 객체	Player
부분 객체의 독립 존재	가능
전체 객체 소멸 후 부분 객체	계속 존재할 수 있음

UML 표기	속이 빈 마름모

집합과 일반 연관은 실제 설계에서 경계가 분명하지 않을 수 있다. 소유 관계를 강조할 필요가 없다면 단순한 연관으로 표현하기도 한다.

합성 관계**(Composition)**
합성 관계는 전체 객체가 부분 객체를 자신의 구성 요소로 소유하며, 부분 객체의 수명이 전체 객체의 수명에 종속되는 강한 포함 관계이다.
Character는 Vector2 객체를 위치 정보로 직접 포함한다.



합성 관계는 전체 객체 쪽에 채워진 마름모를 표시한다.

```
class Character
{
private:
    Vector2 position;
};
```

position은 Character 객체 내부에 직접 포함된다.

시점	position의 상태
Character 생성	함께 생성
Character 사용	구성 요소로 존재
Character 소멸	함께 소멸

합성이 반드시 값 타입 멤버로만 구현되는 것은 아니다. 동적 할당한 객체를 전체 객체가 독점적으로 소유하고 소멸시키는 방식도 합성으로 볼 수 있다. 판단 기준은 문법의 모양이 아니라 소유권과 수명이다.

<관계별 특징>

구분	일반화	의존	연관	집합	합성
UML 표기	빈 삼각형	점선 화살표	실선	빈 마름모	채워진 마름모
기본 의미	한 종류임	일시적으로 사용	지속적으로 연결	독립 객체를 모아 관리	구성 요소를 소유
C++ 표현	상속	매개변수, 지역 변수	포인터·참조 멤버	외부 객체의 주소 저장	값 멤버 또는 독점 소유
부분 객체 독립 존재	해당 없음	가능	가능	가능	일반적으로 불가능
코드 예	Player : Character	Attack(Weapon&)	Weapon* weapon	Player* members[4]	Vector2 position

다중성**(Multiplicity)**
다중성은 클래스 사이에 연결될 수 있는 객체의 개수를 나타낸다.

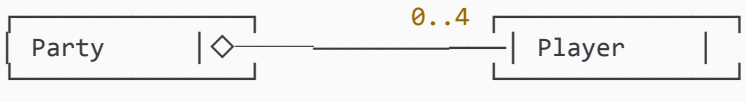
표기	의미
1	정확히 하나
0..1	없거나 하나
* 또는 0..*	없거나 여러 개
1..*	하나 이상
1..4	하나 이상 네 개 이하

현재 장착된 무기가 없을 수도 있고 하나만 있을 수 있다면 0..1로 표현한다.




```
Weapon* weapon = nullptr;
```

파티가 최대 네 명의 플레이어를 관리한다면 0..4로 표시할 수 있다.




```
Player* members[4] = {};
```

다중성은 연결되는 객체의 수를 표현하지만 배열이나 컨테이너의 구체적인 구현 방법까지 결정하지는 않는다.

3.11.3 시퀀스 다이어그램(Sequence Diagram)

클래스 다이어그램은 프로그램의 정적인 구성을 보여 주지만, 실행 중 객체가 동작하는 순서는 표현하지 않는다.
시퀀스 다이어그램은 객체들이 시간의 흐름에 따라 어떤 메시지를 주고받는지 나타낸다. 시간은 위에서 아래로 흐른다.

```
위
 │
 │  시간의 흐름
 ▼
아래
```

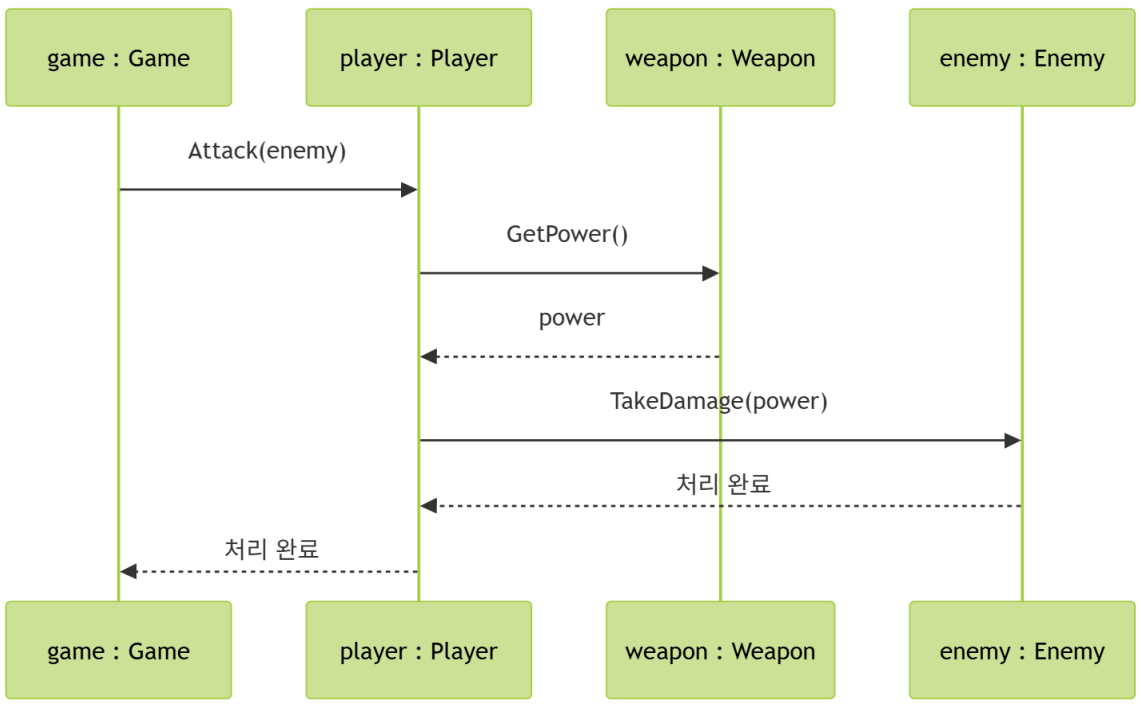
시퀀스 다이어그램은 참여자, 생명선, 메시지, 반환 메시지와 실행 구간으로 구성한다.

요소	의미
참여자(Participant)	기능 호출에 참여하는 객체나 외부 대상
생명선(Lifeline)	객체가 호출 과정에 존재함을 나타내는 세로선
메시지(Message)	다른 객체의 기능을 호출하는 화살표
반환 메시지(Return Message)	호출 결과가 돌아오는 것을 나타내는 점선 화살표
실행 구간(Activation)	객체가 실제 기능을 처리하는 구간

객체는 객체이름 : 클래스이름 형식으로 표시한다.

player : Player
weapon : Weapon
enemy : Enemy

공격 과정의 시퀀스 다이어그램
Game이 Player에게 적을 공격하도록 요청한다. 플레이어는 장착한 무기에서 공격력을 얻고, 적에게 피해 처리를 요청한다.



호출은 Game, Player, Weapon, Enemy 순서로 진행된다.

순서	요청 객체	처리 객체	메시지
1	Game	Player	Attack(enemy)
2	Player	Weapon	GetPower()
3	Weapon	Player	공격력 반환
4	Player	Enemy	TakeDamage(power)
5	Enemy	Player	처리 완료
6	Player	Game	공격 처리 완료

C++ 코드에서는 멤버 함수 호출로 표현한다.

```
void Player::Attack(Enemy& enemy)
{
    if (weapon == nullptr)
    {
        return;
    }

    int power = weapon->GetPower();
    enemy.TakeDamage(power);
}
int Weapon::GetPower() const
{
    return power;
}
void Enemy::TakeDamage(int damage)
{
    if (damage <= 0)
    {
        return;
    }

    hp -= damage;

    if (hp < 0)
    {
        hp = 0;
    }
}
```

시퀀스 다이어그램의 메시지와 C++ 호출은 서로 대응한다.

시퀀스 다이어그램	C++ 코드
Attack(enemy)	player.Attack(enemy)
GetPower()	weapon->GetPower()
TakeDamage(power)	enemy.TakeDamage(power)
power 반환	return power;

반환 결과가 중요할 때는 반환 메시지를 표시한다. 반환값이 없거나 제어가 돌아오는 과정이 중요하지 않다면 생략할 수 있다.
시퀀스 다이어그램을 사용하면 객체의 협력 구조를 확인할 수 있다.

확인 내용	설계 질문
호출 순서	어떤 객체가 먼저 동작하는가
요청 주체	어떤 객체가 기능을 요청하는가
처리 책임	실제 작업을 수행하는 객체는 무엇인가
협력 범위	한 기능에 몇 개의 객체가 참여하는가
호출 집중	특정 객체에 요청이 지나치게 집중되는가
불필요한 왕복	객체 사이에 불필요한 호출이 반복되는가

두 다이어그램의 역할을 비교하면 차이가 분명하다.

구분	클래스 다이어그램	시퀀스 다이어그램
관점	정적인 구조	동적인 동작
주요 대상	클래스와 관계	객체와 메시지
표현 내용	상태, 기능, 상속, 포함과 연결	호출 순서와 반환
확인 질문	무엇으로 구성되는가	어떤 순서로 협력하는가

3.11.4 UML에서 C++ 코드로(From UML to C++ Code)

UML은 코드와 분리된 장식이 아니다. 구현할 객체의 상태와 기능, 클래스 사이의 관계를 코드 작성 전에 정리하는 도구로 사용할 수 있다.
캐릭터 객체에 필요한 조건을 정리해 보자.

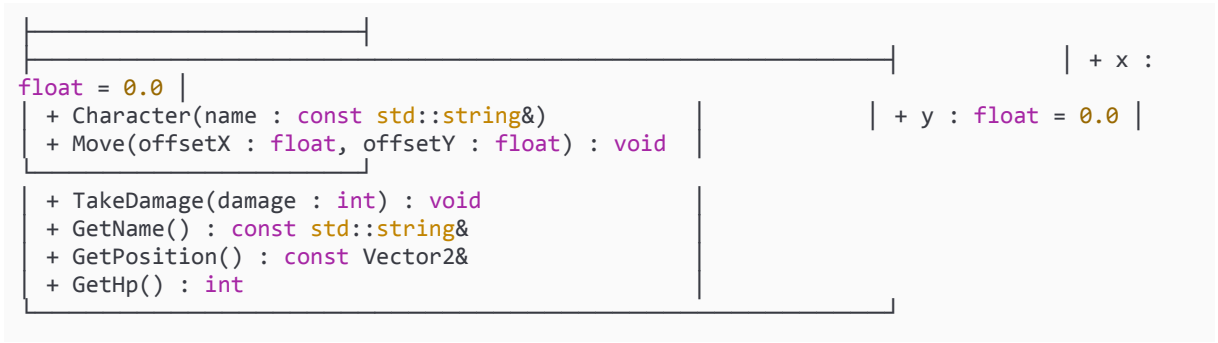
- 캐릭터는 이름, 위치와 체력을 가진다.
- 지정된 거리만큼 이동할 수 있다.
- 피해를 받으면 체력이 감소하지만 0보다 작아지지 않는다.
- 외부에서는 이름, 위치와 현재 체력을 확인할 수 있다.

설계 대상은 Character이며 상태와 기능은 표로 구분할 수 있다.

구분	내용
클래스 이름	Character
상태	이름, 위치, 체력
기능	이동, 피해 처리, 이름 확인, 위치 확인, 체력 확인

클래스 다이어그램





Character는 Vector2 객체를 값 타입 멤버로 포함하므로 두 클래스는 합성 관계이다.

<UML과 C++ 코드의 대응>

UML	C++
Character	class Character
- name : std::string	private: std::string name;
- position : Vector2	private: Vector2 position;
- hp : int = 100	private: int hp = 100;
+ Move(...) : void	public: void Move(...);
+ TakeDamage(...) : void	public: void TakeDamage(...);
+ GetName() : const std::string&	public: const std::string& GetName() const;
+ GetPosition() : const Vector2&	public: const Vector2& GetPosition() const;
+ GetHp() : int	public: int GetHp() const;

```cpp
// Vector2.h
#pragma once

struct Vector2
{
    float x = 0.0f, y = 0.0f;
};
```

```cpp
// Character.h
#pragma once

#include <string>
#include "Vector2.h"

class Character
{
public:
    Character(const std::string& name);

    void Move(float offsetX, float offsetY);
    void TakeDamage(int damage);

    const std::string& GetName() const;
    const Vector2& GetPosition() const;
    int GetHp() const;

private:
    std::string name;
    Vector2 position;
    int hp = 100;
};
```

```cpp
// Character.cpp
#include "Character.h"

Character::Character(const std::string& name)
    : name(name)
{
```

```
}

void Character::Move(float offsetX, float offsetY)
{
    position.x += offsetX;
    position.y += offsetY;
}

void Character::TakeDamage(int damage)
{
    if (damage <= 0)
    {
        return;
    }
    if (hp -= damage; hp < 0)
    {
        hp = 0;
    }
}

const std::string& Character::GetName() const
{
    return name;
}

const Vector2& Character::GetPosition() const
{
    return position;
}

int Character::GetHp() const
{
    return hp;
}
```

```
// main.cpp
#include <iostream>
#include "Character.h"

int main()
{
    Character character("Warrior");

    character.Move(10.0f, 20.0f);
    character.TakeDamage(30);

    const Vector2& position = character.GetPosition();

    std::cout << "Name: "
              << character.GetName() << '\n';

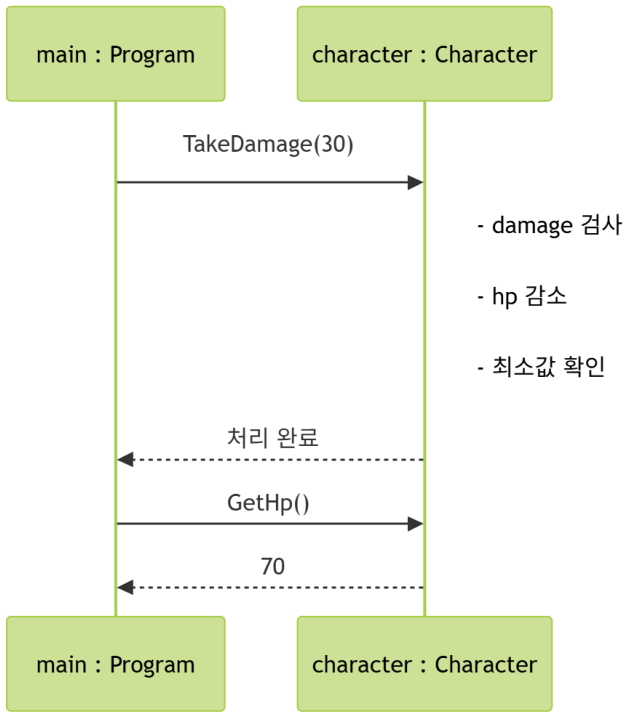
    std::cout << "Position: "
              << position.x << ", "
              << position.y << '\n';

    std::cout << "HP: "
              << character.GetHp() << '\n';

    return 0;
}

// 실행 결과
Name: Warrior
Position: 10, 20
HP: 70
```

동작을 시퀀스 다이어그램으로 표현
main()은 캐릭터에게 피해 처리를 요청하고 현재 체력을 확인한다.



이 흐름은 두 개의 멤버 함수 호출로 구현한다.

```
character.TakeDamage(30);
std::cout << character.GetHp() << '\n';
```

UML을 먼저 작성하면 구현 전에 설계상의 문제를 확인할 수 있다.

확인 항목	질문
객체의 상태	클래스에 필요한 데이터는 무엇인가
공개 범위	어떤 기능을 외부에 공개해야 하는가
상태 변경	객체의 상태를 변경하는 기능은 무엇인가
유효한 상태	잘못된 상태가 만들어질 가능성은 없는가
상속 관계	공통 특성을 상위 클래스로 분리할 수 있는가
포함 관계	객체가 어떤 객체를 구성 요소로 가지는가
연결 관계	어떤 객체가 다른 객체를 지속적으로 알아야 하는가
객체 협력	어떤 객체가 누구에게 기능을 요청하는가
호출 순서	여러 객체가 어떤 순서로 동작하는가

앞으로 클래스의 구조가 중요한 예제에서는 클래스 다이어그램을 먼저 제시한다. 여러 객체가 협력하는 예제에는 시퀀스 다이어그램을 함께 사용하여 호출 순서를 표현한다. UML로 정리한 구조와 동작은 C++ 코드로 구현하면서 각 문법과 설계의 의미를 확인한다.

3.12 학습 정리

객체지향 모델링은 프로그램에 필요한 객체를 식별하고, 각 객체의 상태와 책임, 객체 사이의 관계와 협력을 설계하는 과정이다. UML은 객체의 구조와 동작을 다이어그램으로 표현하는 모델링 언어이다.

클래스는 객체의 상태와 기능을 정의한 사용자 정의 타입이다. 객체는 클래스에 따라 생성되어 각각의 상태를 가지며, 특정 클래스에 따라 생성된 객체임을 강조할 때 인스턴스라고 한다.

절차적 구조는 작업의 처리 순서와 함수를 중심으로 프로그램을 구성한다. 객체지향 구조는 관련된 상태와 기능을 책임을 가진 객체에 모으고, 객체 사이의 요청과 협력으로 프로그램을 구성한다. 독립적으로 설계된 객체의 관계를 형성하고 조합하여 더 큰 객체와 프로그램 구조로 확장할 수 있다.

관련된 상태와 기능을 하나의 객체에 모으면 응집도를 높일 수 있고, 객체가 공개된 기능을 통해 협력하도록 구성하면 객체 사이의 결합도를 낮출 수 있다. 객체의 내부 구현에 대한 의존이 줄어들면 기존 객체를 재사용하기 쉬워지고, 요구 사항이 변경될 때 수정 범위가 줄어 프로그램의 유연성과 안정성을 높일 수 있다.

객체지향 프로그래밍의 주요 특징은 추상화, 캡슐화, 상속과 다형성이다. 추상화는 목적에 필요한 특징만 선택하여 표현하는 것이며, 캡슐화는 관련된 상태와 기능을 하나의 객체에 모으고 공개된 기능을 통해 상태 변경 규칙을 관리하는 것이다. 상속은 기존 클래스의 특성을 바탕으로 새로운 클래스를 구성하는 것이며, 다형성은 공통된 인터페이스를 통해 서로 다른 객체를 동일한 방식으로 사용할 수 있게 한다.

C++의 class와 struct는 모두 클래스 타입을 정의한다. class의 기본 접근 수준은 private이고 struct는 public이다. 단순한 데이터와 값의 표현에는 struct, 상태와 변경 규칙을 관리하는 객체에는 class를 우선 고려한다.

접근 제어는 public, protected, private을 사용하여 멤버의 접근 범위를 제한한다. 공개된 멤버는 객체의 인터페이스를 구성하고, 비공개 멤버는 객체 내부의 상태와 구현을 표현한다.

객체가 항상 만족해야 하는 상태 조건을 불변식이라고 한다. 캡슐화의 목적은 데이터를 숨기는 데 그치지 않고, 객체가 자신의 상태와 상태 변경 규칙을 책임지도록 만드는 데 있다.

게터와 세터를 기계적으로 추가한다고 캡슐화가 완성되는 것은 아니다. 상태 변경에 규칙이 있다면 값 자체를 설정하는 함수보다 객체의 역할과 의도를 나타내는 기능을 제공해야 한다. 비상수 참조 반환은 외부에서 내부 상태를 직접 변경할 수 있으므로 주의해야 한다.

멤버 함수는 자신을 호출한 객체의 상태를 처리한다. this는 현재 객체의 주소이고, *this는 현재 객체를 의미한다. 객체 자신을 참조로 반환하면 같은 객체를 계속 사용할 수 있으며 메서드 체이닝을 구현할 수 있다.

상수 멤버 함수는 호출 객체의 상태를 변경하지 않는다는 의도를 나타낸다. 함수의 매개변수 목록 뒤에 const를 붙여 선언하며, 상수 객체에서는 상수 멤버 함수만 호출할 수 있다.

비정적 데이터 멤버는 객체마다 별도로 존재하고, 정적 데이터 멤버는 클래스 전체에서 하나만 존재한다. 정적 멤버 함수에는 this가 없으므로 특정 객체의 비정적 멤버에 직접 접근할 수 없다.

클래스 내부에는 중첩 클래스, 중첩 열거형과 타입 별칭을 선언할 수 있다. friend 함수와 friend 클래스는 비공개 멤버에 접근할 수 있지만, 내부 구현에 대한 의존성을 높일 수 있으므로 제한적으로 사용한다.

클래스 정의와 멤버 함수 선언은 주로 헤더 파일에 작성하고, 멤버 함수의 구현은 소스 파일에 분리한다. 선언과 정의의 함수 이름, 반환 타입, 매개변수, const와 함수 한정자는 서로 일치해야 한다.

전방 선언은 타입의 이름만 컴파일러에 알리는 선언이다. 포인터와 참조를 선언할 때 사용할 수 있지만, 값 타입 멤버, 객체 생성, 멤버 함수 호출처럼 전체 타입 정보가 필요한 작업에는 헤더를 포함해야 한다.

클래스 다이어그램은 클래스의 상태와 기능, 일반화, 의존, 연관, 집합과 합성 관계를 표현한다. 시퀀스 다이어그램은 객체가 시간의 흐름에 따라 주고받는 요청과 처리 순서를 표현한다. UML 관계는 C++ 문법의 모양만으로 판단하지 않고 객체의 역할, 소유권과 수명을 함께 고려해야 한다.

3.13 학습 과제

과제 **1.** 일상생활 객체 분석

- 일상생활의 관리 영역 하나 선정
- 서로 관계있는 객체 **5**개 이상 식별
- 객체별 작성 내용
    - 객체 이름
    - 역할과 책임
    - 상태
    - 기능
    - 상태 변경 규칙
- 객체 사이의 관계와 협력 분석
- 관계 자체에 상태와 기능이 필요하면 새로운 객체 추가

과제 **2.** 게임 시스템 객체 분석

- 실제 출시된 게임 하나 선정
- 게임 제목과 장르 작성(필수)
- 분석할 게임 시스템의 범위 작성
- 서로 관계있는 객체 **5**개 이상 식별
- 게임에 실제로 등장하는 구체적인 명칭 사용
- Mob, Monster, Enemy, Item과 같은 포괄적인 이름만 사용하지 않음
- 객체별 작성 내용
    - 객체 이름
    - 역할과 책임
    - 상태
    - 기능
    - 상태 변경 규칙
- 객체 사이의 관계와 협력 분석
- 서로 다른 게임의 객체를 혼합하지 않음

과제 **3.** 일상생활 객체의 **UML** 설계

- 과제 1의 객체를 UML 클래스 다이어그램으로 표현

- 클래스 이름, 속성, 연산과 접근 수준 표시
- 일반화, 의존, 연관, 집합과 합성 관계 표시
- 관계의 방향과 다중성 표시

과제 **4.** 게임 시스템 객체의 **UML** 설계
- 과제 2의 객체를 UML 클래스 다이어그램으로 표현
- 클래스 이름, 속성, 연산과 접근 수준 표시
- 일반화, 의존, 연관, 집합과 합성 관계 표시
- 관계의 방향과 다중성 표시
- 실제 게임 시스템에서의 객체 관계 반영

과제 **5.** 일상생활 객체의 **C++** 클래스 구현
- 과제 3의 UML을 C++ 클래스로 구현
- 속성을 멤버 변수로 구현
- 연산을 멤버 함수로 구현
- 객체의 역할에 따라 class와 struct 선택
- 상태와 변경 규칙을 고려하여 접근 수준 지정
- UML의 객체 관계를 C++ 코드에 반영

과제 **6.** 게임 시스템 객체의 **C++** 클래스 구현
- 과제 4의 UML을 C++ 클래스로 구현
- 실제 게임 객체의 구체적인 이름 유지
- 속성을 멤버 변수로 구현
- 연산을 멤버 함수로 구현
- 상태와 변경 규칙을 고려하여 접근 수준 지정
- UML의 상속, 사용, 연결과 소유 관계를 C++ 코드에 반영

04. 생성자와 소멸자, 초기화

클래스 타입으로 객체를 생성하면 객체를 저장할 공간이 마련되고 각 멤버가 초기화된다. 생성자는 이 과정에 참여하여 객체의 초기 상태를 설정한다. 소멸자는 객체의 수명이 끝날 때 호출되어 객체가 사용하던 자원을 정리한다.

객체를 만든 뒤 멤버 변수에 값을 하나씩 대입할 수도 있다. 그러나 초기화되지 않은 객체가 잠시 존재하며, 필요한 설정 함수를 호출하지 않으면 잘못된 상태로 사용될 수 있다. 생성자를 사용하면 객체가 생성되는 시점에 필요한 값을 전달하고, 처음부터 사용할 수 있는 상태를 구성할 수 있다.

C++은 객체를 초기화하는 여러 문법을 제공한다. 괄호를 사용한 직접 초기화, 등호를 사용한 복사 초기화, 중괄호를 사용한 리스트 초기화는 형태뿐만 아니라 생성자 선택과 형 변환 규칙에서도 차이가 있다.

컴파일러가 기본 생성자와 소멸자를 작성하는 경우도 있다. 사용자가 생성자를 선언하면 컴파일러가 제공하던 기본 생성자가 더 이상 만들어지지 않을 수 있으며, 멤버의 구성에 따라 컴파일러가 생성한 함수가 사용할 수 없는 상태가 되기도 한다.

이 장에서는 객체의 생성과 소멸 과정을 이해하고 생성자, 소멸자, 멤버 초기화 리스트와 여러 초기화 문법을 사용한다. = default, = delete, explicit, std::initializer_list를 이용하여 객체의 생성 방법을 제어하고, const와 constexpr를 반영한 클래스 인터페이스도 구성한다.

4.1 생성자 (constructor)

생성자는 객체가 생성될 때 호출되는 특별한 멤버 함수이다. 생성자의 이름은 클래스 이름과 같으며 반환 타입을 작성하지 않는다.

```
struct Point
{
    int x, y;
};
class Rectangle
{
public:
    Rectangle()
    {
        position.x = 0;
        position.y = 0;
        width = 1;
        height = 1;
    }
private:
    Point position;
    int width;
    int height;
};
```

Rectangle 객체를 선언하면 Rectangle() 생성자가 호출된다.

```
Rectangle rectangle;
```

생성자에는 반환 타입을 작성하지 않는다.

```
Rectangle();        // 생성자
// void Rectangle();   // 오류
// int Rectangle();    // 오류
```

생성자는 일반 함수처럼 값을 반환하지 않는다. 객체의 저장 공간이 마련되고 멤버가 초기화되는 과정에 참여하여 객체의 초기 상태를 완성한다.

4.1.1 생성자의 역할

생성자의 가장 기본적인 역할은 멤버 변수를 초기화하는 것이다.

```
class Circle
{
public:
    Circle()
    {
        center.x = 0;
        center.y = 0;
```

```
        radius = 1;
    }
private:
    Point center;
    int radius;
};
```

생성자는 전달받은 값이 객체의 상태 규칙을 만족하는지도 검사할 수 있다.

```
class Circle
{
public:
    Circle(int x, int y, int radius)
    {
        center.x = x;
        center.y = y;
        if (radius > 0)
        {
            this->radius = radius;
        }
        else
        {
            this->radius = 1;
        }
    }

private:
    Point center;
    int radius;
};
```

반지름은 0보다 커야 한다. 생성자에서 이 규칙을 적용하면 잘못된 반지름을 가진 객체가 만들어지는 것을 방지할 수 있다.

```
Circle first(10, 20, 30);
Circle second(10, 20, -5);
```

second에 전달한 반지름은 음수이므로 생성자는 반지름을 1로 설정한다.

생성자는 객체가 만들어질 때 필요한 작업을 담당한다.

- 멤버 변수의 초기값 설정
- 전달받은 값의 유효성 검사
- 객체 타입 멤버의 생성
- 객체가 사용할 자원의 준비
- 객체의 상태 규칙 설정

생성자는 객체의 모든 기능을 처리하는 함수가 아니다. 객체의 생성과 초기 상태 구성에 필요한 작업에 집중해야 한다.

4.1.2 기본 생성자 (default constructor)

인수 없이 호출할 수 있는 생성자를 기본 생성자라고 한다.

```
class Rectangle
{
public:
    Rectangle()
    {
        position.x = 0;
        position.y = 0;
        width = 1;
        height = 1;
    }

private:
    Point position;
    int width;
    int height;
};
```

객체를 인수 없이 선언하면 기본 생성자가 호출된다.

```
Rectangle first;
Rectangle second;
```

기본 생성자를 호출하기 위해 빈 괄호를 붙이지 않는다.

```
Rectangle rectangle;
```

빈 괄호를 붙이면 객체 선언이 아니라 함수 선언으로 해석된다.

```
Rectangle rectangle();
```

이 코드는 Rectangle 객체를 반환하고 매개변수가 없는 rectangle이라는 함수를 선언한다.

중괄호를 사용하면 객체 생성임을 분명하게 표현할 수 있다.

```
Rectangle rectangle{};
```

4.1.3 암시적으로 선언되는 기본 생성자 (Implicitly-declared Default Constructor)

클래스에 생성자가 하나도 선언되지 않으면 컴파일러가 기본 생성자를 암시적(implicit) 으로 선언할 수 있다.

```
class Rectangle
{
private:
    Point position;
    int width;
    int height;
};
```

이 클래스에는 사용자가 작성한 생성자가 없지만 인수 없이 객체를 선언할 수 있다.

```
Rectangle rectangle;
```

컴파일러가 소스 파일에 실제 생성자 코드를 삽입하는 것은 아니다. 컴파일 과정에서 필요한 기본 생성자를 암시적으로 선언하고, 프로그램에서 사용되면 그 정의를 생성한다.

컴파일러가 기본 생성자를 제공하는 것은 모든 멤버 변수를 0으로 초기화한다는 뜻이 아니다.

```
Rectangle rectangle;
```

rectangle을 기본 초기화하면 기본형 멤버인 width와 height에는 정해지지 않은 값이 남을 수 있다. position.x와 position.y도 자동으로 0이 되지 않는다.

컴파일러가 생성한 기본 생성자는 각 멤버를 해당 타입의 기본 초기화 규칙에 따라 초기화한다. 클래스 타입 멤버에는 그 클래스의 기본 생성자가 호출되지만, 기본형 멤버에는 별도의 초기값이 설정되지 않을 수 있다.

사용자가 생성자를 하나라도 선언하면 컴파일러는 기본 생성자를 자동으로 선언하지 않는다.

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;
        this->width = width;
        this->height = height;
    }

private:
    Point position;
    int width;
```

```
    int height;
};
```

이 클래스에는 매개변수가 있는 생성자만 존재한다.

```
// Rectangle first;   // 오류

Rectangle second(10, 20, 100, 50);
```

인수 없는 객체 생성도 허용하려면 기본 생성자를 직접 선언해야 한다.

```
class Rectangle
{
public:
    Rectangle()
    {
        position.x = 0;
        position.y = 0;
        width = 1;
        height = 1;
    }

    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;
        this->width = width;
        this->height = height;
    }

private:
    Point position;
    int width;
    int height;
};
```

기본 생성자의 의미는 두 가지로 구분해야 한다.

- 인수 없이 호출할 수 있는 생성자
- 사용자가 생성자를 선언하지 않아 컴파일러가 암시적으로 제공한 생성자

사용자가 직접 작성한 Rectangle()도 기본 생성자이며, 컴파일러가 제공한 Rectangle()도 기본 생성자이다. 두 생성자의 초기화 동작은 서로 다를 수 있다.

4.1.4 매개변수가 있는 생성자 (Parameterized Constructor)

객체마다 서로 다른 초기 상태가 필요하면 생성자에 매개변수를 선언한다.

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;

        if (width > 0)
        {
            this->width = width;
        }
        else
        {
            this->width = 1;
        }

        if (height > 0)
        {
            this->height = height;
```

```
        }
        else
        {
            this->height = 1;
        }
    }

private:
    Point position;
    int width;
    int height;
};
```

위치와 크기를 전달하여 객체를 생성한다.

```
Rectangle first(10, 20, 100, 50);
Rectangle second(0, 0, 640, 480);
```

생성자 매개변수는 객체를 완성하는 데 필요한 값을 클래스의 인터페이스로 표현한다.

객체가 반드시 위치와 크기를 가져야 한다면 기본 생성자를 제공하지 않고 매개변수가 있는 생성자만 제공할 수 있다.

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height);

private:
    Point position;
    int width;
    int height;
};

Rectangle rectangle(10, 20, 100, 50);

// Rectangle invalidRectangle;   // 오류
```

이 설계에서는 위치와 크기가 지정되지 않은 Rectangle 객체를 만들 수 없다.

4.1.5 생성자 오버로딩 (Constructor Overloading)

생성자도 매개변수의 개수나 타입이 다르면 오버로딩할 수 있다.

```
class Rectangle
{
public:
    Rectangle()
    {
        position.x = 0;
        position.y = 0;
        width = 1;
        height = 1;
    }

    Rectangle(int width, int height)
    {
        position.x = 0;
        position.y = 0;
        this->width = width;
        this->height = height;
    }

    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;
        this->width = width;
```

```
        this->height = height;
    }

private:
    Point position;
    int width;
    int height;
};
```

객체 생성에 사용한 인수에 맞는 생성자가 선택된다.

```
Rectangle first;
Rectangle second(100, 50);
Rectangle third(10, 20, 100, 50);
```

- first는 기본 생성자를 호출한다.
- second는 크기만 받는 생성자를 호출한다.
- third는 위치와 크기를 받는 생성자를 호출한다.

생성자 오버로딩을 사용하면 객체를 여러 방법으로 만들 수 있다. 그러나 생성자마다 같은 초기화 코드가 반복될 수 있다.

```
Rectangle()
{
    position.x = 0;
    position.y = 0;
    width = 1;
    height = 1;
}

Rectangle(int width, int height)
{
    position.x = 0;
    position.y = 0;
    this->width = width;
    this->height = height;
}
```

중복되는 초기화 코드는 기본 멤버 초기값과 위임 생성자를 사용하여 줄일 수 있다.
기본 인수를 사용하는 생성자와 다른 생성자를 함께 선언할 때는 호출이 모호해지지 않는지 확인해야 한다.

```
class Sample
{
public:
    Sample();
    Sample(int value = 0);
};
```

두 생성자 모두 인수 없이 호출할 수 있다.

```
// Sample sample;   // 오류: 호출할 생성자가 모호함
```

생성자 오버로딩은 각 생성 방법의 목적이 분명하고 호출 형태가 서로 구분될 때 사용한다.

<생성자 오버로딩 성립 조건>

구분	성립 여부	설명
매개변수의 개수	성립	매개변수 개수가 다르면 서로 다른 생성자로 구분
매개변수의 타입	성립	매개변수 타입이 다르면 서로 다른 생성자로 구분
매개변수 타입의 순서	성립	타입의 배치 순서가 다르면 서로 다른 생성자로 구분
반환 타입	해당 없음	생성자에는 반환 타입이 없음
매개변수의 이름	성립하지 않음	매개변수 이름은 함수 구분 기준이 아님

4.1.6 변환 생성자 (Converting Constructor)

하나의 인수로 호출할 수 있는 생성자는 다른 타입의 값을 클래스 타입 객체로 변환하는 데 사용될 수 있다.

```
class Length
{
public:
    Length(int value)
    {
        if (value >= 0)
        {
            this->value = value;
        }
        else
        {
            this->value = 0;
        }
    }

private:
    int value;
};
```

int 값을 사용하여 Length 객체를 생성할 수 있다.

```
Length first(100);
```

복사 초기화 문법을 사용하면 int가 Length로 암시적으로 변환될 수 있다.

```
Length second = 200;
```

컴파일러는 200을 인수로 Length(int) 생성자를 호출하여 Length 객체를 만든다.

함수의 매개변수가 Length 타입일 때도 암시적 변환이 발생할 수 있다.

```
void PrintLength(const Length& length)
{
    std::cout << "PrintLength: " << length.Value() << '\n';
}
```

```
PrintLength(300);
```

300으로 임시 Length 객체가 생성되고 그 객체가 함수에 전달된다.

암시적 변환이 항상 바람직한 것은 아니다. 단위나 의미가 다른 값이 실수로 전달되어도 변환이 허용될 수 있다. explicit을 지정하면 명시적인 객체 생성만 허용할 수 있다.

```
class Length
{
public:
    explicit Length(int value)
    {
        this->value = value;
    }

private:
    int value;
};

Length first(100);
Length second{200};

// Length third = 300;   // 오류
// PrintLength(400);     // 오류

PrintLength(Length(400));
```

explicit은 이후 단원에서 자세히 다룬다.

4.1.7 객체 생성과 초기화

객체의 생성은 저장 공간을 마련한 뒤 생성자 본문만 실행하는 과정이 아니다.
객체가 생성될 때 처리되는 순서는 크게 세 단계로 구분할 수 있다.

```
1. 객체의 저장 공간 마련
2. 기반 클래스와 멤버 초기화
3. 생성자 본문 실행
```

생성자 본문에 작성한 대입문은 멤버의 초기화가 끝난 뒤 실행된다.

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
    {
        position.x = x;
        position.y = y;
        this->width = width;
        this->height = height;
    }

private:
    Point position;
    int width;
    int height;
};
```

position, width, height는 생성자 본문이 실행되기 전에 이미 초기화 과정을 거친다. 생성자 본문의 코드는 그 멤버에 값을 대입한다.
멤버를 직접 초기화하려면 멤버 초기화 리스트를 사용한다.

```
Rectangle(int x, int y, int width, int height)
    : position{ x, y }, width(width), height(height)
{
}
```

멤버 초기화 리스트는 4.4절에서 다룬다.

4.2 소멸자 (Destructor)

소멸자(destructor)는 객체의 수명이 끝날 때 호출되는 특별한 멤버 함수이다.
소멸자의 이름은 클래스 이름 앞에 ~를 붙여 작성하며 반환 타입과 매개변수를 사용하지 않는다.

```
class Rectangle
{
public:
    Rectangle();
    ~Rectangle();

private:
    Point position;
    int width;
    int height;
};
```

소멸자는 클래스마다 하나만 선언할 수 있다. 매개변수를 가질 수 없으므로 오버로딩할 수 없다.

```
~Rectangle();
```

4.2.1 소멸자의 역할

소멸자는 객체가 사용하던 자원을 정리한다.

```
- 동적으로 할당한 메모리 해제
- 열린 파일 닫기
- 운영체제 자원 반환
- 잠금 해제
- 객체가 등록한 콜백이나 연결 해제
```

단순한 값만 가지는 클래스에는 사용자가 소멸자를 직접 작성하지 않아도 되는 경우가 많다.

```
class Rectangle
{
private:
    Point position;
    int width;
    int height;
};
```

Rectangle 객체가 소멸하면 position, width, height도 함께 소멸한다. 기본형 멤버는 별도의 정리 작업이 필요하지 않는다.
std::string, std::vector와 같은 표준 라이브러리 타입은 자신의 소멸자에서 내부 자원을 정리한다. 이러한 타입을 멤버로 사용하면 클래스가 직접 자원 정리 코드를 작성하지 않아도 되는 경우가 많다.

컴파일러는 사용자가 소멸자를 선언하지 않았을 때 소멸자를 암시적으로 선언할 수 있다.

```
class Rectangle
{
private:
    Point position;
    int width;
    int height;
};
```

이 클래스는 개념적으로 컴파일러가 소멸자를 제공한 것처럼 동작한다.

```
class Rectangle
{
public:
    ~Rectangle() = default;

private:
    Point position;
    int width;
    int height;
};
```

컴파일러가 제공하는 소멸자는 각 멤버의 소멸자를 호출한다.

4.2.2 객체의 소멸 시점

객체의 저장 위치와 생성 방법에 따라 소멸 시점이 달라진다.
지역 객체는 선언된 블록을 벗어날 때 소멸한다.

```
#include <iostream>
#include <string>

class Trace
{
public:
    Trace(const std::string& name)
    {
        this->name = name;
        std::cout << name << " 생성\n";
    }

    ~Trace()
    {
        std::cout << name << " 소멸\n";
    }
```

```
private:
    std::string name;
};

int main()
{
    Trace trace("local");

    std::cout << "main 실행 중\n";
}
```

실행 결과는 객체의 생성과 소멸 시점을 보여 준다.

```
local 생성
main 실행 중
local 소멸
```

전역 객체와 정적 객체는 프로그램이 종료될 때 소멸한다. 동적으로 생성한 객체는 delete를 실행할 때 소멸한다.

4.2.3 지역 객체의 소멸

지역 객체는 자신이 선언된 블록의 끝에서 소멸한다.

```
void Test()
{
    Trace first("first");

    {
        Trace second("second");
    }

    Trace third("third");
}
```

second는 내부 블록이 끝날 때 먼저 소멸한다. first와 third는 Test() 함수가 끝날 때 소멸한다.
같은 블록에 선언된 지역 객체는 생성된 순서의 반대로 소멸한다.

```
void Test()
{
    Trace first("first");
    Trace second("second");
    Trace third("third");
}
```

생성 순서는 first, second, third이며 소멸 순서는 third, second, first이다.
함수가 중간에 return을 실행하더라도 이미 생성된 지역 객체는 소멸한다.

```
void Test(bool stop)
{
    Trace trace("trace");

    if (stop)
    {
        return;
    }
}
```

예외로 블록을 벗어나는 경우에도 정상적으로 생성이 완료된 지역 객체는 소멸한다. 이러한 수명 관리 방식은 자원 관리를 객체의 수명과 연결하는 RAII의 기반이 된다.

4.2.4 동적 객체의 소멸

new로 생성한 객체는 delete를 실행할 때 소멸한다.

```
Trace* trace = new Trace("dynamic");

delete trace;
trace = nullptr;
```

delete trace는 Trace의 소멸자를 호출한 뒤 객체의 메모리를 해제한다.
new[]로 생성한 객체 배열은 delete[]로 해제해야 한다.

```
class Sample
{
public:
    Sample()
    {
        std::cout << "생성: " << this << '\n';
    }
    ~Sample()
    {
        std::cout << "소멸: " << this << '\n';
    }
};

int main()
{
    Sample* samples = new Sample[3];
    delete[] samples;
}
```

배열의 각 요소에 대해 생성자가 호출되며, delete[]를 실행하면 각 요소의 소멸자가 역순으로 호출된다.
new[]로 생성한 배열에 delete를 사용하면 정의되지 않은 동작이 발생한다.

```
Sample* samples = new Sample[3];

// delete samples;   // 오류
delete[] samples;
```

동적으로 생성한 객체는 자동으로 소멸하지 않는다. delete를 실행하지 않으면 프로그램이 실행되는 동안 해당 객체의 자원을 계속 차지할 수 있다.
모던 C++에서는 직접 new와 delete를 관리하기보다 표준 컨테이너와 스마트 포인터를 사용하여 객체의 수명과 자원 해제를 자동화하는 방식을 우선한다.

4.2.5 생성과 소멸 순서

객체가 여러 개 생성되면 생성된 순서의 반대로 소멸한다.

```
int main()
{
    Trace first("first");
    Trace second("second");
    Trace third("third");
}

first 생성
second 생성
third 생성
third 소멸
second 소멸
first 소멸
```

배열 요소도 앞에서부터 생성되고 뒤에서부터 소멸한다.

객체가 다른 객체를 멤버로 가지면 멤버 객체가 먼저 생성된 뒤 바깥 객체의 생성자 본문이 실행된다.

```
class Border
{
public:
    Border()
```

```
    {
        std::cout << "Border 생성\n";
    }

    ~Border()
    {
        std::cout << "Border 소멸\n";
    }
};

class Window
{
public:
    Window()
    {
        std::cout << "Window 생성\n";
    }

    ~Window()
    {
        std::cout << "Window 소멸\n";
    }

private:
    Border border;
};

Window window;
```

실행 순서는 멤버 객체가 바깥 객체보다 먼저 생성되고 나중에 소멸한다.

```
Border 생성
Window 생성
Window 소멸
Border 소멸
```

객체의 생성 과정에서는 내부 구성 요소를 먼저 준비해야 한다. 소멸 과정에서는 바깥 객체의 소멸자 본문을 실행한 뒤 내부 구성 요소를 정리한다.

<객체를 구성하는 부분의 생성과 소멸 순서>

구성 요소	생성 순서	소멸 순서
기반 클래스	가장 먼저 생성	가장 나중에 소멸
멤버 객체	선언된 순서대로 생성	선언된 역순으로 소멸
객체 자신의 생성자·소멸자 본문	생성 과정의 마지막에 실행	소멸 과정의 처음에 실행

4.2.6 소멸자 작성 시 주의점

소멸자는 정상적인 객체 정리 과정에서 호출되므로 예외를 외부로 전달하지 않는 것이 원칙이다.
소멸자에서 처리할 작업은 실패 가능성이 낮고 확실하게 정리할 수 있어야 한다. 복잡한 저장 작업이나 오류 처리가 필요한 작업을 소멸자에 모두 넣으면 프로그램 종료 과정에서 문제를 처리하기 어려워진다.

소멸자는 일반적으로 직접 호출하지 않는다.

```
Rectangle rectangle;

// rectangle.~Rectangle();   // 직접 호출하지 않음
```

소멸자를 직접 호출하면 객체의 실제 수명이 끝나지 않았는데 소멸자만 실행될 수 있다. 이후 객체가 다시 소멸하면 같은 자원을 두 번 해제하는 문제가 발생할 수 있다.

4.3 초기화 방식

초기화는 객체가 처음 만들어질 때 값을 설정하는 과정이다. 이미 존재하는 객체의 값을 변경하는 대입과 구분된다.

```
int first = 10;   // 초기화

int second;
second = 10;      // 대입
```

클래스 객체도 사용한 문법에 따라 초기화 방식과 선택되는 생성자가 달라질 수 있다.

4.3.1 기본 초기화 (Default Initialization)

초기값 없이 객체를 선언하면 기본 초기화(default initialization)가 적용된다.

```
int count;
Rectangle rectangle;
```

클래스 타입에는 기본 생성자가 호출된다.

```
Rectangle rectangle;
```

기본형 지역 변수에는 정해지지 않은 값이 남을 수 있다.

```
int count;
double ratio;
```

count와 ratio를 값을 설정하지 않은 상태에서 읽으면 정의되지 않은 동작이 발생할 수 있다.

집합체의 기본형 멤버도 기본 초기화만으로는 0이 되지 않는다.

```
struct Point
{
    int x;
    int y;
};

Point point;
```

point.x와 point.y에는 정해지지 않은 값이 남을 수 있다.

4.3.2 값 초기화 (Value Initialization)

빈 괄호나 빈 중괄호를 사용하면 값 초기화(value initialization)가 적용될 수 있다.

```
int count{};       → count 0
double ratio{};    → ratio 0.0
Point point{};     → point.x, point.y 0
```

기본형은 0으로 초기화된다. 클래스 타입에 기본 생성자가 있으면 해당 생성자가 호출된다.

```
Rectangle rectangle{};
```

동적 객체에도 값 초기화를 적용할 수 있다.

```
int* value = new int();
Point* point = new Point();

delete value;
delete point;
```

new int()로 생성한 정수는 0으로 초기화된다. new int만 사용하면 초기값이 설정되지 않는다.

```
int* first = new int;     // 초기값 없음
int* second = new int();  // 0으로 초기화
```

```
delete first;
delete second;
```

4.3.3 직접 초기화 (Direct Initialization)

괄호나 중괄호를 사용하여 생성자에 인수를 직접 전달하면 직접 초기화(direct initialization)가 적용된다.

```
Rectangle first(10, 20, 100, 50);
Rectangle second{10, 20, 100, 50};
```

두 표현 모두 생성자를 직접 선택하여 호출한다. 중괄호 초기화는 축소 변환을 제한하고 std::initializer_list 생성자를 우선적으로 고려한다는 차이가 있다.
직접 초기화는 explicit 생성자도 호출할 수 있다.

```
class Length
{
public:
    explicit Length(int value) : value(value)
    {
    }
private:
    int value;
};

Length first(100);
Length second{200};
```

4.3.4 복사 초기화 (Copy Initialization)

등호를 사용하여 객체를 초기화하면 복사 초기화 가 적용된다.

```
int value = 10;
Rectangle first = Rectangle(10, 20, 100, 50);
```

초기화 문장에 등호가 있지만 이미 존재하는 객체에 값을 대입하는 것은 아니다. 새로운 객체를 생성하는 초기화 문법이다.
다른 객체로 초기화할 수도 있다.

```
Rectangle original(10, 20, 100, 50);
Rectangle copy = original;
```

이 경우 복사 생성자가 사용될 수 있다. 복사 생성자와 객체 복사의 세부 동작은 복사와 이동을 다루는 장에서 설명한다.
변환 생성자가 explicit이 아니면 다른 타입의 값도 복사 초기화에 사용할 수 있다.

```
class Length
{
public:
    Length(int value) : value(value)
    {
    }
private:
    int value;
};

Length length = 100;
```

생성자에 explicit을 지정하면 이 문법은 허용되지 않는다.

4.3.5 중괄호 초기화 (List Initialization)

중괄호를 사용한 초기화를 리스트 초기화 라고 한다.

```
int value{10};
Point point{10, 20};
Rectangle rectangle{10, 20, 100, 50};
```

중괄호 초기화는 기본형, 구조체, 배열과 클래스 객체에 공통된 형태로 사용할 수 있다.
중괄호 초기화는 축소 변환을 허용하지 않는다.

```
int first = 3.14;
int second(3.14);

// int third{3.14};   // 오류
```

first와 second에는 소수 부분이 제거된 값이 저장된다. third는 값이 손실될 수 있으므로 컴파일 오류가 발생한다.
변수의 값을 사용할 때도 컴파일러가 값 손실 가능성을 판단하면 오류가 발생할 수 있다.

```
double value = 3.14;

// int number{value};   // 오류
```

중괄호 초기화는 잘못된 형 변환을 컴파일 시점에 찾는 데 도움이 된다.
빈 중괄호는 기본값을 사용한 초기화에 적합하다.

```
int count{};
Point position{};
Rectangle rectangle{};
```

4.3.6 집합 초기화 (Aggregate Initialization)

집합체(aggregate)는 멤버를 선언된 순서대로 직접 초기화할 수 있는 타입이다.
생성자를 직접 선언하지 않고 공개 멤버만 가지는 단순한 구조체는 일반적으로 집합체로 사용할 수 있다.

```
struct Point
{
    int x, y;
};
Point first{10, 20};
Point second{30, 40};
```

배열도 집합 초기화를 사용할 수 있다.

```
int values[5]{1, 2, 3, 4, 5};
```

일부 값만 작성하면 나머지 요소는 0으로 초기화된다.

```
int values[5]{1, 2};
```

저장되는 값은 1, 2, 0, 0, 0이다.

집합체가 다른 집합체를 멤버로 가질 수도 있다.

```
struct Size
{
    int width;
    int height;
};

struct Area
{
    Point position;
    Size size;
};

Area area{{10, 20}, {100, 50}};
```

집합 초기화는 단순한 데이터 타입에 적합하다. 객체의 상태 규칙을 검사하거나 생성 방법을 제한해야 한다면 생성자를 가진 class로 설계하는 것이 좋다.

객체를 선언할 때 사용한 문법에 따라 적용되는 초기화 방식과 생성자 선택 규칙이 달라진다.

<객체의 초기화 방식>

초기화 방식	문법	주요 동작
기본 초기화	T object;	클래스의 기본 생성자를 호출, 기본형 지역 변수에는 정해지지 않은 값이 남을 수 있음
값 초기화	T object{};	기본형과 집합체의 멤버를 0에 해당하는 값으로 초기화, 또는 클래스의 기본 생성자 호출
직접 초기화	T object(value);	인수에 맞는 생성자를 직접 선택, explicit 생성자도 호출 가능
복사 초기화	T object = value;	암시적 변환을 허용, explicit 생성자는 사용할 수 없음
직접 리스트 초기화	T object{value};	축소 변환 제한, 생성자 또는 집합 초기화 적용
복사 리스트 초기화	T object = {value};	축소 변환 제한, explicit 생성자 사용에 제약
집합 초기화	T object{...};	집합체 멤버를 선언된 순서대로 초기화

4.4 멤버 초기화 리스트 (Member Initializer List)

멤버 초기화 리스트(member initializer list)는 생성자 본문이 실행되기 전에 멤버를 직접 초기화한다.
생성자의 매개변수 목록 뒤에 콜론을 작성하고 초기화할 멤버를 나열한다.

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
    }

private:
    Point position;
    int width;
    int height;
};
```

객체를 생성하면서 위치와 크기를 전달한다.

```
Rectangle rectangle({10, 20}, 100, 50);
```

position, width, height는 생성자 본문이 실행되기 전에 초기화된다.

4.4.1 대입과 초기화의 차이

생성자 본문에서 멤버에 값을 저장하면 초기화가 아니라 대입이다.

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
    {
        this->position = position;
        this->width = width;
        this->height = height;
    }

private:
    Point position;
    int width;
    int height;
};
```

생성자 본문이 실행되기 전에 멤버들은 이미 초기화 과정을 거친다. 생성자 본문의 대입문은 초기화된 멤버의 값을 다시 변경한다.
멤버 초기화 리스트를 사용하면 전달받은 값으로 멤버를 직접 초기화한다.

```
Rectangle(Point position, int width, int height)
    : position(position), width(width), height(height)
{
}
```

기본형 멤버에서는 성능 차이가 거의 없을 수 있지만, 클래스 타입 멤버에서는 불필요한 기본 생성과 대입을 피할 수 있다.

```
#include <string>

class Label
{
public:
    Label(const std::string& text)
    {
        this->text = text;
    }
private:
    std::string text;
};
```

이 생성자에서는 **text** 멤버의 기본 생성자가 먼저 호출된 뒤 전달받은 문자열이 대입된다.
멤버 초기화 리스트를 사용하면 문자열을 전달받은 값으로 직접 초기화한다.

```
class Label
{
public:
    Label(const std::string& text) : text(text)
    {
    }
private:
    std::string text;
};
```

생성자 본문이 실행될 때는 모든 멤버의 초기화가 이미 끝난 상태이다. 생성자 본문에서 멤버에 값을 저장하면 초기화된 멤버에 새로운 값을 대입하며, 멤버 초기화 리스트를 사용하면 전달받은 값으로 멤버를 직접 초기화한다.

두 방식은 작성 위치만 다른 것이 아니라 실행 시점과 처리 과정이 다르므로, 생성자에서는 특별한 이유가 없다면 멤버 초기화 리스트를 사용하는 것이 좋다.

<멤버 초기화와 대입의 차이>

구분	멤버 초기화 리스트	생성자 본문의 대입
실행 시점	생성자 본문 실행 전	생성자 본문 실행 중
처리 대상	아직 초기값이 결정되지 않은 멤버	이미 초기화된 멤버
클래스 타입 멤버	지정한 생성자를 직접 호출	기본 생성 후 대입 연산 수행
const 멤버	사용 가능	사용 불가
참조 멤버	사용 가능	사용 불가
기본 생성자가 없는 객체 멤버	사용 가능	사용 불가

4.4.2 멤버 초기화 순서

멤버는 멤버 초기화 리스트에 작성한 순서가 아니라 클래스에 선언된 순서대로 초기화된다.

```
class Rectangle
{
public:
    Rectangle(int width, int height)
        : height(height), width(width)
    {
    }
private:
    int width;
```

```
    int height;
};
```

초기화 리스트에는 height가 먼저 작성되어 있지만 실제로는 width가 먼저 초기화된다. 멤버 선언 순서가 width, height이기 때문이다.
초기화 리스트도 멤버 선언 순서와 같게 작성하면 실제 초기화 순서를 쉽게 파악할 수 있다.

```
Rectangle(int width, int height)
    : width(width), height(height)
{
}
```

한 멤버의 초기값이 앞서 초기화되는 다른 멤버에 의존한다면 선언 순서가 특히 중요하다.

```
class Size
{
public:
    Size(int width) : width(width), area(width * width)
    {
    }
private:
    int width;
    int area;
};
```

width가 먼저 선언되어 있으므로 area를 초기화할 때 사용할 수 있다.
멤버 선언 순서가 반대라면 초기화 리스트의 순서를 바꾸어도 area가 먼저 초기화된다.

```
class Size
{
public:
    Size(int width) : width(width), area(width * width)
    {
    }
private:
    int area;
    int width;
};
```

초기화 리스트의 순서만 보고 판단하면 오류를 찾기 어렵다. 멤버 선언 순서와 초기화 리스트의 순서를 일치시키는 것이 좋다.

4.4.3 객체 타입 멤버 초기화

클래스가 객체 타입 멤버를 가지면 그 멤버의 생성자가 먼저 호출된다.

```
struct Point
{
    Point(int x, int y) : x(x), y(y)
    {
    }
    int x, y;
};
```

Point는 인수 없는 기본 생성자를 제공하지 않는다.

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
        : position(x, y), width(width), height(height)
    {
    }
private:
    Point position;
    int width, height;
};
```

position은 Point(x, y) 생성자를 사용하여 초기화된다.
생성자 본문에서 position을 대입하는 방식은 사용할 수 없다.

```
class Rectangle
{
public:
    Rectangle(int x, int y, int width, int height)
    {
        // position은 생성자 본문 이전에 초기화되어야 함
    }
private:
    Point position;
    int width, height;
};
```

Point에는 기본 생성자가 없으므로 Rectangle의 생성자 본문이 실행되기 전에 position을 초기화할 방법이 없다. 멤버 초기화 리스트에서 Point 생성자를 호출해야 한다.

4.4.4 상수 멤버 초기화

const 멤버는 객체가 생성될 때 반드시 초기화해야 한다. 생성자 본문에서 값을 대입할 수 없다.

```
class ShapeId
{
public:
    ShapeId(int id) : id(id)
    {
    }
private:
    const int id;
};
```

생성자 본문에서 값을 대입하면 오류가 발생한다.

```
class ShapeId
{
public:
    ShapeId(int id)
    {
        // this->id = id;   // 오류
    }
private:
    const int id;
};
```

const 멤버는 생성자 본문이 실행되기 전에 초기화되어야 한다.

4.4.5 참조 멤버 초기화

참조 멤버도 선언과 동시에 대상을 결정해야 하므로 멤버 초기화 리스트에서 초기화해야 한다.

```
class PointView
{
public:
    PointView(Point& point) : point(point)
    {
    }
    void Move(int x, int y)
    {
        point.x = x;
        point.y = y;
    }
private:
    Point& point;
};
```

```
Point position{10, 20};
PointView view(position);

view.Move(30, 40);
```

view의 point는 position을 참조한다.

참조 멤버는 다른 객체를 소유하지 않는다. 참조하는 객체가 먼저 소멸하면 참조 멤버는 유효하지 않은 대상을 가리키게 된다. 참조 관계를 사용할 때는 두 객체의 수명을 함께 고려해야 한다.

4.4.6 기반 클래스 초기화

파생 클래스 객체가 생성되면 기반 클래스 부분이 먼저 초기화된다. 파생 클래스 생성자는 멤버 초기화 리스트에서 기반 클래스 생성자를 호출할 수 있다.

```
class Shape
{
public:
    Shape(Point position) : position(position)
    {
    }
protected:
    Point position;
};

class Rectangle : public Shape
{
public:
    Rectangle(Point position, int width, int height)
        : Shape(position), width(width), height(height)
    {
    }

private:
    int width, height;
};
```

Rectangle 객체를 생성하면 Shape(position)이 먼저 호출되고 width, height가 초기화된 뒤 Rectangle 생성자 본문이 실행된다.

기반 클래스 초기화와 상속 구조는 상속을 다루는 장에서 자세히 설명한다.

4.5 생성자 기능의 확장

C++11 이후에는 기본 멤버 초기값, 위임 생성자, explicit, = default, = delete, std::initializer_list를 사용하여 객체 생성 방법을 더 분명하게 표현할 수 있다.

4.5.1 기본 멤버 초기값 (Default Member Initializer)

멤버를 선언하면서 기본값을 지정할 수 있다.

```
class Rectangle
{
private:
    Point position{0, 0};
    int width = 1;
    int height = 1;
};
```

생성자가 해당 멤버를 별도로 초기화하지 않으면 기본 멤버 초기값이 사용된다.

```
class Rectangle
{
public:
    Rectangle() = default;
```

```
    Rectangle(int width, int height)
        : width(width), height(height)
    {
    }
private:
    Point position{0, 0};
    int width = 1, height = 1;
};
```

기본 생성자는 모든 기본 멤버 초기값을 사용한다.

```
Rectangle first;
```

크기만 받는 생성자는 width와 height를 전달받은 값으로 초기화하고, position에는 {0, 0}이 사용된다.

```
Rectangle second(100, 50);
```

생성자의 멤버 초기화 리스트에서 값을 지정하면 그 값이 기본 멤버 초기값보다 우선한다.

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
    }

private:
    Point position{0, 0};
    int width = 1, height = 1;
};
```

기본 멤버 초기값은 여러 생성자에서 공통으로 사용하는 초기값을 한곳에 표현할 수 있다.

4.5.2 위임 생성자 (Delegating Constructor)

한 생성자가 같은 클래스의 다른 생성자를 호출하도록 만들 수 있다. 이를 위임 생성자(delegating constructor)라고 한다.

```
class Rectangle
{
public:
    Rectangle()
        : Rectangle({0, 0}, 1, 1)
    {
    }
    Rectangle(int width, int height)
        : Rectangle({0, 0}, width, height)
    {
    }
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
        if (this->width <= 0)
        {
            this->width = 1;
        }
        if (this->height <= 0)
        {
            this->height = 1;
        }
    }
private:
    Point position;
    int width, height;
};
```

기본 생성자와 크기만 받는 생성자는 위치와 크기를 모두 받는 생성자에 초기화 작업을 위임한다.
유효성 검사와 공통 초기화 코드는 하나의 생성자에 모을 수 있다. 위임 생성자는 같은 클래스의 생성자 하나만 호출해야 한다.

```
Rectangle()
    : Rectangle({0, 0}, 1, 1)
{
}
```

위임 생성자에서는 다른 생성자 호출과 멤버 초기화를 함께 작성할 수 없다.

```
// Rectangle()
//     : Rectangle({0, 0}, 1, 1), width(10)   // 오류
// {
// }
```

위임받은 생성자가 모든 멤버의 초기화를 담당한다.

4.5.3 explicit

하나의 인수로 호출할 수 있는 생성자는 암시적 타입 변환에 사용될 수 있다.

```
class Length
{
public:
    Length(int value) : value(value)
    {
    }
private:
    int value;
};

Length length = 100;
```

의도하지 않은 암시적 변환을 막으려면 생성자에 explicit을 지정한다.

```
class Length
{
public:
    explicit Length(int value) : value(value)
    {
    }
private:
    int value;
};

Length first(100);
Length second{200};

// Length third = 300;   // 오류
```

explicit 생성자는 직접 초기화에는 사용할 수 있지만 암시적 변환에는 사용할 수 없다.
여러 개의 매개변수를 가진 생성자도 기본 인수 때문에 하나의 인수로 호출할 수 있다면 암시적 변환 생성자가 될 수 있다.

```
class Length
{
public:
    explicit Length(int value, int scale = 1)
        : value(value * scale)
    {
    }
private:
    int value;
};
```

클래스 타입으로의 암시적 변환이 명확한 의미를 가지지 않는다면 단일 인수 생성자에는 explicit을 지정하는 것이 좋다.

4.5.4 = default

컴파일러는 클래스의 구성에 따라 기본 생성자, 소멸자와 복사·이동에 관련된 특별 멤버 함수를 암시적으로 선언할 수 있다.

= default는 컴파일러가 제공하는 기본 동작을 명시적으로 사용하겠다는 뜻이다.

```
class Rectangle
{
public:
    Rectangle() = default;
    ~Rectangle() = default;

private:
    Point position{0, 0};
    int width = 1;
    int height = 1;
};
```

Rectangle() = default;는 컴파일러가 기본 생성자의 동작을 작성하도록 요청한다.
~Rectangle() = default;는 컴파일러가 소멸자의 기본 동작을 작성하도록 요청한다.

= default는 함수가 아무 작업도 하지 않는다는 뜻이 아니다. 컴파일러가 각 멤버의 생성과 소멸 규칙에 맞는 기본 동작을 생성한다.

```
#include <string>

class Label
{
public:
    Label() = default;
    ~Label() = default;

private:
    std::string text;
};
```

컴파일러가 생성한 소멸자는 **text** 멤버의 소멸자를 호출한다.
사용자가 다른 생성자를 선언한 뒤에도 기본 생성자를 명시적으로 제공할 수 있다.

```
class Point
{
public:
    Point() = default;

    Point(int x, int y) : x(x), y(y) { }
private:
    int x = 0, y = 0;
};

Point first;
Point second(10, 20);
```

= default로 선언한 함수가 항상 사용할 수 있는 것은 아니다. 멤버나 기반 클래스가 필요한 동작을 제공하지 않으면 컴파일러가 해당 함수를 삭제된 함수로 정의할 수 있다.

```
class Point
{
public:
    Point(int x, int y) : x(x), y(y) { }
private:
    int x, y;
};

class Rectangle
{
public:
    Rectangle() = default;
private:
    Point position;
```

```
    int width = 1, height = 1;
};
```

Point에는 기본 생성자가 없다. 따라서 컴파일러는 Rectangle의 기본 생성자를 정상적으로 만들 수 없다.

```
// Rectangle rectangle;   // 오류
```

= default는 기본 생성자를 강제로 만들어 내는 문법이 아니다. 클래스의 멤버와 기반 클래스가 해당 기본 동작을 지원할 때 컴파일러가 그 동작을 작성하도록 요청한다.

4.5.5 = delete

= delete는 특정 함수를 사용할 수 없도록 명시한다.

위치와 크기 없이 생성되는 사각형을 허용하지 않으려면 기본 생성자를 삭제할 수 있다.

```
class Rectangle
{
public:
    Rectangle() = delete;

    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
    }

private:
    Point position;
    int width, height;
};

// Rectangle first;   // 오류

Rectangle second({10, 20}, 100, 50);
```

특정 타입의 인수를 받는 생성자를 삭제할 수도 있다.

```
class Length
{
public:
    explicit Length(int value) : value(value) { }

    Length(double value) = delete;

private:
    int value;
};

Length first(100);

// Length second(3.14);   // 오류
```

복사를 허용하지 않는 클래스에서는 복사 생성자와 복사 대입 연산자를 삭제할 수 있다.

```
class Resource
{
public:
    Resource() = default;

    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;
};
```

= default와 = delete는 특별 멤버 함수를 직접 구현하지 않고 그 동작을 명시할 때 사용한다. = default는 컴파일러가 기본 동작을 정의하도록 요청하며, 멤버나 기반 클래스 때문에 해당 동작을 만들 수 없으면 함수가 삭제되고, = delete는 특정 생성·복사·이동·대입을 의도적으로 금지한다.

<"= default" 와 "= delete">

문법	의미	사용 목적
Function() = default;	컴파일러가 기본 동작을 정의하도록 요청	기본 생성, 복사, 이동과 소멸 동작을 명시적으로 사용
Function() = delete;	해당 함수의 호출을 금지	특정 생성, 변환, 복사 또는 대입을 허용하지 않음

4.5.6 std::initializer_list

std::initializer_list를 매개변수로 받는 생성자는 중괄호 안의 여러 값을 전달받을 수 있다.

```
#include <initializer_list>

class Matrix2x2
{
public:
    Matrix2x2(std::initializer_list<int> values)
    {
        int index = 0;
        for (int value : values)
        {
            if (index >= 4)
            {
                break;
            }
            elements[index] = value;
            ++index;
        }

        while (index < 4)
        {
            elements[index] = 0;
            ++index;
        }
    }

private:
    int elements[4];
};
```

중괄호 안의 값이 std::initializer_list<int>로 전달된다.

```
Matrix2x2 first{1, 2, 3, 4};
Matrix2x2 second{1, 2};
```

second의 나머지 요소는 0으로 설정된다.

std::initializer_list 생성자가 다른 생성자와 함께 존재하면 중괄호 초기화에서 우선적으로 선택되는 경우가 많다.

```
class NumberSet
{
public:
    NumberSet(int count, int value)
    {
    }

    NumberSet(std::initializer_list<int> values)
    {
    }
};

NumberSet first(10, 20);
NumberSet second{10, 20};
```

- first는 NumberSet(int, int)를 호출한다.
- second는 NumberSet(std::initializer_list<int>)를 호출한다.

괄호와 중괄호가 서로 다른 생성자를 선택할 수 있으므로 std::initializer_list 생성자를 추가할 때는 기존 생성자와의 관계를 확인해야 한다.

4.6 const 정확성과 클래스 인터페이스(Const Correctness)

const 정확성은 객체를 변경하는 기능과 변경하지 않는 기능을 타입과 인터페이스에 반영하는 것이다. 객체의 상태를 조회하는 함수는 상수 멤버 함수로 선언하고, 읽기만 하는 객체는 상수 참조로 전달하며, 객체 내부를 반환할 때도 호출 객체의 상수성을 유지해야 한다.

클래스의 공개 인터페이스에 상수성을 반영하면 함수 선언만으로 객체의 상태를 변경하는 함수인지 구분할 수 있다. 컴파일러도 상수 객체를 변경하려는 코드를 검사하므로 잘못된 상태 변경을 줄일 수 있다.

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
    }

    void MoveTo(Point position);
    void Resize(int width, int height);

    int Area() const;
    const Point& Position() const;

private:
    Point position;
    int width, height;
};
```

MoveTo()와 Resize()는 객체의 상태를 변경하므로 일반 멤버 함수로 선언한다. Area()와 Position()은 객체의 상태를 조회하므로 함수 선언 뒤에 const를 지정한다.

4.6.1 상수 멤버 함수와 this(Const Member Function and this)

객체의 상태를 변경하지 않는 멤버 함수는 상수 멤버 함수로 선언할 수 있다. 상수 멤버 함수는 매개변수 목록 뒤에 const를 붙인다.

```
class Rectangle
{
public:
    int Area() const
    {
        return width * height;
    }

private:
    Point position;
    int width, height;
};
```

Area() 뒤의 const는 반환값이 아니라 멤버 함수를 호출한 객체에 적용된다.

```
int Area() const;
```

상수 멤버 함수 안에서는 일반 데이터 멤버를 변경할 수 없다.

```
int Area() const
{
    // width = 100;    // 오류
    return width * height;
}
```

상태를 변경하는 일반 멤버 함수도 호출할 수 없다.

```
int Area() const
```

```
{
    // Resize(100, 50);    // 오류
    return width * height;
}
```

멤버 함수가 호출되면 호출 객체의 주소가 this를 통해 전달된다. 일반 멤버 함수에서는 this가 변경 가능한 객체를 가리키고, 상수 멤버 함수에서는 변경할 수 없는 객체를 가리키는 것으로 취급된다.

<멤버 함수와 this가 가리키는 객체>

멤버 함수	this가 가리키는 객체	일반 멤버 변경
일반 멤버 함수	변경 가능한 객체	가능
상수 멤버 함수	변경할 수 없는 객체	불가능

상수 멤버 함수에서 this를 사용해도 같은 제한이 적용된다.

```
int Area() const
{
    // this->width = 100;    // 오류
    return this->width * this->height;
}
```

함수 선언에서 const가 놓이는 위치에 따라 적용 대상이 달라진다.

```
const Point& Position() const;
```

<멤버 함수 선언에 사용한 const>

위치	적용 대상	의미
반환 타입의 const	반환되는 Point 객체	반환된 객체를 통해 상태를 변경할 수 없음
매개변수 목록 뒤의 const	멤버 함수를 호출한 객체	함수가 호출 객체의 일반 멤버를 변경하지 않음

상수 멤버 함수는 호출 객체의 논리적 상태를 변경하지 않는다는 의미를 인터페이스에 표현한다. 다만 객체가 포인터나 참조로 외부 객체를 가리키고 있다면, 상수 멤버 함수라고 해서 그 외부 객체까지 자동으로 상수가 되는 것은 아니다.

```
class TargetView
{
public:
    void ResetTarget() const
    {
        if (target != nullptr)
        {
            target->Reset();
        }
    }
private:
    Target* target = nullptr;
};
```

ResetTarget()은 TargetView의 포인터 멤버 자체를 변경하지 않지만, 포인터가 가리키는 외부 Target 객체는 변경할 수 있다. 상수 멤버 함수가 어떤 상태를 변경하지 않아야 하는지는 클래스의 의미와 인터페이스 설계에 따라 판단해야 한다.

4.6.2 상수 객체(Const Object)

객체를 const로 선언하면 해당 객체를 통해 상태를 변경할 수 없다.

```
const Rectangle rectangle({10, 20}, 100, 50);
```

상수 객체에서는 상수 멤버 함수만 호출할 수 있다.

```
const Rectangle rectangle({10, 20}, 100, 50);
int area = rectangle.Area();
```

상태를 변경하는 일반 멤버 함수는 호출할 수 없다.

```
const Rectangle rectangle({10, 20}, 100, 50);
// rectangle.MoveTo({30, 40});    // 오류
// rectangle.Resize(200, 100);    // 오류
```

일반 객체는 일반 멤버 함수와 상수 멤버 함수를 모두 호출할 수 있다.

```
Rectangle rectangle({10, 20}, 100, 50);
rectangle.MoveTo({30, 40});
int area = rectangle.Area();
```

<호출 객체와 멤버 함수>

호출 객체	일반 멤버 함수	상수 멤버 함수
일반 객체	호출 가능	호출 가능
상수 객체	호출 불가능	호출 가능

상수 객체도 생성 과정에서는 생성자를 통해 초기화된다.

```
const Rectangle rectangle({10, 20}, 100, 50);
```

생성자가 실행되는 동안 객체의 초기 상태를 구성할 수 있다. 생성이 끝난 뒤에는 rectangle을 통해 상태를 변경할 수 없다.
const는 객체가 처음부터 값을 가지지 못하게 하는 것이 아니라, 초기화가 끝난 객체를 해당 이름이나 참조를 통해 변경하지 못하도록 제한한다.

4.6.3 const 오버로딩(Const Overloading)

멤버 함수는 매개변수 목록 뒤의 const 여부에 따라 오버로딩할 수 있다.

```
class Rectangle
{
public:
    Point& Position()
    {
        return position;
    }

    const Point& Position() const
    {
        return position;
    }

private:
    Point position;
    int width, height;
};
```

두 함수는 이름과 매개변수가 같지만 함수 뒤의 const가 다르다.

```
Point& Position();
const Point& Position() const;
```

일반 객체에서는 일반 멤버 함수가 선택된다.

```
Rectangle rectangle({10, 20}, 100, 50);
rectangle.Position().x = 30;
rectangle.Position().y = 40;
```

일반 버전은 Point&를 반환하므로 위치를 변경할 수 있다. 상수 객체에서는 상수 멤버 함수가 선택된다.

```
const Rectangle rectangle({10, 20}, 100, 50);
int x = rectangle.Position().x;
int y = rectangle.Position().y;
// rectangle.Position().x = 30;    // 오류
```

상수 버전은 const Point&를 반환하므로 위치를 읽을 수 있지만 변경할 수 없다.

<일반 객체와 상수 객체에서 선택되는 함수>

호출 객체	선택되는 함수	반환 타입
Rectangle	일반 멤버 함수	Point&
const Rectangle	상수 멤버 함수	const Point&

반환 타입만 다른 함수는 오버로딩할 수 없다.

```
// Point& Position();
// const Point& Position();    // 오류
```

그러나 멤버 함수 뒤의 const는 호출 객체에 대한 제한을 나타내며 함수 타입을 구분하는 요소이므로 오버로딩할 수 있다.

```
Point& Position();
const Point& Position() const;
```

이러한 오버로딩은 컨테이너나 문자열처럼 일반 객체에서는 요소 변경을 허용하고, 상수 객체에서는 읽기만 허용해야 하는 타입에서 활용한다.

```
class IntArray
{
public:
    int& operator[](std::size_t index);
    const int& operator[](std::size_t index) const;
};
```

4.6.4 상수 참조 매개변수와 반환값 (Const Reference Parameters and Return Values)

객체를 복사하지 않고 전달하면서 함수 안에서 변경하지 않도록 제한하려면 상수 참조를 사용한다.

```
void PrintArea(const Rectangle& rectangle)
{
    std::cout << rectangle.Area() << '\n';
}
```

const Rectangle&는 전달받은 객체를 복사하지 않고 읽기 전용으로 참조한다.

```
Rectangle rectangle({10, 20}, 100, 50);
PrintArea(rectangle);
```

상수 객체도 전달할 수 있다.

```
const Rectangle rectangle({10, 20}, 100, 50);
PrintArea(rectangle);
```

상수 참조를 통해서는 상수 멤버 함수만 호출할 수 있다.

```
void PrintArea(const Rectangle& rectangle)
{
    int area = rectangle.Area();

    // rectangle.MoveTo({0, 0});    // 오류
```

```
}
```

크기가 큰 객체나 복사를 피해야 하는 객체를 읽기 전용으로 전달할 때 const T&를 활용한다. 작은 기본형은 값으로 전달하는 편이 자연스러울 수 있으므로 모든 매개변수를 상수 참조로 선언할 필요는 없다.

```
void Resize(int width, int height);
void PrintArea(const Rectangle& rectangle);
```

함수가 객체 내부의 상태를 참조로 반환할 때도 상수성을 유지해야 한다.

```
const Point& Position() const
{
    return position;
}
```

상수 멤버 함수가 일반 참조를 반환하면 상수 객체의 내부 상태를 외부에서 변경할 수 있게 된다.

```
class Rectangle
{
public:
    // 잘못된 인터페이스
    // Point& Position() const
    // {
    //     return position;
    // }

private:
    Point position;
};
```

일반 객체와 상수 객체에서 접근 권한을 구분하려면 두 함수를 함께 제공한다.

```
class Rectangle
{
public:
    Point& Position()
    {
        return position;
    }

    const Point& Position() const
    {
        return position;
    }

private:
    Point position;
    int width, height;
};
```

참조 반환은 객체 내부를 복사하지 않고 접근할 수 있지만, 반환된 참조는 원래 객체의 수명에 의존한다.

```
const Point& GetPosition(const Rectangle& rectangle)
{
    return rectangle.Position();
}
```

rectangle이 소멸하면 반환된 참조도 더 이상 사용할 수 없다. 내부 객체의 참조를 반환하는 인터페이스는 복사 비용뿐만 아니라 객체의 수명과 외부 변경 가능성도 함께 고려해야 한다.

<값 반환과 참조 반환>

반환 방식	특징	주의점
Point	독립된 값을 반환	객체가 복사되거나 이동될 수 있음

Point&	내부 객체를 수정 가능하게 참조	외부에서 객체 상태를 직접 변경할 수 있음
const Point&	내부 객체를 읽기 전용으로 참조	원본 객체의 수명에 의존

작고 복사 비용이 낮은 타입은 값으로 반환하는 것이 단순할 수 있다. 내부 객체를 직접 수정하도록 허용해야 하는지, 읽기만 허용해야 하는지에 따라 반환 방식을 결정한다.

4.6.5 논리적 상수성과 mutable(Logical Constness and mutable)

상수 멤버 함수는 객체의 논리적인 상태를 변경하지 않아야 한다. 논리적 상태는 객체가 외부에 나타내는 의미와 동작을 결정하는 상태이다. 사각형의 위치와 크기는 객체의 기하학적 상태에 포함된다.

```
class Rectangle
{
private:
    Point position;
    int width, height;
};
```

Area()가 호출 횟수를 기록하거나 계산 결과를 캐시에 저장하더라도 사각형의 위치와 크기는 바뀌지 않는다. 이러한 내부 정보는 객체의 논리적인 상태와 구분할 수 있다.
상수 멤버 함수에서도 변경해야 하는 내부 구현 멤버에는 mutable을 지정할 수 있다.

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height){}
    int Area() const
    {
        ++areaQueryCount;
        return width * height;
    }
    int AreaQueryCount() const
    {
        return areaQueryCount;
    }
private:
    Point position;
    int width, height;
    mutable int areaQueryCount = 0;
};
```

상수 객체에서도 Area()를 호출할 수 있다.

```
const Rectangle rectangle({10, 20}, 100, 50);
int first = rectangle.Area();
int second = rectangle.Area();
std::cout << rectangle.AreaQueryCount() << '\n';
```

Area()는 위치와 크기를 변경하지 않지만 조회 횟수는 증가시킨다. areaQueryCount는 사각형의 기하학적 의미에 영향을 주지 않는 관리 정보이므로 mutable로 선언할 수 있다.

계산 비용이 큰 값을 캐시에 저장할 때도 mutable을 활용할 수 있다.

```
class Rectangle
{
public:
    int Area() const
    {
        if (!areaCached)
        {
            cachedArea = width * height;
            areaCached = true;
        }
```

```
        return cachedArea;
    }

private:
    Point position;
    int width, height;

    mutable bool areaCached = false;
    mutable int cachedArea = 0;
};
```

크기가 변경되면 캐시를 무효화해야 한다.

```
void Resize(int width, int height)
{
    if (width > 0)
    {
        this->width = width;
    }
    if (height > 0)
    {
        this->height = height;
    }

    areaCached = false;
}
```

mutable은 상수성을 무시하기 위한 일반적인 수단이 아니다. 객체의 논리적 상태를 변경하지 않는 내부 정보에 제한적으로 사용해야 한다.

<물리적 상수성과 논리적 상수성>

구분	의미
물리적 상수성	객체의 저장된 비트와 데이터 멤버를 변경하지 않음
논리적 상수성	객체가 외부에 나타내는 의미와 관찰 가능한 상태를 변경하지 않음

상수 멤버 함수는 기본적으로 일반 멤버의 물리적 변경을 막는다. mutable을 사용하면 논리적 상태를 유지하면서 필요한 내부 구현 정보를 변경할 수 있다.

4.6.6 상수성을 반영한 인터페이스

클래스의 멤버 함수는 객체의 상태 변경 여부를 기준으로 구분한다.

- 상태를 변경하는 함수는 일반 멤버 함수로 선언
- 상태를 조회하는 함수는 상수 멤버 함수로 선언
- 읽기 전용 객체 매개변수는 상수 참조로 전달
- 내부 상태를 반환할 때 호출 객체의 상수성을 유지
- 논리적 상태와 구현을 위한 내부 정보를 구분
- mutable은 논리적 상태를 변경하지 않는 정보에만 사용

Rectangle의 인터페이스에 상수성을 반영한다.

```
class Rectangle
{
public:
    Rectangle(Point position, int width, int height)
        : position(position), width(width), height(height)
    {
    }

    void MoveTo(Point position)
    {
        this->position = position;
    }

    void Resize(int width, int height)
    {
```

```
        if (width > 0)
        {
            this->width = width;
        }

        if (height > 0)
        {
            this->height = height;
        }
    }

    int Area() const
    {
        ++areaQueryCount;
        return width * height;
    }

    Point& Position()
    {
        return position;
    }
    const Point& Position() const
    {
        return position;
    }

    int AreaQueryCount() const
    {
        return areaQueryCount;
    }

private:
    Point position;
    int width, height;
    mutable int areaQueryCount = 0;
};
```

일반 객체에서는 상태 변경과 조회가 모두 가능하다.

```
Rectangle rectangle({10, 20}, 100, 50);
rectangle.MoveTo({30, 40});
rectangle.Resize(200, 100);
rectangle.Position().x = 50;
std::cout << rectangle.Area() << '\n';
```

상수 객체에서는 조회 기능만 사용할 수 있다.

```
const Rectangle rectangle({10, 20}, 100, 50);
std::cout << rectangle.Area() << '\n';
std::cout << rectangle.Position().x << '\n';
// rectangle.MoveTo({30, 40});     // 오류
// rectangle.Resize(200, 100);     // 오류
// rectangle.Position().x = 50;    // 오류
```

상수성을 인터페이스에 반영하면 함수 선언만으로 상태 변경 가능성을 파악할 수 있다.

<클래스 인터페이스의 const 선언>

선언	의미
const Rectangle rectangle;	객체를 통해 상태를 변경할 수 없음
int Area() const;	호출 객체의 논리적 상태를 변경하지 않는 멤버 함수
const Rectangle& rectangle	객체를 복사하지 않고 읽기 전용으로 참조
const Point& Position() const;	상수 멤버 함수가 내부 객체를 읽기 전용 참조로 반환

mutable int count;	논리적 상태에 영향을 주지 않는 멤버를 상수 멤버 함수에서도 변경 가능

상수 멤버 함수는 단순히 컴파일 오류를 피하기 위해 붙이는 문법이 아니다. 객체를 변경하는 연산과 관찰하는 연산을 인터페이스에서 구분하고, 상수 객체와 상수 참조를 통해 사용할 수 있는 기능의 범위를 명확하게 표현한다.

객체의 논리적인 상태를 변경하지 않는 멤버 함수는 상수 멤버 함수로 선언하고, 외부에서 읽기만 하는 객체는 상수 참조로 전달한다. 내부 상태를 참조로 반환할 때도 호출 객체의 상수성을 유지하도록 일반 버전과 상수 버전을 구분한다. 이러한 규칙을 일관되게 적용하면 클래스의 의도가 명확해지고 잘못된 상태 변경을 줄일 수 있다.

4.7 constexpr와 컴파일 타임 초기화

프로그램의 일부 값은 컴파일 시점에 결정할 수 있다. 컴파일러가 값을 미리 계산하면 해당 값을 배열 크기, 템플릿 인수와 정적 검사에 사용할 수 있다.
constexpr는 변수나 함수가 상수 표현식에 참여할 수 있음을 나타낸다.

4.7.1 상수 표현식 (Constant Expression)

상수 표현식(constant expression)은 컴파일 시점에 값을 계산할 수 있는 표현식이다.

```
constexpr int width = 100;
constexpr int height = 50;
constexpr int area = width * height;
```

area는 컴파일 시점에 5000으로 계산될 수 있다.

일반 변수의 값은 실행 시점에 변경될 수 있으므로 상수 표현식으로 사용할 수 없다.

```
int width = 100;

// constexpr int area = width * 50;   // 오류
```

const 객체라고 해서 항상 컴파일 타임 상수인 것은 아니다.

```
int ReadWidth();

const int width = ReadWidth();
```

width는 변경할 수 없지만 값은 실행 시점에 결정된다.

```
constexpr int width = 100;
```

constexpr 변수는 상수 표현식으로 초기화되어야 한다.

4.7.2 constexpr 변수

constexpr 변수는 선언과 동시에 초기화해야 한다.

```
constexpr int width = 100;
constexpr int height = 50;
```

초기값은 컴파일 시점에 계산할 수 있어야 한다.

```
int GetRuntimeValue();

// constexpr int value = GetRuntimeValue();   // 오류
```

constexpr 변수는 const의 성질도 가지므로 값을 변경할 수 없다.

```
constexpr int width = 100;

// width = 200;   // 오류
```

4.7.3 constexpr 함수

constexpr 함수는 상수 표현식 안에서 호출될 수 있는 함수이다.

```
constexpr int CalculateArea(int width, int height)
{
    return width * height;
}
```

상수 인수를 전달하면 컴파일 시점에 계산할 수 있다.

```
constexpr int area = CalculateArea(100, 50);
```

constexpr 함수는 반드시 컴파일 시점에만 실행되는 함수가 아니다. 실행 시점에 결정된 값을 전달하면 일반 함수처럼 실행될 수 있다.

```
#include <iostream>

int main()
{
    int width, height;
    std::cin >> width >> height;

    int area = CalculateArea(width, height);
    std::cout << area << '\n';
}
```

함수가 constexpr로 선언되었다고 해서 모든 호출이 컴파일 시점에 평가되는 것은 아니다. 호출 위치와 인수가 상수 표현식의 조건을 만족하면 컴파일 시점에 평가될 수 있다.

4.7.4 constexpr 생성자

클래스 객체를 컴파일 시점에 생성하려면 생성자와 필요한 멤버 함수가 constexpr 조건을 만족해야 한다.

```
struct Point
{
    int x, y;
    constexpr Point(int x, int y) : x(x), y(y) { }
};

class Rectangle
{
public:
    constexpr Rectangle(Point pos, int width, int height)
        : position(pos), width(width), height(height)
    {
    }
    constexpr int Area() const
    {
        return width * height;
    }
    constexpr Point Position() const
    {
        return position;
    }

private:
    Point position;
    int width, height;
};
```

Rectangle 객체를 컴파일 시점에 생성할 수 있다.

```
constexpr Rectangle rectangle({10, 20}, 100, 50);
constexpr int area = rectangle.Area();
```

컴파일 시점에 조건을 검사할 수도 있다.

```
static_assert(rectangle.Area() == 5000);
```

같은 생성자를 실행 시점에도 사용할 수 있다.

```
int width, height;
std::cin >> width >> height;

Rectangle rectangle({0, 0}, width, height);
std::cout << rectangle.Area() << '\n';
```

constexpr 생성자는 컴파일 시점 전용 생성자가 아니다. 상수 표현식에 필요한 조건을 만족하면 컴파일 시점에 사용할 수 있고, 그렇지 않으면 일반 생성자처럼 실행된다.

4.7.5 리터럴 타입 (Literal Type)

상수 표현식에서 객체를 사용하려면 해당 타입이 리터럴 타입(literal type)의 요건을 만족해야 한다.

기본형과 일부 단순 클래스 타입은 리터럴 타입으로 사용할 수 있다. 클래스 타입은 멤버와 생성자, 소멸자 등이 컴파일 타임 평가에 적합해야 한다.

```
struct Point
{
    int x, y;
    constexpr Point(int x, int y) : x(x), y(y) { }
};
```

Point는 상수 표현식에서 객체를 생성하고 사용할 수 있다.

```
constexpr Point origin(0, 0);
```

동적 자원 관리나 실행 시점에만 사용할 수 있는 기능을 포함한 클래스는 컴파일 타임 객체로 사용하기 어려울 수 있다.
리터럴 타입의 세부 조건은 C++ 표준 버전에 따라 확장되어 왔다. 클래스의 모든 기능을 constexpr로 만들기보다 컴파일 시점 계산이 실제로 필요한 타입과 함수에 적용하는 것이 좋다.

4.7.6 컴파일 시점과 실행 시점 평가

constexpr는 컴파일 시점 평가가 가능하다는 뜻이며 항상 컴파일 시점 평가를 강제하지 않는다.

```
constexpr int Square(int value)
{
    return value * value;
}

constexpr int first = Square(10);
```

first는 컴파일 시점에 계산되어야 한다.

```
int value;
std::cin >> value;

int second = Square(value);
```

second는 실행 시점에 계산된다.

컴파일 시점 평가를 요구하려면 결과를 constexpr 변수의 초기값이나 static_assert에 사용해야 한다.

```
constexpr int result = Square(10);
static_assert(Square(5) == 25);
```

4.7.7 즉시 함수 (Immediate Function) 와 consteval

C++20의 consteval은 함수 호출이 반드시 컴파일 시점에 평가되어야 함을 나타낸다. 이러한 함수를 즉시 함수 라고 한다.

```
consteval int Square(int value)
{
    return value * value;
}
```

상수 값을 전달한 호출은 허용된다.

```
constexpr int result = Square(10);
int value = Square(20);
```

두 호출 모두 컴파일 시점에 계산된다.
실행 시점에 결정되는 값을 전달할 수 없다.

```
int value;
std::cin >> value;

// int result = Square(value);   // 오류
```

constexpr 함수는 컴파일 시점과 실행 시점에 모두 사용할 수 있지만, consteval 함수는 반드시 컴파일 시점에 평가되어야 한다.

4.7.8 정적 초기화 (Static Initialization) 와 constinit

C++20의 constinit은 정적 저장 기간 또는 스레드 저장 기간을 가진 변수가 정적 초기화되도록 요구한다.

```
constinit int objectCount = 0;
```

objectCount는 프로그램 실행 전에 초기화가 완료되어야 한다.
constinit은 변수를 읽기 전용으로 만들지 않는다.

```
constinit int objectCount = 0;

void IncreaseCount()
{
    ++objectCount;
}
```

값을 변경하지 못하게 하려면 const나 constexpr를 함께 고려해야 한다.

```
constinit const int maximumCount = 100;
```

실행 시점 함수의 결과로 초기화할 수 없다.

```
int ReadCount();

// constinit int objectCount = ReadCount();   // 오류
```

constinit은 여러 전역 객체의 초기화 순서 때문에 발생할 수 있는 문제를 줄이는 데 사용할 수 있다. 일반 지역 변수에는 사용할 수 없다.

```
void Test()
{
    // constinit int value = 0;   // 오류
}
```

const, constexpr, consteval, constinit은 목적이 다르다.

<const, constexpr, consteval, constinit의 차이>

키워드	주요 적용 대상	의미	변경 가능 여부
const	변수, 객체, 멤버 함수	객체를 통한 변경을 제한	선언에 따라 변경 불가
constexpr	변수, 함수, 생성자	상수 표현식에서 사용할 수 있도록 선언	constexpr 변수는 변경 불가
consteval	함수	모든 호출을 컴파일 시점에 평가하도록 요구	해당 없음

constinit	정적·스레드 저장 기간 변수	정적 초기화를 보장	변경 가능

네 키워드는 모두 상수나 초기화와 관련되어 있지만, 변경 제한·컴파일 타임 평가·정적 초기화라는 서로 다른 목적을 가진다.

4.8 학습 정리

생성자는 객체가 생성될 때 호출되어 멤버를 초기화하고 객체의 초기 상태를 구성한다. 생성자의 이름은 클래스 이름과 같으며 반환 타입을 작성하지 않는다. 인수 없이 호출할 수 있는 생성자를 기본 생성자라고 한다.

클래스에 생성자가 하나도 선언되지 않으면 컴파일러가 기본 생성자를 암시적으로 선언할 수 있다. 컴파일러가 기본 생성자를 제공하더라도 기본형 멤버가 자동으로 0으로 초기화되는 것은 아니다. 사용자가 생성자를 하나라도 선언하면 기본 생성자가 자동으로 제공되지 않는다.

소멸자는 객체의 수명이 끝날 때 호출되어 객체가 사용하던 자원을 정리한다. 지역 객체는 선언된 블록을 벗어날 때 소멸하고, 동적 객체는 delete를 실행할 때 소멸한다. 같은 범위에서 생성된 객체는 생성 순서의 반대로 소멸한다.

객체는 기본 초기화, 값 초기화, 직접 초기화, 복사 초기화와 리스트 초기화 등의 방식으로 초기화할 수 있다. 중괄호 초기화는 축소 변환을 제한하며 집합체의 멤버를 순서대로 초기화하는 데도 사용된다.

멤버 초기화 리스트는 생성자 본문이 실행되기 전에 멤버를 직접 초기화한다. 멤버의 실제 초기화 순서는 초기화 리스트의 작성 순서가 아니라 클래스의 멤버 선언 순서에 따른다. 상수 멤버, 참조 멤버와 기본 생성자가 없는 객체 타입 멤버는 멤버 초기화 리스트에서 초기화해야 한다.

기본 멤버 초기값은 여러 생성자에서 공통으로 사용할 초기값을 제공한다. 위임 생성자는 다른 생성자에 초기화 작업을 맡겨 중복을 줄인다. explicit은 생성자를 통한 암시적 변환을 제한하며, = default는 컴파일러의 기본 동작을 명시적으로 요청한다. = delete는 특정 생성이나 복사와 같은 동작을 금지한다.

상수 멤버 함수는 호출 객체의 논리적인 상태를 변경하지 않으며 상수 객체와 상수 참조를 통해 호출할 수 있다. 일반 객체와 상수 객체에 서로 다른 접근 권한을 제공하려면 const 오버로딩을 사용하고, 객체 내부를 참조로 반환할 때도 호출 객체의 상수성을 유지해야 한다. mutable은 논리적 상태에 영향을 주지 않는 내부 정보를 상수 멤버 함수에서 변경할 때 제한적으로 사용한다.

constexpr 변수와 함수는 상수 표현식에 참여할 수 있다. constexpr 함수와 생성자는 조건에 따라 컴파일 시점 또는 실행 시점에 평가될 수 있으며, consteval 함수 호출은 반드시 컴파일 시점에 평가되어야 한다. constinit은 정적 또는 스레드 저장 기간을 가진 변수가 정적 초기화되도록 보장한다.

4.9 학습 과제

과제 **1.** 생성자와 초기 상태

- Point, Rectangle, Circle 클래스 작성
- Rectangle 상태
  - 위치
  - 너비
  - 높이
- Circle 상태
  - 중심점
  - 반지름
- 기본 생성자 작성
- 매개변수가 있는 생성자 작성
- 생성자 오버로딩 적용
- 너비, 높이와 반지름이 0 이하이면 1로 설정
- 객체 생성 후 상태 출력

과제 **2.** 초기화 방식 비교

- 기본형 변수와 Point 객체 선언
- 기본 초기화 적용
- 값 초기화 적용
- 직접 초기화 적용
- 복사 초기화 적용
- 중괄호 초기화 적용
- 각 초기화 결과 확인
- 축소 변환이 발생하는 코드 작성
- 괄호 초기화와 중괄호 초기화의 차이 설명

과제 **3.** 생성과 소멸 순서 확인

- 생성자와 소멸자에서 객체 이름 출력
- 같은 블록에 객체 3개 생성
- 중첩 블록에 객체 생성
- 객체 배열 생성
- new로 동적 객체 생성
- new[]로 동적 객체 배열 생성

- 생성 순서와 소멸 순서 기록
- delete와 delete[] 사용

과제 **4.** 멤버 초기화 리스트
- Point에 매개변수가 있는 생성자만 작성
- Rectangle이 Point를 멤버로 포함
- 멤버 초기화 리스트로 위치와 크기 초기화
- 상수 멤버 추가
- 참조 멤버를 가진 보조 클래스 작성
- 멤버 선언 순서와 실제 초기화 순서 확인
- 생성자 본문의 대입과 멤버 초기화 리스트의 차이 설명

과제 **5.** 생성자 기능 확장
- 기본 멤버 초기값 적용
- 위임 생성자로 생성자 중복 제거
- 단일 인수 생성자에 explicit 적용
- 기본 생성자에 = default 적용
- 허용하지 않을 생성자에 = delete 적용
- std::initializer_list를 받는 클래스 작성
- 괄호와 중괄호에 따라 선택되는 생성자 확인

과제 **6. const**와 **constexpr** 적용
- 상태 변경 함수와 상태 조회 함수 구분
- 조회 함수에 const 적용
- 일반 객체와 상수 객체의 호출 차이 확인
- 일반 참조와 상수 참조를 반환하는 함수 오버로딩
- 상수 참조 매개변수를 받는 출력 함수 작성
- Rectangle 생성자와 Area()에 constexpr 적용
- static_assert로 넓이 검사
- 실행 시점에 입력받은 크기로 넓이 계산
- constexpr 평가와 실행 시점 평가의 차이 설명

05. 복사, 이동과 값 의미

객체를 다른 객체로 초기화하거나 이미 존재하는 객체에 대입하면 객체의 상태가 전달된다. 기본형이나 값을 직접 저장하는 단순한 클래스는 각 멤버를 그대로 복사해도 독립된 객체를 만들 수 있다. 그러나 동적 메모리, 파일, 소켓과 같은 자원을 직접 소유하는 객체는 포인터나 핸들의 값만 복사해서는 올바르게 동작하지 않는다.

자원을 소유하는 객체에 기본 복사를 사용하면 두 객체가 하나의 자원을 함께 가리킬 수 있다. 한 객체를 변경했는데 다른 객체의 상태까지 바뀌거나, 객체가 소멸할 때 같은 자원을 두 번 해제하는 문제가 발생한다. 값처럼 사용할 타입은 복사본이 원본과 독립적으로 동작하도록 복사 정책을 설계해야 한다.

복사 생성자와 복사 대입 연산자는 객체의 복사 방법을 정의한다. 객체가 자원을 직접 소유한다면 복사 과정에서 새로운 자원을 준비하고 원본의 내용을 복제해야 한다. 소멸자, 복사 생성자와 복사 대입 연산자는 서로 다른 시점에 호출되지만 하나의 자원 소유 정책을 함께 구현한다.

C++11부터는 이동 생성자와 이동 대입 연산자를 사용하여 자원의 내용을 복제하지 않고 소유권을 새로운 객체로 이전할 수 있다. 이동을 이해하려면 표현식이 객체의 정체성을 나타내는지, 임시 값을 나타내는지, 자원을 이전할 수 있는 상태인지를 구분하는 값 범주도 알아야 한다.

이 장에서는 값 의미와 객체 정체성을 구분하고, 기본 복사와 복사 정책을 학습한다. 동적 배열을 소유하는 IntArray를 단계적으로 구현하며 복사 생성자, 복사 대입 연산자, 이동 생성자와 이동 대입 연산자의 동작을 확인한다. 이어서 Rule of Three, Rule of Five와 Rule of Zero를 통해 특별 멤버 함수의 설계 기준을 정리하고, std::move, noexcept와 복사 생략을 이용해 객체를 효율적으로 전달하고 반환하는 방법을 살펴본다.

5.1 값 의미와 객체 정체성

프로그램에서 값과 객체는 밀접하게 연결되어 있지만 같은 개념은 아니다. 값은 계산과 비교의 대상이 되는 내용이고, 객체는 값을 저장하는 메모리 영역과 수명을 가진 실체이다.

```
int first = 10;
int second = 10;
```

first와 second는 모두 값 10을 저장하지만 서로 다른 메모리 위치에 존재하는 별개의 객체이다.

```
std::cout << std::boolalpha;
std::cout << (first == second) << '\n';
std::cout << (&first == &second) << '\n';

// 결과
// true
// false
```

첫 번째 비교는 두 객체에 저장된 값이 같은지 확인하고, 두 번째 비교는 두 주소가 같은지 확인한다. 값이 같다는 사실과 같은 객체라는 사실은 구분해야 한다.

5.1.1 값 동일성

두 객체가 프로그램에서 같은 내용으로 취급될 수 있으면 값이 동일하다고 본다.

```
struct Point { int x, y; };
Point first{10, 20};
Point second{10, 20};
```

first와 second는 별개의 객체이지만 같은 좌표를 나타낸다. 연산자 오버로딩을 사용하지 않아도 각 멤버를 직접 비교하여 값 동일성을 확인할 수 있다.

```
bool sameValue =  first.x == second.x && first.y == second.y;
```

어떤 상태를 값 동일성에 포함할지는 타입의 의미에 따라 결정한다. 게임 캐릭터가 식별자, 이름, 체력과 위치를 가진다고 가정해 보자.

```
class Character
{
private:
    int id;
    std::string name;
    int hp;
    Point position;
```

```
};
```

이름과 체력이 같아도 id가 다르면 서로 다른 캐릭터이다. 위치를 비교하는 기능에서는 position만 같으면 같은 위치로 판단할 수 있다. 객체의 모든 멤버가 같아야 값이 동일한 것은 아니며, 타입이 표현하는 의미와 비교 목적에 따라 기준을 정한다.

5.1.2 객체 정체성

객체 정체성은 두 표현식이 같은 객체를 나타내는지를 의미한다.

```
Point point{10, 20};
Point& first = point;
Point& second = point;
```

first와 second는 모두 point를 참조하므로 같은 객체를 나타낸다.

```
Point point{10, 20};
Point& first = point;
Point& second = point;
```

한 참조를 통해 값을 변경하면 같은 객체를 참조하는 다른 이름에서도 변경된 값이 보인다.

```
first.x = 100;
std::cout << point.x << '\n';
std::cout << second.x << '\n';
```

두 출력 모두 100이다.

객체의 상태가 변경되어도 객체의 정체성은 유지된다.

```
Character player(1, "Knight");
player.TakeDamage(30);
player.Move(10, 20);
```

체력과 위치가 변경되어도 player는 계속 같은 객체이다. 객체의 주소는 현재 실행 중인 메모리 위치를 구분하지만, 항상 영구적인 식별자가 되는 것은 아니다. 컨테이너의 재할당으로 객체가 이동할 수 있고, 파일이나 네트워크에 객체를 저장할 때 메모리 주소를 그대로 사용할 수도 없다. 프로그램에서 지속적인 정체성이 필요하면 별도의 식별자를 사용한다.

5.1.3 값 의미와 참조 의미

값 의미(value semantics)를 가지는 객체를 복사하면 같은 값을 가진 새로운 객체가 만들어지고, 복사본과 원본을 독립적으로 변경할 수 있다.

```
Point original{10, 20};
Point copy = original;
copy.x = 100;
```

copy.x를 변경해도 original.x는 10으로 유지된다.
참조나 포인터를 복사하면 새로운 대상 객체가 만들어지는 것이 아니라 같은 객체를 가리키는 별도의 참조 수단이 만들어진다.

```
Point point{10, 20};
Point* first = &point;
Point* second = first;
second->x = 100;
```

first와 second는 같은 point를 가리키므로 first->x도 100이다.

```
값 의미
원본 객체 ── 값 복사 ──▶ 독립된 복사본

참조 의미
첫 번째 포인터 ─┐
               ├──▶ 하나의 대상 객체
두 번째 포인터 ─┘
```

값 의미와 참조 의미 중 어느 방식이 더 좋은 것은 아니다. 좌표, 날짜, 문자열처럼 독립된 값을 표현하는 타입은 값 의미가 자연스럽다. 게임 월드에 하나만 존재하는 플레이어 객체나 장면 객체를 여러 위치에서 참조해야 한다면 참조 의미가 자연스러울 수 있다.

5.1.4 값 의미를 가지는 타입

값 의미를 가지는 타입은 객체가 자신의 값을 소유하고, 복사된 객체가 원본과 독립적으로 동작하도록 설계한다.

```
std::string first = "Player";
std::string second = first;

second.append("_Two");
```

std::string은 내부적으로 동적 메모리를 사용할 수 있지만 second를 변경해도 first의 문자열은 유지된다. 사용자는 내부 구현을 알지 못해도 문자열 객체를 독립된 값처럼 복사하고 반환할 수 있다.

값 의미를 가지는 타입은 일반적으로 다음 성질을 제공한다.

- 복사본이 원본과 독립적으로 동작함
- 객체의 값에 따라 동일성을 판단할 수 있음
- 함수의 인수와 반환값으로 자연스럽게 사용할 수 있음
- 객체가 자신의 자원을 스스로 관리함
- 객체의 수명이 끝나면 소유한 자원도 정리됨

C++에서 값 의미를 가진 타입은 별도의 언어 분류가 아니라 설계 관점의 표현이다. class와 struct 모두 값 의미를 가지도록 설계할 수 있다.

5.1.5 자원을 소유하는 객체

객체가 동적 메모리, 파일, 소켓, 운영체제 핸들과 같은 자원의 사용과 해제를 책임지면 해당 자원을 소유한다고 표현한다.

동적 정수 배열을 관리하는 IntArray를 생각해 보자.

```
class IntArray
{
public:
    explicit IntArray(std::size_t size)
        : size(size), data(size > 0 ? new int[size]{} : nullptr)
    { }
    ~IntArray()
    {
        delete[] data;
    }
private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

IntArray 객체는 생성할 때 배열을 할당하고 소멸할 때 배열을 해제한다. data 멤버에는 주소값만 저장되지만, 클래스의 관점에서는 그 주소가 가리키는 동적 배열까지 객체가 관리해야 할 상태에 포함된다.

자원을 소유하는 객체를 복사할 때 size와 data만 그대로 복사하면 두 객체가 같은 배열을 가리키게 된다. IntArray를 값처럼 사용하려면 복사본이 새로운 배열을 소유하도록 복사 동작을 직접 설계해야 한다.

구분	값 동일성	객체 정체성
판단 기준	객체가 나타내는 내용	같은 객체를 나타내는지 여부
확인 방법	상태 비교	주소와 참조 관계 확인
복사 후 관계	같은 값을 가질 수 있음	서로 다른 객체가 됨
상태 변경	독립된 복사본에는 영향을 주지 않음	같은 객체를 참조하면 변경이 공유됨
설계 관점	어떤 상태가 객체의 값을 구성하는지 결정	객체의 수명과 참조 관계를 관리

5.2 기본 복사와 복사 정책

클래스에 복사 방법을 별도로 선언하지 않으면 컴파일러는 조건에 따라 각 멤버를 대응하는 멤버로 복사하는 동작을 제공한다. 기본형이나 값을 직접 소유하는 멤버에는 자연스러운 동작이지만, 자원을 가리키는 포인터 멤버에는 문제가 생길 수 있다.

5.2.1 멤버 단위 복사

앞에서 값 동일성을 설명할 때 사용한 Point 를 다시 살펴보자. 이번에는 객체를 복사할 때 각 데이터 멤버가 어떻게 처리되는지 확인해 보자.

객체를 복사하면 각 멤버의 값이 대응하는 멤버로 복사된다.

```
struct Point { int x, y; };

Point first{10, 20};
Point second = first;
```

개념적으로 다음 작업이 수행된 것과 비슷하다.

```
second.x = first.x;
second.y = first.y;
```

각 객체가 x와 y를 직접 저장하므로 기본 복사만으로 독립적인 객체가 만들어진다.
클래스 타입 멤버는 해당 타입의 복사 동작을 사용한다.

```
class Player
{
private:
    std::string name;
    Point position;
};
```

Player를 복사하면 name에는 std::string의 복사 동작이 적용되고, position에는 Point의 복사 동작이 적용된다. 객체 전체를 단순히 바이트로 복제하는 것이 아니라 각 멤버가 자신의 복사 규칙에 따라 처리된다.

5.2.2 포인터 멤버와 얕은 복사

동적 배열을 소유하는 IntArray에 원소를 변경하고 읽는 함수를 추가한다.

```
class IntArray
{
public:
    explicit IntArray(std::size_t size)
        : size(size), data(size > 0 ? new int[size]{} : nullptr)
    { }
    ~IntArray()
    {
        delete[] data;
    }
    void Set(std::size_t index, int value)
    {
        if (index < size)
        {
            data[index] = value;
        }
    }
    int Get(std::size_t index) const
    {
        if (index < size)
        {
            return data[index];
        }
        return 0;
    }
private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

복사 생성자를 선언하지 않았으므로 컴파일러가 제공하는 복사 동작이 사용될 수 있다.

```
IntArray first(3);
first.Set(0, 10);
first.Set(1, 20);
first.Set(2, 30);
IntArray second = first;
```

멤버 단위 복사는 size와 data의 값을 그대로 복사한다.

```
first
├─ size: 3
└─ data ──────┐
             ▼
          [10][20][30]
             ▲
second       │
├─ size: 3   │
└─ data ─────┘
```

first.data와 second.data는 같은 배열을 가리킨다. 포인터가 가리키는 배열의 내용은 복제되지 않고 주소값만 복사되었다. 이처럼 객체가 가리키는 자원의 내용을 복제하지 않고 포인터나 핸들 값만 복사하는 동작을 얕은 복사(shallow copy)라고 한다.

```
second.Set(0, 100);
std::cout << first.Get(0) << '\n';
```

출력값은 100이다. 두 객체가 같은 배열을 공유하므로 복사본의 변경이 원본에도 나타난다.

5.2.3 깊은 복사

깊은 복사(deep copy)는 원본 객체가 소유한 자원의 내용을 새로운 자원에 복제한다.

```
first
├─ size: 3
└─ data ──→ [10][20][30]

second
├─ size: 3
└─ data ──→ [10][20][30]
```

두 배열의 내용은 같지만 서로 다른 메모리 영역에 존재한다. 한 객체의 원소를 변경하거나 객체가 소멸해도 다른 객체는 영향을 받지 않는다.

깊은 복사에는 다음 작업이 필요하다.

- 원본과 같은 크기의 자원을 준비한다.
- 원본 자원의 내용을 새로운 자원으로 복사한다.
- 복사본이 새로운 자원을 소유하도록 설정한다.
- 각 객체가 자신의 자원만 해제하도록 유지한다.
- 이 작업은 복사 생성자와 복사 대입 연산자에서 구현한다.

5.2.4 공유, 비소유와 복사 금지 정책

포인터 멤버가 있다고 해서 항상 깊은 복사를 해야 하는 것은 아니다. 포인터의 역할과 클래스의 의미에 따라 복사 정책을 선택한다.

독립된 깊은 복사
복사본이 원본과 같은 값을 가지면서 독립적인 자원을 소유한다. 배열, 문자열, 이미지 버퍼처럼 값 의미를 제공할 타입에 적합하다.

공유 소유
여러 객체가 하나의 자원을 공동으로 소유한다. 마지막 소유자가 사라질 때 자원을 해제하도록 별도의 관리가 필요하다. 이후 장에서 std::shared_ptr를 이용한 공유 소유권을 다룬다.

비소유 참조
객체가 자원을 소유하지 않고 다른 객체를 가리키기만 한다.

```
class PlayerView
{
```

```
public:
    explicit PlayerView(Player* player) : player(player) { }
private:
    Player* player = nullptr;  // 소유하지 않음
};
```

PlayerView를 복사하면 두 뷰가 같은 Player를 가리키는 것이 자연스럽다. 뷰는 Player를 해제하지 않으며, 참조 대상이 먼저 소멸하지 않도록 수명을 관리해야 한다.

복사 금지
파일 핸들, 잠금 객체, 고유한 장치 연결처럼 복제 의미를 정의하기 어렵거나 소유자가 하나여야 하는 자원은 복사를 금지할 수 있다.

```
class UniqueResource
{
public:
    UniqueResource(const UniqueResource&) = delete;
    UniqueResource& operator=(const UniqueResource&) = delete;
};
```

복사 정책은 클래스의 구현 세부 사항에 그치지 않는다. 객체를 값처럼 복사할 수 있는지, 하나의 자원을 공유하는지, 대상을 참조만 하는지, 복사를 허용하지 않는지는 클래스 사용 방법을 결정하는 공개 인터페이스의 일부이다.

복사 정책	복사 결과	대표적인 용도
깊은 복사	독립된 자원과 같은 내용	배열, 문자열, 이미지 버퍼
공유 소유	하나의 자원을 함께 소유	공유 데이터, 캐시된 리소스
비소유 참조	같은 대상을 참조	뷰, 관찰자, 임시 접근 객체
복사 금지	복사 시 컴파일 오류	고유 핸들, 잠금, 단일 연결

5.2.5 자명한 복사와 바이트 단위 복사

C에서 구조체를 복사할 때 memcpy()를 사용한 경험이 있다면 C++ 객체도 같은 방식으로 복사할 수 있다고 생각하기 쉽다.

```
Point first{10, 20};
Point second{};

std::memcpy(&second, &first, sizeof(Point));
```

Point처럼 자명하게 복사 가능한(trivially copyable) 타입은 객체 표현을 바이트 단위로 복사할 수 있다. 타입이 자명하게 복사 가능한지는 타입 특성으로 확인할 수 있다.

```
#include <type_traits>

static_assert(std::is_trivially_copyable_v<Point>);
```

그러나 일반적인 C++ 객체 복사에 memcpy()를 사용해서는 안 된다.

```
struct Player
{
    std::string name;
    int hp;
};
```

std::string은 내부 자원을 관리한다. Player를 바이트 단위로 복사하면 두 문자열 객체의 내부 표현이 강제로 복제되어 자원 관리 규칙이 깨질 수 있다.

```
static_assert(!std::is_trivially_copyable_v<Player>);
```

객체 복사는 각 멤버의 복사 생성자나 복사 대입 연산자를 사용한다. memcpy()는 객체의 의미를 이해하지 않고 메모리 표현만 복제하므로 자명하게 복사 가능한 타입과 제한된 저수준 처리에서만 사용한다.

바이트 표현이 같다고 해서 값이 같다고 판단할 수도 없다. 패딩 바이트가 포함될 수 있으므로 객체의 값 비교는 타입이 정의한 비교 기준을 사용해야 한다.

5.2.6 이중 해제와 메모리 오류

얕은 복사를 수행한 IntArray 객체가 같은 배열을 소유한다고 가정하면 소멸 과정에서 문제가 발생한다.

```
{
    IntArray first(3);
    IntArray second = first;
}
```

블록을 벗어날 때 second와 first의 소멸자가 차례로 호출된다.

```
second 소멸 → 배열 해제
first 소멸  → 이미 해제된 배열을 다시 해제
```

같은 동적 메모리를 두 번 해제하는 동작을 이중 해제(double deletion)라고 한다. 이 동작의 결과는 정의되지 않으며 프로그램이 비정상 종료되거나 힙 메모리가 손상될 수 있다.

잘못된 복사와 소유권 처리는 여러 오류로 이어진다.

```
- 하나의 자원을 두 객체가 각각 소유한다고 판단하여 이중 해제가 발생함
- 한 객체가 자원을 해제한 뒤 다른 객체가 해당 주소를 계속 사용하는 댕글링 포인터가 생김
- 복사 대입에서 기존 주소를 잃어버려 메모리가 누수됨
- 비소유 포인터를 소유 포인터로 잘못 판단하여 해제함
- 배열에 delete를 사용하거나 단일 객체에 delete[]를 사용함
```

깊은 복사만으로 모든 문제가 해결되는 것은 아니다. 먼저 클래스가 자원을 소유하는지와 어떤 복사 정책을 제공할지 결정하고, 그 정책에 맞는 특별 멤버 함수를 구현해야 한다.

5.3 복사 생성자

복사 생성자(copy constructor)는 같은 타입의 다른 객체를 사용하여 새로운 객체를 초기화한다.

```
IntArray first(3);
IntArray second(first);
```

second는 새로 생성되는 객체이고 first는 복사의 원본이다.

5.3.1 복사 생성자의 선언

복사 생성자는 일반적으로 같은 클래스 타입의 상수 참조를 매개변수로 받는다.

```
class IntArray
{
public:
    IntArray(const IntArray& other);
};
```

복사 생성자의 이름은 클래스 이름과 같고 반환 타입을 작성하지 않는다.

```
IntArray(const IntArray& other);
```

구성	의미
IntArray	생성자 이름
const IntArray&	원본 객체를 읽기 전용 참조로 전달
other	원본 객체를 나타내는 매개변수 이름

const 참조를 사용하면 원본을 다시 복사하지 않고 접근할 수 있으며, 복사 과정에서 원본을 변경하지 않는다는 의도도 표현한다.

5.3.2 암시적으로 선언되는 복사 생성자

앞에서 사용한 Point에는 복사 생성자가 선언되어 있지 않다. 같은 타입을 복사 생성자의 관점에서 다시 살펴보자.

Point에는 사용자가 작성한 복사 생성자가 없지만 객체를 복사할 수 있다.

```
struct Point { int x, y; };

Point first{10, 20};
Point second(first);
```

컴파일러가 제공하는 복사 생성자는 각 직접 기반 클래스와 비정적 데이터 멤버를 대응하는 원본의 하위 객체로 초기화한다. Point에서는 x와 y가 각각 복사된다.

암시적으로 선언되었다고 해서 항상 사용할 수 있는 것은 아니다. 멤버나 기반 클래스가 복사를 지원하지 않으면 복사 생성자가 삭제된 함수로 정의될 수 있다.

```
class NonCopyable
{
public:
    NonCopyable() = default;
    NonCopyable(const NonCopyable&) = delete;
};

class Holder
{
private:
    NonCopyable value;
};
```

Holder의 value를 복사할 수 없으므로 Holder의 암시적 복사 생성자도 사용할 수 없다.

```
Holder first;
// Holder second = first;  // 오류
```

5.3.3 복사 생성자의 호출 시점

기존 객체를 이용하여 새로운 객체를 만들 때 복사 생성자가 호출될 수 있다.

```
IntArray original(3);
IntArray first(original);   // 직접 초기화
IntArray second = original; // 복사 초기화
```

두 선언은 초기화 문법이 다르지만 모두 새로운 객체를 생성한다.

함수의 매개변수를 값으로 전달할 때도 매개변수 객체를 만들기 위해 복사가 발생할 수 있다.

```
void PrintArray(IntArray array)
{
}

IntArray values(3);
PrintArray(values);
```

읽기만 하는 큰 객체는 상수 참조로 전달하여 복사를 피할 수 있다.

```
void PrintArray(const IntArray& array)
{
}
```

함수에서 객체를 값으로 반환할 때도 복사 생성자나 이동 생성자가 관련될 수 있다. 다만 모던 C++에서는 복사 생략 규칙으로 실제 호출이 생략되는 경우가 많다.

이미 존재하는 객체에 다른 객체를 대입하는 것은 복사 생성이 아니다.

```
IntArray first(3);
IntArray second(5);

second = first;
```

second는 이미 생성되어 있으므로 복사 대입 연산자가 사용된다.

코드	대상 객체의 상태	사용되는 기능
IntArray b(a);	b를 새로 생성	복사 생성자
IntArray b = a;	b를 새로 생성	복사 생성자
b = a;	b가 이미 존재	복사 대입 연산자

5.3.4 사용자 정의 복사

IntArray가 깊은 복사를 수행하도록 복사 생성자를 작성한다.

```
IntArray::IntArray(const IntArray& other)
 : size(other.size), data(other.size > 0 ? new int[other.size] : nullptr)
{
    for (std::size_t i = 0; i < size; ++i)
    {
        data[i] = other.data[i];
    }
}
```

복사 생성자는 원본의 size를 복사하고 같은 크기의 새로운 배열을 할당한다. 이후 원본 배열의 원소를 새로운 배열로 복사한다.

```
원본 크기 확인
      ↓
새 배열 할당
      ↓
원소 복사
      ↓
복사본이 새 배열 소유
```

다음 코드에서 second는 first와 다른 배열을 소유한다.

```
IntArray first(3);
first.Set(0, 10);
first.Set(1, 20);
first.Set(2, 30);

IntArray second = first;
second.Set(0, 100);

std::cout << first.Get(0) << '\n';
std::cout << second.Get(0) << '\n';

// 결과.
// 10
// 100
```

복사본의 원소를 변경해도 원본은 유지된다.

5.3.5 복사 생성자의 매개변수

복사 생성자의 매개변수를 값으로 받을 수는 없다.

```
// 올바른 복사 생성자가 될 수 없음
IntArray(IntArray other);
```

other 매개변수 자체를 만들려면 인수 객체를 먼저 복사해야 한다. 그 복사에도 같은 생성자가 필요하므로 복사 생성자를 호출하기 위해 다시 복사 생성자를 호출해야 하는 구조가 된다.

일반적인 복사 생성자는 상수 참조를 사용한다.

```
IntArray(const IntArray& other);
```

상수 참조를 사용하면 다음 객체를 모두 복사할 수 있다.

```
IntArray normal(3);
const IntArray fixed(3);

IntArray first(normal);
IntArray second(fixed);
```

비상수 참조를 받는 복사 생성자도 문법적으로 가능한 경우가 있지만, 상수 객체와 일부 임시 결과를 복사할 수 없으므로 일반적인 값 타입에는 적합하지 않다.

```
// 특별한 이유가 없다면 사용하지 않음
IntArray(IntArray& other);
```

5.4 복사 대입 연산자

복사 대입 연산자(copy assignment operator)는 이미 존재하는 객체의 값을 다른 객체의 값으로 교체한다.

```
IntArray first(3);
IntArray second(5);
second = first;
```

second는 이미 자신의 배열을 소유하고 있다. 복사 대입은 기존 자원을 잃거나 원본의 자원을 함께 소유하지 않도록 처리해야 한다.

5.4.1 복사 대입

복사 대입 연산자는 일반적으로 같은 타입의 상수 참조를 받고 현재 객체의 참조를 반환한다.

```
IntArray& operator=(const IntArray& other);
```

현재 객체의 참조를 반환하면 연속 대입을 사용할 수 있다.

```
first = second = third;
```

대입은 오른쪽부터 처리된다. second = third가 second의 참조를 반환하고, 그 결과가 first에 대입된다.

복사 생성자와 복사 대입 연산자는 대상 객체가 이미 존재하는지에 따라 구분한다.

구분	복사 생성자	복사 대입 연산자
대상 객체	새로 생성됨	이미 존재함
호출 예	IntArray b = a;	b = a;
기존 자원	없음	있을 수 있음
주요 작업	새 자원 준비와 내용 복사	기존 자원을 교체하고 내용 복사
반환값	없음	일반적으로 현재 객체의 참조

5.4.2 기존 자원의 교체

대입 대상 객체가 이미 자원을 소유하고 있다면 기존 자원을 먼저 고려해야 한다.

```
IntArray first(3);
IntArray second(100);
second = first;
```

second.data가 가리키던 100개 원소의 배열을 해제하지 않은 채 주소를 덮어쓰면 기존 배열에 다시 접근할 방법이 없어져 메모리가 누수된다.

```
// 잘못된 구현
size = other.size;
data = new int[other.size];
```

새 배열의 주소를 data에 바로 대입하면 이전 주소를 잃어버린다.
반대로 기존 배열을 먼저 해제하는 구현도 주의해야 한다.

```
// 아직 안전하지 않은 구현
delete[] data;

data = new int[other.size];
size = other.size;
```

새 배열 할당이 실패하면 대상 객체는 기존 값을 잃고 일관된 상태를 유지하기 어려울 수 있다. 자기 자신을 대입하는 경우에는 원본 데이터까지 먼저 해제하게 된다.

5.4.3 자기 대입

객체가 자기 자신에게 대입될 수도 있다.

```
array = array;
```

복사 대입에서 기존 자원을 먼저 해제하면 other가 가리키는 원본 데이터도 함께 사라진다.

```
IntArray& IntArray::operator=(const IntArray& other)
{
    delete[] data;

    size = other.size;
    data = new int[size];

    for (std::size_t i = 0; i < size; ++i)
    {
        data[i] = other.data[i];
    }

    return *this;
}
```

this와 &other가 같은 주소이면 delete[] data 이후 other.data도 해제된 메모리를 가리킨다.

자기 대입 여부를 확인할 수 있다.

```
if (this == &other)
{
    return *this;
}
```

자기 대입 검사는 잘못된 처리 순서를 가리는 용도로만 사용해서는 안 된다. 새로운 자원을 안전하게 준비한 뒤 기존 자원을 교체하도록 전체 대입 과정을 구성해야 한다.

5.4.4 새 자원을 먼저 준비하는 대입

복사 대입은 원본을 이용해 새 자원을 먼저 준비하고, 준비가 완료된 뒤 기존 자원을 교체하도록 작성할 수 있다.

```
IntArray& IntArray::operator=(const IntArray& other)
{
    if (this == &other)
    {
        return *this;
    }
    int* newData = other.size > 0 ? new int[other.size] :
```

```
nullptr;
    for (std::size_t i = 0; i < other.size; ++i)
    {
        newData[i] = other.data[i];
    }

    delete[] data;
    data = newData;
    size = other.size;
    return *this;
}
```

처리 순서는 다음과 같다.

```
새 배열 할당
      ↓
원본 원소 복사
      ↓
기존 배열 해제
      ↓
새 배열과 크기 적용
```

새 배열 할당에 실패하면 delete[] data에 도달하지 않으므로 대상 객체의 기존 상태는 유지된다. 이 예제는 int 배열을 복사하므로 원소 대입에서 예외가 발생하지 않는다. 복사 과정 자체가 실패할 수 있는 일반적인 클래스 타입은 RAII를 이용해 임시 자원도 안전하게 관리해야 한다.

5.4.5 복사-교환 기법

복사-교환 기법(copy-and-swap idiom)은 원본의 복사본을 먼저 만든 뒤 대상 객체의 상태와 교환한다.

```
#include <utility>

class IntArray
{
public:
    IntArray& operator=(IntArray other)
    {
        Swap(other);
        return *this;
    }
    void Swap(IntArray& other) noexcept
    {
        std::swap(size, other.size);
        std::swap(data, other.data);
    }

private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

매개변수 other는 값으로 전달되므로 복사 생성자를 사용해 독립된 복사본이 만들어진다.

```
원본 객체
   ↓ 복사 생성
other
   ↓ 상태 교환
대상 객체는 새 값을 소유
other는 대상의 이전 값을 소유
   ↓ 함수 종료
other의 소멸자가 이전 자원 해제
```

복사본 생성에 실패하면 대입 대상은 변경되지 않는다. 자기 대입에서도 별도의 검사 없이 올바른 결과를 만들 수 있다.
복사-교환 기법은 앞의 operator=(const IntArray&) 구현에 추가하는 단계가 아니라 대입 연산자를 구현하는 다른 전략이다. 다음 두 형태 중 하나를 선택한다.

```
IntArray& operator=(const IntArray& other); // 복사 대입을 직접 구현
IntArray& operator=(IntArray other);        // 값 매개변수와 교환 사용
```

값 매개변수 방식은 구현이 단순하고 안전하지만, 기존 배열의 저장 공간을 재사용할 수 있는 상황에서도 새로운 자원을 준비할 수 있다. 타입의 크기와 사용 빈도, 구현의 단순성과 성능 요구를 함께 고려하여 선택한다.

5.5 Rule of Three

5.5.1 Rule of Three의 의미

클래스가 소멸자, 복사 생성자, 복사 대입 연산자 중 하나를 직접 작성해야 한다면 나머지 두 함수도 필요한지 함께 검토해야 한다. 이 설계 원칙을 Rule of Three라고 한다.

동적 배열을 직접 소유하는 클래스는 소멸자에서 배열을 해제해야 한다. 소멸자가 필요하다는 사실은 기본 멤버 복사만으로 자원 소유권을 처리하기 어렵다는 신호가 된다.

```
~IntArray();
IntArray(const IntArray& other);
IntArray& operator=(const IntArray& other);
```

Rule of Three는 세 함수를 무조건 모두 직접 작성하라는 규칙이 아니다. 하나의 함수를 직접 작성할 이유가 생겼다면 나머지 함수에 컴파일러의 기본 동작을 사용해도 자원 소유 규칙이 유지되는지 검토하라는 원칙이다.

5.5.2 세 특별 멤버 함수의 관계

세 함수는 객체 수명의 서로 다른 지점에서 같은 자원 소유 정책을 적용한다.

특별 멤버 함수	호출 상황	자원 관리 역할
소멸자	객체의 수명이 끝남	소유한 자원 해제
복사 생성자	새 객체를 기존 객체로 초기화	독립된 자원을 준비하고 내용 복사
복사 대입 연산자	기존 객체의 값을 교체	기존 자원을 정리하고 새 내용 복사

```
객체 생성
   └ 복사 생성자: 원본과 독립된 자원 준비

객체 사용 중 값 교체
   └ 복사 대입 연산자: 기존 자원을 새 값으로 교체

객체 수명 종료
   └ 소멸자: 현재 소유한 자원 해제
```

소멸자만 작성하고 복사 함수를 기본 동작에 맡기면 두 객체가 같은 자원을 소유할 수 있다. 복사 생성자만 작성하고 복사 대입을 기본 동작에 맡기면 초기화는 안전하지만 대입에서 얕은 복사가 발생할 수 있다. 세 함수는 하나의 소유권 규칙을 일관되게 유지해야 한다.

5.5.3 직접 자원 관리의 부담

동적 메모리를 직접 소유하는 클래스는 많은 상황을 고려해야 한다.

```
- 생성 과정에서 확보한 자원의 정확한 해제
- 복사본의 독립적인 자원 소유
- 대입 대상의 기존 자원 보존
- 자기 대입에서 올바른 동작
- 자원 준비에 실패시  기존 객체 상태 유지
- 이동 지원을 위한 소유권 이전 규칙
- 특별 멤버 함수의 추가와 수정이 일관성 유지
```

가능하면 자원 관리를 전용 타입 하나에 맡기고, 일반적인 클래스에서는 std::string, std::vector와 같이 자원 관리를 이미 구현한 타입을 멤버로 사용한다. 이러한 방향은 뒤에서 설명할 Rule of Zero로 이어진다.

5.6 값 범주와 우측값 참조

C++의 표현식은 타입뿐만 아니라 값 범주**(value category)** 를 가진다. 값 범주는 표현식이 객체나 함수의 정체성을 나타내는지, 값을 계산하는지, 기존 객체의 자원을 재사용할 수 있는 표현식인지를 구분한다.

같은 타입의 표현식이라도 값 범주는 서로 다를 수 있다. 값 범주는 참조의 결합, 함수 오버로딩, 복사와 이동 생성자의 선택 등에 영향을 준다.

5.6.1 값 범주의 관계

C++11 이후의 기본 값 범주는 lvalue, xvalue(expiring value), prvalue(pure rvalue)이다. 이 세 범주를 성질에 따라 묶어 glvalue(generalized lvalue)와 rvalue로 구분한다.

<C++11 이후 값 범주의 관계>

```
expression        glvalue           rvalue
├─ lvalue         ├─ lvalue         ├─ xvalue
├─ xvalue         └─ xvalue         └─ prvalue
└─ prvalue
```

```
- glvalue: 객체나 함수의 정체성을 나타내는 표현식
- rvalue: 이동 가능한 객체나 임시 계산 결과를 묶은 범주
- xvalue: glvalue이면서 rvalue 인 표현식
```

실제 코드에서는 lvalue, prvalue, xvalue의 구분이 참조의 결합과 복사·이동 연산의 선택에 직접 영향을 준다.

5.6.2 lvalue (좌측값)

lvalue는 객체나 함수의 정체성을 나타내는 표현식이다. 이름을 가진 변수 표현식은 일반적으로 lvalue이다.

```
int value = 10;
value = 20;
std::cout << value << '\n';
```

표현식 value는 같은 객체에 반복해서 접근할 수 있다.
참조를 반환하는 함수 호출도 lvalue가 될 수 있다.

```
int& Select(int& value)
{
    return value;
}

int number = 10;
Select(number) = 100;
```

Select(number)는 number 객체를 나타내므로 대입의 왼쪽에 사용할 수 있다.
const 객체를 나타내는 표현식도 lvalue이다.

```
const int maxCount = 100;
```

maxCount는 수정할 수 없는 lvalue이다. 수정 가능 여부와 값 범주는 별개의 성질이다.

5.6.3 prvalue (pure rvalue: 순수우측값)

prvalue는 값을 계산하거나 결과 객체를 초기화하는 표현식이다.

```
42
3.14
first + second
IntArray(10)
```

리터럴과 기본형의 산술 결과, 값을 반환하는 대부분의 함수 호출은 prvalue를 만든다.

```
int Add(int first, int second)
{
    return first + second;
```

```
}

int result = Add(10, 20);
```

Add(10, 20)은 결과 값을 계산하는 prvalue이다.
클래스 타입의 prvalue는 결과 객체를 직접 초기화할 수 있다.

```
IntArray CreateArray(std::size_t size)
{
    return IntArray(size);
}
```

C++17 이후에는 이 코드를 반드시 별도의 임시 객체를 먼저 만들고 이동하는 과정으로 이해할 필요가 없다. IntArray(size)가 반환 결과 객체를 직접 초기화할 수 있다.

5.6.4 xvalue (eXpiring value: 소멸예정 값)

xvalue는 정체성을 가진 객체를 나타내지만 자원을 이전할 수 있는 상태로 취급되는 표현식이다.

```
IntArray source(100);
IntArray target(std::move(source));
```

이름 source를 사용한 표현식은 lvalue이다. std::move(source)는 같은 source 객체를 나타내면서 값 범주를 xvalue로 변환한다.

```
source
├─ 객체의 정체성 있음
└─ 표현식은 lvalue

std::move(source)
├─ 같은 객체의 정체성 있음
└─ 이동 대상으로 사용할 수 있는 xvalue
```

xvalue는 객체가 곧 소멸한다는 사실을 보장하지 않는다. 이동 생성자나 이동 대입 연산자가 해당 객체의 자원을 가져갈 수 있도록 표현식의 값 범주를 바꾼다.

5.6.5 rvalue (우측값)

rvalue는 prvalue와 xvalue를 포함하는 범주이다.

```
10 + 20             // prvalue이므로 rvalue
std::move(source)   // xvalue이므로 rvalue
```

과거에는 대입문의 오른쪽에 놓이는 값을 rvalue라고 간단히 설명하기도 했다. 그러나 모던 C++에서는 대입문의 위치보다 객체의 정체성과 이동 가능성을 기준으로 값 범주를 구분한다.

```
int first = 10;
int second = 20;
first = second;
```

오른쪽에 놓인 second는 이름 있는 객체 표현식이므로 lvalue이다. 대입문의 오른쪽에 있다는 이유만으로 rvalue가 되는 것은 아니다.

5.6.6 rvalue reference(우측값 참조)

rvalue reference 는 &&로 선언하며 rvalue에 결합한다.

```
int&& value = 10;
```

10은 prvalue이고 value는 그 결과 객체를 참조한다.
클래스의 이동 생성자는 같은 타입의 우측값 참조를 매개변수로 받는다.

```
IntArray(IntArray&& other) noexcept;
```

우측값 참조를 사용하면 복사가 필요한 일반 객체와 자원을 이전할 수 있는 객체를 함수 오버로딩으로 구분할 수 있다.

```
void Process(const IntArray& array); // 읽기 또는 복사에 사용
void Process(IntArray&& array);      // 이동 가능한 객체에 사용
```

5.6.7 이름이 있는 우측값 참조

우측값 참조 타입으로 선언한 변수라도 그 이름을 사용한 표현식은 lvalue이다.

```
IntArray&& reference = IntArray(10);
```

변수 reference의 선언 타입은 IntArray&&이지만 표현식 reference는 이름을 통해 같은 객체에 반복해서 접근할 수 있으므로 lvalue이다.

```
IntArray copied(reference);              // lvalue이므로 복사
IntArray moved(std::move(reference));    // xvalue로 변환하여
이동
```

이 규칙은 이동 생성자 내부에서도 중요하다.

```
IntArray::IntArray(IntArray&& other)
{
    // 표현식 other는 lvalue
}
```

other의 타입은 우측값 참조이지만 이름 other를 사용한 표현식은 lvalue이다. other의 클래스 타입 멤버를 이동하려면 std::move(other.member)처럼 다시 이동 가능한 표현식으로 변환해야 한다.

5.6.8 임시 객체와 수명

계산 결과나 함수 반환 과정에서 이름 없는 임시 객체가 만들어질 수 있다.

```
IntArray array = IntArray(10);
```

실제 객체 생성 과정은 복사 생략 규칙의 영향을 받지만, IntArray(10)은 결과 객체를 초기화하는 임시 값 표현식이다.
상수 lvalue 참조는 임시 객체에 결합하여 수명을 연장할 수 있다.

```
const IntArray& reference = IntArray(10);
```

해당 임시 객체는 reference의 수명이 끝날 때까지 유지된다.
우측값 참조도 직접 초기화한 임시 객체의 수명을 연장할 수 있다.

```
IntArray&& reference = IntArray(10);
```

참조를 함수 밖으로 반환한다고 해서 지역 객체의 수명이 연장되지는 않는다.

```
const IntArray& CreateInvalidReference()
{
    IntArray local(10);
    return local;  // 잘못된 참조 반환
}
```

함수가 끝나면 local이 소멸하므로 반환된 참조는 더 이상 유효하지 않다.

값 범주	정체성	이동 가능성	대표 표현식
lvalue	있음	그대로는 일반적으로 이동 대상이 아님	이름 있는 변수, 참조 반환 함수
prvalue	결과 값을 계산	있음	리터럴, 산술 결과, 값 반환 함수
xvalue	있음	있음	std::move(object)
glvalue	있음	lvalue와 xvalue를 포함	객체나 함수의 정체성을 나타내는 표현식

rvalue	prvalue 또는 이동 가능한 정체성	있음	prvalue와 xvalue

5.7 이동 생성자와 이동 대입 연산자

깊은 복사는 새로운 자원을 준비하고 원본 자원의 내용을 모두 복제한다. 큰 배열을 복사하면 배열 크기에 비례하는 작업이 필요하다.

```
복사
원본 배열 ── 원소 전체 복제 ──▸ 새 배열
```

원본 객체의 기존 값을 더 이상 유지할 필요가 없다면 배열의 내용을 복제하지 않고 포인터와 크기를 새로운 객체로 넘길 수 있다.

```
이동
원본 객체의 자원 ── 소유권 이전 ──▸ 새 객체
원본 객체는 자원을 소유하지 않는 상태로 변경
```

5.7.1 이동 생성자

이동 생성자(move constructor)는 같은 타입의 우측값 참조를 받아 새로운 객체를 초기화한다.

```
// 이동 생성자 선언.
IntArray(IntArray&& other) noexcept;
```

IntArray의 이동 생성자를 구현한다.

```
// 이동 생성자 구현.
IntArray::IntArray(IntArray&& other) noexcept
    : size(other.size),
      data(other.data)
{
    other.size = 0;
    other.data = nullptr;
}
```

새 객체는 other의 배열 주소와 크기를 가져온다. 이후 other.data를 nullptr로 바꾸어 두 객체가 같은 배열을 소유하지 않게 한다.

```
IntArray source(1000);
IntArray target(std::move(source));
```

```
target
├─ size: 1000
└─ data ──▸ [동적 배열]


source
├─ size: 0
└─ data: nullptr
```

배열 원소 1000개를 복사하지 않고 두 멤버의 값만 이전한다.

5.7.2 이동 대입 연산자

이동 대입 연산자(move assignment operator)는 이미 존재하는 객체의 자원을 정리한 뒤 다른 객체의 자원을 넘겨받는다.

```
// 이동 대입 연산자 선언.
IntArray& operator=(IntArray&& other) noexcept;

// 이동 대입 연산자 구현.
IntArray& IntArray::operator=(IntArray&& other) noexcept
{
    if (this == &other)
    {
        return *this;
```

```
    }

    delete[] data;
    size = other.size;
    data = other.data;
    other.size = 0;
    other.data = nullptr;

    return *this;
}
```

이동 대입에서는 대상 객체가 기존에 소유하던 배열을 먼저 해제해야 한다.

```
IntArray first(100);
IntArray second(1000);
first = std::move(second);
```

```
first의 기존 배열 해제
         ↓
second의 size와 data를 first로 이전
         ↓
second를 빈 상태로 변경
```

자기 이동 대입은 일반적인 코드에서는 드물지만 발생할 수 있다.

```
array = std::move(array);
```

예제에서는 자기 대입 검사를 사용하여 기존 값을 유지한다. 타입에 따라 자기 이동 대입 후 유효한 빈 상태를 만들도록 설계할 수도 있지만, 문서화되지 않은 결과에 의존하지 않는 편이 좋다.

5.7.3 자원 소유권 이전

이동은 객체의 모든 바이트를 무조건 복사하는 동작이 아니다. 하나의 객체가 소유하던 자원을 다른 객체가 소유하도록 각 멤버를 처리한다. 포인터 멤버는 주소를 넘긴 뒤 원본을 nullptr로 만든다.

```
data = other.data;
other.data = nullptr;
```

크기 멤버도 자원을 소유하지 않는 상태와 일치하도록 변경한다.

```
size = other.size;
other.size = 0;
```

클래스 타입 멤버는 해당 타입의 이동 기능을 사용한다.

```
class Player
{
public:
    Player(Player&& other) noexcept
        : name(std::move(other.name))
        , items(std::move(other.items))
    { }

private:
    std::string name;
    std::vector<int> items;
};
```

other는 이름이 있는 표현식이므로 other.name과 other.items도 그대로 사용하면 lvalue로 처리된다. 각 멤버의 이동 생성자를 사용하려면 std::move()가 필요하다.

실제 Player처럼 자원 관리 타입만 멤버로 가지는 클래스는 이동 생성자를 직접 작성하지 않고 컴파일러의 기본 동작을 사용하는 편이 더 좋다. 이 예제는 이름이 있는 우측값 참조의 멤버를 이동하는 원리를 보여 주기 위한 것이다.

5.7.4 이동 후 객체의 상태

이동된 원본 객체는 수명이 끝난 객체가 아니다. 여전히 존재하며 정상적으로 소멸할 수 있어야 한다. 새로운 값을 대입하여 다시 사용할 수도 있어야 한다.
IntArray는 이동 후 원본을 명시적인 빈 배열 상태로 만든다.

```
other.size = 0;
other.data = nullptr;
```

따라서 다음 코드는 안전하다.

```
IntArray source(10);
IntArray target(std::move(source));

source = IntArray(5);
```

사용자 정의 타입은 이동 후 상태를 문서화할 수 있다. IntArray는 IsEmpty()가 true가 되는 것으로 정한다.

표준 라이브러리의 많은 타입은 이동 후에도 유효하지만 값은 지정되지 않은 상태(valid but unspecified state)가 된다. 정상적으로 소멸하거나 새로운 값을 대입할 수 있지만, 이동 전의 원소나 크기가 유지된다고 가정해서는 안 된다. 각 타입이 별도로 보장하는 연산이 있다면 그 보장을 따른다.

5.7.5 복사와 이동의 선택

인수가 lvalue이면 복사 생성자가 선택된다.

```
IntArray source(10);
IntArray first(source);
```

인수가 rvalue이면 이동 생성자가 선택될 수 있다.

```
IntArray second(IntArray(10));
IntArray third(std::move(source));
```

두 생성자는 매개변수 타입으로 구분된다.

```
IntArray(const IntArray& other); // 복사
IntArray(IntArray&& other) noexcept; // 이동
```

대입에서도 같은 원리가 적용된다.

```
IntArray target(3);
IntArray source(10);

target = source;              // 복사 대입
target = std::move(source);  // 이동 대입
```

구분	복사	이동
원본 표현식	주로 lvalue	rvalue 또는 std::move() 결과
자원 처리	새로운 자원을 만들고 내용 복제	기존 자원의 소유권 이전
원본 상태	유지	유효하지만 값이 달라질 수 있음
비용	자원 크기에 비례할 수 있음	포인터와 크기 이전 정도로 끝날 수 있음
대표 함수	복사 생성자, 복사 대입	이동 생성자, 이동 대입

5.7.6 상수 객체와 이동

std::move()는 객체의 const를 제거하지 않는다.

```
const IntArray source(10);
```

```
IntArray target(std::move(source));
```

std::move(source)의 타입은 const IntArray&&이다. 일반적인 이동 생성자는 비상수 우측값 참조를 받는다.

```
IntArray(IntArray&& other) noexcept;
```

이동 생성자는 원본의 포인터를 nullptr로 바꾸는 등 원본 상태를 변경해야 하므로 const IntArray&&에는 결합할 수 없다.

복사 생성자가 존재하면 상수 참조에 결합하여 복사가 선택될 수 있다.

```
IntArray(const IntArray& other);
std::move(nonConstObject) → T&&       → 이동 가능
std::move(constObject)    → const T&& → 일반적으로 복사
```

const T&&를 받는 별도의 함수를 만들 수도 있지만 원본을 변경할 수 없어 자원 소유권을 이전하기 어렵다. 이동할 객체에는 불필요한 const를 사용하지 않는다.

5.8 std::move와 noexcept

5.8.1 std::move의 의미

std::move()는 객체의 자원을 실제로 이동하는 함수가 아니다. 전달받은 표현식을 이동 가능한 rvalue로 취급하도록 변환한다.

```
IntArray source(1000);
IntArray copied(source);
IntArray moved(std::move(source));
```

source는 이름 있는 객체이므로 lvalue이다. std::move(source)는 같은 객체를 나타내는 xvalue를 만든다.
개념적으로는 다음 형 변환과 비슷하다.

```
static_cast<IntArray&&>(source)
```

실제 std::move()는 참조와 타입 한정자를 보존하도록 템플릿으로 구현되어 있으므로 위 코드는 개념을 설명하기 위한 단순화이다.

5.8.2 std::move가 이동을 수행하지 않는 이유

std::move()를 호출해도 이동 생성자나 이동 대입 연산자가 없다면 복사가 선택될 수 있다.

```
class CopyOnly
{
public:
    CopyOnly() = default;
    CopyOnly(const CopyOnly&) = default;
};
CopyOnly first;
CopyOnly second(std::move(first));));
```

CopyOnly에는 이동 생성자가 없지만 복사 생성자의 const CopyOnly&는 rvalue에도 결합할 수 있으므로 복사 생성자가 사용될 수 있다.
함수 오버로딩에서도 실제 호출은 오버로드 해석으로 결정된다.

```
void Process(const IntArray& array);
void Process(IntArray&& array);
IntArray values(10);
Process(values);            // const IntArray& 버전
Process(std::move(values)); // IntArray&& 버전
```

std::move()는 이동을 명령하지 않는다. 이동이 가능한 표현식을 만들어 이동 함수가 선택될 기회를 제공한다. 실제 자원 이전은 선택된 이동 생성자나 이동 대입 연산자가 수행한다.

5.8.3 noexcept 이동

이동 생성자와 이동 대입 연산자가 예외를 밖으로 전달하지 않는다면 noexcept를 명시할 수 있다.

```
IntArray(IntArray&& other) noexcept;
IntArray& operator=(IntArray&& other) noexcept;
```

IntArray의 이동은 포인터와 크기를 이전하고 원본을 빈 상태로 바꾸는 작업만 수행하므로 예외를 발생시킬 이유가 없다.

```
IntArray::IntArray(IntArray&& other) noexcept
    : size(other.size)
    , data(other.data)
{
    other.size = 0;
    other.data = nullptr;
}
```

noexcept는 설명용 주석이 아니라 함수 계약의 일부이다. noexcept 함수 밖으로 예외가 빠져나가면 프로그램은 std::terminate()를 호출한다. 실제로 예외가 발생할 수 있는 이동 함수에 무조건 noexcept를 붙이면 안 된다.

클래스 타입 멤버의 이동 생성자가 예외를 발생시키는지에 따라 조건부로 noexcept를 지정할 수 있다.
std::is_nothrow_move_constructible_v<T>는 T의 이동 생성자가 예외를 발생시키지 않는지 컴파일 시점에 확인한다.

```
class Wrapper
{
public:
    Wrapper(Wrapper&&) noexcept(
        std::is_nothrow_move_constructible_v<std::string>
    ) = default;

private:
    std::string value;
};
```

Wrapper의 이동 생성자는 value를 이동하여 초기화한다. 따라서 std::string의 이동 생성자가 noexcept이면 Wrapper의 이동 생성자도 noexcept가 된다.

이동 생성자에 noexcept를 지정할 때는 멤버와 기반 클래스의 이동 연산도 예외를 발생시키지 않는지 확인해야 한다.

5.8.4 std::move_if_noexcept

std::move_if_noexcept()는 이동이 예외를 발생시키지 않거나 복사를 사용할 수 없는 타입이면 이동 가능한 참조를 제공한다. 이동이 예외를 발생시킬 수 있고 복사가 가능하면 상수 lvalue 참조를 제공하여 복사가 선택될 수 있게 한다.

```
T target(std::move_if_noexcept(source));
```

개념적인 선택 기준은 다음과 같다.

```
이동 생성자가 noexcept인가?
├─ 예 → 이동 가능
└─ 아니오
   ├─ 복사 가능 → 복사 가능하도록 const 참조 제공
   └─ 복사 불가 → 이동 사용
```

std::move_if_noexcept()를 일반 코드에서 자주 직접 호출할 필요는 없다. 컨테이너가 원소를 재배치하면서 복사와 이동 중 더 안전한 방법을 선택하는 원리를 이해하는 데 활용할 수 있다.

5.8.5 컨테이너 재배치와 이동

std::vector는 저장 공간이 부족하면 더 큰 메모리를 확보하고 기존 원소를 새 공간으로 재배치한다.

```
기존 저장 공간
[A][B][C]

더 큰 저장 공간
[A][B][C][ ][ ][ ]
```

원소 타입의 이동 생성자가 예외를 발생시키지 않는다고 보장되면 기존 원소를 이동하여 재배치할 수 있다.
IntArray(IntArray&& other) noexcept;

이동 도중 일부 원소의 자원이 이미 이전된 뒤 예외가 발생하면 원래 컨테이너 상태를 복구하기 어려울 수 있다. 이동이 예외를 발생시킬 수 있고 복사가 가능하면 컨테이너는 기존 원소를 유지하기 위해 복사를 선택할 수 있다.

복사를 금지한 이동 전용 타입은 이동 생성자가 예외를 발생시킬 수 있어도 이동할 수밖에 없다. 이 경우 컨테이너가 제공할 수 있는 예외 안전성의 수준이 달라질 수 있다.

이동 함수의 noexcept 여부는 단순한 문서 정보가 아니라 표준 컨테이너가 더 효율적이고 안전한 경로를 선택하는 데 영향을 준다.

5.8.6 반환값에 std::move를 사용할 때의 문제

지역 객체를 값으로 반환할 때는 일반적으로 std::move()를 붙이지 않는다.

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);

    return result;
}
```

컴파일러는 이름 있는 반환값 최적화(NRVO)를 적용하여 result를 호출자의 결과 객체 위치에 직접 생성할 수 있다. NRVO가 적용되지 않아도 지역 객체는 이동 대상으로 처리될 수 있다.

불필요한 std::move()를 붙이면 NRVO를 방해할 수 있다.

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);

    return std::move(result);  // 권장하지 않음
}
```

std::move(result)는 이름 있는 지역 객체 자체가 아니라 xvalue 표현식을 반환하므로 result를 결과 위치에 직접 생성하는 NRVO의 대상이 되지 않는다.

복사 대신 이동이 발생할 수 있지만, 아무 이동도 필요 없는 경우보다 작업이 늘어난다. 지역 객체를 반환할 때는 return result;를 사용하고 복사 생략과 암시적 이동 규칙에 맡긴다.

5.9 특별 멤버 함수의 설계

C++ 클래스에는 객체의 생성, 복사, 이동과 소멸을 담당하는 특별 멤버 함수가 있다.

특별 멤버 함수	기본 형태	역할
기본 생성자	T()	객체의 기본 초기화
소멸자	~T()	객체 수명 종료 처리
복사 생성자	T(const T&)	기존 객체로 새 객체 생성
복사 대입 연산자	T& operator=(const T&)	기존 객체에 다른 객체의 상태 복사
이동 생성자	T(T&&)	기존 객체의 자원을 새 객체로 이전
이동 대입 연산자	T& operator=(T&&)	기존 객체의 자원을 다른 기존 객체로 이전

컴파일러가 제공하는 기본 동작을 사용할지, 직접 구현할지, 호출을 금지할지는 클래스의 멤버 구성과 설계 의도에 따라 결정한다.

5.9.1 특별 멤버 함수의 암시적 선언과 삭제

사용자가 특별 멤버 함수를 선언하지 않으면 컴파일러가 조건에 따라 해당 함수를 암시적으로 선언할 수 있다. 암시적으로 선언된 함수도 멤버나 기반 클래스의 동작 때문에 삭제된 함수가 될 수 있다.

기본 생성자
앞의 Point를 이번에는 기본 생성자의 관점에서 살펴보자. 사용자가 생성자를 하나도 선언하지 않았으므로 컴파일러가 기본 생성자를 암시적으로 선언할 수 있다.

```
struct Point { int x, y; };
Point point;
```

사용자가 매개변수가 있는 생성자를 선언하면 기본 생성자가 자동으로 선언되지 않는다.

```
class Length
{
public:
    explicit Length(int value);
};

// Length length;  // 오류
```

소멸자
사용자가 소멸자를 선언하지 않으면 컴파일러가 소멸자를 암시적으로 선언한다. 각 멤버와 기반 클래스의 소멸자가 호출되도록 구성된다.

복사 생성자와 복사 대입 연산자
사용자가 해당 복사 함수를 선언하지 않으면 멤버 단위 복사를 수행하는 함수가 암시적으로 선언될 수 있다. 복사할 수 없는 멤버가 있으면 삭제될 수 있다.

```
class Locked
{
public:
    Locked() = default;
    Locked(const Locked&) = delete;
};

class Object
{
private:
    Locked lock;
};

// Object copy = object;  // Object의 복사 생성자도 사용할 수 없음
```

참조 멤버나 일부 상수 멤버를 가진 클래스는 암시적 복사 대입 연산자를 사용할 수 없을 수 있다. 대입은 이미 결합된 참조 대상을 다른 객체로 바꾸거나 상수 멤버에 값을 대입할 수 없기 때문이다.

이동 생성자와 이동 대입 연산자
이동 함수는 다음 특별 멤버 함수가 사용자에 의해 선언되지 않은 경우에 암시적으로 선언될 수 있다.

```
- 복사 생성자
- 복사 대입 연산자
- 이동 생성자
- 이동 대입 연산자
- 소멸자
```

사용자가 소멸자를 직접 선언하면 이동 생성자와 이동 대입 연산자가 자동으로 선언되지 않을 수 있다.

```
class Data
{
public:
    ~Data()
    {
    }

private:
    std::string value;
};
```

Data의 빈 소멸자가 특별한 작업을 하지 않더라도 사용자가 소멸자를 선언했다는 사실이 이동 함수의 암시적 선언에 영향을 준다. 기본 소멸 동작이 충분하다면 소멸자를 선언하지 않는 것이 좋다. 인터페이스상 명시해야 한다면 필요한 이동 함수를 = default로 선언할 수 있다.

```
class Data
{
public:
```

```
    Data() = default;
    ~Data() = default;

    Data(const Data&) = default;
    Data& operator=(const Data&) = default;
    Data(Data&&) noexcept = default;
    Data& operator=(Data&&) noexcept = default;

private:
    std::string value;
};
```

특별 멤버 함수의 정확한 생성 조건은 서로 영향을 주므로, 복사와 이동 정책이 중요한 클래스에서는 의도한 함수를 명시하는 편이 좋다.

함수	컴파일러 기본 동작이 제공될 수 있는 조건의 핵심	사용할 수 없게 되는 대표적인 원인
기본 생성자	사용자가 다른 생성자를 선언하지 않음	멤버나 기반 클래스를 기본 생성할 수 없음
소멸자	사용자가 소멸자를 선언하지 않음	멤버나 기반 클래스의 소멸자를 사용할 수 없음
복사 생성자	사용자가 복사 생성자를 선언하지 않음	멤버나 기반 클래스를 복사할 수 없음
복사 대입	사용자가 복사 대입을 선언하지 않음	참조·상수 멤버 또는 대입 불가능한 하위 객체
이동 생성자	복사·이동 함수와 소멸자를 사용자가 선언하지 않음	멤버나 기반 클래스를 이동할 수 없음
이동 대입	복사·이동 함수와 소멸자를 사용자가 선언하지 않음	멤버나 기반 클래스에 이동 대입할 수 없음

이 표는 설계에 필요한 핵심 규칙을 요약한 것이다. 템플릿, 상속과 접근 제어가 포함되면 삭제 여부를 결정하는 조건이 더 세분화될 수 있다.

5.9.2 Rule of Five

C++11 이후 자원을 직접 소유하는 클래스는 이동 생성자와 이동 대입 연산자까지 함께 검토해야 한다. 소멸자, 복사 생성자, 복사 대입 연산자, 이동 생성자와 이동 대입 연산자의 관계를 Rule of Five라고 한다.

```
class IntArray
{
public:
    ~IntArray();

    IntArray(const IntArray& other);
    IntArray& operator=(const IntArray& other);

    IntArray(IntArray&& other) noexcept;
    IntArray& operator=(IntArray&& other) noexcept;
};
```

함수	역할
소멸자	소유한 자원 해제
복사 생성자	새 객체에 독립된 자원 복제
복사 대입 연산자	기존 자원을 교체하며 내용 복제
이동 생성자	새 객체로 자원 소유권 이전
이동 대입 연산자	기존 자원을 정리하고 소유권 이전

Rule of Five도 다섯 함수를 무조건 모두 직접 구현하라는 의미는 아니다. 직접 자원 관리를 선택했다면 복사와 이동을 포함한 객체의 모든 주요 수명 연산을 함께 검토해야 한다.

5.9.3 Rule of Zero

자원 관리를 이미 올바르게 구현한 타입을 멤버로 사용하면 클래스가 특별 멤버 함수를 직접 작성하지 않아도 된다. 이를 Rule of Zero라고 한다.

```
class Inventory
{
public:
    void AddItem(const std::string& item)
    {
        items.push_back(item);
    }

private:
    std::string owner;
    std::vector<std::string> items;
};
```

std::string과 std::vector가 자신의 동적 메모리를 관리하고 복사와 이동을 제공한다. Inventory는 소멸자, 복사 생성자, 복사 대입, 이동 생성자와 이동 대입을 직접 작성할 필요가 없다.

```
Inventory first;
Inventory second = first;
Inventory third = std::move(first);
```

컴파일러가 각 멤버의 복사와 이동 동작을 조합하여 Inventory의 특별 멤버 함수를 제공할 수 있다.
Rule of Zero는 특별 멤버 함수를 절대 작성하지 말라는 의미가 아니다. 자원 소유를 전용 타입에 맡기고 일반적인 업무 클래스는 상태와 기능 설계에 집중하라는 원칙이다.

```
• 직접 자원 관리
- 업무 클래스가 new, delete와 특별 멤버 함수를 모두 관리

• Rule of Zero
- 자원 관리 타입이 소유권과 수명 관리
- 업무 클래스는 자원 관리 타입을 멤버로 사용
```

5.9.4 복사 금지 객체

객체를 복사하면 안 되는 타입은 복사 생성자와 복사 대입 연산자를 삭제한다.

```
class UniqueFile
{
public:
    UniqueFile() = default;

    UniqueFile(const UniqueFile&) = delete;
    UniqueFile& operator=(const UniqueFile&) = delete;
};
```

복사를 시도하면 컴파일 오류가 발생한다.

```
UniqueFile first;
UniqueFile second;

// UniqueFile third = first;  // 오류
// second = first;            // 오류
```

고유한 파일 핸들, 잠금 객체, 하나의 장치 연결, 복제할 수 없는 시스템 자원 등에 적용할 수 있다. 싱글톤은 생성자의 접근을 제한하고 복사 생성자와 복사 대입 연산자를 삭제하여 객체 복제를 막는 방식으로 구성할 수 있다.

5.9.5 이동 전용 객체

복사는 금지하지만 이동은 허용할 수 있다.

```
class UniqueBuffer
{
```

```
public:
    explicit UniqueBuffer(std::size_t size)
        : size(size), data(size > 0 ? new int[size]{} : nullptr)
    {
    }

    ~UniqueBuffer()
    {
        delete[] data;
    }

    UniqueBuffer(const UniqueBuffer&) = delete;
    UniqueBuffer& operator=(const UniqueBuffer&) = delete;

    UniqueBuffer(UniqueBuffer&& other) noexcept
        : size(other.size),
          data(other.data)
    {
        other.size = 0;
        other.data = nullptr;
    }

    UniqueBuffer& operator=(UniqueBuffer&& other) noexcept
    {
        if (this == &other)
        {
            return *this;
        }

        delete[] data;
        size = other.size;
        data = other.data;
        other.size = 0;
        other.data = nullptr;

        return *this;
    }

private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

이동 전용 객체는 하나의 소유자만 유지하면서 함수 반환이나 컨테이너 저장에 필요한 소유권 이전을 허용한다.

```
UniqueBuffer CreateBuffer()
{
    return UniqueBuffer(1024);
}

UniqueBuffer buffer = CreateBuffer();
```

표준 라이브러리의 std::unique_ptr도 대표적인 이동 전용 타입이다.

5.9.6 = default와 = delete의 적용

= default와 **= delete**는 특별 멤버 함수를 직접 구현하지 않고 의도한 동작을 명시할 때 활용한다. **= default**는 컴파일러가 기본 동작을 정의하도록 요청하며, 멤버나 기반 클래스 때문에 해당 동작을 만들 수 없으면 함수가 삭제된다. **= delete**는 특정 생성, 복사, 이동 또는 대입을 의도적으로 금지한다.
기본 복사가 클래스의 의미와 일치하면 명시적으로 기본 동작을 사용할 수 있다.

```
struct Point
{
    int x = 0, y = 0;

    Point() = default;
    Point(const Point&) = default;
    Point& operator=(const Point&) = default;
```

```
};
```

복사할 수 없는 멤버가 있으면 **= default**로 요청한 복사 함수도 삭제될 수 있다.

```
class NonCopyable
{
public:
    NonCopyable() = default;
    NonCopyable(const NonCopyable&) = delete;
};

class Holder
{
public:
    Holder() = default;
    Holder(const Holder&) = default; // 결과적으로 삭제됨.

private:
    NonCopyable value;
};
```

복사만 금지하고 이동은 허용할 수도 있다.

```
class Resource
{
public:
    Resource() = default;

    Resource(const Resource&) = delete;
    Resource& operator=(const Resource&) = delete;

    Resource(Resource&&) noexcept = default;
    Resource& operator=(Resource&&) noexcept = default;
};
```

기본 동작을 명시할 때는 각 멤버의 복사와 이동이 클래스 전체의 의미와 일치하는지 확인해야 한다. 원시 포인터가 자원을 소유하는 클래스에 복사 생성자를 **= default**로 지정하면 포인터 주소만 복사되어 얕은 복사가 발생한다.

<특별 멤버 함수를 적용한 IntArray - 복사와 이동 기능을 모두 적용>

```
// IntArray.h
#pragma once

#include <cstddef>

class IntArray
{
public:
    IntArray() = default;
    explicit IntArray(std::size_t size);
    ~IntArray();

    IntArray(const IntArray& other);
    IntArray& operator=(const IntArray& other);

    IntArray(IntArray&& other) noexcept;
    IntArray& operator=(IntArray&& other) noexcept;

    std::size_t GetSize() const;
    bool IsEmpty() const;

    void Set(std::size_t index, int value);
    int Get(std::size_t index) const;

private:
    std::size_t size = 0;
    int* data = nullptr;
};
```

```cpp
// IntArray.cpp
#include "IntArray.h"

IntArray::IntArray(std::size_t size)
    : size(size), data(size > 0 ? new int[size]{} : nullptr)
{
}

IntArray::~IntArray()
{
    delete[] data;
}

IntArray::IntArray(const IntArray& other)
    : size(other.size)
    , data(other.size > 0 ? new int[other.size] : nullptr)
{
    for (std::size_t i = 0; i < size; ++i)
    {
        data[i] = other.data[i];
    }
}

IntArray& IntArray::operator=(const IntArray& other)
{
    if (this == &other)
    {
        return *this;
    }

    int* newData = other.size > 0 ? new int[other.size] : nullptr;
    for (std::size_t i = 0; i < other.size; ++i)
    {
        newData[i] = other.data[i];
    }

    delete[] data;
    data = newData;
    size = other.size;

    return *this;
}

IntArray::IntArray(IntArray&& other) noexcept
    : size(other.size), data(other.data)
{
    other.size = 0;
    other.data = nullptr;
}

IntArray& IntArray::operator=(IntArray&& other) noexcept
{
    if (this == &other)
    {
        return *this;
    }

    delete[] data;
    size = other.size;
    data = other.data;
    other.size = 0;
    other.data = nullptr;

    return *this;
}

std::size_t IntArray::GetSize() const
{
    return size;
```

```
}

bool IntArray::IsEmpty() const
{
    return size == 0;
}

void IntArray::Set(std::size_t index, int value)
{
    if (index < size)
    {
        data[index] = value;
    }
}

int IntArray::Get(std::size_t index) const
{
    if (index < size)
    {
        return data[index];
    }

    return 0;
}

// main.cpp
#include <iostream>
#include <utility>

#include "IntArray.h"

void PrintArray(const IntArray& array)
{
    for (std::size_t i = 0; i < array.GetSize(); ++i)
    {
        std::cout << array.Get(i) << ' ';
    }

    std::cout << '\n';
}

int main()
{
    IntArray original(3);
    original.Set(0, 10);
    original.Set(1, 20);
    original.Set(2, 30);

    IntArray copied = original;
    copied.Set(0, 100);

    std::cout << "original: ";
    PrintArray(original);

    std::cout << "copied: ";
    PrintArray(copied);

    IntArray moved = std::move(copied);

    std::cout << "moved: ";
    PrintArray(moved);

    std::cout << std::boolalpha;
    std::cout << "copied is empty: "
              << copied.IsEmpty() << '\n';

    original = original;
    moved = IntArray(2);
```

```
    std::cout << "moved size: "
              << moved.GetSize() << '\n';
}

// 출력
// original: 10 20 30
// copied: 100 20 30
// moved: 100 20 30
// copied is empty: true
// moved size: 2
```

original과 copied는 서로 다른 배열을 소유한다. copied를 이동한 뒤에는 moved가 배열을 소유하고 copied는 빈 상태가 된다.
이 클래스는 복사와 이동의 원리를 확인하기 위해 Rule of Five를 직접 구현한다. 실제 프로그램에서는 같은 기능을 std::vector<int>에 맡겨 Rule of Zero를 적용하는 편이 안전하고 간결하다.

5.10 복사 생략과 값 반환

복사 생략(copy elision)은 객체를 초기화하거나 반환하는 과정에서 중간 객체의 복사나 이동을 생략하는 규칙이다. 객체를 값으로 반환한다고 해서 항상 임시 객체를 여러 번 만들고 복사하는 것은 아니다.

5.10.1 반환값 최적화

함수가 이름 없는 클래스 객체를 반환할 수 있다.

```
IntArray CreateArray(std::size_t size)
{
    return IntArray(size);
}
```

호출자는 반환값으로 객체를 초기화한다.

```
IntArray values = CreateArray(1000);
```

반환 객체를 함수 내부의 임시 저장 공간에 먼저 만들고 호출자에게 복사하는 대신 호출자의 결과 객체가 놓일 위치에 직접 생성할 수 있다.

```
CreateArray()의 반환 결과
          ↓ 직접 초기화
values
```

이러한 형태를 일반적으로 반환값 최적화(RVO)와 연결하여 설명한다.

5.10.2 이름 있는 반환값 최적화

이름이 있는 지역 객체를 반환할 때 적용할 수 있는 복사 생략을 이름 있는 반환값 최적화(Named Return Value Optimization: NRVO)라고 한다.

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);

    return result;
}
```

컴파일러는 result를 호출자의 결과 객체 위치에 직접 생성할 수 있다.

NRVO는 반드시 적용되는 규칙은 아니다. 여러 지역 객체 중 하나를 조건에 따라 반환하면 적용하기 어려울 수 있다.

```
IntArray SelectArray(bool selectFirst)
{
    IntArray first(10);
    IntArray second(20);
    if (selectFirst)
    {
        return first;
    }
```

```
    return second;
}
```

서로 다른 지역 객체가 반환 후보이므로 하나의 결과 위치에 어떤 객체를 직접 생성할지 정하기 어렵다. NRVO가 적용되지 않으면 이동 생성자가 사용될 수 있다.

5.10.3 보장된 복사 생략

C++17부터 같은 타입의 prvalue로 객체를 초기화하는 일부 상황에서는 복사나 이동 생성자를 호출하지 않고 결과 객체를 직접 초기화한다.

```
IntArray CreateArray(std::size_t size)
{
    return IntArray(size);
}

IntArray values = CreateArray(1000);
```

이 과정을 반드시 다음처럼 이해하면 부정확하다.

```
임시 IntArray 생성 → 반환 객체로 이동 → values로 다시 이동
```

반환 표현식의 prvalue가 최종 결과 객체를 직접 초기화할 수 있다. 이 경우 복사와 이동 생성자가 삭제되어 있어도 해당 초기화가 가능할 수 있다.

```
class Result
{
public:
    Result() = default;

    Result(const Result&) = delete;
    Result(Result&&) = delete;
};

Result CreateResult()
{
    return Result();
}

Result result = CreateResult();
```

다만 결과 객체의 소멸자는 접근 가능하고 삭제되지 않아야 한다.

5.10.4 지역 객체 반환과 암시적 이동

이름 있는 지역 객체의 NRVO가 적용되지 않더라도 반환 과정에서 이동 생성자가 선택될 수 있다.

```
IntArray CreateArray(std::size_t size)
{
    IntArray result(size);

    return result;
}
```

처리 가능성은 다음처럼 정리할 수 있다.

```
NRVO 적용
└ result가 결과 위치에 직접 생성됨

NRVO 미적용
└ 반환 대상 지역 객체를 이동 가능한 대상으로 처리하여 이동 시도
   └ 이동이 불가능하면 조건에 따라 복사 시도
```

따라서 지역 객체를 반환하기 위해 직접 std::move()를 붙일 필요가 없다.

```cpp
return result;            // 권장
return std::move(result); // NRVO를 방해할 수 있음
```

지역 객체가 아닌 함수 매개변수, 다른 객체의 멤버 또는 전역 객체를 반환하는 경우에는 적용되는 규칙이 다를 수 있다. 소유권을 이전하려는 인터페이스인지 별도로 판단해야 한다.

5.10.5 값 반환의 비용

큰 객체를 값으로 반환하면 항상 느리다는 설명은 모던 C++에 그대로 적용되지 않는다.

```cpp
IntArray CreateArray(std::size_t size)
{
    return IntArray(size);
}
```

복사 생략으로 반환 객체를 직접 생성할 수 있고, 복사 생략이 적용되지 않는 상황에서도 이동을 사용할 수 있다. 값 반환은 소유권과 수명을 분명하게 표현한다.

```cpp
IntArray CreateArray(std::size_t size);
```

호출자는 반환된 객체를 자신의 값으로 소유한다. 지역 객체의 포인터나 참조를 반환하여 수명 오류를 만드는 것보다 안전하다.

```cpp
// 잘못된 설계.
const IntArray& CreateArray()
{
    IntArray local(10);
    return local;
}
```

작은 값 타입과 새로 생성한 소유 객체는 값으로 반환하는 방식을 우선 검토한다.

5.10.6 불필요한 임시 객체 줄이기

복사 생략이 모든 복사 비용을 자동으로 없애는 것은 아니다. 함수의 매개변수와 반환 형식, 호출 방식에 따라 불필요한 복사가 발생할 수 있다.
읽기만 하는 큰 객체는 상수 참조로 전달한다.

```cpp
void PrintArray(const IntArray& array);
```

함수가 전달받은 값을 자신의 멤버로 소유해야 한다면 값을 매개변수로 받은 뒤 이동하는 방식을 사용할 수 있다.

```cpp
class Player
{
public:
    explicit Player(std::string name)
        : name(std::move(name))
    {
    }
private:
    std::string name;
};
```

lvalue를 전달하면 매개변수 객체를 만들기 위해 한 번 복사하고 멤버로 이동한다.

```cpp
std::string name = "Knight";
Player first(name);
```

rvalue를 전달하면 매개변수와 멤버 사이에서 이동을 활용할 수 있다.

```cpp
Player second("Mage");
```

이 방식은 항상 최적이라는 뜻은 아니다. lvalue 호출이 매우 많고 복사를 한 번만 수행해야 한다면 상수 참조와 우측값 참조 오버로드를 각각 제공하는 방법도 있다. 인터페이스의 단순성과 성능 요구를 함께 고려한다.

상황	일반적인 선택
작은 값 전달	값 전달
큰 객체를 읽기만 함	const 참조 전달
함수가 전달받은 객체의 복사본을 소유	값 전달 후 이동 또는 별도 오버로드
새 값을 생성하여 반환	값 반환
기존 객체 일부를 읽기 전용으로 노출	수명이 보장되는 const 참조 반환
소유권 이전	우측값 참조와 이동

단순히 복사를 피하려고 포인터나 참조를 무분별하게 저장하면 객체 수명과 소유권 문제가 더 커질 수 있다. 값 의미가 자연스러운 타입은 값으로 사용하고, 공유나 비소유 관계가 필요한 경우에만 참조 의미를 선택한다.

5.11 학습 정리

객체의 값이 같다는 사실과 같은 객체를 나타낸다는 사실은 구분해야 한다. 값 의미를 가지는 타입은 복사한 객체가 원본과 독립적인 값을 가지도록 설계한다. 자원을 직접 소유하는 객체는 포인터 값뿐만 아니라 포인터가 가리키는 자원까지 복사 의미에 포함해야 한다.

컴파일러가 제공하는 복사는 각 멤버의 복사 동작을 사용한다. 값을 직접 저장하는 멤버에는 자연스럽지만, 자원을 소유하는 원시 포인터에는 얕은 복사와 이중 해제 문제가 발생할 수 있다. 깊은 복사, 공유 소유, 비소유 참조와 복사 금지 중 타입의 의미에 맞는 복사 정책을 선택해야 한다.
복사 생성자는 기존 객체로 새로운 객체를 초기화하고, 복사 대입 연산자는 이미 존재하는 객체의 값을 교체한다. 자원을 직접 소유하는 클래스는 소멸자, 복사 생성자와 복사 대입 연산자를 하나의 정책으로 검토해야 하며 이를 Rule of Three라고 한다.

표현식의 값 범주는 객체의 정체성과 이동 가능성을 나타낸다. 이름 있는 객체 표현식은 일반적으로 lvalue이고, 계산 결과는 prvalue가 될 수 있으며, std::move()는 객체를 이동 가능한 xvalue로 취급하게 한다. 우측값 참조 타입의 변수도 이름을 사용한 표현식은 lvalue이다.

이동 생성자와 이동 대입 연산자는 자원의 내용을 복제하지 않고 소유권을 이전한다. 이동된 원본 객체는 여전히 유효하며 정상적으로 소멸하거나 새로운 값을 대입할 수 있어야 한다. 표준 라이브러리 타입의 이동 후 값은 별도의 보장이 없다면 지정되지 않은 상태로 취급한다.

std::move()는 자원을 직접 이동하지 않고 오버로드 해석에서 이동 함수가 선택될 수 있도록 값 범주를 변환한다. 이동 함수가 예외를 발생시키지 않는다면 noexcept를 명시하여 표준 컨테이너가 재배치 과정에서 이동을 활용할 수 있게 한다.

C++11 이후에는 Rule of Five를 통해 복사와 이동을 함께 검토한다. 자원 관리 타입을 멤버로 사용하면 특별 멤버 함수를 직접 작성하지 않는 Rule of Zero를 적용할 수 있다. 복사를 허용하지 않는 타입은 복사 함수를 = delete로 선언하고, 기본 동작이 의미에 맞으면 = default를 활용한다.

모던 C++은 반환값 최적화, 이름 있는 반환값 최적화와 보장된 복사 생략을 제공한다. 새 값을 만드는 함수는 값으로 반환하고, 이름 있는 지역 객체를 반환할 때 불필요한 std::move()를 사용하지 않는다.

<gvalue, rvalue, lvalue, xvalue, prvalue 요약>

값범주		의미	객체 식별	move 가능	decltype ((expr))	샘플코드

glvalue
- 객체 식별(identity) 기준
- 객체 식별 가능한 표현식
- lvalue ∪ xvalue
	lvalue	glvalue 중에서 move 불가	0	X	T&	value; // lvalue : 이름 있는 객체의 표현식
						GetReference(); // lvalue : int& 반환 함수 호출 결과
						reference; // lvalue 참조 변수
						rvalueReference; // 이름이 있는 rvalue 참조 변수

	xvalue
	glvalue ∩ rvalue
		glvalue, rvaue 중에서 객체 식별 가능하고, move 가능한 표현식	0	0	T&&	std::move(value); // xvalue : 객체 식별과 move 가능
						GetRvalueReference(); // xvalue : int&& 반환 함수 호출 결과

rvalue
- move 기준
- move 가능한 표현식
- prvalue ∪ xvalue
	prvalue	rvalue 중에서 값을 생성하는 표현식	X	0	T	100 ; // 리터럴
						value + 10; // 연산 결과
						GetValue(); // int 반환

5.12 학습 과제

과제 **1.** 동적 배열의 깊은 복사
정수 배열을 동적으로 관리하는 IntArray 클래스를 작성한다.
필수 데이터 멤버

```
std::size_t size;
int* data;
```

필수 기능

- 인수로 전달한 크기만큼 동적 배열 생성
- 모든 원소를 0으로 초기화
- 소멸자에서 배열 해제
- Set()과 Get()으로 원소 변경 및 조회
- GetSize()로 배열 크기 반환
- 복사 생성자에서 깊은 복사
- 복사 대입 연산자에서 깊은 복사
- 자기 대입 안전성 보장
- 서로 다른 크기의 배열 대입 지원

```
// 확인 코드
IntArray first(3);
first.Set(0, 10);
first.Set(1, 20);
first.Set(2, 30);

IntArray second = first;
second.Set(0, 100);

std::cout << first.Get(0) << '\n';
std::cout << second.Get(0) << '\n';

IntArray third(10);
third = first;

std::cout << third.GetSize() << '\n';

third = third;
std::cout << third.Get(1) << '\n';
```

원본과 복사본은 서로 다른 배열을 소유해야 한다. 자기 대입 후에도 데이터가 유지되어야 하며 프로그램 종료 시 이중 해제가 발생하지 않아야 한다.

과제 **2.** 이동을 지원하는 **ImageBuffer**
픽셀 데이터를 동적으로 저장하는 ImageBuffer 클래스를 작성한다.
상태

- 이미지 너비
- 이미지 높이
- 픽셀 수만큼 동적 할당한 unsigned int 배열

필수 생성자

```
ImageBuffer(int width, int height);
```

너비와 높이가 0 이하이면 빈 이미지로 생성한다.

필수 특별 멤버 함수

- 소멸자
- 복사 생성자
- 복사 대입 연산자
- 이동 생성자
- 이동 대입 연산자

이동 생성자와 이동 대입 연산자에는 noexcept를 지정한다. 이동 후 원본 객체는 너비와 높이가 0이고 픽셀 포인터가 nullptr인 상태로 만든다.

```
// 필수 멤버 함수
int GetWidth() const;
int GetHeight() const;
bool IsEmpty() const;
void SetPixel(int x, int y, unsigned int color);
unsigned int GetPixel(int x, int y) const;
```

좌표가 범위를 벗어나면 SetPixel()은 아무 작업도 하지 않고 GetPixel()은 0을 반환한다.
확인 항목

- 복사본의 픽셀을 변경해도 원본이 유지되는가
- 이동 후 대상 객체가 원래 픽셀을 소유하는가
- 이동 후 원본에 새로운 이미지를 대입할 수 있는가
- 자기 복사 대입과 자기 이동 대입에서 오류가 발생하지 않는가
- 프로그램 종료 시 이중 해제와 메모리 누수가 없는가

과제 **3.** 복사와 이동 호출 추적
특별 멤버 함수가 호출될 때 함수 이름과 객체 번호를 출력하는 TraceObject 클래스를 작성한다.
출력할 함수

- 기본 생성자
- 매개변수가 있는 생성자
- 복사 생성자
- 복사 대입 연산자
- 이동 생성자
- 이동 대입 연산자
- 소멸자

```
class TraceObject
{
private:
    int id;
};
```

다음 코드를 실행하고 호출 순서를 기록한다.

```
TraceObject first(1);
TraceObject second = first;
TraceObject third = std::move(first);

second = third;
third = TraceObject(4);
```

값을 반환하는 함수도 작성한다.

```
TraceObject CreateObject(int id);
```

Debug와 Release 구성에서 실행 결과를 비교한다. 복사 생략은 C++ 언어 규칙과 컴파일러의 선택에 영향을 받으므로, 출력 횟수가 예상과 다를 수 있는 이유를 설명한다.

과제 **4.** 복사 정책 분석
각 클래스에 적합한 복사 정책을 선택하고 이유를 작성한다.

1. 게임 좌표를 저장하는 Position
2. 하나의 파일 핸들을 소유하는 FileHandle
3. 플레이어를 가리키는 화면 표시용 PlayerView
4. 여러 객체가 공동으로 사용하는 텍스처 리소스
5. 독립된 픽셀 데이터를 가져야 하는 Image
6. 동시에 하나의 스레드만 소유해야 하는 잠금 객체

선택할 수 있는 정책은 다음과 같다.

- 깊은 복사
- 공유 소유
- 비소유 참조
- 복사 금지
- 이동 전용

각 타입의 복사 생성자와 복사 대입 연산자를 = default, = delete 또는 사용자 정의 구현 중 어떤 방식으로 구성할지도 함께 작성한다.

과제 **5. Rule of Zero**를 적용한 캐릭터 인벤토리
직접 동적 메모리를 사용하지 않고 std::string과 std::vector를 사용하여 CharacterInventory 클래스를 작성한다.
상태

- 캐릭터 이름
- 골드
- 아이템 이름 목록

```
//필수 기능
void AddItem(const std::string& item);
bool RemoveItem(const std::string& item);
bool Contains(const std::string& item) const;
void AddGold(int amount);
void Print() const;
```

소멸자, 복사 생성자, 복사 대입 연산자, 이동 생성자와 이동 대입 연산자를 직접 작성하지 않는다.
원본 인벤토리를 복사한 뒤 복사본에서 아이템을 추가하거나 제거해도 원본 목록은 유지되어야 한다. 시작 아이템을 가진 인벤토리를 값으로 반환하는 함수도 작성한다.

```
CharacterInventory CreateStarterInventory(const std::string& characterName);
```

함수에서 지역 객체를 만들고 값으로 반환한다. 반환문에는 std::move()를 사용하지 않는다.

과제 **6.** 이동 전용 로그 세션
하나의 로그 파일 연결을 소유한다고 가정한 LogSession 클래스를 설계한다. 실제 파일 입출력 대신 세션 번호와 열림 상태만 관리해도 된다.

설계 조건

- 생성자에서 세션을 열린 상태로 만듦
- 소멸자에서 세션을 닫음
- 복사 생성자 삭제
- 복사 대입 연산자 삭제
- 이동 생성자 허용
- 이동 대입 연산자 허용
- 이동 후 원본은 닫힌 상태로 변경
- IsOpen()으로 상태 확인

```
LogSession first(1);
// LogSession second = first;  // 컴파일 오류
LogSession second = std::move(first);
```

복사가 금지되는 이유와 이동은 허용할 수 있는 이유를 코드 주석으로 설명한다. 함수에서 LogSession을 값으로 반환하고 복사 생략 또는 이동을 통해 호출자에게 전달되는 과정도 확인한다.

06. 연산자 오버로딩과 사용자 정의 타입

기본형은 언어가 제공하는 연산자를 사용하여 값을 계산하고 비교한다.

```
int first = 10;
int second = 20;
int sum = first + second;
bool same = first == second;
```

사용자 정의 타입도 타입의 의미에 맞는 연산을 가질 수 있다. 3차원 벡터는 두 벡터를 더하거나 스칼라를 곱할 수 있고, 문자열과 동적 배열은 인덱스를 이용해 원소에 접근할 수 있다. 버전 정보는 순서를 비교할 수 있으며, 좌표나 날짜는 스트림을 통해 출력할 수 있다.
일반 멤버 함수로도 이러한 기능을 구현할 수 있다.

```
Vector3 result = first.Add(second);
```

벡터 덧셈을 +로 표현하면 계산의 구조가 더 분명하게 드러난다.

```
Vector3 result = first + second;
```

연산자 오버로딩은 클래스와 열거형에 기존 C++ 연산자의 동작을 정의하는 기능이다. 새로운 연산자 기호를 만드는 기능은 아니며, 연산자의 우선순위와 피연산자 개수도 변경하지 않는다. 기본형에서 사용하던 연산자의 의미를 사용자 정의 타입에 자연스럽게 확장할 때 활용한다.
5장에서는 값 의미를 가지는 타입이 복사와 이동을 통해 독립적인 값을 구성하는 방법을 살펴보았다. 이 장에서는 그 값에 산술 연산, 비교, 인덱스 접근, 형 변환과 스트림 입출력을 제공하여 완성된 사용자 정의 타입으로 구성한다.

6.1 연산자 오버로딩의 목적

연산자 오버로딩은 문법을 짧게 만드는 기능이 아니다. 타입이 나타내는 개념에 맞는 연산을 제공하여 코드가 문제 영역의 의미를 직접 표현하도록 만드는 기능이다.

6.1.1 사용자 정의 타입과 연산

3차원 벡터를 나타내는 Vector3를 정의한다. Vector3는 수학 벡터의 각 성분에 자주 직접 접근하므로 x, y, z를 public으로 둔다. 각 성분을 직접 변경해도 타입의 유효성이 손상되지 않는다.

```
class Vector3
{
public:
    Vector3(float x, float y, float z) : x(x), y(y), z(z)
    { }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

일반 함수로 벡터 덧셈을 구현할 수 있다.

```
Vector3 Add(const Vector3& first, const Vector3& second);
```

호출 코드는 함수 이름을 통해 연산의 목적을 나타낸다.

```
Vector3 result = Add(first, second);
```

벡터 덧셈은 수학에서 사용하는 +와 의미가 일치하므로 연산자로 표현할 수 있다.

```
Vector3 result = first + second;
```

연산자 표현은 여러 계산이 연결될 때 장점이 더 분명해진다.

```
Vector3 position = start + velocity * deltaTime;
```

같은 계산을 일반 함수로만 표현하면 호출이 중첩된다.

```
Vector3 position = Add(start, Multiply(velocity, deltaTime));
```

두 표현 중 어느 것이 항상 우수한 것은 아니다. 타입에서 자연스럽게 기대할 수 있는 연산은 연산자로 제공하고, 동작의 의미가 특별하거나 여러 해석이 가능한 경우에는 이름이 있는 함수를 사용하는 편이 좋다.

연산자 오버로딩은 값 타입에서 특히 유용하다.

```
- 벡터와 행렬의 산술 연산
- 복소수와 분수의 계산
- 날짜와 시간 간격의 덧셈과 뺄셈
- 문자열과 경로의 연결
- 버전과 식별자의 비교
- 컨테이너와 고정 크기 배열의 원소 접근
- 스트림을 이용한 사용자 정의 타입의 입출력
```

6.1.2 기존 연산자 의미 유지

사용자 정의 타입에 연산자를 제공할 때는 기본형에서 형성된 기대를 유지해야 한다.

+는 두 값을 사용하여 새로운 값을 만든다.

```
Vector3 result = first + second;
```

이 연산으로 **first**나 **second**가 변경될 것이라고 예상하지 않는다.
+=는 왼쪽 객체의 값을 변경한다.

```
first += second;
```

==는 두 값이 같은지를 확인하며, 비교 과정에서 피연산자는 변경을 안한다.

```
bool same = first == second;
```

[]는 순서나 키를 이용해 요소에 접근한다.

```
vec[0] = 10.0f;
```

<<는 값을 출력 스트림에 기록하고 스트림을 반환한다.

```
std::cout << vec << '\n';
```

<연산자에 기대되는 일반적인 의미>

연산자	일반적인 의미	피연산자 변경
+, -, *, /	계산 결과를 새로운 값으로 생성	변경하지 않음
+=, -=, *=, /=	오른쪽 값을 현재 객체에 반영	왼쪽 객체 변경
==, !=	값의 동등성 비교	변경하지 않음
<, <=, >, >=	값의 순서 비교	변경하지 않음
[]	요소 접근	반환 참조에 따라 가능
++, --	현재 값을 한 단계 증가 또는 감소	현재 객체 변경
<<, >>	스트림 출력과 입력	스트림 또는 입력 대상 변경

연산자의 의미를 크게 바꾸면 표현은 짧아져도 코드를 이해하기 어려워진다.

```
// 피해야 할 설계의 예
Player result = first + second;
```

Player 두 객체의 덧셈이 어떤 의미인지 분명하지 않다면 **MergeParty(), CombineStats()**처럼 목적을 나타내는 함수를 사용하는 편이 낫다.

연산 비용도 연산자에서 예상할 수 있는 범위를 고려해야 한다. **+**가 네트워크 요청이나 파일 저장을 수행한다면 호출자는 식만 보고 큰 비용과 외부 효과를 알아차리기 어렵다.

6.1.3 오버로딩 규칙과 오버로딩할 수 없는 연산자

연산자 오버로딩에는 언어가 정한 제한이 있다. 하나 이상의 피연산자가 사용자 정의 타입이어야 한다
기본형끼리의 연산을 다시 정의할 수 없다.

```
// int operator+(int first, int second);    // 허용되지 않음
```

클래스나 열거형이 하나 이상 포함되어야 한다.

```
Vector3 operator+(const Vector3& first, const Vector3& second);
Vector3 operator*(const Vector3& vec3, float scalar);
```

새로운 연산자 기호를 만들 수 없다

```
**
<->
%%
```

C++에 존재하지 않는 기호를 새로운 연산자로 정의할 수 없다.

피연산자 개수를 변경할 수 없다
단항 -는 피연산자 하나, 이항 -는 피연산자 두 개를 가진다. 오버로딩하더라도 이 구조는 유지된다.

```
Vector3 operator-() const;                          // 단항
Vector3 operator-(Vector3 left, const Vector3& right); // 이항
```

우선순위와 결합 방향을 변경할 수 없다

```
Vector3 result = first + second * scalar;
```

사용자 정의 타입에서도 *가 +보다 먼저 계산된다. 연산자 오버로딩은 기존 문법의 우선순위를 바꾸지 않는다.
기본 연산자의 평가 특성을 모두 유지하는 것은 아니다.

&&와 ||를 오버로딩하면 기본 논리 연산자의 단락 평가를 기대할 수 없다. 두 피연산자가 모두 평가된 뒤 함수가 호출될 수 있으므로 일반적인 클래스에서는 오버로딩하지 않는 편이 좋다.

<오버로딩 불가능한 연산자>

연산자 또는 문법	의미
::	범위 지정
.	직접 멤버 접근
.*	멤버 포인터를 이용한 직접 접근
?:	조건 연산자
sizeof	객체 또는 타입의 크기 확인
alignof	타입의 정렬 요구 사항 확인
typeid	런타임 타입 정보 확인
noexcept	표현식의 예외 발생 가능성 확인

형 변환 문법 자체를 오버로딩할 수는 없지만 변환 생성자와 형 변환 연산자를 통해 사용자 정의 변환을 제공할 수 있다.
C++20 기준으로 다음 연산자는 멤버 함수로 구현해야 한다.

연산자	기능
=	대입
[]	인덱스 또는 키(key)를 이용한 원소 접근
()	함수 호출

(*)., ->	포인터 형태의 멤버 접근

함수 호출 연산자는 함수 객체와 람다를 다루는 장에서 자세히 설명한다.

6.2 멤버 연산자와 비 멤버 연산자

연산자 함수는 멤버 함수 또는 비 멤버 함수로 정의할 수 있다. 구현 위치에 따라 피연산자를 전달하는 방식과 암시적 형 변환이 적용되는 범위가 달라진다.

6.2.1 멤버 함수로 구현하는 연산자

멤버 함수로 구현한 이항 연산자는 왼쪽 피연산자를 현재 객체로 사용한다.

```
class Vector3
{
public:
    Vector3& operator+=(const Vector3& rhs)
    {
        x += rhs.x;
        y += rhs.y;
        z += rhs.z;
        return *this;
    }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

호출할 때는 기본형의 복합 대입 연산자 **"+="** 와 같은 형태로 사용한다.

```
first += second;
```

함수 호출 형태로 표현하면 first.operator+=(second)와 같다.

```
first.operator+=(second);
```

first는 멤버 함수를 호출한 현재 객체이고, second는 매개변수로 전달된다.

단항 연산자는 현재 객체 하나에 적용되므로 별도의 피연산자 매개변수를 받지 않는다.

```
class Vector3
{
public:
    Vector3 operator-() const
    {
        return Vector3(-x, -y, -z);
    }
public:
    float x=0.0f, y=0.0f, z=0.0f;
};

Vector3 reversed = -position;
```

이 표현은 개념적으로 position.operator-()를 호출한다.

현재 객체의 상태를 변경하지 않는 연산자는 상수 멤버 함수로 선언한다.

```
Vector3 operator-() const;
bool operator==(const Vector3& rhs) const;
float operator[](std::size_t index) const;
```

현재 객체를 변경하는 연산자는 일반 멤버 함수로 선언한다.

```
Vector3& operator+=(const Vector3& rhs);
float& operator[](std::size_t index);
```

대입, 복합 대입과 인덱스 접근처럼 왼쪽 객체의 상태 변경이나 원소 접근과 밀접한 연산은 멤버 함수로 구현하는 것이 자연스럽다.

6.2.2 비 멤버 함수로 구현하는 연산자

비 멤버 연산자 함수는 모든 피연산자를 매개변수로 받는다.

```
Vector3 operator+(Vector3 left, const Vector3& right)
{
    left += right;
    return left;
}
```

이제 **+=** 연산자를 쓸 수 있다.

```
Vector3 result = first + second;
```

함수 호출 형태로 표현하면 operator+(first, second)와 같다.

```
Vector3 result = operator+(first, second);
```

left는 first의 복사본이다. 복사본에 right를 더한 뒤 반환하므로 원본 피연산자는 변경되지 않는다.

비 멤버 함수는 왼쪽과 오른쪽 피연산자를 모두 매개변수로 받으므로 피연산자를 대칭적으로 처리하기 쉽다.

```
Vector3 operator*(Vector3 vec3, float scalar)
{
    vec3 *= scalar;
    return vec3;
}

Vector3 operator*(float scalar, Vector3 vec3)
{
    vec3 *= scalar;
    return vec3;
}
```

```
Vector3 first = vec3Value * 2.0f;
Vector3 second = 2.0f * vec3Value;
```

vec3 * scalar와 scalar * vec3 는 피연산자의 순서가 다르므로 각각의 연산자 함수가 필요하다. C++은 한쪽 순서의 연산자를 정의했다고 해서 반대 순서의 연산자를 자동으로 생성하지 않는다.

scalar * vec3 는 왼쪽 피연산자가 float이므로 Vector3의 멤버 함수로 구현할 수 없다. 모든 피연산자를 매개변수로 받는 비 멤버 함수로 정의해야 한다.

위 함수는 공개된 operator*=를 사용하므로 Vector3의 멤버에 직접 접근하지 않는다. 따라서 friend가 필요하지 않다. 비 멤버 함수가 클래스의 비공개 멤버에 직접 접근해야 한다면 friend로 선언할 수 있다.

클래스와 같은 이름 공간에 비 멤버 연산자를 선언하면 인수 의존 이름 탐색을 통해 피연산자의 타입과 관련된 연산자를 찾을 수 있다. 지금은 클래스와 관련된 비 멤버 연산자를 클래스 선언과 가까운 위치에 함께 선언한다고 이해하면 충분하다.

6.2.3 멤버 함수와 비 멤버 함수의 선택

연산자를 멤버 함수와 비 멤버 함수 중 어느 형태로 구현할지는 연산의 성격과 피연산자의 배치를 기준으로 판단한다.

복합 대입처럼 왼쪽 객체를 직접 변경하는 연산은 멤버 함수가 자연스럽다.

```
first += second;
```

두 피연산자를 같은 위치에서 처리하거나 왼쪽 피연산자가 클래스 타입이 아닐 수 있는 이항 연산은 비 멤버 함수가 적합하다.

```
Vector3 result1 = vec3 * 2.0f;
Vector3 result2 = 2.0f * vec3;
```

피연산자의 순서를 바꾸어 구현을 재사용할 수 있는지는 연산의 의미에 따라 결정한다. 벡터와 스칼라의 곱은 순서를 바꾸어도 같은 의미를 가질 수 있지만, 행렬 곱셈처럼 순서에 따라 결과가 달라지는 연산은 피연산자를 바꾸어서는 안 된다.

```
Matrix result = first * second;
```

행렬의 곱셈, first * second와 second * first의 결과는 서로 다르다.

<멤버 함수와 비 멤버 함수의 선택>

기준	멤버 함수	비 멤버 함수
왼쪽 피연산자	현재 객체로 고정	매개변수로 전달
왼쪽 피연산자의 암시적 변환	제한적	적용 가능
비공개 멤버 접근	직접 가능	friend 또는 공개 인터페이스 필요
대칭적인 이항 연산	덜 유연함	적합함
상태 변경 연산	적합함	멤버가 자연스러움

friend는 비 멤버 연산자를 구현하기 위한 필수 문법이 아니다. 공개된 멤버 함수와 공개 상태만으로 연산을 구현할 수 있다면 일반 비 멤버 함수로 정의한다.

6.3 복합 대입 연산자와 산술 연산자

복합 대입 연산자를 먼저 구현하고 이항 산술 연산자가 이를 재사용하도록 구성하면 연산 규칙을 한곳에 모을 수 있다.

6.3.1 복합 대입 연산자

Vector3에 덧셈, 뺄셈과 스칼라 곱의 복합 대입 연산자를 추가한다.

```
class Vector3
{
public:
    Vector3& operator+=(const Vector3& rhs)
    {
        x += rhs.x;
        y += rhs.y;
        z += rhs.z;
        return *this;
    }
    Vector3& operator-=(const Vector3& rhs)
    {
        x -= rhs.x;
        y -= rhs.y;
        z -= rhs.z;
        return *this;
    }
    Vector3& operator*=(float scalar)
    {
        x *= scalar;
        y *= scalar;
        z *= scalar;
        return *this;
    }

public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

복합 대입 연산자는 현재 객체를 변경하고 현재 객체의 참조를 반환한다.

```
first += second;
first *= 2.0f;
```

참조를 반환하면 연속된 표현을 구성할 수 있다.

```
(first += second) *= 2.0f;
```

이 표현은 사용할 수 있지만 복잡한 연산을 한 문장에 과도하게 연결하면 읽기 어려울 수 있다. 참조 반환의 주된 목적은 기본형의 복합 대입 연산자와 같은 반환 규칙을 제공하는 데 있다.

복합 대입 연산자는 객체의 유효한 상태를 유지해야 한다. 분수 타입의 /=에서 분모가 0이 되거나, 자원 크기를 나타내는 타입에서 음수가 되는 연산은 검증이 필요하다.

```
class Size
{
public:
    Size& operator-=(int amount)
    {
        if (amount > value)
        {
            value = 0;
        }
        else
        {
            value -= amount;
        }
        return *this;
    }

private:
    int value = 0;
};
```

연산자라고 해서 클래스의 상태 규칙을 우회해서는 안 된다.

6.3.2 이항 산술 연산자

이항 +는 왼쪽 피연산자를 값으로 받아 복사본을 만든 뒤 +=를 적용할 수 있다.

```
Vector3 operator+(Vector3 left, const Vector3& right)
{
    left += right;
    return left;
}

Vector3 result = first + second;
```

left는 first의 복사본이다. 복사본을 변경하여 반환하므로 원본 피연산자는 유지된다.

5장에서 학습한 복사와 이동 규칙이 이 구현에 적용된다. 왼쪽 피연산자가 lvalue이면 left를 만들 때 복사가 발생할 수 있고, 임시 객체이면 이동을 활용할 수 있다. 반환 과정에서는 복사 생략이나 이동이 적용될 수 있다.
같은 방식으로 뺄셈과 스칼라 곱을 구현한다.

```
Vector3 operator-(Vector3 left, const Vector3& right)
{
    left -= right;
    return left;
}

Vector3 operator*(Vector3 vec3, float scalar)
{
    vec3 *= scalar;
    return vec3;
}
```

스칼라가 왼쪽에 놓이는 연산은 피연산자의 순서를 바꾸어 재사용할 수 있다.

```
Vector3 operator*(float scalar, Vector3 vec3)
{
```

```
    vec3 *= scalar;
    return vec3;
}
```

```
Vector3 first = vec3 * 2.0f;
Vector3 second = 2.0f * vec3;
```

두 함수가 모두 필요하다. C++은 vec3 * scalar를 정의했다고 해서 scalar * vec3를 자동으로 생성하지 않는다.
교환 법칙이 성립하지 않는 연산은 피연산자의 순서를 바꾸면 안 된다.

```
Matrix result = first * second;
```

행렬 곱셈은 first * second와 second * first의 결과가 다르다. 연산자의 수학적 성질도 타입의 의미에 맞게 유지해야 한다.

6.3.3 단항 연산자

단항 연산자는 하나의 피연산자에 적용된다.

```
Vector3 reversed = -vec3;
```

Vector3의 부호를 반전하는 단항 -를 멤버 함수로 구현할 수 있다.

```
class Vector3
{
public:
    Vector3 operator-() const
    {
        return Vector3(-x, -y, -z);
    }

public:
    float x=0.0f, y=0.0f, z=0.0f;
};
```

현재 객체를 변경하지 않고 새로운 벡터를 반환하므로 상수 멤버 함수로 선언한다.
단항 +는 객체를 그대로 값으로 반환하는 의미를 제공할 수 있다.

```
Vector3 operator+() const
{
    return *this;
}
```

타입에서 단항 +의 의미가 필요하지 않다면 굳이 제공할 필요는 없다.
논리 부정 !은 객체가 가진 상태의 반대 의미가 자연스러울 때 사용할 수 있다.

```
class ResourceHandle
{
public:
    bool operator!() const
    {
        return handle == nullptr;
    }

public:
    void* handle = nullptr;
};
```

다만 유효성 확인에는 explicit operator bool()이 더 자연스러운 경우가 많다. 형 변환 연산자 절에서 자세히 살펴본다.

6.3.4 증가와 감소 연산자

증가와 감소 연산자는 전위형과 후위형을 구분한다.

```
++counter;  // 전위 증가
```

```
counter++;  // 후위 증가
```

전위 연산자는 매개변수를 받지 않는다.

```
class Counter
{
public:
    Counter& operator++()
    {
        ++value;
        return *this;
    }

public:
    int value = 0;
};
```

후위 연산자는 전위형과 구분하기 위해 사용하지 않는 int 매개변수를 받는다.

```
class Counter
{
public:
    Counter operator++(int)
    {
        Counter previous = *this;
        ++(*this);
        return previous;
    }

public:
    int value = 0;
};
```

int 매개변수는 후위 연산자를 구분하기 위한 표시이며 호출할 때 값을 전달하지 않는다.
전위 증가 연산자는 증가된 현재 객체의 참조를 반환한다.

```
Counter operator++(int);
```

후위 증가 연산자는 증가 전 값을 반환해야 하므로 복사본을 값으로 반환한다.

```
Counter operator++(int);
```

<전위형과 후위형 증가 연산자>

구분	선언	반환값
전위 증가	T& operator++()	증가된 현재 객체
후위 증가	T operator++(int)	증가하기 전 값의 복사본
전위 감소	T& operator--()	감소된 현재 객체
후위 감소	T operator--(int)	감소하기 전 값의 복사본

후위 연산자는 이전 값을 보존해야 하므로 전위 연산자보다 복사 비용이 추가될 수 있다. 이전 값이 필요하지 않다면 전위형을 사용하는 편이 단순하다.
증가와 감소의 의미가 자연스러운 타입에만 제공한다. 날짜의 하루 증가, 반복자의 다음 요소 이동과 카운터 증가에는 적합하지만, 벡터 전체를 증가시키는 의미가 분명하지 않다면 제공하지 않는다.

6.4 비교 연산자

비교 연산자는 객체의 값 동일성과 순서를 표현한다. 어떤 상태를 비교에 포함할지는 타입의 의미에 따라 결정해야 한다.

6.4.1 동등 비교

두 객체가 같은 값을 나타내는지 확인하려면 operator==를 정의한다.

```
struct Point
{
    int x, y;
    bool operator==(const Point& rhs) const
    {
        return x == rhs.x && y == rhs.y;
    }
};

Point first{10, 20};
Point second{10, 20};
bool same = first == second;
```

비교 과정에서 객체를 변경하지 않으므로 상수 멤버 함수로 선언한다.

C++20에서는 적절한 operator==가 존재하면 != 표현식을 ==의 결과를 반전하는 형태로 처리할 수 있다.

```
bool different = first != second;
```

"==" 는 객체의 메모리 표현이 같은지를 비교하는 기능이 아니다. 객체가 나타내는 의미가 같은지를 비교한다.

```
class Character
{
private:
    int id;
    std::string name;
    int hp;
};
```

캐릭터의 동등성을 고유 식별자인 id로 판단할 수도 있고 모든 상태가 같은지를 기준으로 판단할 수도 있다. 비교 기준이 여러 가지라면 SameIdentity(), SameState()처럼 이름이 있는 함수를 제공하는 편이 명확하다.

부동소수점 값은 계산 오차 때문에 정확한 동등 비교가 적합하지 않을 수 있다.

```
bool Vector3::NearlyEquals(const Vector3& rhs, float epsilon) const
{
    return std::fabs(x - rhs.x) <= epsilon
        && std::fabs(y - rhs.y) <= epsilon
        && std::fabs(z - rhs.z) <= epsilon;
}
```

operator==를 허용 오차 비교로 정의하면 허용 오차의 선택과 추이성 문제가 생길 수 있다. 정확한 비교가 필요한지 근사 비교가 필요한지 구분하고, 근사 비교는 이름이 있는 함수로 제공하는 편이 좋다.

6.4.2 순서 비교

값 사이에 자연스러운 순서가 있다면 <, <=, >, >=를 제공할 수 있다.
버전 정보는 주 버전, 부 버전과 패치 버전을 차례로 비교할 수 있다.

```
struct Version
{
    int major = 0;
    int minor = 0;
    int patch = 0;
};
```

직접 <를 구현하면 각 비교 조건을 작성해야 한다.

```
bool operator<(const Version& rhs) const
{
    if (major != rhs.major)
    {
        return major < rhs.major;
    }
    if (minor != rhs.minor)
    {
```

```
        return minor < rhs.minor;
    }
    return patch < rhs.patch;
}
```

이 순서는 멤버를 사전식으로 비교한다.

```
Version first{1, 4, 2};
Version second{1, 5, 0};
bool older = first < second;
```

순서 비교는 정렬과 연관 컨테이너의 키 비교에 사용될 수 있다. 비교 함수는 일관된 순서를 제공해야 한다.

- 객체가 자기 자신보다 작다고 판단하지 않음
- a < b와 b < a가 동시에 참이 되지 않음
- a < b이고 b < c이면 a < c가 성립
- 동등한 값에 대해 비교 결과가 일관됨

모든 타입에 순서가 필요한 것은 아니다. 3차원 벡터를 크기, x 좌표 또는 사전식 순서 중 어떤 기준으로 정렬할지 명확하지 않다면 <를 정의하지 않고 비교 기준을 별도 함수 객체로 제공하는 편이 낫다.

```
struct CompareVectorByLength
{
    bool operator()(
        const Vector3& first,
        const Vector3& second
    ) const;
};
```

함수 호출 연산자와 비교 함수 객체는 알고리즘 장에서 자세히 다룬다.

6.4.3 삼방향 비교 연산자

C++20의 삼방향 비교 연산자 <=>는 두 값을 한 번 비교하여 작음, 같음과 큼의 관계를 표현한다.

```
#include <compare>

struct Version
{
    int major = 0;
    int minor = 0;
    int patch = 0;

    std::strong_ordering operator<=> (const Version& rhs) const
    {
        if (major != rhs.major)
        {
            return major <=> rhs.major;
        }
        if (minor != rhs.minor)
        {
            return minor <=> rhs.minor;
        }
        return patch <=> rhs.patch;
    }

    bool operator==(const Version& rhs) const
    {
        return major == rhs.major
            && minor == rhs.minor
            && patch == rhs.patch;
    }
};
```

비교 결과는 0과 직접 비교할 수 있다.

```
Version first{1, 4, 2};
Version second{1, 5, 0};
auto result = first <=> second;
if (result < 0)
{
    std::cout << "first가 이전 버전\n";
}
```

컴파일러는 **<=>**를 이용해 **<**, **<=**, **>**, **>=** 등의 비교 표현을 처리할 수 있다.

```
bool older = first < second;
bool newer = first > second;
```

삼방향 비교 결과의 종류는 타입의 순서 특성에 따라 달라진다.

비교 범주	의미	대표 사례
std::strong_ordering	동등성과 순서가 완전하게 일치	정수, 버전 번호
std::weak_ordering	순서는 있지만 서로 구별되는 값이 동등한 순서로 취급될 수 있음	대소문자를 무시한 문자열 비교
std::partial_ordering	비교할 수 없는 값이 존재할 수 있음	NaN을 포함하는 부동소수점 비교

부동소수점형은 NaN 때문에 완전한 순서를 만들 수 없으므로 부분 순서를 가진다.

6.4.4 기본 비교 연산자 생성

모든 멤버를 선언 순서대로 비교해도 타입의 의미와 일치한다면 비교 연산자를 기본 동작으로 생성할 수 있다.

```
#include <compare>
struct Version
{
    int major = 0;
    int minor = 0;
    int patch = 0;
    auto operator<=>(const Version&) const = default;
};
```

멤버는 선언된 순서대로 비교된다.

```
major → minor → patch

Version first{1, 4, 2};
Version second{1, 5, 0};
std::cout << std::boolalpha;
std::cout << (first < second) << '\n';
std::cout << (first == second) << '\n';
```

기본 생성된 삼방향 비교 연산자는 적절한 동등 비교도 함께 제공할 수 있다.
operator==만 필요한 타입은 동등 비교를 기본 생성할 수 있다.

```
struct Point
{
    int x, y;
    bool operator==(const Point&) const = default;
};
```

기본 비교는 모든 비교 대상 멤버를 포함한다. 캐시, 통계, 디버깅 번호처럼 값의 의미에 포함하지 않아야 하는 멤버가 있다면 기본 비교가 적합하지 않을 수 있다.

```
class Rectangle
{
public:
    Point position;
```

```
    int width;
    int height;
    mutable int areaQueryCount = 0;
};
```

areaQueryCount를 사각형의 값 동일성에 포함하지 않으려면 비교 연산자를 직접 구현해야 한다.

6.5 첨자 연산자 (subscript operator)

첨자 연산자(인덱스 연산자)는 []를 오버로딩하면 사용자 정의 타입의 원소에 배열과 같은 문법으로 접근할 수 있다. 첨자로 사용하는 값은 정수 인덱스에 한정되지 않으며 타입의 의미에 따라 키나 열거형 등을 사용할 수도 있다.

6.5.1 일반 객체의 원소 접근

Vector3의 좌표를 배열 형태로 저장한다.

```
class Vector3
{
public:
    float& operator[](std::size_t index)
    {
        return values[index];
    }
public:
    float values[3] = {};
};
```

반환 타입이 float&이므로 읽기와 쓰기가 모두 가능하다.

```
Vector3 vec3;
vec3[0] = 10.0f;
vec3[1] = 20.0f;
vec3[2] = 30.0f;
```

값으로 반환하면 읽기는 가능하지만 요소를 직접 변경할 수 없다.

```
float operator[](std::size_t index)
{
    return values[index];
}

// vec3[0] = 10.0f;    // 값을 반환하므로 대입할 수 없음
```

요소 접근 연산자는 실제 요소의 참조를 반환한다.

6.5.2 상수 객체의 원소 접근

일반 객체와 상수 객체의 접근 권한을 구분하려면 const 오버로딩을 사용한다.

```
class Vector3
{
public:
    float& operator[](std::size_t index)
    {
        return values[index];
    }
    const float& operator[](std::size_t index) const
    {
        return values[index];
    }
public:
    float values[3] = {};
};
```

일반 객체에서는 수정 가능한 참조가 반환된다.

```
Vector3 vec3;
vec3[0] = 10.0f;
```

상수 객체에서는 읽기 전용 참조가 반환된다.

```
const Vector3 vec3(10.0f, 20.0f, 30.0f);

float x = vec3[0];
// vec3[0] = 100.0f;    // 오류
```

<첨자 연산자의 const 오버로딩>

호출 객체	선택되는 함수	반환 타입
Vector3	일반 버전	float&
const Vector3	상수 버전	const float&

기본형처럼 작은 원소는 상수 버전에서 값으로 반환해도 된다.

```
float operator[](std::size_t index) const;
```

그러나 일반 버전과 상수 버전의 반환 형태를 일관되게 유지하고 클래스 타입 원소의 복사를 피하기 위해 상수 참조를 반환하는 방식이 널리 사용된다.

6.5.3 범위 검사와 at 형태의 함수

operator[]는 빠른 요소 접근을 제공하며 범위를 검사하지 않는 형태로 설계한다.

```
vec3[3] = 100.0f;    // 유효 범위를 벗어남
```

Vector3의 유효한 인덱스는 0, 1, 2이다. 범위를 벗어난 접근은 정의되지 않은 동작을 일으킬 수 있다.

범위를 검사하는 별도 함수는 At() 형태로 제공할 수 있다.

```
#include <stdexcept>

class Vector3
{
public:
    float& At(std::size_t index)
    {
        if (index >= 3)
        {
            throw std::out_of_range("Vector3 index");
        }
        return values[index];
    }
    const float& At(std::size_t index) const
    {
        if (index >= 3)
        {
            throw std::out_of_range("Vector3 index");
        }
        return values[index];
    }

public:
    float values[3] = {};
};
```

At()은 인덱스가 유효한 범위를 벗어나면 std::out_of_range 예외를 발생시킨다. 호출하는 쪽에서는 try-catch를 사용하여 해당 예외를 처리할 수 있다.

```
try
```

```
{
    vec3.At(3) = 100.0f;
}
catch (const std::out_of_range& ex)
{
    std::cout << ex.what() << '\n';
}
```

catch의 ex는 발생한 예외 객체를 참조하며, what()을 통해 예외 메시지를 확인할 수 있다. 예외 처리의 구체적인 구조와 동작은 이후 장에서 자세히 다룬다.

표준 컨테이너도 비슷한 인터페이스를 제공한다.

함수	범위 검사	일반적인 목적
operator[]	하지 않음	인덱스가 유효하다고 보장된 빠른 접근
at()	수행	외부 입력이나 계산 결과처럼 범위를 확인해야 하는 접근

모든 인덱스 타입에 정수만 사용할 수 있는 것은 아니다. 연관 컨테이너나 사용자 정의 매핑 타입은 문자열이나 열거형을 인덱스로 받을 수도 있다.

```
Value& operator[](const std::string& key);
```

첨자 연산자의 매개변수 타입은 클래스가 제공하는 요소 접근 의미에 따라 정한다.

6.6 역참조와 멤버 접근 연산자

역참조 연산자 *와 멤버 접근 연산자 ->를 오버로딩하면 사용자 정의 타입에 포인터와 유사한 접근 문법을 제공할 수 있다. 포인터 래퍼, 반복자, 스마트 포인터처럼 다른 객체를 간접적으로 나타내는 타입에서 활용된다.

스마트 포인터는 대표적인 활용 사례이다. 내부에 포인터를 보관하면서 *와 ->를 제공하면 사용자는 내부 포인터를 직접 꺼내지 않고도 일반 포인터와 비슷한 형태로 대상 객체에 접근할 수 있다.

같은 구조를 여러 타입에 사용할 수 있도록 간단한 클래스 템플릿을 사용한다. 템플릿은 이후 장에서 다루고, 여기서는 template<typename T>에서 T가 나중에 지정할 객체의 타입을 나타낸다고 이해하면 충분하다.

```
class Player
{
public:
    void Move()
    {
        std::cout << Player::Move << '\n';
    }
public:
    int hp = 100;
};
```

```
template<typename T>
class SimplePtr
{
public:
    explicit SimplePtr(T* ptr) : _ptr{ptr} { }
    ~SimplePtr()
    {
        delete _ptr;
    }
    T& operator*() const
    {
        return *_ptr;
    }
    T* operator->() const
    {
        return _ptr;
    }
public:
```

```
    T* _ptr{};
};
```

SimplePtr는 생성자에서 전달받은 포인터를 보관하고 소멸할 때 delete한다. 하나의 포인터를 여러 SimplePtr 객체가 동시에 소유하지 않도록 복사 생성자와 복사 대입 연산자는 삭제한다.

_ptr는 연산자 오버로딩 전후의 접근 방법을 비교하기 위해 public으로 두었다. 실제 자원 관리 클래스에서는 내부 포인터를 외부에 직접 공개하지 않는 것이 일반적이다.

6.6.1 역참조 연산자

역참조 연산자 *는 포인터가 가리키는 대상 객체 자체에 접근한다.
SimplePtr<Player> 객체의 내부 포인터에 직접 접근하면 다음과 같이 작성할 수 있다.

```
SimplePtr<Player> player(new Player);

(*player._ptr).Move();
player._ptr->hp = 80;
```

SimplePtr에 역참조 연산자를 정의하면 내부 포인터를 직접 사용하지 않고 대상 객체에 접근할 수 있다.

```
T& operator*() const
{
    return *_ptr;
}
```

SimplePtr<Player>에서 T는 Player가 되므로 반환 타입은 Player&가 된다.

```
SimplePtr<Player> player(new Player);
(*player).Move();
(*player).hp = 80;
```

player는 개념적으로 operator()를 호출한 결과이다.

```
(*player).Move()
        ↓
(player.operator*()).Move()
        ↓
(*player._ptr).Move()
```

operator*()가 대상 객체의 참조 T&를 반환하므로 복사본이 아니라 실제 대상 객체에 접근한다.

```
Player& object = *player;
object.hp = 80;
object.Move();
```

operator*() 뒤의 const는 SimplePtr 객체 자체의 상태를 변경하지 않는다는 뜻이다.

```
T& operator*() const;
```

반환 타입은 T&이므로 가리키는 대상 객체까지 상수가 되는 것은 아니다.

6.6.2 멤버 접근 연산자

멤버 접근 연산자 ->는 포인터가 가리키는 대상 객체의 멤버에 접근한다.
SimplePtr<Player>가 보관한 내부 포인터를 직접 사용하면 _ptr를 거쳐 대상 객체의 멤버에 접근해야 한다.

```
SimplePtr<Player> player(new Player);
player._ptr->Move();
player._ptr->hp = 80;
```

멤버 접근 연산자를 정의하면 SimplePtr 객체 자체에 ->를 사용할 수 있다.

```
T* operator->() const
{
    return _ptr;
}
```

SimplePtr<Player>에서 T는 Player이므로 operator->()는 Player*를 반환한다.

```
SimplePtr<Player> player(new Player);
player->Move();
player->hp = 80;
```

player->Move()는 operator->()가 반환한 포인터를 통해 Player의 멤버 함수에 접근한다.

```
player->Move()
      ↓
player.operator->()->Move()
      ↓
player._ptr->Move()
```

멤버 변수 접근도 같은 구조이다.

```
player->hp
      ↓
player.operator->()->hp
      ↓
player._ptr->hp
```

따라서 내부 포인터를 직접 사용하는 표현을 단순하게 표현할 수 있다.

```
player._ptr->Move();
player._ptr->hp = 80;
```

⇒

```
player->Move();
player->hp = 80;
```

operator*와 operator->는 같은 대상에 서로 다른 형태로 접근한다.

```
(*player).Move();
player->Move();
```

<역참조와 멤버 접근 연산자>

연산자	일반적인 역할	일반적인 반환 타입
*	가리키는 대상 객체에 접근	T&
->	가리키는 대상 객체의 멤버에 접근	T*

두 연산자는 스마트 포인터에만 사용되는 기능은 아니다. 반복자, 포인터 래퍼, 프록시 객체처럼 다른 객체를 간접적으로 나타내는 사용자 정의 타입에서도 활용할 수 있다.

<SimplePtr 사용 예>

```
#include <iostream>

class Player
{
public:
    void Move()
    {
        std::cout << "Player Move\n";
    }
public:
    int hp = 100;
};

template<typename T>
class SimplePtr
{
```

```
public:
    SimplePtr(const SimplePtr&) = delete;
    SimplePtr& operator=(const SimplePtr&) = delete;

    explicit SimplePtr(T* ptr) : _ptr{ptr} { }
    ~SimplePtr()
    {
        delete _ptr;
    }
    T& operator*() const
    {
        return *_ptr;
    }
    T* operator->() const
    {
        return _ptr;
    }
public:
    T* _ptr{};
};

int main()
{
    SimplePtr<Player> player(new Player);

    player._ptr->hp = 80;
    player._ptr->Move();

    (*player).hp = 70;
    (*player).Move();

    player->hp = 60;
    player->Move();

    std::cout << player->hp << '\n';
}

// 실행 결과:
// Player Move
// Player Move
// Player Move
// 60
```

SimplePtr는 역참조 연산자와 멤버 접근 연산자의 동작을 확인하기 위한 단순한 예제이다. 실제 고유 소유권 관리에는 표준 라이브러리의 std::unique_ptr를 사용한다.

6.7 형 변환 연산자

사용자 정의 타입은 변환 생성자와 형 변환 연산자를 통해 다른 타입과 변환할 수 있다. 암시적 변환은 표현을 간단하게 만들지만 예상하지 못한 함수 선택과 정보 손실을 일으킬 수 있으므로 변환 정책을 신중하게 정해야 한다.

6.7.1 사용자 정의 형 변환

형 변환 연산자는 객체를 다른 타입의 값으로 변환한다.

```
class Length
{
public:
    explicit Length(double meters) : meters(meters)
    {
    }
    operator double() const
    {
        return meters;
    }

private:
```

```
    double meters = 0.0;
};
```

형 변환 연산자는 반환 타입을 별도로 작성하지 않는다.

```
operator double() const;
```

변환할 타입이 함수 이름의 일부가 된다.

```
Length length(3.5);
double meters = length;
```

컴파일러는 length.operator double()을 사용하여 double 값을 만든다.

형 변환 연산자는 매개변수를 받지 않으며 멤버 함수로 선언한다. 객체의 상태를 변경하지 않는 변환은 상수 멤버 함수로 작성한다.
암시적 형 변환을 허용하면 함수 호출에서도 변환이 발생할 수 있다.

```
void PrintMeters(double value);
Length length(3.5);
PrintMeters(length);
```

편리하지만 변환이 눈에 드러나지 않으므로 타입의 의미가 명확하고 정보 손실이 없는 경우에만 암시적 변환을 고려한다.

6.7.2 explicit 형 변환 연산자

암시적 변환을 제한하려면 형 변환 연산자에 explicit을 지정한다.

```
class Length
{
public:
    explicit Length(double meters) : meters(meters)
    {
    }
    explicit operator double() const
    {
        return meters;
    }

private:
    double meters = 0.0;
};
```

암시적 대입은 허용되지 않는다.

```
Length length(3.5);
// double meters = length;    // 오류
```

명시적으로 변환하면 사용할 수 있다.

```
double meters = static_cast<double>(length);
```

직접 초기화 문법도 명시적 변환을 사용할 수 있다.

```
double meters(length);
```

정보가 줄어들거나 비용이 크거나 변환의 의미가 특별하다면 explicit을 사용하는 편이 안전하다.

```
class FileSize
{
public:
    explicit operator int() const;
};
```

파일 크기가 표현 범위를 벗어날 수 있다면 int로의 암시적 변환은 위험하다. ToInt(), Bytes()처럼 이름이 있는 함수가 더 적합할 수도 있다.

6.7.3 explicit operator bool

객체가 유효한 상태인지 확인할 때 explicit operator bool()을 자주 사용한다.

```
class ResourceHandle
{
public:
    explicit ResourceHandle(void* handle) : handle(handle)
    {
    }
    explicit operator bool() const noexcept
    {
        return handle != nullptr;
    }

private:
    void* handle = nullptr;
};
```

explicit 형 변환 연산자이지만 조건식에서는 사용할 수 있다.

```
ResourceHandle resource(GetResource());
if (resource)
{
    std::cout << "자원을 사용할 수 있음\n";
}
while (resource)
{
    break;
}
```

명시적인 bool 값이 필요하면 형 변환을 작성한다.

```
bool valid = static_cast<bool>(resource);
```

일반적인 암시적 변환은 제한된다.

```
// int value = resource;    // 오류
```

과거에는 포인터 변환을 이용해 조건식 사용을 구현하기도 했지만, 모던 C++에서는 explicit operator bool()을 사용하여 의도를 분명하게 표현할 수 있다.

operator!()를 함께 제공할 수도 있지만 if (!resource)는 operator bool()의 결과를 반전하여 처리할 수 있으므로 반드시 필요하지는 않다.

6.7.4 변환 생성자와 형 변환 연산자의 선택

변환 생성자는 다른 타입의 값으로 현재 클래스 객체를 만든다.

```
class Length
{
public:
    explicit Length(double meters) : meters(meters)
    {
    }

private:
    double meters = 0.0;
};

Length length(3.5);
```

형 변환 연산자는 현재 클래스 객체를 다른 타입의 값으로 변환한다.

```
class Length
```

```
{
public:
    explicit operator double() const
    {
        return meters;
    }

private:
    double meters = 0.0;
};

double meters = static_cast<double>(length);
```

<변환 생성자와 형 변환 연산자>

구분	선언 위치	변환 방향
변환 생성자	변환 결과 타입	다른 타입 → 현재 클래스
형 변환 연산자	변환 원본 타입	현재 클래스 → 다른 타입

두 타입을 모두 수정할 수 있다면 변환의 책임을 어디에 둘지 결정해야 한다. 변환 결과 타입이 입력값으로 객체를 자연스럽게 생성할 수 있으면 변환 생성자를 사용하고, 원본 타입이 자신의 값을 다른 표현으로 제공하는 것이 자연스러우면 형 변환 연산자를 사용한다.

같은 변환 경로를 변환 생성자와 형 변환 연산자 양쪽에 모두 제공하면 함수 선택이 모호해질 수 있다.

```
class Celsius;
class Fahrenheit
{
public:
    Fahrenheit(const Celsius& value);
};
class Celsius
{
public:
    operator Fahrenheit() const;
};
```

Celsius에서 Fahrenheit로 변환하는 경로가 두 개이므로 어떤 함수를 사용할지 모호해질 수 있다. 하나의 변환 책임만 선택하는 편이 좋다.

암시적 변환은 변환의 의미가 분명하고 안전할 때 사용한다. 정보 손실이나 단위·의미의 변화가 발생하거나 실행 비용이 크고, 변환이 실패할 수 있거나 여러 변환 기준이 존재한다면 명시적 변환이나 이름이 있는 함수를 사용한다.

<명시적 변환 또는 이름이 있는 함수를 사용하는 경우>

```
- 정보 손실이 발생함
- 단위나 의미가 달라짐
- 실행 비용이 큼
- 실패할 수 있음
- 여러 변환 기준이 존재함
```

```
int ToInt() const;
std::string ToString() const;
Duration ToSeconds() const;
```

연산자 오버로딩과 형 변환은 코드의 의도를 더 직접적으로 표현할 때 활용하며, 호출자가 알아야 할 중요한 동작을 숨기지 않아야 한다.

6.8 스트림 연산자

사용자 정의 타입에 스트림 연산자를 제공하면 기본형과 같은 형태로 std::cout과 std::cin을 사용할 수 있다.

6.8.1 출력 연산자

출력 연산자 <<의 왼쪽 피연산자는 std::ostream 객체이다.

```
std::cout << vec3;
```

왼쪽 피연산자가 Vector3 객체가 아니므로 operator<<는 Vector3의 멤버 함수로 구현할 수 없다. 스트림과 출력할 객체를 매개변수로 받는 비 멤버 함수로 정의한다.

```
std::ostream& operator<<(std::ostream& output, const Vector3& vec3);
```

출력 과정에서 Vector3 객체를 변경하지 않으므로 상수 참조 "const Vector3&" 를 사용한다.

```
std::ostream& operator<<(std::ostream& output, const Vector3&
vec3)
{
    output << '('
           << vec3.GetX() << ", "
           << vec3.GetY() << ", "
           << vec3.GetZ() << ')';

    return output;
}
```

스트림의 참조를 반환하면 여러 출력을 연결할 수 있다.

```
std::cout << "position: " << position << '\n';
```

처리 순서는 연결된 각 출력 연산자가 같은 스트림을 반환하는 구조이다.

```
std::cout
→ "position: " 출력
→ position 출력
→ '\n' 출력
```

출력 연산자는 보통 줄 바꿈을 포함하지 않는다. 호출자가 여러 값을 한 줄에 출력하거나 원하는 위치에서 줄을 바꿀 수 있도록 값의 표현만 기록한다.

```
std::cout << first << ' ' << second << '\n';
```

6.8.2 입력 연산자

입력 연산자 >>의 왼쪽 피연산자도 std::istream 객체이므로 비 멤버 함수로 구현한다.

```
std::istream& operator>>(std::istream& input, Vector3& vec3);
```

입력 대상의 값을 변경해야 하므로 Vector3 객체를 비상수 참조로 받는다.

```
std::istream& operator>>(std::istream& input, Vector3& vec3)
{
    float x=0.0f, y=0.0f, z=0.0f;
    if (input >> x >> y >> z)
    {
        vec3 = Vector3(x, y, z);
    }
    return input;
}
```

입력값을 임시 변수에 먼저 저장하고 세 값의 입력이 모두 성공한 뒤 객체에 반영한다. 중간 입력에서 실패하면 기존 vec3의 값이 유지된다.

```
Vector3 position;

std::cin >> position;

if (std::cin)
{
    std::cout << position << '\n';
}
```

Vector3의 성분이 공개되어 있으므로 입력 연산자도 friend가 필요하지 않다.

입력 형식에 괄호와 쉼표를 포함할 수도 있다.

```
(10, 20, 30)
```

형식이 복잡해지면 괄호와 쉼표 같은 구분 문자를 검사하고 입력 실패 상태를 설정해야 한다. 단순한 학습 예제에서는 공백으로 구분한 세 값을 입력하고, 파일 형식이나 입력 규칙이 복잡하면 Parse()와 같은 이름이 있는 함수를 사용하는 편이 좋다.

입력 연산자는 객체의 유효한 상태를 유지해야 한다. 너비와 높이가 양수여야 하는 Size 타입은 입력값을 바로 객체에 저장하지 않고 검증한 뒤 반영한다.

6.8.3 비 멤버 함수와 friend 사용

스트림 연산자는 비 멤버 함수이므로 비공개 멤버에 직접 접근할 수 없다. 공개 접근자 함수를 사용하면 friend 없이 구현할 수 있다.

```
std::ostream& operator<<(std::ostream& output,const Vector3&
vec3)
{
    output << vec3.GetX() << ' '
           << vec3.GetY() << ' '
           << vec3.GetZ();
    return output;
}
```

출력 형식을 클래스 인터페이스와 밀접하게 관리하고 비공개 멤버에 직접 접근해야 한다면 friend로 선언할 수 있다.

```
class Vector3
{
public:
    float values[3] = {};

private:
    friend std::ostream& operator<<(std::ostream& output, const Vector3& vec3);
    friend std::istream& operator>>(std::istream& input, Vector3& vec3);
};
```

friend 선언은 함수가 클래스의 멤버가 된다는 뜻이 아니다. 함수는 여전히 비 멤버이며 왼쪽 피연산자인 스트림을 첫 번째 매개변수로 받는다. 스트림 연산자를 클래스가 선언된 이름 공간에 함께 선언하면 타입과 관련된 함수로 관리하기 쉽다.

```
namespace Math
{
    class Vector3;
    std::ostream& operator<<(std::ostream& output, const
Vector3& vec3);
}
```

friend를 사용하지 않고 공개 인터페이스로 구현할 수 있다면 그 방법을 우선 검토한다. 비공개 표현에 직접 의존하는 연산자는 멤버 배치가 바뀔 때 함께 수정해야 한다.

6.9 연산자 오버로딩의 설계 기준

연산자 오버로딩은 타입의 사용 방법을 크게 바꾼다. 구현할 수 있다는 이유만으로 모든 연산자를 제공하지 않고, 타입의 의미와 호출자의 기대를 기준으로 선택해야 한다.

6.9.1 자연스러운 의미와 일관성

연산자만 보고도 동작을 예측할 수 있어야 한다.

```
Vector3 result = first + second;
```

이 표현은 두 벡터의 성분별 덧셈으로 새로운 벡터를 만든다고 예상할 수 있다.

```
Version newer = version + 1;
```

버전에서 + 1이 주 버전, 부 버전 또는 패치 버전 중 무엇을 증가시키는지 불분명하다. NextMajor(), NextMinor(), NextPatch()처럼 구체적인 함수가 더 적합하다.
복합 대입과 이항 산술 연산자의 관계도 일관되어야 한다.

```
first += second;
Vector3 result = first + second;
```

a + b의 결과는 a의 복사본에 += b를 적용한 값과 같아야 한다.
동등 비교와 순서 비교도 서로 모순되지 않아야 한다.

```
if (first == second)
{
    // first < second와 second < first는 모두 거짓이어야 함
}
```

타입이 단위를 포함한다면 단위 사이의 관계를 분명히 해야 한다.

```
Meters distance;
Seconds time;
```

서로 다른 의미의 값을 단순히 더할 수 있게 만들기보다 올바른 물리적 연산만 제공한다.

```
MetersPerSecond speed = distance / time;
```

6.9.2 예상하지 못한 부작용 피하기

값을 계산하거나 비교하는 연산에서 피연산자를 몰래 변경하지 않는다.

```
Vector3 result = first + second;
```

이 연산이 first의 값을 바꾸면 기본형의 +와 다른 의미가 된다. 상태 변경이 목적이면 +=를 사용한다.

비교 연산자는 파일 출력, 로그 기록이나 통계 증가처럼 관찰 가능한 부작용을 만들지 않는 편이 좋다.

```
bool same = first == second;
```

연산 비용과 실패 가능성을 숨기지 않아야 한다.

- +에서 파일을 읽거나 네트워크 요청 수행
- ==에서 대규모 데이터베이스 조회
- []에서 새로운 요소를 암시적으로 생성
- 형 변환에서 큰 데이터 복사
- 출력 연산자에서 객체 상태 변경

연관 컨테이너의 operator[]처럼 요소를 생성하는 의미가 잘 알려진 인터페이스도 있지만, 사용자 정의 타입에서는 이러한 부작용을 문서와 타입 의미로 분명하게 전달해야 한다.

논리 연산자 &&와 ||를 오버로딩하면 단락 평가를 유지할 수 없으므로 일반적인 타입에서는 피한다.

```
if (first && ExpensiveCheck())
{
}
```

first가 사용자 정의 타입이고 operator&&가 오버로딩되어 있다면 ExpensiveCheck()가 실행되지 않을 것이라고 가정할 수 없다.

6.9.3 일반 함수가 더 적합한 경우

동작을 여러 의미로 해석할 수 있거나 연산자의 일반적인 기대와 맞지 않으면 이름이 있는 함수를 사용한다.

```
vec3.Normalize();
vec3.Length();
vec3.Dot(rhs);
vec3.Cross(rhs);
vec3.NearlyEquals(rhs, epsilon);
```

벡터의 내적과 외적을 * 하나로 모두 표현할 수는 없다. 어떤 연산인지 함수 이름으로 구분하는 편이 명확하다.

```
float dot = first.Dot(second);
Vector3 cross = first.Cross(second);
```

파일 저장과 문자열 변환도 일반 함수가 적합하다.

```
object.Save(path);
std::string text = object.ToString();
```

형 변환이 실패할 수 있다면 암시적 형 변환보다 결과를 분명하게 표현하는 함수가 낫다.

```
std::optional<int> TryToInt() const;
```

연산자를 제공할지는 연산의 자연스러움과 의미 전달을 기준으로 판단한다.

<연산자 제공 여부를 판단하는 기준>

```
- 타입에서 자연스럽게 기대되는 연산
- 기본형의 같은 연산자와 비슷한 의미
- 예상 가능한 피연산자의 변경 여부
- 자연스러운 반환 타입
- 숨겨지지 않는 연산 비용과 실패 가능성
- 일반 함수 이름보다 명확한 의미 전달
```

6.10 종합 예제: Vector3

Vector3를 값 의미를 가지는 사용자 정의 타입으로 완성한다. 산술 연산, 단항 부호 반전, 동등 비교, 근사 비교, 인덱스 접근, 범위 검사와 스트림 입출력을 제공한다.

```
// Vector3.h
#pragma once
#include <cstddef>
#include <iosfwd>

class Vector3
{
public:
    static constexpr float EPSILON = 0.0001f;
public:
    Vector3() = default;
    Vector3(float x, float y, float z) : values{x, y, z} { }
    float GetX() const;
    float GetY() const;
    float GetZ() const;

    Vector3& operator+=(const Vector3& rhs);
    Vector3& operator-=(const Vector3& rhs);
    Vector3& operator*=(float scalar);
    Vector3 operator+() const;
    Vector3 operator-() const;

    bool operator==(const Vector3& rhs) const;
    bool NearlyEquals(const Vector3& rhs, float epsilon=EPSILON) const;

    float& operator[](std::size_t index);
    const float& operator[](std::size_t index) const;

    float& At(std::size_t index);
    const float& At(std::size_t index) const;

public:
    float values[3] = {};
    friend std::istream& operator>>(std::istream& input, Vector3& vec3);
};

Vector3 operator+(Vector3 left, const Vector3& right);
Vector3 operator-(Vector3 left, const Vector3& right);
Vector3 operator*(Vector3 vec3, float scalar);
```

```
Vector3 operator*(float scalar, Vector3 vec3);

std::ostream& operator<<(std::ostream& output, const Vector3& vec3);
std::istream& operator>>(std::istream& input, Vector3& vec3);
```

```
// Vector3.cpp
#include "Vector3.h"
#include <cmath>
#include <stdexcept>
#include <iostream>

float Vector3::GetX() const
{
    return values[0];
}

float Vector3::GetY() const
{
    return values[1];
}

float Vector3::GetZ() const
{
    return values[2];
}

Vector3& Vector3::operator+=(const Vector3& rhs)
{
    values[0] += rhs.values[0];
    values[1] += rhs.values[1];
    values[2] += rhs.values[2];

    return *this;
}

Vector3& Vector3::operator-=(const Vector3& rhs)
{
    values[0] -= rhs.values[0];
    values[1] -= rhs.values[1];
    values[2] -= rhs.values[2];

    return *this;
}

Vector3& Vector3::operator*=(float scalar)
{
    values[0] *= scalar;
    values[1] *= scalar;
    values[2] *= scalar;

    return *this;
}

Vector3 Vector3::operator+() const
{
    return *this;
}

Vector3 Vector3::operator-() const
{
    return Vector3(-values[0], -values[1], -values[2]);
}

bool Vector3::operator==(const Vector3& rhs) const
{
    return values[0] == rhs.values[0]
        && values[1] == rhs.values[1]
        && values[2] == rhs.values[2];
}

bool Vector3::NearlyEquals(const Vector3& rhs, float epsilon) const
{
```

```
    return std::fabs(values[0] - rhs.values[0]) <= epsilon
        && std::fabs(values[1] - rhs.values[1]) <= epsilon
        && std::fabs(values[2] - rhs.values[2]) <= epsilon;
}

float& Vector3::operator[](std::size_t index)
{
    return values[index];
}

const float& Vector3::operator[](std::size_t index) const
{
    return values[index];
}

float& Vector3::At(std::size_t index)
{
    if (index >= 3)
    {
        throw std::out_of_range("Vector3 index");
    }

    return values[index];
}

const float& Vector3::At(std::size_t index) const
{
    if (index >= 3)
    {
        throw std::out_of_range("Vector3 index");
    }

    return values[index];
}

Vector3 operator+(Vector3 left, const Vector3& right)
{
    left += right;
    return left;
}

Vector3 operator-(Vector3 left, const Vector3& right)
{
    left -= right;
    return left;
}

Vector3 operator*(Vector3 vec3, float scalar)
{
    vec3 *= scalar;
    return vec3;
}

Vector3 operator*(float scalar, Vector3 vec3)
{
    vec3 *= scalar;
    return vec3;
}

std::ostream& operator<<(std::ostream& output, const Vector3& vec3)
{
    output << '('
           << vec3.GetX() << ", "
           << vec3.GetY() << ", "
           << vec3.GetZ() << ')';

    return output;
}

std::istream& operator>>(std::istream& input, Vector3& vec3)
{
    float x=0.0f, y=0.0f, z=0.0f;
```

```
    if (input >> x >> y >> z)
    {
        vec3.values[0] = x;
        vec3.values[1] = y;
        vec3.values[2] = z;
    }

    return input;
}
```

```
// main.cpp
#include <iostream>
#include <stdexcept>
#include "Vector3.h"

int main()
{
    Vector3 position(10.0f, 20.0f, 30.0f);
    Vector3 velocity(2.0f, 1.0f, -1.0f);

    Vector3 moved = position + velocity * 3.0f;
    Vector3 reversed = -velocity;

    std::cout << "position: " << position << '\n';
    std::cout << "velocity: " << velocity << '\n';
    std::cout << "moved: " << moved << '\n';
    std::cout << "reversed: " << reversed << '\n';

    moved[1] = 100.0f;
    std::cout << "changed: " << moved << '\n';

    std::cout << std::boolalpha;
    std::cout << "same: " << (position == moved) << '\n';

    float accumulated = 0.0f;
    for (int i = 0; i < 10; ++i)
    {
        accumulated += 0.1f;
    }

    Vector3 calculated(accumulated, 0.0f, 0.0f);
    Vector3 expected(1.0f, 0.0f, 0.0f);

    std::cout << "exact: "
              << (calculated == expected) << '\n';
    std::cout << "nearly: "
              << calculated.NearlyEquals(expected) << '\n';

    try
    {
        std::cout << moved.At(3) << '\n';
    }
    catch (const std::out_of_range& error)
    {
        std::cout << error.what() << '\n';
    }

    Vector3 input;

    std::cout << "x y z 입력: ";
    if (std::cin >> input)
    {
        std::cout << "input: " << input << '\n';
    }
}
```

```
// 결과: 입력으로 1 2 3을 전달한 실행 결과의 예
// position: (10, 20, 30)
// velocity: (2, 1, -1)
// moved: (16, 23, 27)
// reversed: (-2, -1, 1)
```

```
// changed: (16, 100, 27)
// same: false
// exact: false
// nearly: true
// Vector3 index
// x y z 입력: 1 2 3
// input: (1, 2, 3)
```

Vector3는 동적 자원을 직접 관리하지 않으므로 복사 생성자, 이동 생성자와 대입 연산자를 직접 작성하지 않는다. 고정 크기 배열의 각 원소는 컴파일러가 제공하는 기본 특별 멤버 함수로 올바르게 복사되고 이동된다.

산술 연산자는 복합 대입 연산자를 재사용한다. +, -, *는 새로운 값을 반환하고 원본을 변경하지 않으며, +=, -=, *=는 현재 객체를 변경한다. operator==는 세 좌표를 정확하게 비교하고 NearlyEquals()는 부동소수점 계산 결과를 허용 오차로 비교한다. 서로 다른 비교 의미를 하나의 연산자에 섞지 않고 함수 이름으로 구분한다.

operator[]는 빠른 접근을 제공하고 At()은 범위를 검사한다. 일반 객체와 상수 객체에서 접근 권한을 구분하기 위해 두 함수 모두 const 오버로딩을 제공한다.

출력 연산자는 공개 접근자를 사용하므로 friend가 필요하지 않다. 입력 연산자는 세 값을 한 번에 내부 배열에 반영하기 위해 friend로 선언하였다. 입력을 임시 변수에 먼저 저장하므로 일부 값만 입력된 경우 기존 객체의 상태가 변경되지 않는다.

6.11 학습 정리

연산자 오버로딩은 클래스와 열거형에 C++의 기존 연산자 의미를 정의한다. 새로운 연산자를 만들거나 연산자의 우선순위, 결합 방향과 피연산자 개수를 변경할 수는 없다. 하나 이상의 피연산자는 사용자 정의 타입이어야 하며 기본형끼리의 연산을 다시 정의할 수 없다.

멤버 연산자는 왼쪽 피연산자를 현재 객체로 사용한다. 복합 대입, 첨자와 상태 변경 연산은 멤버 함수로 구현하는 것이 자연스럽다. 비 멤버 연산자는 모든 피연산자를 매개변수로 받아 대칭적인 이항 연산과 왼쪽 피연산자의 형 변환을 지원하기 쉽다. 비공개 멤버 접근이 필요할 때만 friend를 제한적으로 활용한다.

복합 대입 연산자는 현재 객체를 변경하고 참조를 반환한다. 이항 산술 연산자는 왼쪽 값을 복사한 뒤 복합 대입 연산자를 적용하여 구현할 수 있다. 단항 연산자는 현재 값을 변경하지 않는 새로운 결과를 반환할 수 있으며, 증가와 감소 연산자는 전위형과 후위형의 반환 의미를 구분해야 한다.

동등 비교는 객체가 나타내는 값의 의미를 기준으로 구현한다. 자연스러운 순서가 있는 타입만 순서 비교를 제공하고, C++20의 삼방향 비교 연산자와 기본 비교 생성을 활용하면 반복되는 비교 코드를 줄일 수 있다. 모든 멤버가 값 의미에 포함되는지 확인한 뒤 기본 비교를 사용한다.

첨자 연산자는 요소의 참조를 반환하여 배열과 같은 접근을 제공한다. 일반 객체에는 수정 가능한 참조를, 상수 객체에는 읽기 전용 참조를 반환하도록 const 오버로딩을 구성한다. 빠른 비검사 접근과 범위를 확인하는 At()을 구분할 수 있다.

역참조 연산자 *는 포인터와 유사한 객체가 가리키는 대상 객체 자체에 접근하고, 멤버 접근 연산자 ->는 대상 객체의 멤버에 직접 접근하는 문법을 제공한다. operator*는 일반적으로 대상 객체의 참조를 반환하고, operator->는 대상 객체의 포인터를 반환한다. 스마트 포인터, 반복자와 포인터 래퍼처럼 다른 객체를 간접적으로 나타내는 사용자 정의 타입에서 활용할 수 있다.

형 변환 연산자는 객체를 다른 타입으로 변환하며 반환 타입을 별도로 작성하지 않는다. 예상하지 못한 암시적 변환을 막으려면 explicit을 지정한다. 객체의 유효성을 조건식에서 확인할 때는 explicit operator bool()을 활용한다. 변환 생성자와 형 변환 연산자를 같은 변환 경로에 중복해서 제공하지 않는다.

스트림 연산자는 왼쪽 피연산자가 스트림이므로 비 멤버 함수로 구현한다. 스트림의 참조를 반환하면 입출력을 연결할 수 있다. 입력은 임시 값에 먼저 저장하고 검증이 끝난 뒤 객체에 반영하여 객체의 유효한 상태를 유지한다.

연산자 오버로딩은 기본 연산자의 자연스러운 의미와 일관성을 유지할 때 활용한다. 동작이 특별하거나 여러 의미로 해석될 수 있거나 비용과 실패 가능성을 드러내야 한다면 이름이 있는 일반 함수가 더 적합하다.

6.12 학습 과제

과제 **1. Vector2** 산술 연산
2차원 벡터를 나타내는 Vector2 클래스를 작성한다.
상태: float x, y

필수 연산자:

```
+=   벡터 덧셈 대입
-=   벡터 뺄셈 대입
*=   스칼라 곱 대입
+    벡터 덧셈
-    벡터 뺄셈과 단항 부호 반전
*    벡터와 스칼라의 곱
==   정확한 좌표 비교
```

```
<<   (x, y) 형식 출력
```

vec2 * scalar와 scalar * vec2 를 모두 지원한다. 이항 산술 연산자는 복합 대입 연산자를 재사용한다. 부동소수점 근사 비교를 위한 NearlyEquals()도 작성한다.

과제 **2. Money** 값 타입
금액을 나타내는 Money 클래스를 작성한다. 부동소수점형을 사용하지 않고 최소 화폐 단위를 long long으로 저장한다.
필수 기능:

- 금액 덧셈과 뺄셈
- 복합 대입 연산
- 동등 비교와 순서 비교
- 삼방향 비교 연산자
- 출력 연산자
- 음수 금액을 허용할지 정책 결정

```
Money first(1500);
Money second(2300);
Money total = first + second;
std::cout << total << '\n';
std::cout << std::boolalpha
          << (first < second) << '\n';
```

금액 출력 형식과 음수 처리 정책을 클래스 설명 주석으로 기록한다.

과제 **3. Version** 비교
주 버전, 부 버전과 패치 버전을 저장하는 Version 구조체를 작성한다.

```
struct Version
{
    int major;
    int minor;
    int patch;
};
```

operator<=>를 기본 생성하여 모든 비교 연산을 지원한다. 멤버 선언 순서가 비교 순서에 어떤 영향을 주는지 확인한다.
문자열 출력 형식은 3자를 점(dot)과 결합해서 구성한다.

```
1.4.2
```

std::set<Version>에 여러 버전을 저장하고 정렬 결과를 확인하는 코드는 표준 컨테이너를 학습한 뒤 확장 과제로 사용할 수 있다.

과제 **4. FixedArray** 인덱스 접근
정수 10개를 저장하는 FixedArray 클래스를 작성한다.

필수 기능:

```
int& operator[](std::size_t index);
const int& operator[](std::size_t index) const;
int& At(std::size_t index);
const int& At(std::size_t index) const;
```

operator[]는 범위를 검사하지 않고 At()은 범위를 검사한다. 범위를 벗어나면 std::out_of_range를 발생시킨다.
일반 객체와 상수 객체에서 반환 타입이 어떻게 달라지는지 확인하는 코드를 작성한다.

과제 **5. ResourceHandle**의 명시적 **bool** 변환
자원 번호와 열림 상태를 관리하는 ResourceHandle 클래스를 작성한다.
필수 기능:

- 기본 객체는 닫힌 상태
- 양수 자원 번호를 전달하면 열린 상태
- Close()를 호출하면 닫힌 상태
- explicit operator bool()로 유효성 확인
- operator!()는 작성하지 않고 !handle이 동작하는지 확인

```
ResourceHandle handle(10);

if (handle)
{
    std::cout << "open\n";
```

```
}

handle.Close();

if (!handle)
{
    std::cout << "closed\n";
}
```

int value = handle;이 허용되지 않는 이유를 설명한다.

과제 **6. Date** 스트림 입출력
연도, 월과 일을 저장하는 Date 클래스를 작성한다.
출력 형식
2026-08-01

입력 형식
2026 8 1

입력 연산자는 값을 임시 변수에 먼저 저장한다. 월이 1~12가 아니거나 일이 1~31이 아니면 스트림에 failbit를 설정하고 기존 Date 객체의 상태를 유지한다.
동등 비교와 삼방향 비교 연산자도 제공하여 날짜 순서를 비교한다.

과제 **7.** 연산자와 일반 함수 선택
게임 프로그램에서 사용할 CharacterStats 클래스를 설계한다.
상태는 체력, 공격력, 방어력과 이동 속도로 구성한다.
제시된 기능을 연산자로 제공할지 일반 함수로 제공할지 결정하고 이유를 작성한다.

- 두 능력치 묶음 합산
- 모든 능력치에 배율 적용
- 두 캐릭터의 전투력 비교
- 체력 회복
- 피해 적용
- 버프 병합
- 문자열 출력

연산자로 선택한 기능은 실제 코드를 작성하고, 일반 함수로 선택한 기능은 함수 이름과 매개변수 형식을 설계한다. 기본 연산자의 의미와 예상하지 못한 부작용 여부를 기준으로 판단한다.

07. 객체 수명, RAII와 소유권

프로그램에서 객체를 생성하는 것만큼 중요한 문제는 객체가 언제까지 존재하고 누가 정리하는지 결정하는 것이다. 지역 변수는 범위를 벗어나면 자동으로 소멸하지만 new로 생성한 객체는 delete를 호출할 때까지 남는다. 파일, 소켓과 잠금 같은 자원도 사용을 마친 뒤 반드시 닫거나 해제해야 한다.

```
void Process()
{
    int* value = new int(100);

    // 작업 중 함수가 종료되면 어떻게 되는가?

    delete value;
}
```

정상적으로 마지막 줄까지 실행되면 메모리가 해제된다. 그러나 중간에서 return하거나 예외가 발생하면 delete가 실행되지 않을 수 있다. 자원 해제를 사람이 모든 실행 경로에서 직접 관리하면 코드가 복잡해지고 누락되기 쉽다.

C++은 객체의 수명과 자원 관리를 연결하는 RAII(Resource Acquisition Is Initialization)를 사용한다. 객체가 생성될 때 자원을 획득하고 소멸할 때 자원을 해제하면, 범위를 벗어나거나 예외가 발생해도 소멸자가 정리 작업을 수행한다.

```
void Process()
{
    std::unique_ptr<int> value = std::make_unique<int>(100);

    // 범위를 벗어나면 자동으로 메모리 해제
}
```

RAII는 동적 메모리에만 적용되는 기법이 아니다. 파일, 네트워크 연결, 잠금, 그래픽 자원과 운영체제 핸들처럼 명시적인 정리가 필요한 모든 자원에 적용할 수 있다.

이 장에서는 저장 기간과 객체 수명을 구분하고 원시 동적 메모리 관리에서 발생하는 문제를 살펴본다. 이어서 RAII와 예외 안전성을 학습하고, std::unique_ptr, std::shared_ptr와 std::weak_ptr를 이용하여 고유 소유권, 공유 소유권과 비소유 참조를 표현한다. 마지막으로 게임 장면과 텍스처 리소스를 예로 들어 값, 참조와 스마트 포인터를 선택하는 기준을 정리한다.

7.1 저장 기간과 객체 수명

객체를 올바르게 사용하려면 저장 공간이 얼마나 유지되는지와 그 공간에서 객체가 언제 존재하는지 구분해야 한다.
저장 기간**(storage duration)** 은 객체가 사용할 저장 공간이 유지되는 기간을 나타낸다. C++의 저장 기간은 자동, 정적, 스레드와 동적 저장 기간으로 구분한다.

객체 수명**(object lifetime)** 은 객체가 초기화되어 해당 타입의 객체로 사용할 수 있게 된 시점부터 소멸이 끝난 시점까지이다.
일반적인 변수에서는 저장 기간과 객체 수명이 함께 시작하고 끝나는 것처럼 보이지만 두 개념은 같지 않다. 저장 공간이 존재하더라도 객체의 초기화가 끝나지 않았거나 이미 소멸했다면 해당 객체를 사용할 수 없다.

7.1.1 자동 저장 기간(Automatic Storage Duration)

함수나 블록 안에 선언한 일반 지역 객체는 자동 저장 기간을 가진다.

```
void Update()
{
    int frameCount = 0;
    std::string message = "Update";
}
```

함수 실행이 선언 지점에 도달하면 객체가 초기화된다. 블록을 벗어나면 객체가 선언된 순서의 반대로 소멸한다.

```
class TraceObject
{
public:
    explicit TraceObject(const std::string& name)
        : name(name)
    {
        std::cout << name << " 생성\n";
    }
```

```
    ~TraceObject()
    {
        std::cout << name << " 소멸\n";
    }

private:
    std::string name;
};
```

```
void TestScope()
{
    TraceObject first("first");
    TraceObject second("second");
}

// 실행 결과
// first 생성
// second 생성
// second 소멸
// first 소멸
```

나중에 생성된 지역 객체가 먼저 소멸한다. 이 역순 소멸 규칙은 여러 자원의 의존 관계를 정리할 때 중요하다.

중첩 블록을 사용하면 더 좁은 수명을 만들 수 있다.

```
void Update()
{
    TraceObject outer("outer");

    {
        TraceObject inner("inner");
    }

    std::cout << "Update 계속\n";
}
```

inner는 내부 블록이 끝나는 즉시 소멸하고 outer는 함수가 끝날 때 소멸한다. 필요한 범위에만 객체를 선언하면 자원을 더 일찍 정리할 수 있다.

함수 매개변수도 자동 저장 기간을 가진다. 값 매개변수는 함수 호출 과정에서 생성되고 함수가 끝날 때 소멸한다.

```
void PrintName(std::string name)
{
    std::cout << name << '\n';
}
```

참조 매개변수는 새로운 대상 객체를 만들지 않는다.

```
void PrintName(const std::string& name)
{
    std::cout << name << '\n';
}
```

참조 변수 자체의 사용 범위는 함수 내부 이지만, 참조 대상의 수명은 원래 객체의 저장 기간에 따라 결정된다.

7.1.2 정적 저장 기간(Static Storage Duration)

이름 공간 범위에 선언한 객체와 static으로 선언한 객체는 정적 저장 기간을 가진다. 프로그램 실행 동안 저장 공간이 유지된다.

```
int totalScore = 0;
class GameConfig
{
};
GameConfig globalConfig;
```

이름 공간 범위의 객체는 일반적으로 main()이 시작되기 전에 초기화되고 프로그램 종료 과정에서 소멸한다.

함수 안의 정적 지역 객체도 정적 저장 기간을 가진다.

```
int NextId()
{
    static int id = 0;
    return ++id;
}
```

id는 NextId()를 호출할 때마다 새로 생성되지 않는다. 한 번 초기화된 뒤 프로그램이 끝날 때까지 값을 유지한다.

클래스 객체를 정적 지역 객체로 만들 수도 있다.

```
GameConfig& GetConfig()
{
    static GameConfig config;
    return config;
}
```

config는 함수 실행이 처음으로 선언 지점에 도달할 때 초기화된다. C++11 이후 정적 지역 객체의 초기화는 여러 스레드가 동시에 접근하더라도 한 번만 수행되도록 보장된다. 객체의 구체적인 동시 사용 규칙은 동시성 장에서 다룬다.

정적 저장 기간 객체의 생성과 소멸 순서는 주의해야 한다. 하나의 번역 단위 안에서는 초기화가 완료된 순서의 반대로 소멸하지만, 서로 다른 소스 파일에 정의된 정적 객체 사이의 초기화 순서에 의존하면 문제가 발생할 수 있다.

```
// 서로 다른 소스 파일의 전역 객체가
// 상대 객체의 생성 순서에 의존하지 않도록 설계한다.
```

필요한 시점에 정적 지역 객체를 생성하는 함수 형태는 전역 객체 사이의 초기화 순서 의존성을 줄이는 데 활용할 수 있다.

7.1.3 스레드 저장 기간(Thread Storage Duration)

thread_local로 선언한 객체는 스레드 저장 기간을 가진다. 프로그램 전체에 하나만 존재하는 정적 객체와 달리, 각 스레드마다 별도의 객체가 만들어진다.

```
thread_local int currentWorkerId = 0;
```

각 스레드는 자신의 currentWorkerId를 사용한다. 한 스레드에서 값을 변경해도 다른 스레드의 객체에는 영향을 주지 않는다.
함수 안에서도 사용할 수 있다.

```
int NextThreadLocalId()
{
    thread_local int id = 0;
    return ++id;
}
```

thread_local 객체는 각 스레드마다 초기화되고 스레드가 종료될 때 소멸한다. 구체적인 초기화 시점은 선언 위치와 사용 방식에 따라 달라질 수 있다. 함수 안의 thread_local 객체는 해당 스레드의 실행이 처음으로 선언 지점에 도달할 때 초기화된다. 주 스레드의 스레드 저장 기간 객체는 프로그램 종료 과정에서 정리된다.

thread_local은 스레드마다 독립된 상태가 필요한 경우에 활용할 수 있다.

- 스레드별 작업 번호
- 스레드별 임시 메모리
- 스레드별 오류 상태
- 스레드별 난수 생성기

스레드 저장 기간이 데이터 경쟁을 자동으로 해결하는 것은 아니다. thread_local 객체 자체는 스레드마다 분리되지만, 그 객체가 공용 자원을 가리키거나 다른 공유 상태에 접근한다면 별도의 동기화가 필요하다.

7.1.4 동적 저장 기간(Dynamic Storage Duration)

동적 저장 기간을 가진 저장 공간은 프로그램 실행 중 명시적인 요청에 따라 확보된다. 일반적으로 new 표현식으로 객체를 생성하고 delete 표현식으로 소멸한다.

```
int* value = new int(100);
std::cout << *value << '\n';
delete value;
```

new int(100)은 정수 객체를 위한 저장 공간을 확보하고 값 100으로 초기화한다. delete value는 객체의 수명을 끝내고 저장 공간을 반환한다.
클래스 객체도 동적으로 생성할 수 있다.

```
class Enemy
{
public:
    Enemy()
    {
        std::cout << "Enemy 생성\n";
    }
    ~Enemy()
    {
        std::cout << "Enemy 소멸\n";
    }
};
```

```
Enemy* enemy = new Enemy;
// enemy 사용
delete enemy;
```

동적 객체는 객체를 생성한 블록이 끝나도 자동으로 소멸하지 않는다.

```
Enemy* CreateEnemy()
{
    Enemy* enemy = new Enemy;
    return enemy;
}
```

반환된 포인터를 누가 소유하고 누가 delete를 호출할지 인터페이스만 보고 분명하지 않다. 이러한 불명확한 소유권은 메모리 누수와 이중 해제의 원인이 된다.

동적 저장 기간은 주로 네 가지 상황에서 필요하다.

- 객체의 수명이 생성한 함수보다 길어야 하는 경우
- 실행 중 개수가 결정되는 객체를 관리하는 경우
- 서로 다른 객체가 소유권을 전달하거나 공유해야 하는 경우
- 다형적인 객체를 기반 클래스 포인터로 관리하는 경우

다만 실행 중 크기가 결정된다는 이유만으로 직접 new[]를 사용할 필요는 없다. 원소 집합은 일반적으로 std::vector, 문자열은 std::string, 단일 동적 객체는 스마트 포인터와 같은 자원 관리 타입을 우선 사용한다.

7.1.5 저장 기간과 객체 수명의 차이

저장 기간은 저장 공간의 유지 기간이고 객체 수명은 그 공간에 특정 타입의 객체가 존재하는 기간이다.

<저장 기간과 객체 수명>

구분	의미	시작과 종료의 기준
저장 기간	객체가 사용할 저장 공간이 유지되는 기간	선언 형태와 메모리 할당 방식
객체 수명	초기화된 객체를 해당 타입으로 사용할 수 있는 기간	객체 초기화 완료와 소멸 완료

일반 지역 객체는 선언 지점에서 초기화되어 수명이 시작되고 블록을 벗어날 때 소멸한다.

```
void Test()
{
    TraceObject object("local");  // 초기화 완료 후 수명 시작
    object.Print();
}                                 // 소멸 후 수명 종료
```

생성자가 실행되는 동안에는 객체의 초기화가 진행 중이다. 생성자가 정상적으로 끝나야 완성된 객체로 사용할 수 있다. 생성자에서 예외가 발생하면 객체의 수명은 정상적으로 시작되지 않으며, 해당 객체의 소멸자는 호출되지 않는다. 대신 이미 생성된 멤버 객체는 자동으로 소멸한다.

소멸자가 실행되면 객체의 수명이 끝나는 과정이 시작된다. 소멸이 끝난 객체를 가리키던 포인터나 참조가 남아 있어도 해당 객체를 사용할 수 없다.

```
Enemy* enemy = nullptr;
{
    Enemy localEnemy;
    enemy = &localEnemy;
}
// enemy->Update();    // 오류가 발생할 수 있는 잘못된 접근
```

enemy 변수에는 주소가 남아 있지만 그 주소의 Enemy 객체는 이미 소멸했다. 포인터 값이 nullptr가 아니라는 사실만으로 객체가 살아 있다고 판단할 수 없다.

저장 기간과 객체 수명의 구분은 여러 수명 문제를 이해하는 기반이 된다.

- 지역 객체를 가리키는 포인터나 참조를 함수 밖에서 사용하면 안 되는 이유
- 동적 객체에 delete가 필요한 이유
- 컨테이너 재배치 뒤 기존 포인터가 무효화될 수 있는 이유
- 예외가 발생했을 때 어떤 지역 객체가 소멸하는지
- 스마트 포인터가 객체 수명을 어떻게 소유권과 연결하는지

7.2 동적 메모리와 자원

원시 new와 delete는 동적 객체의 생성과 소멸 원리를 이해하는 데 필요하다. 그러나 실제 프로그램에서 자원 소유를 원시 포인터만으로 표현하면 해제 책임을 추적하기 어렵다.
이 절의 원시 동적 메모리 예제는 문제의 원리를 확인하기 위한 것이다. 뒤에서는 같은 문제를 RAII와 스마트 포인터로 해결한다.

7.2.1 new와 delete

new 표현식은 객체에 필요한 저장 공간을 확보하고 객체를 초기화한 뒤 포인터를 반환한다.

```
int* first = new int;
int* second = new int(100);
int* third = new int{};
```

<초기화 결과>

표현식	초기화
new int	기본 초기화, 값이 정해지지 않음
new int(100)	값 100으로 직접 초기화
new int{}	값 초기화, 0으로 초기화

사용이 끝난 단일 객체는 delete로 소멸한다.

```
delete first;
delete second;
delete third;
```

클래스 객체에서는 new가 생성자를 호출하고 delete가 소멸자를 호출한다.

```
class Resource
{
public:
    explicit Resource(int id) : id(id)
    {
        std::cout << "Resource " << id << " 생성\n";
    }

    ~Resource()
    {
        std::cout << "Resource " << id << " 소멸\n";
```

```
    }

private:
    int id;
};
```

```
Resource* resource = new Resource(1);
delete resource;
```

delete한 뒤 포인터 변수 자체가 자동으로 nullptr가 되지는 않는다.

```
Resource* resource = new Resource(1);
delete resource;
resource = nullptr;
```

nullptr에 delete를 사용해도 아무 작업도 수행하지 않는다.

```
Resource* resource = nullptr;
delete resource;    // 안전함
```

그러나 같은 포인터에 두 번 delete를 호출하는 것은 허용되지 않는다.

```
Resource* resource = new Resource(1);
delete resource;
// delete resource;    // 정의되지 않은 동작
```

delete를 호출할 수 있다는 사실과 호출해야 할 책임이 누구에게 있는지는 별개의 문제이다. 원시 포인터만으로는 소유 포인터와 비소유 포인터를 타입에서 구분할 수 없다.

```
Resource* CreateResource();       // 호출자가 delete해야 하는가?
Resource* FindResource(int id);   // 내부 객체를 빌려주는가?
```

스마트 포인터와 참조를 사용하면 이러한 의도를 인터페이스에 더 분명하게 표현할 수 있다.

7.2.2 new[]와 delete[]

동적 배열은 new[]로 생성한다.

```
std::size_t count = 5;
int* values = new int[count]{};
```

각 원소는 0으로 값 초기화된다.

```
for (std::size_t i = 0; i < count; ++i)
{
    values[i] = static_cast<int>(i * 10);
}
```

동적 배열은 delete[]로 해제해야 한다.

```
delete[] values;
```

new[]와 delete[]를 대응하지 않으면 정의되지 않은 동작이 발생한다.

```
int* values = new int[10];
// delete values;    // 잘못된 해제
delete[] values;
```

클래스 배열을 생성하면 각 원소의 생성자가 호출된다.

```
Resource* resources = new Resource[3]{
    Resource(1),
```

```
    Resource(2),
    Resource(3)
};
```

배열을 해제하면 각 원소의 소멸자가 역순으로 호출된다.

```
delete[] resources;
```

동적 배열의 크기 정보는 원시 포인터 타입에 포함되지 않는다.

```
void PrintValues(const int* values, std::size_t count);
```

포인터와 원소 개수를 함께 관리해야 하며, 둘이 일치하지 않으면 범위를 벗어난 접근이 발생할 수 있다. 이러한 이유로 일반적인 가변 길이 배열은

std::vector를 사용하고, 고정 길이 배열은 std::array를 사용하는 편이 안전하다.
동적 배열의 고유 소유권이 꼭 필요하다면 std::unique_ptr<T[]>를 사용할 수 있다.

7.2.3 할당 실패와 std::bad_alloc

new가 필요한 저장 공간을 확보하지 못하면 기본적으로 std::bad_alloc 예외를 발생시킨다.

```
#include <new>
try
{
    int* values = new int[veryLargeCount];
    delete[] values;
}
catch (const std::bad_alloc& error)
{
    std::cerr << error.what() << '\n';
}
```

할당 실패를 nullptr로 확인하는 C 방식과 다르다.

```
int* value = new int;
// 기본 new가 정상적으로 반환했다면 value는 nullptr가 아님
```

예외를 사용하지 않고 실패 시 nullptr를 받으려면 std::nothrow를 지정할 수 있다.

```
#include <new>
int* values = new (std::nothrow) int[veryLargeCount];
if (values == nullptr)
{
    std::cerr << "메모리 할당 실패\n";
    return;
}
delete[] values;
```

std::nothrow를 사용하면 호출 지점마다 실패 여부를 직접 검사해야 한다. 검사를 빠뜨리면 nullptr 역참조 문제가 발생할 수 있다.

일반적인 C++ 코드는 RAII와 예외 전파를 중심으로 구성한다.

- 객체와 자원을 RAII 타입으로 관리
- 할당 실패는 예외로 전달
- 예외가 발생해도 이미 생성된 지역 객체가 자원을 정리
- 실제로 복구할 수 있는 경계에서 예외 처리

할당 실패를 무조건 모든 함수에서 잡아야 하는 것은 아니다. 현재 함수가 복구 방법을 알지 못한다면 상위 호출자에게 예외가 전달되도록 두는 편이 낫다.

7.2.4 메모리 누수(Memory Leak)

동적으로 확보한 메모리를 더 이상 사용할 수 없는데도 해제하지 않으면 메모리 누수가 발생한다.

```
void Leak()
{
    int* value = new int(100);

    // delete value;가 없음
}
```

함수가 끝나면 지역 포인터 value는 사라지지만 동적으로 생성한 정수 객체는 자동으로 해제되지 않는다. 주소를 잃었으므로 정상적인 방법으로 다시 delete할 수도 없다.

포인터에 새 주소를 덮어써도 기존 메모리가 누수된다.

```
int* value = new int(10);
value = new int(20);    // 첫 번째 주소를 잃음
delete value;
```

반복문에서 누수가 발생하면 사용 메모리가 계속 증가한다.

```
for (int i = 0; i < 1000; ++i)
{
    Resource* resource = new Resource(i);
    // 해제하지 않음
}
```

조기 반환도 해제 코드를 건너뛸 수 있다.

```
bool Process(bool valid)
{
    int* values = new int[100]{};
    if (!valid)
    {
        return false;    // values 누수
    }
    delete[] values;
    return true;
}
```

예외가 발생해도 같은 문제가 생긴다.

```
void Process()
{
    int* values = new int[100]{};
    LoadData();    // 예외가 발생할 수 있음
    delete[] values;
}
```

LoadData()에서 예외가 발생하면 마지막 delete[]가 실행되지 않는다.

모든 반환과 예외 경로마다 직접 해제 코드를 추가하는 방식은 유지보수하기 어렵다.

```
void Process()
{
    std::unique_ptr<int[]> values =
        std::make_unique<int[]>(100);
    LoadData();
}
```

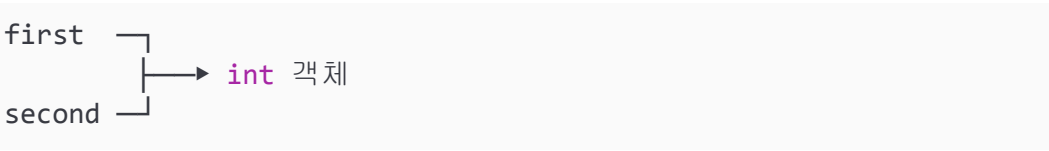
RAII 객체를 사용하면 정상 종료, 조기 반환과 예외 발생 모두에서 소멸자가 자원을 정리한다.

7.2.5 이중 해제와 댕글링 포인터

하나의 동적 객체를 두 포인터가 가리킬 수 있다.

```
int* first = new int(100);
int* second = first;
```

두 포인터가 있다고 해서 객체가 두 개가 된 것은 아니다.



한 포인터로 객체를 해제하면 다른 포인터도 더 이상 유효한 객체를 가리키지 않는다.

```
delete first;
first = nullptr;
// std::cout << *second;    // 댕글링 포인터 역참조
```

소멸한 객체의 주소를 계속 저장하는 포인터를 댕글링 포인터(dangling pointer)라고 한다.
second에 다시 delete를 호출하면 같은 메모리를 두 번 해제한다.

```
// delete second;    // 이중 해제
```

포인터 하나를 nullptr로 바꾸어도 다른 별칭 포인터는 자동으로 변경되지 않는다.

```
delete first;
first = nullptr;
// second에는 이전 주소가 그대로 남음
```

지역 객체를 가리키는 포인터도 객체가 소멸하면 댕글링 포인터가 된다.

```
const std::string* GetNamePointer()
{
    std::string name = "Player";
    return &name;    // 잘못된 반환
}
```

함수가 끝나면 name이 소멸하므로 반환된 포인터를 사용할 수 없다.
참조도 같은 수명 문제를 가진다.

```
const std::string& GetNameReference()
{
    std::string name = "Player";
    return name;    // 잘못된 반환
}
```

댕글링 포인터와 이중 해제를 피하려면 해제 책임과 소유권을 명확히 해야 한다.

- 하나의 자원에 대한 해제 책임을 명확히 정함
- 고유 소유권은 std::unique_ptr로 표현
- 실제 공유 수명이 필요할 때만 std::shared_ptr 사용
- 소유하지 않는 접근은 참조나 원시 포인터로 표현하고 수명을 보장
- 지역 객체의 주소나 참조를 반환하지 않음
- 해제 뒤 남은 별칭 포인터를 사용하지 않음

7.2.6 메모리 이외의 자원

프로그램이 관리해야 하는 자원은 메모리만이 아니다.

자원	획득	해제
파일	열기	닫기
네트워크 소켓	생성·연결	종료·닫기
스레드	시작	결합 또는 분리
뮤텍스	잠금	잠금 해제
그래픽 자원	버퍼·텍스처 생성	API에 맞는 해제 함수 호출
운영체제 핸들	API 호출로 생성	대응하는 닫기 함수 호출

데이터베이스 연결	연결 획득	연결 반환 또는 종료

파일을 직접 관리하는 코드를 생각해 보자.

```
#include <cstdio>
void SaveData()
{
    std::FILE* file = std::fopen("data.txt", "w");
    if (file == nullptr)
    {
        return;
    }
    std::fputs("Player\n", file);
    std::fclose(file);
}
```

중간에 여러 반환 경로가 추가되면 모든 경로에서 std::fclose()를 호출해야 한다.

```
void SaveData(bool valid)
{
    std::FILE* file = std::fopen("data.txt", "w");
    if (file == nullptr)
    {
        return;
    }
    if (!valid)
    {
        std::fclose(file);
        return;
    }
    std::fputs("Player\n", file);
    std::fclose(file);
}
```

예외가 발생하면 직접 작성한 정리 코드까지 실행되지 않을 수 있다.

자원마다 획득과 해제 함수가 다르지만 공통 문제는 같다.

```
자원을 획득함
→ 모든 종료 경로에서 정확히 한 번 해제해야 함
```

RAII는 자원의 종류와 관계없이 이 규칙을 객체의 생성자와 소멸자에 연결한다.

7.3 RAII

RAII(Resource Acquisition Is Initialization)는 자원의 획득과 해제를 객체의 초기화와 소멸에 연결하는 C++의 자원 관리 방식이다.

```
객체 생성 → 자원 획득
객체 사용 → 자원 사용
객체 소멸 → 자원 해제
```

자원을 소유하는 객체가 범위를 벗어나면 소멸자가 자동으로 호출된다. 정상 종료뿐 아니라 조기 반환과 예외로 범위를 벗어나는 경우에도 생성이 완료된 지역 객체는 소멸한다.

7.3.1 자원 획득과 객체 초기화

RAII 객체는 생성이 완료되면 사용할 수 있는 유효한 자원을 소유하도록 구성한다.

```
class FileHandle
{
public:
    FileHandle(const char* fileName, const char* mode)
        : file(std::fopen(fileName, mode))
    {
        if (file == nullptr)
```

```
        {
            throw std::runtime_error("파일을 열 수 없습니다.");
        }
    }
    ~FileHandle()
    {
        std::fclose(file);
    }
private:
    std::FILE* file = nullptr;
};
```

생성자는 파일을 열고 소유권을 FileHandle 객체에 저장한다. 파일 열기에 실패하면 완성되지 않은 객체를 반환하지 않고 생성 실패를 알린다. 예외 처리 방법은 7.4절에서 자세히 다룬다.

```
void SaveData()
{
    FileHandle file("data.txt", "w");
    // file이 생성되었다면 유효한 파일을 소유함
}
```

이 FileHandle은 생성이 완료된 객체가 유효한 파일을 소유한다는 불변식을 유지한다.

```
생성이 완료된 FileHandle 객체는 유효한 파일을 소유한다.
```

빈 상태도 허용해야 한다면 어떤 연산이 가능한지 명확히 정의한다. 스마트 포인터는 빈 상태를 허용하며 nullptr와 비교하거나 논리 조건식에서 확인할 수 있다.

자원 획득을 생성자와 연결하면 객체를 생성한 뒤 별도의 Open() 호출을 빠뜨리는 문제를 줄일 수 있다.

```
// 두 단계 초기화
FileHandle file;
file.Open("data.txt");
```

두 단계 초기화가 항상 잘못된 것은 아니지만, Open()을 호출하기 전의 객체 상태와 실패 이후의 상태까지 설계해야 한다. 생성과 동시에 자원이 필요한 타입이라면 생성자에서 획득하는 편이 단순하다.

7.3.2 결정적 소멸(Deterministic Destruction)

자동 저장 기간 객체는 범위를 벗어나는 시점에 소멸한다. 소멸 시점을 코드 구조로 예측할 수 있으므로 결정적 소멸이라고 표현한다.

```
void SaveData()
{
    FileHandle file("data.txt", "w");
    // 작업
}   // 이 지점에서 file 소멸
```

가비지 컬렉션을 사용하는 환경에서는 더 이상 참조되지 않는 객체가 언제 정리될지 즉시 알 수 없는 경우가 있다. C++의 자동 객체는 범위가 끝나는 시점에 소멸자가 호출되므로 파일 닫기, 잠금 해제와 같은 작업을 객체 수명에 연결할 수 있다.

결정적 소멸은 메모리 사용량을 줄이는 목적만 있는 것이 아니다.

- 잠금을 필요한 구간에서만 유지
- 파일 내용을 기록한 뒤 즉시 닫음
- 그래픽 자원을 정해진 시점에 반환
- 트랜잭션 범위가 끝날 때 커밋 또는 롤백
- 임시 상태를 원래 상태로 복구

소멸자는 객체가 소유한 자원을 정리하는 마지막 안전 장치이다. 사용자가 별도의 정리 함수를 호출하지 않아도 객체 수명이 끝나면 정리가 수행되어야 한다.

7.3.3 범위와 자원 해제

RAII 객체의 수명은 범위로 제어할 수 있다.

```
void UpdateGame()
```

```
{
    LoadGameState();

    {
        FileHandle log("frame.log", "w");

        WriteFrameLog(log);
    }   // log 파일 닫힘

    ContinueGame();
}
```

함수 전체가 끝날 때까지 파일을 유지할 필요가 없다면 내부 블록을 사용하여 더 일찍 해제할 수 있다.

조기 반환에서도 소멸자가 호출된다.

```
bool SavePlayer(bool valid)
{
    FileHandle file("player.dat", "wb");

    if (!valid)
    {
        return false;
    }

    WritePlayer(file);
    return true;
}
```

valid가 false여도 함수가 반환되기 전에 file의 소멸자가 호출된다.
반복문 안에 선언한 객체는 각 반복이 끝날 때 소멸한다.

```
for (const std::string& name : fileNames)
{
    FileHandle file(name.c_str(), "rb");

    ProcessFile(file);
}
```

한 파일의 처리가 끝나면 해당 파일이 닫히고, 이어지는 반복에서 새 파일을 연다.

RAII 객체를 가능한 좁은 범위에 선언하면 자원 점유 시간을 줄이고 소유 관계를 쉽게 추적할 수 있다.

7.3.4 자원 래퍼 클래스(Resource Wrapper Class)

std::FILE*을 소유하는 이동 전용 FileHandle을 구성한다.

```
#pragma once

#include <cstdio>
#include <stdexcept>
#include <utility>

class FileHandle
{
public:
    FileHandle(const char* fileName, const char* mode)
        : file(std::fopen(fileName, mode))
    {
        if (file == nullptr)
        {
            throw std::runtime_error("파일을 열 수 없습니다.");
        }
    }

    ~FileHandle()
    {
```

```
        Close();
    }

    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;

    FileHandle(FileHandle&& other) noexcept
        : file(other.file)
    {
        other.file = nullptr;
    }

    FileHandle& operator=(FileHandle&& other) noexcept
    {
        if (this == &other)
        {
            return *this;
        }

        Close();

        file = other.file;
        other.file = nullptr;

        return *this;
    }

    void Write(const char* text)
    {
        if (file == nullptr)
        {
            throw std::logic_error("파일이 열려 있지 않습니다.");
        }

        if (std::fputs(text, file) == EOF)
        {
            throw std::runtime_error("파일 쓰기에
실패했습니다.");
        }
    }

    bool IsOpen() const
    {
        return file != nullptr;
    }

    std::FILE* Get() const
    {
        return file;
    }

private:
    void Close() noexcept
    {
        if (file != nullptr)
        {
            std::fclose(file);
            file = nullptr;
        }
    }

    std::FILE* file = nullptr;
};
```

파일 하나는 하나의 객체만 소유해야 하므로 복사를 금지한다.

```
FileHandle first("data.txt", "w");
// FileHandle second = first;    // 오류
```

이동은 파일의 소유권을 새로운 객체로 이전한다.

```
FileHandle first("data.txt", "w");
FileHandle second = std::move(first);

std::cout << std::boolalpha;
std::cout << first.IsOpen() << '\n';
std::cout << second.IsOpen() << '\n';
```

이동 후 first는 빈 상태이고 second가 파일을 소유한다.

```
false
true
```

이 클래스는 5장에서 학습한 이동 전용 자원 객체이다. 복사와 이동은 단순한 성능 기능이 아니라 자원 소유권을 표현하는 수단이다.

자원 래퍼의 공개 인터페이스에서 원시 핸들을 노출해야 한다면 소유권을 넘기는지 단순히 빌려주는지 구분해야 한다.

```
std::FILE* Get() const;
```

Get()이 반환한 포인터는 FileHandle이 계속 소유한다. 호출자는 std::fclose()를 호출해서는 안 되며 FileHandle보다 오래 보관해서도 안 된다.

7.3.5 Rule of Zero와 RAII

자원을 직접 관리하는 저수준 클래스는 소멸자와 이동 연산을 직접 구현해야 할 수 있다. 그러나 일반적인 업무 클래스는 이미 RAII를 구현한 타입을 멤버로 사용하여 특별 멤버 함수를 직접 작성하지 않는 편이 좋다.

```
class SaveSystem
{
public:
    explicit SaveSystem(const std::string& path)
        : path(path)
    {
    }

private:
    std::string path;
    std::vector<std::string> pendingFiles;
};
```

std::string과 std::vector가 자신의 메모리를 관리한다. SaveSystem은 소멸자, 복사 생성자와 이동 생성자를 직접 작성하지 않아도 된다.
스마트 포인터도 RAII 타입이다.

```
class GameScene
{
private:
    std::vector<std::unique_ptr<Enemy>> enemies;
};
```

GameScene이 소멸하면 enemies가 소멸하고, 각 std::unique_ptr<Enemy>가 자신이 소유한 Enemy를 소멸한다. GameScene은 직접 반복하면서 delete할 필요가 없다.

Rule of Zero는 자원을 관리하지 않는다는 뜻이 아니다. 자원 관리 책임을 검증된 RAII 타입에 맡기고 상위 클래스는 자신의 역할과 상태 규칙에 집중한다는 의미이다.

```
저수준 자원 래퍼
FileHandle, unique_ptr, vector, string
        ↓
상위 업무 객체
SaveSystem, GameScene, Player
```

직접 자원 관리가 필요한 코드는 작은 자원 래퍼에 한정하고, 나머지 클래스는 그 래퍼를 값 멤버로 사용하는 구조가 유지보수에 유리하다.

7.4 예외 처리

예외(exception)는 함수가 정상적인 결과를 만들 수 없을 때 오류 정보를 호출자에게 전달하는 기능이다. 예외가 발생하면 현재 함수의 나머지 코드는 실행되지 않고, 처리할 수 있는 catch를 찾을 때까지 호출 경로를 거슬러 올라간다.
예외는 반환값과 분리된 오류 전달 경로를 제공한다.

```
Texture LoadTexture(const std::string& fileName);
```

성공하면 Texture를 반환하고, 파일을 열 수 없거나 형식이 잘못되면 예외를 발생시킬 수 있다. 호출자는 오류 코드를 매 단계에서 전달하지 않고 복구할 수 있는 경계에서 처리한다.

7.4.1 try, throw와 catch

예외는 throw로 발생시킨다.

```
int Divide(int left, int right)
{
    if (right == 0)
    {
        throw std::invalid_argument("0으로 나눌 수 없습니다.");
    }

    return left / right;
}
```

예외가 발생할 수 있는 코드를 try 블록에서 실행하고 catch에서 처리한다.

```
try
{
    int result = Divide(10, 0);
    std::cout << result << '\n';
}
catch (const std::invalid_argument& error)
{
    std::cerr << error.what() << '\n';
}
```

예외 객체는 일반적으로 값으로 던지고 const 참조로 받는다.

```
throw std::runtime_error("리소스 로드 실패");

catch (const std::runtime_error& error)
{
    std::cerr << error.what() << '\n';
}
```

참조로 받으면 예외 객체를 불필요하게 복사하지 않고 실제 예외 타입의 정보를 유지할 수 있다.
여러 예외 타입을 구분하여 처리할 수 있다.

```
try
{
    LoadGameData();
}
catch (const std::invalid_argument& error)
{
    std::cerr << "잘못된 인수: " << error.what() << '\n';
}
catch (const std::runtime_error& error)
{
    std::cerr << "실행 오류: " << error.what() << '\n';
}
catch (const std::exception& error)
{
    std::cerr << "표준 예외: " << error.what() << '\n';
}
```

구체적인 예외를 먼저 처리하고 더 일반적인 예외를 나중에 처리한다. std::invalid_argument와 std::runtime_error는 모두 std::exception 계층에 있으므로 std::exception을 먼저 두면 뒤의 처리기가 선택될 기회가 없다.
모든 예외를 받는 처리기도 작성할 수 있다.

```
catch (...)
{
    std::cerr << "알 수 없는 예외\n";
}
```

현재 예외를 일부 기록한 뒤 상위 호출자에게 다시 전달하려면 피연산자 없는 throw;를 사용한다.

```
try
{
    LoadGameData();
}
catch (const std::exception& error)
{
    WriteLog(error.what());
    throw;
}
```

다시 던질 때 throw error;를 사용하면 정적 타입의 복사본을 새로 던질 수 있어 원래의 구체적인 예외 정보가 손실될 수 있다.
예외는 일반적인 조건 분기를 대신하는 기능이 아니다.

```
// 피하는 편이 좋은 구조
try
{
    UseItem(index);
}
catch (...)
{
    // 인덱스가 없는 경우를 정상 흐름처럼 처리
}
```

아이템 존재 여부처럼 자주 발생하고 예상 가능한 조건은 반환값, bool 또는 선택 값을 이용하는 편이 명확하다. 파일 손상, 자원 획득 실패처럼 현재 함수가 정상 결과를 만들 수 없는 상황에 예외를 활용한다.

7.4.2 표준 예외(Standard Exceptions)

표준 라이브러리는 <stdexcept>와 관련 헤더에 여러 예외 타입을 제공한다.

예외 타입	대표적인 의미
std::exception	표준 예외 계층의 공통 기반 타입
std::logic_error	프로그램 논리나 사전 조건과 관련된 오류
std::invalid_argument	함수 인수가 유효하지 않음
std::out_of_range	인덱스나 값이 허용 범위를 벗어남
std::length_error	요청한 크기가 허용 가능한 최대값을 넘음
std::runtime_error	실행 환경이나 외부 상태로 인해 작업 실패
std::overflow_error	산술 오버플로와 관련된 오류
std::underflow_error	산술 언더플로와 관련된 오류
std::bad_alloc	동적 메모리 할당 실패

범위 검사를 제공하는 함수는 std::out_of_range를 사용할 수 있다.

```
int& FixedArray::At(std::size_t index)
{
    if (index >= size)
    {
        throw std::out_of_range("인덱스 범위 초과");
    }

    return values[index];
}
```

잘못된 생성 인수에는 std::invalid_argument를 사용할 수 있다.

```
Rectangle::Rectangle(int width, int height)
    : width(width), height(height)
{
    if (width <= 0 || height <= 0)
    {
        throw std::invalid_argument("크기는 양수여야 합니다.");
    }
}
```

파일 열기나 리소스 로드처럼 외부 상태에 따라 실패할 수 있는 작업은 std::runtime_error로 표현할 수 있다.

```
if (file == nullptr)
{
    throw std::runtime_error("파일 열기 실패");
}
```

what()은 오류 설명 문자열을 반환한다.

```
catch (const std::exception& error)
{
    std::cerr << error.what() << '\n';
}
```

표준 예외 타입만으로 의미를 충분히 표현할 수 있으면 새로운 예외 타입을 만들 필요가 없다. 호출자가 오류 종류를 별도로 구분해야 하거나 추가 정보를 제공해야 한다면 사용자 정의 예외를 고려한다.

7.4.3 사용자 정의 예외(User-defined Exception)

게임 리소스 로드 오류를 구분하기 위한 예외 타입을 정의한다.

```
#include <stdexcept>
#include <string>

class ResourceError : public std::runtime_error
{
public:
    ResourceError(
        const std::string& resourceName,
        const std::string& message
    )
        : std::runtime_error(message),
          resourceName(resourceName)
    {
    }

    const std::string& ResourceName() const
    {
        return resourceName;
    }

private:
    std::string resourceName;
};
```

ResourceError는 오류 메시지와 리소스 이름을 함께 저장한다. 상속 문법과 예외 계층의 자세한 구성은 8장에서 다룬다.
리소스 로더에서 예외를 발생시킨다.

```
Texture LoadTexture(const std::string& fileName)
{
    if (!FileExists(fileName))
    {
        throw ResourceError(
            fileName,
            "텍스처 파일을 찾을 수 없습니다."
        );
```

```
    }

    return DecodeTexture(fileName);
}
```

호출자는 리소스 정보를 확인할 수 있다.

```
try
{
    Texture texture = LoadTexture("hero.png");
}
catch (const ResourceError& error)
{
    std::cerr << "resource: "
              << error.ResourceName() << '\n';
    std::cerr << error.what() << '\n';
}
```

사용자 정의 예외는 표준 예외를 기반으로 하되 처리에 필요한 정보만 가지도록 단순하게 구성한다.

- 표준 예외 타입을 기반으로 구성
- 오류를 처리하는 데 필요한 정보만 저장
- 예외 객체 생성 과정 자체가 복잡한 작업을 하지 않음
- 오류 분류가 필요하지 않으면 표준 예외를 그대로 사용
- 문자열 비교로 오류 종류를 판단하지 않도록 타입이나 오류 코드를 제공

7.4.4 스택 되감기(Stack Unwinding)

예외가 발생하면 일치하는 **catch**를 찾을 때까지 현재 호출 경로의 함수가 종료된다. 이 과정에서 생성이 완료된 자동 객체가 역순으로 소멸한다. 이를 스택 되감기라고 한다.

```
void LoadLevel()
{
    TraceObject first("first");
    FileHandle file("level.dat", "rb");
    TraceObject second("second");

    throw std::runtime_error("레벨 로드 실패");
}

try
{
    LoadLevel();
}
catch (const std::exception& error)
{
    std::cerr << error.what() << '\n';
}
```

예외가 발생하면 **second, file, first** 순서로 소멸한다. **FileHandle**의 소멸자가 파일을 닫으므로 별도의 예외 처리 코드가 없어도 자원이 정리된다.

```
first 생성 → 파일 열기 → second 생성 → 예외 발생
second 소멸 → 파일 닫기 → first 소멸 → catch 실행
```

원시 포인터가 가리키는 동적 객체는 자동으로 소멸하지 않는다.

```
void LoadLevel()
{
    Resource* resource = new Resource(1);

    throw std::runtime_error("실패");

    delete resource;
}
```

예외가 발생하면 포인터 변수는 사라지지만 동적 객체는 누수된다.

RAII 타입을 사용하면 스택 되감기에 참여한다.

```
void LoadLevel()
{
    std::unique_ptr<Resource> resource =
        std::make_unique<Resource>(1);

    throw std::runtime_error("실패");
}
```

예외로 함수를 벗어날 때 resource가 소멸하여 Resource 객체도 정리된다.
스택 되감기는 모든 지역 변수를 무조건 소멸하는 것이 아니다. 생성이 완료된 객체만 소멸한다. 선언 지점에 도달하지 않은 객체나 생성 도중 실패한 객체 자체의 소멸자는 호출되지 않는다.

7.4.5 생성자와 예외

생성자가 객체의 유효한 상태를 만들 수 없으면 예외를 발생시켜 생성을 실패하게 할 수 있다.

```
class Texture
{
public:
    explicit Texture(const std::string& fileName)
    {
        if (!Load(fileName))
        {
            throw ResourceError(fileName, "텍스처 로드 실패");
        }
    }
};
```

생성자가 예외로 끝나면 완성된 Texture 객체는 만들어지지 않는다. 따라서 Texture의 소멸자는 호출되지 않는다.
다만 생성자 본문에 들어오기 전에 생성이 완료된 멤버 객체는 자동으로 소멸한다.

```
class LevelPackage
{
public:
    LevelPackage()
        : config("config.txt", "rb"),
          map("missing.map", "rb")
    {
    }

    ~LevelPackage()
    {
        std::cout << "LevelPackage 소멸\n";
    }

private:
    FileHandle config;
    FileHandle map;
};
```

config 파일이 열린 뒤 map 생성에서 예외가 발생하면 생성이 완료된 멤버만 역순으로 정리된다.

```
config 생성 완료 → map 생성 실패
config 소멸 → LevelPackage 객체 생성 실패
LevelPackage 소멸자는 호출되지 않음
```

멤버를 RAII 타입으로 구성하면 생성 도중 실패해도 이미 획득한 자원이 정리된다. 원시 포인터 여러 개를 생성자 본문에서 순서대로 할당하면 중간 실패 시 앞에서 확보한 자원을 직접 정리해야 한다.

```
class BadPackage
{
public:
    BadPackage()
    {
```

```
        first = new Resource(1);
        second = new Resource(2);  // 여기서 예외 가능
    }

private:
    Resource* first = nullptr;
    Resource* second = nullptr;
};
```

second 할당이나 생성에서 예외가 발생하면 BadPackage의 소멸자는 호출되지 않으므로 first가 누수될 수 있다. 멤버를 std::unique_ptr<Resource>로 바꾸면 이미 생성된 멤버가 자동으로 정리된다.

7.4.6 소멸자와 예외

소멸자는 자원을 정리하는 함수이므로 외부로 예외를 전달하지 않도록 설계해야 한다.

```
class Resource
{
public:
    ~Resource() noexcept
    {
        Release();
    }
};
```

스택 되감기 중 소멸자가 실행되는 상황을 생각해 보자. 이미 하나의 예외가 처리기를 찾고 있는데 소멸자에서 또 다른 예외가 밖으로 나오면 프로그램은 계속 안전하게 되감을 수 없다. 이 경우 std::terminate()가 호출될 수 있다.

소멸자에서 실패할 수 있는 작업이 필요하다면 좀 더 상세하게 구조를 고려한다.

- 실패하지 않는 정리 작업만 소멸자에서 수행
- 오류를 기록하되 소멸자 밖으로 예외를 내보내지 않음
- 실패 여부를 호출자가 처리해야 한다면 별도의 Close(), Commit() 또는 Flush() 제공
- 사용자가 명시적 함수를 호출하지 않았을 때 소멸자에서는 가능한 범위의 최종 정리 수행

```
class OutputFile
{
public:
    void Commit()
    {
        if (!FlushData())
        {
            throw std::runtime_error("파일 저장 실패");
        }

        committed = true;
    }

    ~OutputFile() noexcept
    {
        if (!committed)
        {
            TryDiscardWithoutThrowing();
        }
    }

private:
    bool committed = false;
};
```

오류를 반드시 호출자에게 알려야 하는 작업은 명시적 함수에서 수행하고, 소멸자는 예외를 내보내지 않는 정리 경로를 유지한다.

7.5 예외 안전성(Exception Safety)

예외 안전성은 예외를 발생시키지 않는다는 뜻이 아니다. 연산 도중 예외가 발생해도 자원이 누수되지 않고 객체가 약속한 상태를 유지하도록 설계하는 것이다.

```
예외 안전한 함수는 먼저 객체의 불변식 유지
ex)
    Inventory의 itemCount는 실제 아이템 개수와 일치
    FileHandle이 열린 상태라면 유효한 파일을 하나 소유
    Image width * height는 픽셀 저장 공간과 일치
```

예외가 발생한 뒤에도 객체가 소멸 가능하고 공개 멤버 함수를 안전하게 호출할 수 있어야 한다.

7.5.1 예외 발생 후 객체 상태

제시된 AddItem() 함수는 아이템 이름과 개수를 별도로 갱신한다.

```
class Inventory
{
public:
    void AddItem(const std::string& item)
    {
        ++itemCount;
        items.push_back(item);
    }

private:
    int itemCount = 0;
    std::vector<std::string> items;
};
```

++itemCount가 실행된 뒤 push_back()에서 예외가 발생하면 itemCount와 실제 원소 개수가 달라진다. 객체의 불변식이 깨진다.
갱신 순서를 바꾸면 실패 전 상태를 더 쉽게 유지할 수 있다.

```
void AddItem(const std::string& item)
{
    items.push_back(item);
    itemCount = static_cast<int>(items.size());
}
```

push_back()이 실패하면 itemCount는 아직 변경되지 않는다. 성공한 뒤 실제 크기를 반영한다.
더 근본적으로 같은 정보를 두 곳에 중복 저장하지 않는 설계를 선택할 수 있다.

```
class Inventory
{
public:
    std::size_t ItemCount() const
    {
        return items.size();
    }

private:
    std::vector<std::string> items;
};
```

예외 안전성은 try와 catch를 많이 작성하는 문제가 아니라, 실패 가능한 연산의 순서와 객체 상태 표현을 설계하는 문제이다.

일반적으로 예외 발생 후 상태를 기준으로 보장 수준을 구분해서 제공한다.

보장 수준	예외 발생 후 상태
보장 없음	자원 누수나 불변식 손상이 발생할 수 있음
기본 보장	자원 누수가 없고 객체는 유효하지만 값은 변경될 수 있음
강한 보장	연산 전 상태가 유지됨
무 예외 보장	연산이 예외를 발생시키지 않음

모든 함수가 가장 강한 보장을 제공해야 하는 것은 아니다. 연산의 의미, 비용과 구현 가능성을 고려하여 적절한 수준을 정하고 인터페이스와 문서에서 일관되게 유지한다.

7.5.2 기본 보장(Basic Guarantee)

기본 보장은 예외가 발생해도 자원과 객체의 유효성을 유지한다.

- 자원이 누수되지 않음
- 객체의 불변식이 유지됨
- 객체를 소멸하거나 다시 사용할 수 있음
- 일부 상태 변경은 남을 수 있음

여러 작업을 순서대로 적용하는 함수를 생각해 보자.

```
void ApplyItems(
    Inventory& inventory,
    const std::vector<std::string>& items
)
{
    for (const std::string& item : items)
    {
        inventory.AddItem(item);
    }
}
```

세 번째 아이템 추가에서 예외가 발생하면 앞의 두 아이템은 이미 추가되어 있을 수 있다. 연산 전 상태로 되돌아가지는 않지만 Inventory는 여전히 유효하고 자원도 누수되지 않는다. 이 함수는 기본 보장을 제공할 수 있다.

게임 월드에 여러 명령을 순서대로 적용하는 경우에도 이전 명령의 결과를 취소하기 어려울 수 있다. 이런 연산은 부분 성공을 허용하는지, 별도의 롤백이 필요한지 인터페이스에서 정해야 한다.

기본 보장은 약한 보장이 아니라 실용적인 보장이다. 강한 보장을 만들기 위한 복사 비용이 너무 크거나 외부 시스템 작업을 되돌릴 수 없다면 기본 보장을 명확히 제공하는 편이 낫다.

7.5.3 강한 보장(Strong Guarantee)

강한 보장은 연산이 성공하면 전체 결과를 반영하고, 실패하면 연산 전 상태를 유지한다. 트랜잭션과 비슷한 의미를 가진다.

```
성공 → 새 상태 반영
실패 → 이전 상태 유지
```

새 상태를 임시 객체에 먼저 구성한 뒤 성공한 경우에만 현재 객체와 교체할 수 있다.

```
class Inventory
{
public:
    void ReplaceItems(
        const std::vector<std::string>& newItems
    )
    {
        std::vector<std::string> copy = newItems;

        items.swap(copy);
    }

private:
    std::vector<std::string> items;
};
```

copy 생성에서 예외가 발생하면 items는 변경되지 않는다. 복사가 성공한 뒤 swap()으로 상태를 교체한다.
강한 보장을 제공하려면 최종 반영 단계가 예외를 발생시키지 않거나 실패 전에 원래 상태를 유지할 수 있어야 한다.

강한 보장은 상태 복사나 롤백 비용이 큰 작업에서는 제공하기 어렵거나 비용이 커질 수 있다.

- 매우 큰 상태 전체를 복사해야 함
- 파일이나 네트워크에 이미 외부 효과를 기록함
- 여러 객체를 동시에 변경하고 되돌려야 함
- 운영체제나 장치 작업이 롤백을 지원하지 않음

이때는 기본 보장을 제공하고 부분 적용 가능성을 문서화하거나, 명시적인 트랜잭션 객체를 설계할 수 있다.

7.5.4 무 예외 보장(No-throw Guarantee)

무 예외 보장은 주로 소멸과 자원 정리처럼 실패를 밖으로 전달하기 어려운 연산에 적용한다.

- 소멸자
- 자원 해제 함수
- 단순한 상태 조회
- 포인터와 정수 값의 교환
- 예외를 발생시키지 않는 이동 연산

```
class FileHandle
{
public:
    ~FileHandle() noexcept
    {
        Close();
    }

private:
    void Close() noexcept
    {
        if (file != nullptr)
        {
            std::fclose(file);
            file = nullptr;
        }
    }

    std::FILE* file = nullptr;
};
```

무 예외 보장은 단순히 함수 선언에 noexcept를 붙인다고 만들어지는 것이 아니다. 함수가 호출하는 모든 작업이 실제로 예외를 밖으로 전달하지 않아야 한다.

```
void SafeOperation() noexcept
{
    MayThrow();    // 예외가 밖으로 나오면 terminate
}
```

실패 가능성이 있지만 반환값으로 처리할 수 있는 정리 함수는 bool이나 오류 코드를 사용할 수 있다.

```
bool TryClose() noexcept;
```

모든 오류를 무시해서 무 예외 보장을 만드는 것은 올바른 설계가 아니다. 호출자가 반드시 알아야 하는 실패는 별도의 예외 가능 함수나 반환값으로 전달한다.

7.5.5 복사-교환과 강한 보장

5장에서 복사 대입 연산자를 구현할 때 복사-교환 기법을 사용했다.

```
IntArray& IntArray::operator=(const IntArray& other)
{
    if (this == &other)
    {
        return *this;
    }

    IntArray copy(other);
    Swap(copy);

    return *this;
}
```

복사본 생성에서 예외가 발생하면 현재 객체는 변경되지 않는다.

```
1. other의 복사본 생성
```

```
   실패 → 현재 객체 유지
2. 현재 객체와 복사본 교환
   성공 → 새 상태 반영
3. 임시 복사본 소멸
   이전 자원 정리
```

Swap()이 예외를 발생시키지 않으면 복사 대입은 강한 보장을 제공하기 쉽다.

```
void IntArray::Swap(IntArray& other) noexcept
{
    using std::swap;

    swap(size, other.size);
    swap(data, other.data);
}
```

복사-교환 기법은 구현을 단순하게 하지만 매번 새로운 자원을 복사할 수 있다. 기존 버퍼를 재사용할 수 있는 타입에서는 더 효율적인 대입 구현을 선택할 수도 있다. 이 경우에도 중간 실패 뒤 객체의 불변식을 유지해야 한다.
값 매개변수를 사용하는 통합 대입 방식도 있다.

```
IntArray& operator=(IntArray other)
{
    Swap(other);
    return *this;
}
```

lvalue를 전달하면 복사하여 other를 만들고 rvalue를 전달하면 이동하여 만들 수 있다. 이 방식은 복사 대입과 이동 대입을 하나의 함수로 처리하지만, 별도의 복사 대입과 이동 대입을 제공하는 방식과 동시에 사용하지 않는다.

7.5.6 noexcept의 설계 기준

noexcept는 함수가 예외를 밖으로 전달하지 않는다는 계약이다.

```
FileHandle(FileHandle&& other) noexcept;
FileHandle& operator=(FileHandle&& other) noexcept;
void Swap(FileHandle& other) noexcept;
```

noexcept는 실제로 실패하지 않는 이동·교환·정리 함수에 지정한다.

- 소멸자와 자원 해제 함수
- 단순한 이동 생성자와 이동 대입 연산자
- 포인터나 기본형 멤버만 교환하는 Swap()
- 실패 가능성이 없는 조회 함수
- 컨테이너가 재배치에 활용해야 하는 이동 연산

실제로 예외가 발생할 수 있는 함수에는 무조건 붙이지 않는다.

```
void LoadFile(const std::string& fileName);  // 실패 가능
```

noexcept 함수에서 예외가 밖으로 나오면 호출자가 catch로 복구할 기회를 얻지 못하고 std::terminate()가 호출된다.
멤버의 동작에 따라 조건부로 지정할 수도 있다.

```
template <typename T>
void SwapValue(T& first, T& second)
    noexcept(noexcept(std::swap(first, second)))
{
    std::swap(first, second);
}
```

템플릿의 자세한 문법은 이후 장에서 다룬다. 여기서는 내부 연산이 무 예외일 때만 외부 함수도 무 예외로 선언할 수 있다는 원리를 확인한다.
noexcept는 성능을 위한 장식이 아니라 예외 안전성 계약이다. 특히 표준 컨테이너는 원소 타입의 이동 생성자가 noexcept인지에 따라 재배치 과정에서 이동과 복사 중 안전한 방법을 선택할 수 있다.

7.6 std::unique_ptr

std::unique_ptr<T>는 동적 객체 하나에 대한 고유 소유권을 표현하는 스마트 포인터이다. 하나의 unique_ptr만 객체를 소유하며 소유자가 소멸하면 동적 객체도 자동으로 소멸한다.

```
#include <memory>
std::unique_ptr<Enemy> enemy(new Enemy);
```

직접 delete를 호출하지 않는다.

```
void Play()
{
    std::unique_ptr<Enemy> enemy(new Enemy);
    enemy->Update();
}   // Enemy 자동 소멸
```

7.6.1 고유 소유권(Unique Ownership)

std::unique_ptr<Enemy>는 자신이 가리키는 Enemy를 단독으로 소유한다.

```
unique_ptr ──────► Enemy
   소유자 1개
```

원시 포인터와 비슷한 접근 문법을 제공한다.

```
std::unique_ptr<Enemy> enemy = std::make_unique<Enemy>();
enemy->Update();
Enemy& reference = *enemy;
```

빈 스마트 포인터도 만들 수 있다.

```
std::unique_ptr<Enemy> enemy;
```

빈 상태는 조건식에서 확인한다.

```
if (enemy)
{
    enemy->Update();
}
```

nullptr와 비교할 수도 있다.

```
if (enemy == nullptr)
{
    std::cout << "Enemy 없음\n";
}
```

unique_ptr가 소멸하면 내부 포인터에 맞는 삭제 연산을 수행한다.

```
{
    std::unique_ptr<Enemy> enemy =
        std::make_unique<Enemy>();
}
```

블록 끝에서 Enemy의 소멸자가 호출되고 메모리가 해제된다.
동적 객체가 꼭 필요한 경우가 아니라면 일반 값 객체를 우선한다.

```
Enemy enemy;
```

unique_ptr는 고유 소유권이 필요하고 객체를 값이나 지역 변수로 둘 수 없는 상황에 적합하다.

- 객체의 수명이 현재 범위보다 길어질 수 있음
- 소유권을 다른 객체나 함수로 이전해야 함
- 객체의 실제 타입을 실행 중 결정해야 함
- 큰 객체의 주소를 안정적으로 유지해야 함

- 객체가 선택적으로 존재하여 빈 상태가 필요함

7.6.2 std::make_unique

C++14부터 std::make_unique()로 unique_ptr와 동적 객체를 함께 생성할 수 있다.

```
std::unique_ptr<Enemy> enemy = std::make_unique<Enemy>(100,
20);
```

auto를 사용하면 타입을 반복하지 않아도 된다.

```
auto enemy = std::make_unique<Enemy>(100, 20);
```

new를 직접 전달해 unique_ptr를 생성하는 방식보다 make_unique()를 우선한다.

```
std::unique_ptr<Enemy> enemy(new Enemy(100, 20));
```

make_unique()를 사용하면 코드와 소유권 연결이 단순해진다.

- new와 타입 이름의 반복을 줄임
- 생성된 객체를 즉시 스마트 포인터에 연결
- 배열 생성 문법을 일관되게 제공
- 소유권이 없는 원시 포인터가 중간에 드러나는 것을 줄임

생성자 인수는 객체 생성자에 전달된다.

```
class Enemy
{
public:
    Enemy(int hp, int attack)
        : hp(hp), attack(attack)
    {
    }

private:
    int hp;
    int attack;
};
```

```
auto enemy = std::make_unique<Enemy>(100, 20);
```

기본 초기화와 값 초기화의 차이도 동적 객체에 적용된다.
auto value = std::make_unique<int>();

정수 객체는 값 초기화되어 0이 된다.
커스텀 삭제자가 필요한 경우에는 make_unique()를 사용할 수 없으므로 생성자에 원시 핸들과 삭제자를 직접 전달한다.

7.6.3 이동 전용 스마트 포인터

고유 소유권은 두 객체가 동시에 가질 수 없으므로 unique_ptr는 복사할 수 없다.

```
auto first = std::make_unique<Enemy>();
// auto second = first;    // 오류
```

소유권은 이동으로 이전한다.

```
auto first = std::make_unique<Enemy>();
auto second = std::move(first);
```

이동 후 first는 비어 있고 second가 객체를 소유한다.

```
first  ──────→ nullptr
second ──────→ Enemy
```

```
std::cout << std::boolalpha;
std::cout << static_cast<bool>(first) << '\n';
std::cout << static_cast<bool>(second) << '\n';

false
true
```

소유권 이전 뒤 원본은 빈 상태가 된다. 다시 새로운 객체를 소유하도록 대입할 수 있다.

```
first = std::make_unique<Enemy>();
```

클래스가 unique_ptr 멤버를 가지면 컴파일러가 생성하는 복사 연산도 사용할 수 없게 된다.

```
class GameScene
{
private:
    std::unique_ptr<Enemy> boss;
};
```

GameScene을 복사하면 하나의 Enemy를 두 장면이 고유 소유해야 하는 모순이 생기므로 기본 복사는 삭제된다. 이동은 멤버의 이동을 통해 제공될 수 있다.

장면 복사가 필요하다면 복사 의미를 명시적으로 정해야 한다.

- 각 객체를 깊은 복사
- 공유 가능한 리소스만 shared_ptr로 분리
- 장면 복사를 금지하고 이동만 허용
- 복제 목적의 Clone() 함수 제공

스마트 포인터를 선택하면 클래스의 복사 정책에도 영향을 준다.

7.6.4 배열 소유

동적 배열의 고유 소유권은 std::unique_ptr<T[]>로 표현한다.

```
std::unique_ptr<int[]> values = std::make_unique<int[]>(10);
```

원소는 인덱스 연산자로 접근한다.

```
for (std::size_t i = 0; i < 10; ++i)
{
    values[i] = static_cast<int>(i * 10);
}
```

unique_ptr<T[]>는 소멸할 때 delete[]를 사용한다. 단일 객체용 unique_ptr<T>와 배열용 unique_ptr<T[]>가 타입에서 구분된다.

```
std::unique_ptr<int> one = std::make_unique<int>(100);
std::unique_ptr<int[]> many = std::make_unique<int[]>(100);
```

배열 크기는 unique_ptr<T[]>에 저장되지 않는다.

```
class ImageBuffer
{
public:
    ImageBuffer(int width, int height)
        : width(width),
          height(height),
          pixels(std::make_unique<unsigned int[]>(
              static_cast<std::size_t>(width) * height
          ))
    {
    }

private:
    int width;
    int height;
    std::unique_ptr<unsigned int[]> pixels;
```

```
};
```

크기, 반복, 삽입과 삭제가 필요한 일반적인 배열은 std::vector<T>가 더 적합하다. unique_ptr<T[]>는 크기가 생성 시 결정되고 배열 인터페이스가 필요하지 않은 저수준 버퍼에 활용할 수 있다.

7.6.5 get, reset과 release

unique_ptr는 원시 포인터를 확인하거나 소유 객체를 교체하는 함수를 제공한다.

get
get()은 소유권을 유지한 채 내부 원시 포인터를 반환한다.

```
auto enemy = std::make_unique<Enemy>();
Enemy* pointer = enemy.get();
```

pointer는 비소유 포인터이다. 호출자는 delete pointer를 실행해서는 안 된다.

```
UseEnemy(enemy.get());
```

기존 C API나 원시 포인터를 받는 함수에 잠시 전달할 때 활용할 수 있다. 반환된 포인터는 원래 unique_ptr보다 오래 사용할 수 없다.

reset
reset()은 현재 소유 객체를 소멸하고 빈 상태로 만든다.

```
auto enemy = std::make_unique<Enemy>();
enemy.reset();
```

새 원시 포인터를 전달하면 기존 객체를 정리한 뒤 새 객체를 소유한다.

```
enemy.reset(new Enemy(200, 30));
```

일반적으로는 새 스마트 포인터를 대입하는 편이 더 분명하다.

```
enemy = std::make_unique<Enemy>(200, 30);
```

release
release()는 객체를 소멸하지 않고 소유권만 포기하며 원시 포인터를 반환한다.

```
auto enemy = std::make_unique<Enemy>();
Enemy* pointer = enemy.release();
```

이후 enemy는 빈 상태이고 pointer의 해제 책임은 호출자에게 넘어간다.

```
delete pointer;
```

release()의 반환값을 잃으면 즉시 메모리 누수가 발생한다.

```
// enemy.release();    // 반환 포인터를 잃어버림
```

<unique_ptr의 소유권 조작>

함수	객체 소멸	unique_ptr의 소유권	반환값
get()	하지 않음	유지	비소유 원시 포인터
reset()	기존 객체 소멸	비우거나 새 객체 소유	없음
release()	하지 않음	포기	해제 책임이 있는 원시 포인터

release()는 소유권을 원시 포인터 기반 API에 넘겨야 하는 경우에만 제한적으로 사용한다. 단순히 객체에 접근하려면 get()을 사용한다.

7.6.6 사용자 정의 삭제자(Custom Deleter)

unique_ptr는 기본적으로 단일 객체에 delete, 배열에 delete[]를 사용한다. 다른 해제 함수를 호출해야 하는 자원에는 사용자 정의 삭제자를 지정할 수 있다.

std::FILE*은 delete가 아니라 std::fclose()로 닫아야 한다.

```
struct FileCloser
{
    void operator()(std::FILE* file) const noexcept
    {
        if (file != nullptr)
        {
            std::fclose(file);
        }
    }
};
```

삭제자 타입을 두 번째 템플릿 인수로 지정한다.

```
using FilePointer = std::unique_ptr<std::FILE, FileCloser>;

FilePointer OpenFile(
    const char* fileName,
    const char* mode
)
{
    std::FILE* file = std::fopen(fileName, mode);

    if (file == nullptr)
    {
        throw std::runtime_error("파일 열기 실패");
    }

    return FilePointer(file);
}

void Save()
{
    FilePointer file = OpenFile("save.txt", "w");

    std::fputs("save data\n", file.get());
}
```

범위를 벗어나면 FileCloser가 호출되어 파일을 닫는다.
함수 포인터를 삭제자로 사용할 수도 있다.

```
using FilePointer = std::unique_ptr<
    std::FILE,
    int (*)(std::FILE*)
>;

FilePointer file(
    std::fopen("data.txt", "w"),
    &std::fclose
);
```

삭제자 타입은 unique_ptr 타입의 일부이다. 상태가 없는 작은 함수 객체 삭제자는 최적화되어 스마트 포인터 크기에 추가 비용이 없을 수 있지만, 함수 포인터나 상태를 가진 삭제자는 객체 크기를 늘릴 수 있다.

사용자 정의 삭제자는 표준 삭제 방식으로 정리할 수 없는 자원을 RAII로 관리할 때 활용한다.

- C 파일 핸들
- 운영체제 핸들
- 그래픽 API 리소스
- 라이브러리 전용 생성·해제 함수 쌍
- 메모리 풀에서 획득한 객체

이미 해당 자원에 맞는 표준 또는 라이브러리 RAII 래퍼가 있다면 직접 삭제자를 만들기보다 기존 타입을 우선한다.

7.6.7 함수 매개변수와 반환값

함수의 스마트 포인터 타입은 소유권 전달 방식을 표현한다.

값을 반환하여 새 소유권 전달
팩토리 함수는 unique_ptr를 값으로 반환할 수 있다.

```
std::unique_ptr<Enemy> CreateEnemy(int hp, int attack)
{
    return std::make_unique<Enemy>(hp, attack);
}

auto enemy = CreateEnemy(100, 20);
```

반환값은 복사되지 않고 복사 생략 또는 이동으로 호출자에게 소유권이 전달된다.

값 매개변수로 소유권 받기
함수가 객체의 소유권을 가져가면 unique_ptr<T>를 값으로 받는다.

```
class GameScene
{
public:
    void AddEnemy(std::unique_ptr<Enemy> enemy)
    {
        enemies.push_back(std::move(enemy));
    }

private:
    std::vector<std::unique_ptr<Enemy>> enemies;
};
```

호출자는 명시적으로 이동한다.

```
auto enemy = CreateEnemy(100, 20);
scene.AddEnemy(std::move(enemy));
```

호출 뒤 enemy는 빈 상태가 된다. 인터페이스만 보고도 소유권이 GameScene으로 이동한다는 사실을 알 수 있다.
임시 객체는 바로 전달할 수 있다.

```
scene.AddEnemy(CreateEnemy(100, 20));
```

소유권 없이 객체 사용
함수가 객체를 사용하기만 한다면 스마트 포인터 자체보다 객체의 참조를 받는다.

```
void DrawEnemy(const Enemy& enemy);
void UpdateEnemy(Enemy& enemy);

DrawEnemy(*enemy);
UpdateEnemy(*enemy);
```

객체가 없을 수 있는 선택적 비소유 접근은 원시 포인터를 사용할 수 있다.

```
void SetTarget(Enemy* enemy);

SetTarget(enemy.get());
SetTarget(nullptr);
```

const std::unique_ptr<Enemy>&를 매개변수로 받는 방식은 함수가 스마트 포인터 객체 자체의 정책에 의존해야 할 때만 사용한다. 단순히 Enemy를 읽는 목적이라면 const Enemy&가 더 일반적이다.

<unique_ptr와 함수 인터페이스>

인터페이스	의미
std::unique_ptr<T> Create()	새 고유 소유권을 호출자에게 반환
void Take(std::unique_ptr<T>)	호출자로부터 고유 소유권을 받음
void Replace(std::unique_ptr<T>&)	호출자의 스마트 포인터 자체를 변경할 수 있음
void Use(T&)	객체를 변경 가능하게 빌림

void Read(const T&)	객체를 읽기 전용으로 빌림
void Observe(T*)	객체가 없을 수도 있는 비소유 접근

소유권이 필요하지 않은 함수에 스마트 포인터를 강요하지 않는다. 함수가 객체의 수명 정책을 알아야 하는지, 객체 기능만 사용하면 되는지를 기준으로 매개변수 타입을 결정한다.

7.7 std::shared_ptr

std::shared_ptr<T>는 여러 스마트 포인터가 하나의 동적 객체를 공동으로 소유하도록 한다. 마지막 소유자가 사라질 때 객체가 소멸한다.

```
std::shared_ptr<Texture> texture =
    std::make_shared<Texture>("hero.png");
```

shared_ptr를 복사하면 새로운 객체를 복사하는 것이 아니라 같은 Texture의 소유권을 공유한다.

7.7.1 공유 소유권(Shared Ownership)

하나의 텍스처를 여러 스프라이트가 사용할 수 있다.

```
class Sprite
{
public:
    explicit Sprite(std::shared_ptr<Texture> texture)
        : texture(std::move(texture))
    {
    }

private:
    std::shared_ptr<Texture> texture;
};

auto texture =
    std::make_shared<Texture>("character.png");

Sprite idle(texture);
Sprite run(texture);
Sprite attack(texture);
```

네 개의 shared_ptr가 하나의 Texture를 소유한다.

```
texture ─┐
idle    ─┤
run     ─┼──▶ Texture
attack  ─┘
```

각 Sprite가 소멸해도 다른 소유자가 남아 있으면 Texture는 유지된다. 마지막 shared_ptr가 소멸하거나 소유권을 포기하면 Texture가 소멸한다.

공유 소유권은 마지막 사용자를 미리 특정하기 어려운 구조에 적합할 수 있다.

- 여러 객체가 하나의 리소스를 사용하며 어느 객체가 마지막 사용자인지 미리 알기 어려움
- 비동기 작업이 완료될 때까지 객체 수명을 유지해야 함
- 데이터가 여러 시스템에 전달되어 독립적으로 보관됨
- 소유 주체가 하나로 결정되지 않는 공유 데이터

단순히 포인터를 복사하기 편하다는 이유로 사용하면 실제 수명 구조를 숨길 수 있다. 가능하면 고유 소유자를 정하고, 공유 수명이 도메인의 의미와 일치할 때만 사용한다.

7.7.2 참조 횟수와 제어 블록

shared_ptr는 객체 포인터와 제어 블록(control block)을 이용한다.

```
shared_ptr A ─┐
              ├──▶ 객체
shared_ptr B ─┘
```

```
        |
        └──────────▶ 제어 블록
                     강한 참조 횟수
                     약한 참조 횟수
                     삭제자 정보
```

shared_ptr를 복사하면 강한 참조 횟수가 증가한다.

```
auto first = std::make_shared<Texture>("hero.png");
auto second = first;
std::cout << first.use_count() << '\n';
```

두 소유자가 있으므로 일반적으로 2가 출력된다.

한 소유자가 해제되면 참조 횟수가 감소한다.

```
second.reset();
```

강한 참조 횟수가 0이 되면 관리 대상 객체가 소멸한다. 제어 블록은 해당 객체를 관찰하는 weak_ptr가 남아 있다면 약한 참조 정보가 사라질 때까지 유지될 수 있다.

use_count()는 학습이나 진단에 활용할 수 있지만 프로그램의 핵심 로직을 참조 횟수 값에 의존시키지 않는 편이 좋다.

```
if (texture.use_count() == 1)
{
    // 이 값을 근거로 중요한 동작을 결정하지 않음
}
```

다른 코드가 소유권을 복사하거나 해제할 수 있으며, 동시 실행 환경에서는 확인 직후 값이 달라질 수 있다. 공유 여부가 기능의 조건이라면 별도의 상태와 동기화 규칙으로 표현한다.

7.7.3 std::make_shared

shared_ptr는 std::make_shared()로 생성하는 편이 좋다.

```
auto texture = std::make_shared<Texture>("hero.png");
```

직접 생성할 수도 있다.

```
std::shared_ptr<Texture> texture(
    new Texture("hero.png")
);
```

make_shared()는 일반적으로 관리 대상 객체와 제어 블록을 하나의 메모리 할당으로 구성할 수 있다. 이 방식은 생성 비용과 예외 안전성 측면에서 이점을 제공한다.

- 객체와 제어 블록 생성 비용 감소 가능
- 메모리 지역성 향상 가능
- 원시 포인터가 중간에 노출되지 않음
- 객체 생성과 제어 블록 생성의 예외 안전성 확보

생성자 인수는 객체 생성자에 전달된다.

```
auto texture = std::make_shared<Texture>("hero.png", TextureFilter::Linear);
```

사용자 정의 삭제자가 필요하거나 객체와 제어 블록의 메모리 배치를 별도로 제어해야 한다면 shared_ptr 생성자를 직접 사용할 수 있다. make_shared()로 생성한 객체는 강한 참조가 0이 되면 소멸하지만, weak_ptr가 남아 있으면 객체와 제어 블록을 함께 할당한 메모리 영역 자체는 제어 블록이 해제될 때까지 유지될 수 있다. 매우 큰 객체에서 이 차이가 중요하다면 별도 할당 방식을 검토한다.

7.7.4 공유 소유권의 비용

shared_ptr는 고유 소유권보다 더 많은 관리 비용이 필요하다.

- 제어 블록을 위한 메모리
- 복사와 소멸 시 참조 횟수 변경
- 동시 실행을 고려한 참조 횟수 조작

- 실제 소유자가 코드에서 불분명해질 가능성
- 순환 참조 가능성
- 객체 수명이 예상보다 길어질 가능성

shared_ptr 자체의 참조 횟수 관리는 여러 스레드에서 안전하게 갱신되도록 구현되지만, 관리 대상 객체의 멤버 접근까지 자동으로 동기화하지는 않는다.

```
std::shared_ptr<Player> player =  std::make_shared<Player>();
```

여러 스레드가 player가 가리키는 Player의 상태를 동시에 변경한다면 별도의 잠금이나 동기화가 필요하다.

공유 소유권을 사용하기 전에는 여러 주체가 실제로 객체 수명을 함께 책임지는지 확인한다.

```
정말 여러 주체가 객체의 수명을 함께 책임지는가?
```

단지 여러 곳에서 객체를 사용한다면 고유 소유자 하나와 비소유 참조 여러 개로 표현할 수 있다.

```
GameScene ── unique_ptr ──▶ Player
Enemy     ── raw pointer ─▶ Player
UI        ── reference ───▶ Player
```

이 구조에서 GameScene만 Player를 소유하고 Enemy와 UI는 장면이 살아 있는 동안 잠시 접근한다.

7.7.5 잘못된 shared_ptr 생성

같은 원시 포인터로 독립된 shared_ptr를 두 개 만들면 안 된다.

```
Texture* raw = new Texture("hero.png");

std::shared_ptr<Texture> first(raw);
std::shared_ptr<Texture> second(raw);    // 잘못된 생성
```

first와 second는 서로 다른 제어 블록을 만든다.

```
first  ──▶ 제어 블록 A ──▶ Texture
second ──▶ 제어 블록 B ──▶ 같은 Texture
```

각 제어 블록은 자신이 마지막 소유자라고 판단하여 같은 Texture를 두 번 해제한다.
기존 shared_ptr에서 복사해야 한다.

```
auto first = std::make_shared<Texture>("hero.png");
auto second = first;
```

원시 포인터가 필요하면 get()으로 비소유 포인터를 얻을 수 있지만, 그 포인터로 새 shared_ptr를 만들면 안 된다.

```
Texture* raw = first.get();

// std::shared_ptr<Texture> third(raw);    // 오류 설계
```

객체 내부에서 this로 shared_ptr를 직접 만드는 것도 같은 문제를 일으킨다.

```
class Texture
{
public:
    std::shared_ptr<Texture> GetPointer()
    {
        return std::shared_ptr<Texture>(this);  // 잘못된 방식
    }
};
```

이미 외부의 shared_ptr가 객체를 소유하고 있다면 새로운 제어 블록이 생긴다. 객체가 자신의 기존 공유 소유권에 참여해야 한다면 std::enable_shared_from_this를 사용한다.

7.7.6 enable_shared_from_this 개요

객체가 자신을 소유하는 기존 shared_ptr와 같은 제어 블록을 사용하는 shared_ptr를 만들어야 할 수 있다.

```
#include <memory>
class GameObject
    : public std::enable_shared_from_this<GameObject>
{
public:
    std::shared_ptr<GameObject> GetShared()
    {
        return shared_from_this();
    }
};
```

객체는 먼저 shared_ptr로 소유되어야 한다.

```
auto object = std::make_shared<GameObject>();
auto sameObject = object->GetShared();
```

object와 sameObject는 같은 제어 블록을 공유한다.
일반 객체나 unique_ptr로 관리되는 객체에서 shared_from_this()를 호출하면 기존 공유 제어 블록이 없으므로 실패한다.

```
GameObject object;
// auto pointer = object.GetShared();    // 잘못된 사용
```

생성자 안에서 호출하는 것도 피해야 한다. 생성자가 실행되는 동안에는 외부 shared_ptr와의 연결이 아직 완성되지 않았을 수 있다. 앞에서 정의한 GameObject에 생성자를 추가하더라도 shared_from_this()를 호출하지 않는다.

```
GameObject()
{
    // shared_from_this();    // 호출하지 않음
}
```

enable_shared_from_this는 모든 클래스에 기본으로 추가하는 기능이 아니다. 객체가 콜백, 작업 큐나 자식 객체에 자신의 공유 수명을 전달해야 하는 명확한 이유가 있을 때 사용한다.

7.8 std::weak_ptr

std::weak_ptr<T>는 shared_ptr가 관리하는 객체를 소유하지 않고 관찰한다. 약한 포인터가 남아 있어도 관리 대상 객체의 수명은 연장되지 않는다.

```
std::shared_ptr<Texture> texture = std::make_shared<Texture>("hero.png");
std::weak_ptr<Texture> observer = texture;
```

7.8.1 비소유 참조(Non-owning Reference)

weak_ptr는 공유 객체의 존재 여부를 확인할 수 있는 비소유 참조이다.

```
shared_ptr ──────▶ Texture    강한 소유
weak_ptr   ------▶ Texture    비소유 관찰
```

원시 포인터도 비소유 접근에 사용할 수 있지만 객체가 소멸했는지 스스로 확인할 수 없다.

```
Texture* raw = texture.get();
```

texture가 마지막 소유권을 해제하면 raw는 댕글링 포인터가 될 수 있다.
weak_ptr는 제어 블록을 통해 관리 대상 객체가 이미 소멸했는지 확인할 수 있다.

```
std::weak_ptr<Texture> observer = texture;
texture.reset();
if (observer.expired())
{
```

```
    std::cout << "Texture 소멸\n";
}
```

weak_ptr는 관찰 관계를 표현하고 공유 객체 사이의 순환 소유를 끊기 위해 사용한다.

- 캐시가 객체 수명을 불필요하게 연장하지 않음
- 이벤트 관찰자가 대상의 소멸 여부를 확인
- 공유 객체 사이의 순환 소유를 끊음
- 비동기 작업이 실행 시점에 대상의 생존 여부를 확인

단순한 범위 내 비소유 접근은 참조나 원시 포인터가 더 간단하다. weak_ptr는 대상이 shared_ptr로 관리되고 수명 중 소멸 가능성을 확인해야 할 때 사용한다.

7.8.2 만료된 객체 확인

expired()는 관리 대상 객체가 이미 소멸했는지 확인한다.

```
std::weak_ptr<Texture> observer;

{
    auto texture =
        std::make_shared<Texture>("hero.png");

    observer = texture;

    std::cout << observer.expired() << '\n';
}

std::cout << observer.expired() << '\n';

// 실행 결과
// false
// true
```

첫 번째 출력에서는 강한 소유자가 있으므로 false, 블록이 끝난 뒤에는 true가 된다.

use_count()로 강한 참조 횟수를 확인할 수도 있다.

```
std::cout << observer.use_count() << '\n';
```

그러나 expired()와 use_count()는 확인 이후 객체의 생존을 보장하지 않는다. 동시 실행 중 다른 소유자가 사라질 수 있기 때문이다.

```
if (!observer.expired())
{
    // 이 사이에 마지막 shared_ptr가 사라질 수 있음
}
```

실제로 객체를 사용하려면 lock()으로 임시 공유 소유권을 획득한다.

7.8.3 lock을 이용한 접근

lock()은 관리 대상 객체가 살아 있으면 같은 제어 블록을 공유하는 shared_ptr를 반환하고, 이미 소멸했으면 빈 shared_ptr를 반환한다.

```
if (std::shared_ptr<Texture> texture = observer.lock())
{
    texture->Bind();
}
else
{
    std::cout << "Texture를 사용할 수 없습니다.\n";
}
```

반환된 texture가 범위 안에 있는 동안에는 강한 소유권이 하나 추가되므로 객체가 소멸하지 않는다.

```
lock 성공
→ 임시 shared_ptr 생성
```

```
→ 객체 사용 중 수명 유지
→ 블록 끝에서 참조 횟수 감소
```

expired()를 먼저 호출한 뒤 다시 lock()하는 것보다 바로 lock()하고 결과를 검사하는 편이 안전하다.

```
// 권장
if (auto texture = observer.lock())
{
    texture->Bind();
}
```

weak_ptr에는 operator->와 역참조 연산자가 없다. 소유권을 확보하지 않고 객체에 직접 접근하면 접근 도중 객체가 소멸할 수 있기 때문이다.
캐시에서 활용할 수 있다.

```
class TextureCache
{
public:
    std::shared_ptr<Texture> Find(
        const std::string& name
    ) const
    {
        auto found = textures.find(name);

        if (found == textures.end())
        {
            return nullptr;
        }

        return found->second.lock();
    }

private:
    std::map<std::string, std::weak_ptr<Texture>> textures;
};
```

캐시는 텍스처의 위치를 기억하지만 텍스처를 사용하는 객체가 모두 사라졌다면 수명을 계속 유지하지 않는다.

7.8.4 순환 참조(Cyclic Reference)

두 객체가 서로를 shared_ptr로 소유하면 참조 횟수가 0이 되지 않을 수 있다.

```
class SceneNode
{
public:
    std::shared_ptr<SceneNode> parent;
    std::vector<std::shared_ptr<SceneNode>> children;
};
```

부모가 자식을 소유하고 자식도 부모를 소유한다.

```
auto parent = std::make_shared<SceneNode>();
auto child = std::make_shared<SceneNode>();

parent->children.push_back(child);
child->parent = parent;
```

외부의 parent와 child가 사라져도 내부 참조가 남는다.

```
parent ──shared_ptr──▶ child
   ▲                     │
   └─────shared_ptr──────┘
```

강한 참조가 순환하므로 두 객체가 소멸하지 않는다.
관계의 소유 의미를 분석하면 부모가 자식의 수명을 소유하지만 자식은 부모를 관찰하기만 하면 된다. 자식의 부모 포인터를 weak_ptr로 바꾼다.

```
class SceneNode
{
public:
    std::weak_ptr<SceneNode> parent;
    std::vector<std::shared_ptr<SceneNode>> children;
};
```

```
parent ──shared_ptr──▶ child
   ▲                     │
   └──────weak_ptr───────┘
```

부모의 외부 소유권이 사라지면 부모가 소멸하고 children이 정리된다. 자식이 별도로 공유되고 있다면 살아 있을 수 있지만 parent.lock()은 빈 포인터를 반환한다.

```
if (auto parent = child->parent.lock())
{
    parent->Update();
}
```

순환 참조는 shared_ptr의 결함이 아니라 소유 관계를 모두 강한 소유권으로 표현한 설계 문제이다. 각 관계에서 상대 객체의 수명을 책임지는지 판단해야 한다.

```
이 객체가 상대 객체의 수명을 책임지는가?
```

책임지면 shared_ptr 또는 unique_ptr, 관찰만 하면 weak_ptr, 참조나 원시 포인터를 사용한다.

7.9 소유권 설계

스마트 포인터를 사용하는 목적은 원시 포인터를 모두 없애는 것이 아니다. 객체의 수명을 누가 책임지는지 타입과 구조에 표현하는 것이다.

```
소유권이 있는가?
├─ 한 객체만 책임짐 → unique_ptr
├─ 여러 객체가 함께 책임짐 → shared_ptr
└─ 책임지지 않고 접근함
   ├─ 대상이 반드시 존재함 → 참조
   ├─ 대상이 없을 수 있음 → 원시 포인터
   └─ shared_ptr 객체의 소멸 확인 필요 → weak_ptr
```

동적 할당이 필요하지 않다면 값 객체가 가장 단순한 선택이다.

7.9.1 고유 소유권

고유 소유권은 하나의 객체가 자원의 수명을 책임지는 관계이다.

```
class GameScene
{
private:
    std::vector<std::unique_ptr<Enemy>> enemies;
};
```

GameScene이 각 Enemy의 생성 이후 수명을 책임지고 장면이 소멸할 때 함께 정리한다.

```
GameScene ── unique_ptr ──▶ Enemy
```

고유 소유권은 수명 책임이 분명하고 관리 비용이 낮다.

- 소유자가 하나이므로 수명 종료 시점이 분명함
- 참조 횟수 관리 비용이 없음
- 복사를 금지하고 이동으로 소유권 이전 가능
- 합성 관계와 자연스럽게 연결됨
- 순환 소유가 생기기 어려움

동적 객체를 설계할 때는 고유 소유권을 기본 선택으로 생각하는 편이 좋다. 실제로 여러 소유자가 필요하다는 근거가 있을 때 공유 소유권으로 확장한다.

고유 소유자가 바뀌면 unique_ptr를 이동한다.

```
std::unique_ptr<Enemy> enemy = CreateEnemy();
scene.AddEnemy(std::move(enemy));
```

소유권이 이동한 뒤 이전 소유자는 객체를 사용하지 않는다.

고유 소유권과 단순 접근을 구분할 수 있다.

```
class EnemySystem
{
public:
    explicit EnemySystem(GameScene& scene)
        : scene(scene)
    {
    }

private:
    GameScene& scene;    // 비소유 참조
};
```

EnemySystem은 GameScene을 사용하지만 소멸시키지 않는다. GameScene의 수명이 EnemySystem보다 길다는 구조가 별도로 보장되어야 한다.

7.9.2 공유 소유권

공유 소유권은 여러 객체가 하나의 대상 수명을 함께 책임지는 관계이다.

```
class Sprite
{
public:
    explicit Sprite(std::shared_ptr<Texture> texture)
        : texture(std::move(texture))
    {
    }

private:
    std::shared_ptr<Texture> texture;
};
```

여러 Sprite가 하나의 Texture를 공유한다. 마지막 스프라이트와 외부 소유자가 사라질 때 텍스처가 소멸한다.

공유 소유권이 적합한지 판단하려면 단순 사용 관계와 구분해야 한다.

```
여러 객체가 사용한다
≠
여러 객체가 수명을 함께 책임진다
```

장면이 텍스처를 고유 소유하고 모든 스프라이트가 장면보다 먼저 소멸한다면 비소유 참조로도 구성할 수 있다.

```
class Sprite
{
public:
    explicit Sprite(const Texture& texture)
        : texture(texture)
    {
    }

private:
    const Texture& texture;
};
```

반대로 스프라이트가 서로 다른 시스템과 비동기 작업에 전달되어 텍스처의 마지막 사용자를 특정하기 어렵다면 shared_ptr가 자연스러울 수 있다.

공유 소유권은 수명 문제를 쉽게 보이게 해결하지만 소유 구조를 불분명하게 만들 수 있다. 사용 전에는 소유 구조를 구체적으로 설명할 수 있어야 한다.

- 어떤 객체들이 소유권을 공유하는가
- 마지막 소유자가 사라지는 조건은 무엇인가
- 순환 참조 가능성이 있는가
- 객체가 예상보다 오래 유지되어도 되는가
- 공유 비용이 사용 빈도에 적합한가

7.9.3 비소유 포인터와 참조

소유하지 않는 접근에는 참조와 원시 포인터를 사용할 수 있다.
참조는 대상이 반드시 존재해야 하는 관계에 적합하다.

```
class Renderer
{
public:
    explicit Renderer(GraphicsDevice& device)
        : device(device)
    {
    }
private:
    GraphicsDevice& device;
};
```

Renderer는 생성될 때 반드시 GraphicsDevice를 받아야 하며 참조 대상을 나중에 다른 객체로 바꿀 수 없다.
원시 포인터는 대상이 없을 수 있거나 나중에 대상을 바꿀 수 있는 비소유 관계에 적합하다.

```
class Enemy
{
public:
    void SetTarget(Player* target)
    {
        this->target = target;
    }
private:
    Player* target = nullptr;
};
```

target이 nullptr이면 현재 공격 대상이 없음을 나타낸다.

```
if (target != nullptr)
{
    target->TakeDamage(attack);
}
```

원시 포인터가 비소유라는 의미는 C++ 타입 자체가 강제하지 않는다. 프로젝트 규칙, 함수 이름과 문서로 일관되게 표현해야 한다.

```
Player* FindPlayer(int id);             // 비소유 반환
std::unique_ptr<Player> CreatePlayer(); // 소유권 반환
```

함수 이름과 반환 타입을 함께 보면 의도를 구분할 수 있다.
비소유 포인터와 참조의 가장 중요한 조건은 대상 수명이다.
접근자가 살아 있는 동안 대상 객체도 살아 있어야 함

이 조건을 구조적으로 보장할 수 없다면 weak_ptr처럼 소멸 여부를 확인할 수 있는 방식이 필요하다.

7.9.4 값, 참조와 포인터 선택

객체를 함수에 전달하거나 멤버로 저장할 때는 먼저 값으로 소유할 수 있는지 판단한다.

값
독립적인 값이 필요하고 복사·이동 의미가 자연스러우면 값으로 저장한다.

```
class Player
{
private:
    std::string name;
    Vector3 position;
```

```
};
```

name과 position은 Player의 일부이며 별도 동적 할당이 필요하지 않다.

참조
대상이 반드시 존재하고 현재 객체가 소유하지 않으면 참조를 사용할 수 있다.

```
class BattleSystem
{
public:
    explicit BattleSystem(GameWorld& world)
        : world(world)
    {
    }

private:
    GameWorld& world;
};
```

참조 멤버는 생성할 때 반드시 초기화해야 하며 다른 객체를 참조하도록 다시 대입할 수 없다.

원시 포인터
대상이 선택적이거나 변경 가능한 비소유 관계이면 원시 포인터를 사용할 수 있다.
Player* target = nullptr;

unique_ptr
동적으로 생성한 객체를 하나의 소유자가 관리하면 unique_ptr를 사용한다.
std::unique_ptr<Boss> boss;

shared_ptr
여러 독립된 소유자가 수명을 공유해야 하면 shared_ptr를 사용한다.
std::shared_ptr<Texture> texture;

weak_ptr
shared_ptr 객체를 소유하지 않고 소멸 여부를 확인해야 하면 weak_ptr를 사용한다.
std::weak_ptr<Texture> cachedTexture;

<값과 포인터 선택>

표현	소유권	빈 상태	수명 관리
T	값 자체 소유	일반적으로 없음	범위와 포함 객체에 따름
T&	비소유	표현하지 않음	호출자가 대상 수명 보장
T*	비소유로 사용하는 것이 일반적	nullptr 가능	호출자가 대상 수명 보장
std::unique_ptr<T>	고유 소유	가능	마지막 고유 소유자가 해제
std::shared_ptr<T>	공유 소유	가능	마지막 공유 소유자가 해제
std::weak_ptr<T>	비소유 관찰	만료 가능	shared_ptr 소유자에 따름

포인터가 필요해 보일 때도 객체를 값 멤버로 직접 포함할 수 있는지 먼저 검토한다. 값은 소유권과 수명이 가장 분명하고 별도의 동적 할당 비용이 없다.

7.9.5 소유권을 전달하는 함수 인터페이스

함수의 매개변수와 반환 타입은 소유권 의도를 나타낸다.

새 객체 반환

```
std::unique_ptr<Enemy> CreateEnemy();
```

호출자가 새 Enemy의 고유 소유권을 받는다.
공유 객체를 생성하는 팩토리라면 shared_ptr를 반환할 수 있다.

```
std::shared_ptr<Texture> LoadSharedTexture( const std::string& fileName);
```

소유권 이전

```
void AddEnemy(std::unique_ptr<Enemy> enemy);
```

호출자는 std::move()를 사용하여 소유권 이전을 명시한다.

공유 소유권 추가

```
void SetTexture(std::shared_ptr<Texture> texture);
```

함수가 스마트 포인터를 멤버에 저장하면 새로운 공유 소유자가 된다. 값을 받아 이동하여 저장하면 lvalue 호출에서는 참조 횟수가 한 번 증가하고 rvalue 호출에서는 효율적으로 전달할 수 있다.

```
void Sprite::SetTexture(std::shared_ptr<Texture> texture)
{
    this->texture = std::move(texture);
}
```

읽기 전용 접근

```
void Draw(const Texture& texture);
```

함수가 객체를 저장하지 않고 호출 중 읽기만 한다면 스마트 포인터를 받을 필요가 없다.

변경 가능한 접근

```
void Update(Player& player);
```

객체의 상태를 변경하지만 수명은 소유하지 않는다.

선택적 접근

```
void SetTarget(Player* target);
```

nullptr가 대상 없음이라는 의미를 가진다.

스마트 포인터 자체 변경

```
void ResetEnemy(std::unique_ptr<Enemy>& enemy);
```

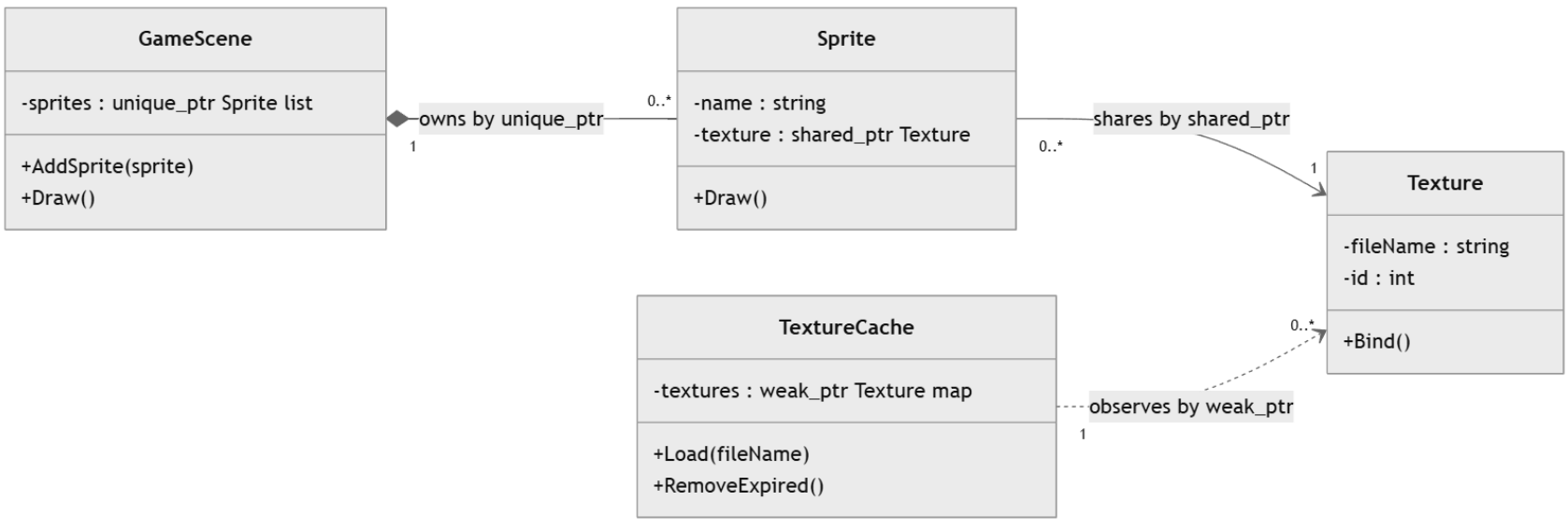
이 함수는 호출자의 소유 포인터를 비우거나 다른 객체로 교체할 수 있다. 일반 객체만 수정하는 함수와 의미가 다르므로 꼭 필요한 경우에만 사용한다.

함수 인터페이스는 소유 여부와 대상의 필수성, 보관 기간을 차례로 판단해 설계한다.

```
1. 함수가 객체를 소유하거나 저장하는가?
2. 소유한다면 고유한가, 공유하는가?
3. 소유하지 않는다면 대상이 반드시 존재하는가?
4. 대상이 호출 이후에도 보관되는가?
5. 대상의 소멸 가능성을 확인해야 하는가?
```

7.9.6 UML로 표현하는 소유 관계

UML 클래스 다이어그램은 객체 사이의 설계 관계를 나타낸다. C++의 구체적인 포인터 타입은 관계 이름이나 스테레오타입으로 보충할 수 있다.



GameScene과 Sprite의 합성 관계는 장면이 스프라이트의 수명을 소유한다는 의미이다. C++ 구현에서는 std::unique_ptr<Sprite>로 표현한다.

```
class GameScene
{
private:
    std::vector<std::unique_ptr<Sprite>> sprites;
};
```

Sprite와 Texture의 관계에는 <<shared_ptr>>를 표시한다. 여러 스프라이트가 같은 텍스처 수명을 공유한다.

```
class Sprite
{
private:
    std::shared_ptr<Texture> texture;
};
```

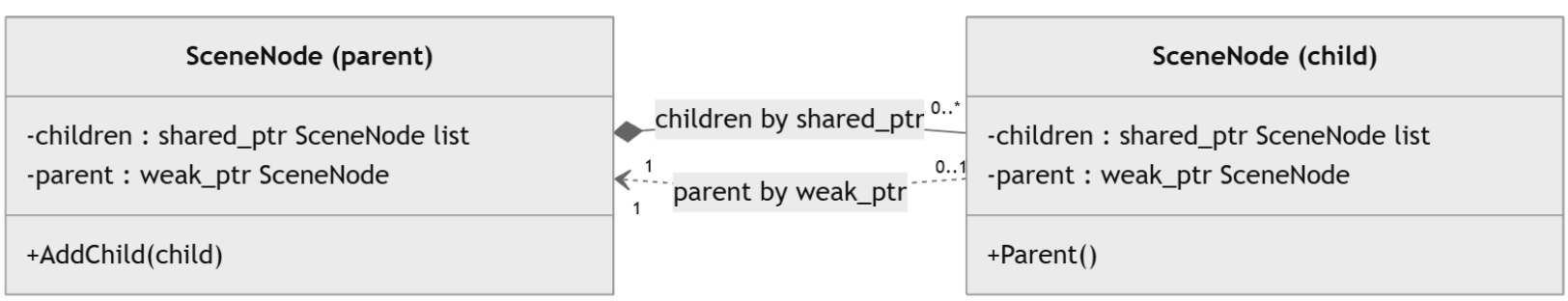
TextureCache는 <<weak_ptr>>로 텍스처를 관찰한다. 캐시에 항목이 남아 있어도 텍스처 수명을 연장하지 않는다.

```
class TextureCache
{
private:
    std::unordered_map<
        std::string,
        std::weak_ptr<Texture>
    > textures;
};
```

UML의 합성, 집약과 연관 관계가 C++ 스마트 포인터 타입과 일대일로 대응하는 것은 아니다.

- 합성은 전체가 부분의 수명을 책임진다는 설계 의미
- unique_ptr는 고유 소유권을 구현하는 한 가지 방법
- shared_ptr는 공유 수명 정책을 나타내므로 관계에 별도 표기 필요
- weak_ptr, 참조와 원시 포인터는 비소유 관계이므로 별도 표기 필요

SceneNode의 부모와 자식 관계도 소유 방향을 구분해야 한다.



ParentNode와 ChildNode는 서로 다른 클래스가 아니라 같은 SceneNode가 부모 역할과 자식 역할로 참여한 모습을 구분해 표시한 것이다.
부모는 자식 목록을 소유하고 자식은 부모를 관찰한다.

```
class SceneNode
{
private:
    std::vector<std::shared_ptr<SceneNode>> children;
    std::weak_ptr<SceneNode> parent;
};
```

관계선의 양쪽을 모두 shared_ptr로 구현하면 순환 참조가 생긴다. UML을 작성할 때 관계의 존재만 표시하지 말고 수명 책임 방향도 함께 기록하는 편이 좋다.

7.9.7 스마트 포인터 선택 기준

스마트 포인터는 동적 할당 필요성과 소유자 수를 기준으로 선택한다.

동적 할당이 필요한가
값 멤버나 지역 객체로 충분하면 스마트 포인터를 사용하지 않는다.

```
Vector3 position;
std::string name;
```

소유자가 하나인가
하나의 객체가 수명을 책임지면 unique_ptr를 사용한다.

```
std::unique_ptr<Enemy> boss;
```

실제로 공유 수명이 필요한가
여러 독립된 객체가 마지막 사용 시점까지 수명을 함께 유지해야 하면 shared_ptr를 사용한다.
std::shared_ptr<Texture> texture;

소유하지 않고 소멸 여부를 확인해야 하는가
공유 객체를 관찰하면서 만료 가능성을 처리해야 하면 weak_ptr를 사용한다.

```
std::weak_ptr<Texture> cachedTexture;
```

소유하지 않고 호출 중에만 사용하는가
대상이 반드시 존재하면 참조, 없을 수 있으면 원시 포인터를 사용한다.

```
void Draw(const Texture& texture);
void SetTarget(Player* target);
```

<스마트 포인터 선택>

상황	일반적인 선택
지역에서만 사용하는 객체	값 객체
클래스가 동적 객체를 단독 소유	std::unique_ptr<T>
팩토리가 고유 소유권을 반환	std::unique_ptr<T> 반환
여러 객체가 실제로 수명을 공유	std::shared_ptr<T>
공유 객체를 비소유로 관찰	std::weak_ptr<T>
반드시 존재하는 비소유 접근	T& 또는 const T&
선택적인 비소유 접근	T* 또는 const T*
가변 길이 원소 집합 소유	std::vector<T> 또는 std::vector<std::unique_ptr<T>>

소유권 의미를 흐리거나 제어 블록을 중복 생성하는 방식은 피한다.

- 모든 동적 객체를 습관적으로 shared_ptr로 관리
- 소유권이 없는 원시 포인터를 delete
- unique_ptr.get() 결과로 새 스마트 포인터 생성
- 같은 원시 포인터로 여러 shared_ptr 생성
- release() 반환값을 잃어버림
- weak_ptr를 lock()하지 않고 사용할 수 있다고 생각함
- 스마트 포인터를 사용했다는 이유만으로 객체 내부의 동시 접근이 안전하다고 판단

스마트 포인터는 올바른 소유권 설계를 대신하지 않는다. 먼저 객체 사이의 수명 책임을 정하고 그 의미에 맞는 타입을 선택한다.

7.10 종합 예제: 게임 장면과 텍스처 소유권

게임 장면은 여러 스프라이트를 소유하고, 여러 스프라이트는 같은 텍스처를 공유할 수 있다. 텍스처 캐시는 이미 로드된 텍스처를 다시 사용할 수 있도록 위치를 기억하지만, 사용되지 않는 텍스처의 수명을 계속 유지할 필요는 없다.

이 구조의 소유권은 세 관계로 나뉜다.

```
GameScene → Sprite       고유 소유권
Sprite → Texture         공유 소유권
TextureCache → Texture   비소유 관찰
```

7.10.1 클래스 구조와 UML

앞의 7.9.6에서 작성한 Mermaid 클래스 다이어그램을 종합 예제의 구조로 사용한다. 다이어그램의 GameScene, Sprite, TextureCache, Texture는 이어지는 구현의 클래스와 대응한다.

GameScene은 std::unique_ptr<Sprite> 목록을 가진다. 장면이 소멸하면 스프라이트도 함께 소멸한다.

각 Sprite는 std::shared_ptr<Texture>를 가진다. 동일한 파일을 사용하는 여러 스프라이트가 텍스처 객체 하나를 공유한다.

TextureCache는 파일 이름과 std::weak_ptr<Texture>를 저장한다. 텍스처가 살아 있으면 공유 포인터를 다시 얻고, 이미 소멸했다면 새 텍스처를 생성한다.

이 다이어그램은 UML 관계와 C++ 구현 타입을 함께 표시한다. 합성 기호는 GameScene의 수명 책임을 나타내고 shared_ptr와 weak_ptr는 관계선의 스테레오타입으로 표현한다.

7.10.2 Texture와 TextureCache

텍스처는 파일 이름과 식별자를 가진다. 예제에서는 실제 그래픽 API 대신 생성, 사용과 소멸 시점을 출력한다.

```
class Texture
{
public:
    explicit Texture(std::string fileName)
        : fileName(std::move(fileName)),
          id(++nextId)
    {
        if (this->fileName.empty())
        {
            throw std::invalid_argument(
                "텍스처 파일 이름이 비어 있습니다."
            );
        }

        std::cout << "Texture 생성: "
                  << this->fileName
                  << " (" << id << ")\n";
    }

    ~Texture()
    {
        std::cout << "Texture 소멸: "
                  << fileName
                  << " (" << id << ")\n";
    }

    void Bind() const
    {
        std::cout << "Texture 바인딩: "
                  << fileName << '\n';
    }

    const std::string& FileName() const
    {
        return fileName;
    }

private:
    inline static int nextId = 0;

    std::string fileName;
    int id;
};
```

TextureCache는 약한 포인터를 저장한다.

```
class TextureCache
{
public:
    std::shared_ptr<Texture> Load(const std::string& fileName)
    {
        auto found = textures.find(fileName);

        if (found != textures.end())
```

```
        {
            if (auto texture = found->second.lock())
            {
                std::cout << "Texture 재사용: "
                          << fileName << '\n';
                return texture;
            }
        }

        auto texture =
            std::make_shared<Texture>(fileName);

        textures[fileName] = texture;
        return texture;
    }

    void RemoveExpired()
    {
        for (auto iterator = textures.begin();
             iterator != textures.end();)
        {
            if (iterator->second.expired())
            {
                iterator = textures.erase(iterator);
            }
            else
            {
                ++iterator;
            }
        }
    }

    std::size_t EntryCount() const
    {
        return textures.size();
    }

private:
    std::unordered_map<
        std::string,
        std::weak_ptr<Texture>
    > textures;
};
```

Load()는 먼저 기존 약한 포인터를 lock()한다. 텍스처가 살아 있으면 공유 소유권을 하나 추가하여 반환한다. 만료되었거나 항목이 없으면 새 텍스처를 만들고 약한 포인터를 저장한다.

캐시는 텍스처를 강하게 소유하지 않는다.

```
Sprite가 모두 소멸
→ Texture의 강한 참조 횟수 0
→ Texture 소멸
→ 캐시에는 만료된 weak_ptr만 남음
```

RemoveExpired()는 만료된 캐시 항목을 제거한다.

7.10.3 Sprite와 공유 텍스처

이 예제에서는 장면에 직접 저장할 객체를 Sprite로 구성한다. 상속을 이용한 여러 게임 객체 타입은 8장에서 다룬다.

```
class Sprite
{
public:
    Sprite(
        std::string name, std::shared_ptr<Texture> texture)
        : name(std::move(name)),
          texture(std::move(texture))
    {
```

```
        if (this->texture == nullptr)
        {
            throw std::invalid_argument(
                "Sprite에는 Texture가 필요합니다."
            );
        }

        std::cout << "Sprite 생성: "
                  << this->name << '\n';
    }

    ~Sprite()
    {
        std::cout << "Sprite 소멸: "
                  << name << '\n';
    }

    void Draw() const
    {
        texture->Bind();
        std::cout << "Sprite 출력: "
                  << name << '\n';
    }

private:
    std::string name;
    std::shared_ptr<Texture> texture;
};

// Sprite 생성자는  Sprite 생성자는 shared_ptr<Texture>를 값으로
받아 멤버로 이동한다.
Sprite(
    std::string name,
    std::shared_ptr<Texture> texture
)
    : name(std::move(name)),
      texture(std::move(texture))
{
}
```

Sprite 생성자는 Sprite 생성자는 shared_ptr<Texture>를 값으로 받아 멤버로 이동한다.

```
Sprite(std::string name, std::shared_ptr<Texture> texture)
    : name(std::move(name)),
      texture(std::move(texture))
{
}
```

호출자가 lvalue shared_ptr를 전달하면 매개변수 생성 과정에서 참조 횟수가 증가하고 멤버로 이동한다. 임시 반환값을 전달하면 불필요한 추가 복사를 줄일 수 있다.

```
auto sprite = std::make_unique<Sprite>(
    "Hero",
    cache.Load("character.png")
);
```

스프라이트는 텍스처 수명을 공유하지만 캐시의 내부 구조에는 의존하지 않는다.

7.10.4 GameScene의 고유 소유권

GameScene은 스프라이트를 고유 소유한다.

```
class GameScene
{
public:
    void AddSprite(std::unique_ptr<Sprite> sprite)
    {
```

```
        if (sprite == nullptr)
        {
            throw std::invalid_argument(
                "빈 Sprite는 추가할 수 없습니다."
            );
        }

        sprites.push_back(std::move(sprite));
    }

    void Draw() const
    {
        for (const auto& sprite : sprites)
        {
            sprite->Draw();
        }
    }

    std::size_t SpriteCount() const
    {
        return sprites.size();
    }

private:
    std::vector<std::unique_ptr<Sprite>> sprites;
};
```

AddSprite()는 unique_ptr<Sprite>를 값으로 받으므로 호출자로부터 소유권을 가져간다.
스프라이트 생성 함수도 고유 소유권을 반환한다.

```
std::unique_ptr<Sprite>
CreateSprite(TextureCache& cache
            , const std::string& name
            , const std::string& textureFile)
{
    return std::make_unique<Sprite>(name, cache.Load(textureFile));
}
```

TextureCache&는 비소유 참조이다. 함수 호출 동안 캐시가 반드시 존재해야 하지만 함수가 캐시 수명을 관리하지 않는다.
호출 코드는 소유권 흐름을 보여 준다.

```
GameScene scene;
scene.AddSprite(CreateSprite(cache, "Hero", "character.png"));
```

CreateSprite()가 새 고유 소유권을 반환하고 AddSprite()가 받아 장면의 목록으로 이동한다.

```
CreateSprite
→ unique_ptr<Sprite> 반환
→ AddSprite가 소유권 받음
→ GameScene이 최종 소유
```

7.10.5 실행 흐름과 객체 소멸

앞에서 단계적으로 작성한 Texture, TextureCache, Sprite와 GameScene을 하나의 소스 파일에서 사용할 수 있도록 완성된 형태로 다시 정리한다. 이 코드는 앞의 클래스 정의와 함께 중복해서 작성하는 것이 아니라 앞의 부분 정의를 대체하는 전체 코드이다.

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <unordered_map>
#include <utility>
#include <vector>

class Texture
{
```

```
public:
    explicit Texture(std::string fileName)
        : fileName(std::move(fileName)),
          id(++nextId)
    {
        if (this->fileName.empty())
        {
            throw std::invalid_argument(
                "텍스처 파일 이름이 비어 있습니다."
            );
        }

        std::cout << "Texture 생성: "
                  << this->fileName
                  << " (" << id << ")\n";
    }

    ~Texture()
    {
        std::cout << "Texture 소멸: "
                  << fileName
                  << " (" << id << ")\n";
    }

    void Bind() const
    {
        std::cout << "Texture 바인딩: "
                  << fileName << '\n';
    }

private:
    inline static int nextId = 0;

    std::string fileName;
    int id;
};

class TextureCache
{
public:
    std::shared_ptr<Texture> Load(
        const std::string& fileName
    )
    {
        auto found = textures.find(fileName);

        if (found != textures.end())
        {
            if (auto texture = found->second.lock())
            {
                std::cout << "Texture 재사용: "
                          << fileName << '\n';
                return texture;
            }
        }

        auto texture =
            std::make_shared<Texture>(fileName);

        textures[fileName] = texture;
        return texture;
    }

    void RemoveExpired()
    {
        for (auto iterator = textures.begin();
             iterator != textures.end();)
        {
            if (iterator->second.expired())
```

```
            {
                iterator = textures.erase(iterator);
            }
            else
            {
                ++iterator;
            }
        }
    }

    std::size_t EntryCount() const
    {
        return textures.size();
    }

private:
    std::unordered_map<
        std::string,
        std::weak_ptr<Texture>
    > textures;
};

class Sprite
{
public:
    Sprite(
        std::string name,
        std::shared_ptr<Texture> texture
    )
        : name(std::move(name)),
          texture(std::move(texture))
    {
        if (this->texture == nullptr)
        {
            throw std::invalid_argument(
                "Sprite에는 Texture가 필요합니다."
            );
        }

        std::cout << "Sprite 생성: "
                  << this->name << '\n';
    }

    ~Sprite()
    {
        std::cout << "Sprite 소멸: "
                  << name << '\n';
    }

    void Draw() const
    {
        texture->Bind();
        std::cout << "Sprite 출력: "
                  << name << '\n';
    }

private:
    std::string name;
    std::shared_ptr<Texture> texture;
};

class GameScene
{
public:
    void AddSprite(std::unique_ptr<Sprite> sprite)
    {
        if (sprite == nullptr)
        {
            throw std::invalid_argument(
```

```
                "빈 Sprite는 추가할 수 없습니다."
            );
        }

        sprites.push_back(std::move(sprite));
    }

    void Draw() const
    {
        for (const auto& sprite : sprites)
        {
            sprite->Draw();
        }
    }

private:
    std::vector<std::unique_ptr<Sprite>> sprites;
};

std::unique_ptr<Sprite> CreateSprite(
    TextureCache& cache,
    const std::string& name,
    const std::string& textureFile
)
{
    return std::make_unique<Sprite>(
        name,
        cache.Load(textureFile)
    );
}

int main()
{
    TextureCache cache;

    try
    {
        {
            GameScene scene;

            scene.AddSprite(CreateSprite(
                cache,
                "HeroIdle",
                "character.png"
            ));

            scene.AddSprite(CreateSprite(
                cache,
                "HeroRun",
                "character.png"
            ));

            scene.AddSprite(CreateSprite(
                cache,
                "Background",
                "forest.png"
            ));

            std::cout << "--- Draw ---\n";
            scene.Draw();

            std::cout << "cache entries: "
                      << cache.EntryCount()
                      << '\n';
        }

        std::cout << "--- Scene 소멸 후 ---\n";

        cache.RemoveExpired();
```

```
        std::cout << "cache entries: "
                  << cache.EntryCount()
                  << '\n';
    }
    catch (const std::exception& error)
    {
        std::cerr << error.what() << '\n';
    }
}
```

두 스프라이트가 같은 character.png를 요청하므로 텍스처는 한 번만 생성된다.

```
Texture 생성: character.png (1)
Sprite 생성: HeroIdle
Texture 재사용: character.png
Sprite 생성: HeroRun
Texture 생성: forest.png (2)
Sprite 생성: Background
--- Draw ---
Texture 바인딩: character.png
Sprite 출력: HeroIdle
Texture 바인딩: character.png
Sprite 출력: HeroRun
Texture 바인딩: forest.png
Sprite 출력: Background
cache entries: 2
```

내부 블록이 끝나면 GameScene이 소멸한다. sprites의 각 unique_ptr가 스프라이트를 소멸하고, 마지막 스프라이트가 가진 공유 소유권이 사라질 때 텍스처가 소멸한다.

```
Sprite 소멸: HeroIdle
Sprite 소멸: HeroRun
Texture 소멸: character.png (1)
Sprite 소멸: Background
Texture 소멸: forest.png (2)
--- Scene 소멸 후 ---
cache entries: 0
```

컨테이너 원소의 구체적인 소멸 순서에 프로그램의 기능을 의존해서는 안 된다. 중요한 사실은 장면이 모든 스프라이트를 고유 소유하고, 각 텍스처가 마지막 공유 소유자가 사라질 때 정리된다는 점이다.

이 예제에는 직접 작성한 delete가 없다. 각 클래스가 자신의 소유권만 표현한다.

```
GameScene
└─ vector<unique_ptr<Sprite>>
   └─ shared_ptr<Texture>

TextureCache
└─ weak_ptr<Texture>
```

장면 생성 도중 예외가 발생해도 이미 GameScene에 추가된 스프라이트와 텍스처는 RAII에 의해 정리된다. 지역 객체와 스마트 포인터를 조합하면 정상 종료와 예외 경로에 같은 소유권 규칙이 적용된다.

7.11 학습 정리

저장 기간은 객체가 사용할 저장 공간이 유지되는 기간이고 객체 수명은 초기화된 객체를 해당 타입으로 사용할 수 있는 기간이다. 자동 객체는 범위를 벗어날 때 소멸하고, 동적 객체는 소유자가 명시적으로 수명을 끝내야 한다.

원시 new와 delete만으로 자원을 관리하면 조기 반환과 예외 경로에서 메모리 누수, 이중 해제와 댕글링 포인터가 발생하기 쉽다. 메모리뿐 아니라 파일, 소켓, 잠금과 그래픽 핸들도 획득과 해제 책임이 필요하다.

RAII는 자원 획득을 객체 초기화에 연결하고 자원 해제를 소멸자에 연결한다. 범위를 벗어나거나 예외로 스택이 되감길 때 생성이 완료된 지역 객체가
역순으로 소멸하므로 자원 래퍼가 자동으로 정리된다. 상위 클래스는 RAII 타입을 멤버로 사용하여 Rule of Zero를 적용하는 편이 좋다.

예외는 현재 함수가 정상 결과를 만들 수 없을 때 오류 정보를 상위 호출자에게 전달한다. 예외 객체는 값으로 던지고 일반적으로 const 참조로 받는다. 생성자에서 예외가 발생하면 완성되지 않은 객체의 소멸자는 호출되지 않지만 이미 생성된 멤버 객체는 소멸한다. 소멸자는 예외를 밖으로 전달하지 않도록 설계한다.

예외 안전성은 실패 뒤에도 자원이 누수되지 않고 객체의 불변식이 유지되는 성질이다. 기본 보장은 객체가 유효한 상태를 유지하며, 강한 보장은 실패하면 연산 전 상태를 유지하고, 무 예외 보장은 함수가 예외를 발생시키지 않는다. noexcept는 실제 동작이 지킬 수 있을 때만 지정한다.

std::unique_ptr는 고유 소유권을 나타내고 복사할 수 없으며 이동으로 소유권을 전달한다. std::shared_ptr는 여러 객체가 수명을 공유하며 마지막 소유자가 사라질 때 객체를 소멸한다. std::weak_ptr는 공유 객체를 소유하지 않고 관찰하며 lock()으로 생존한 객체의 임시 공유 소유권을 얻는다.

스마트 포인터를 선택하기 전에 동적 할당이 필요한지, 수명 책임자가 하나인지 여러 개인지, 단순한 비소유 접근인지 판단해야 한다. 값 객체를 우선하고 고유 소유권을 기본으로 선택하며, 실제 공유 수명이 필요할 때만 shared_ptr를 사용한다. 함수의 매개변수와 반환 타입은 소유권 이전, 공유와 비소유 접근을 드러내도록 구성한다.

7.12 학습 과제

과제 **1.** 저장 기간과 소멸 순서 추적
생성자와 소멸자에서 객체 이름을 출력하는 LifetimeTrace 클래스를 작성한다.
저장 기간을 비교할 수 있도록 여러 위치에 객체를 선언한다.

- 이름 공간 범위의 전역 객체
- 함수의 자동 지역 객체
- 중첩 블록의 지역 객체
- 함수의 정적 지역 객체
- thread_local 객체
- new로 생성한 동적 객체

각 객체의 생성과 소멸 순서를 기록하고 저장 기간을 분류한다. 동적 객체에 delete를 호출하지 않았을 때 소멸자가 실행되지 않는 것도 확인한다.
서로 다른 소스 파일에 전역 객체를 하나씩 정의한 뒤 초기화 순서에 의존하면 안 되는 이유를 설명한다.

과제 **2.** 이동 전용 **FileHandle**
std::FILE*을 소유하는 FileHandle 클래스를 작성한다.
필수 기능

- 생성자에서 파일 열기
- 파일 열기에 실패하면 std::runtime_error 발생
- 소멸자에서 파일 닫기
- 복사 생성자와 복사 대입 연산자 삭제
- 이동 생성자와 이동 대입 연산자 제공
- 이동 함수에 noexcept 지정
- Write()와 IsOpen() 제공
- 이동 후 원본은 빈 상태

```
FileHandle first("log.txt", "w");
first.Write("first\n");
FileHandle second = std::move(first);
second.Write("second\n");
```

조기 반환과 예외가 발생하는 함수에서 파일이 정상적으로 닫히는지 확인한다.

과제 **3.** 예외 안전한 **CharacterInventory**
std::vector<std::string>으로 아이템을 관리하는 CharacterInventory를 작성한다.

```
필수 기능
void AddItem(const std::string& item);
void ReplaceItems( const std::vector<std::string>& items);
std::size_t ItemCount() const;
const std::string& At(std::size_t index) const;
```

At()은 범위를 벗어나면 std::out_of_range를 발생시킨다. ReplaceItems()는 임시 복사본과 swap()을 사용하여 강한 예외 보장을 제공한다.

예외 안전성 관점에서 네 항목을 설명한다.

- AddItem()이 제공하는 예외 보장
- ReplaceItems()가 실패했을 때 기존 목록이 유지되는 이유
- 아이템 개수를 별도 멤버로 중복 저장하지 않는 이유
- swap()에 noexcept가 중요한 이유

과제 **4. unique_ptr**를 사용하는 **GameScene**
GameScene이 여러 Enemy를 고유 소유하도록 작성한다.

```
class GameScene
{
private:
    std::vector<std::unique_ptr<Enemy>> enemies;
};

필수 기능
std::unique_ptr<Enemy> CreateEnemy(
    int id,
    int hp
);
```

```
void AddEnemy(std::unique_ptr<Enemy> enemy);
Enemy* FindEnemy(int id);
const Enemy* FindEnemy(int id) const;
```

CreateEnemy()는 새 고유 소유권을 반환하고 AddEnemy()는 소유권을 받는다. FindEnemy()가 반환하는 원시 포인터는 비소유 포인터임을 설명한다.
GameScene을 복사할 수 없는 이유를 확인하고, 장면 이동을 허용할지 정책을 결정한다.

과제 **5.** 공유 텍스처 캐시
파일 이름으로 텍스처를 검색하는 TextureCache를 작성한다.

```
std::unordered_map<std::string, std::weak_ptr<Texture>>
textures;
```

```
필수 기능
std::shared_ptr<Texture> Load(const std::string& fileName);
void RemoveExpired();
std::size_t EntryCount() const;
```

같은 파일을 두 번 요청하면 살아 있는 기존 텍스처를 반환해야 한다. 모든 외부 shared_ptr가 사라지면 텍스처가 소멸하고 캐시 항목은 만료되어야 한다.

같은 원시 포인터로 두 shared_ptr를 만드는 잘못된 방식도 별도 테스트 코드로 작성하고 사용하면 안 되는 이유를 설명한다.

```
Texture* raw = new Texture("hero.png");
std::shared_ptr<Texture> first(raw);
std::shared_ptr<Texture> second(raw);
```

실제로 실행하여 오류를 유발하지 말고 제어 블록 구조를 그림으로 설명한다.

과제 **6. weak_ptr**를 이용한 **SceneNode**
장면 트리의 노드를 나타내는 SceneNode를 작성한다.
상태

- 노드 이름
- 자식 노드 목록
- 부모 노드

소유 정책

- 부모는 자식을 shared_ptr로 소유
- 자식은 부모를 weak_ptr로 관찰

```
class SceneNode : public std::enable_shared_from_this<SceneNode>
{
private:
    std::string name;
    std::vector<std::shared_ptr<SceneNode>> children;
    std::weak_ptr<SceneNode> parent;
};
```

```
// 필수 기능
```

```
void AddChild(std::shared_ptr<SceneNode> child);
std::shared_ptr<SceneNode> Parent() const;
void PrintPath() const;
```

AddChild()에서는 shared_from_this()를 사용하여 자식의 부모를 설정한다. 객체가 반드시 std::make_shared<SceneNode>()로 생성되어야 하는 이유를 설명한다.
부모와 자식을 모두 shared_ptr로 만들었을 때 순환 참조가 발생하는 구조와 weak_ptr로 변경한 구조를 UML로 작성한다.

과제 **7. ResourceManager**의 예외 처리
리소스 이름으로 데이터를 로드하는 ResourceManager를 작성한다.
오류 조건

- 빈 리소스 이름: std::invalid_argument
- 파일이 없음: 사용자 정의 ResourceError
- 인덱스 범위 초과: std::out_of_range

호출 코드는 구체적인 예외부터 처리하고 마지막에 std::exception을 처리한다.
RAII 추적 객체를 함께 사용하여 예외가 발생했을 때 지역 객체가 소멸하는 순서를 출력한다. 생성자에서 두 개의 자원을 순서대로 획득하다가 두 번째
자원에서 실패하는 클래스도 작성하고, 첫 번째 자원이 자동으로 정리되는지 확인한다.
소멸자에서는 예외를 밖으로 전달하지 않는다.

과제 **8.** 게임 객체 소유권 설계와 **UML**
지정된 객체로 구성된 게임 구조를 설계한다.

- GameApplication
- GameScene
- Player
- Enemy
- Texture
- Sprite
- AudioClip
- AudioPlayer
- TargetSystem
- TextureCache

각 관계는 여섯 가지 표현 중 하나를 선택해 나타낸다.

```
값 멤버
참조
비소유 원시 포인터
unique_ptr
shared_ptr
weak_ptr
```

선택 이유와 대상의 수명 조건을 작성하고 주어진 요구를 설계에 반영한다.

- 애플리케이션은 현재 장면을 고유 소유
- 장면은 플레이어와 적을 소유
- 여러 스프라이트가 텍스처를 공유
- 텍스처 캐시는 사용되지 않는 텍스처 수명을 연장하지 않음
- 적은 플레이어를 공격 대상으로 관찰하지만 소유하지 않음
- 여러 오디오 재생 객체가 하나의 오디오 데이터를 공유할 수 있음

UML 클래스 다이어그램에 합성 관계와 일반 연관 관계를 표시하고 관계선에 <<unique_ptr>>, <<shared_ptr>>, <<weak_ptr>> 또는 <<non-owning>>를 기록한다. 함수 인터페이스도 소유권 의미가 드러나도록 작성한다.

08. 상속과 다형성

게임에 Player, Monster, BossMonster가 있다고 하자. 이 객체들은 서로 다른 방식으로 행동하지만 위치와 활성 상태를 가지며, 체력과 피해 처리 기능을 사용한다는 공통점이 있다.

이러한 공통 상태와 기능을 각 클래스에 반복해서 정의할 수도 있지만, 객체들의 관계를 모델링하면 공통 요소를 기반 클래스로 분리한 계층 구조를 만들 수 있다.

월드에 존재하는 객체의 공통 상태와 기능은 GameObject로 분리하고, 체력과 피해 처리 기능을 가진 객체는 Character로 구체화한다. 플레이어가 사용하는 기능은 PlayerBase, 인공지능으로 행동하는 몹의 공통 기능은 MobBase에서 정의한다. 이러한 관계는 하나의 상속 계층으로 정리할 수 있다.

```
GameObject
└─ Character
   ├─ PlayerBase
   │  └─ Player
   └─ MobBase
      ├─ Monster
      └─ BossMonster
```

Player, Monster, BossMonster는 모두 Character이므로 캐릭터의 공통 상태와 기능을 사용할 수 있다. Monster와 BossMonster는 MobBase를 통해 몹의 공통 행동을 공유하고, Player는 PlayerBase를 통해 플레이어 입력과 관련된 기능을 가진다.

이처럼 여러 객체에서 공통 요소를 찾아 기반 클래스로 분리하고, 각 객체의 차이를 파생 클래스에서 표현하는 과정을 일반화라고 한다.

GameObject는 게임 세계에 존재하는 객체의 공통 기반이다. Character는 체력과 전투 상태를 가진 객체를 표현한다. PlayerBase는 플레이어가 조작하는 캐릭터 계열의 공통 기반이며, MobBase는 게임 로직이나 인공지능이 제어하는 몹 계열의 공통 기반이다. Player, Monster, BossMonster는 실제로 생성할 수 있는 구체 클래스이다.

PlayerBase 아래에 현재 Player 하나만 존재하더라도 중간 기반 클래스는 의미가 있다. 여러 종류의 플레이어 캐릭터를 추가하거나 플레이어 계열에만 필요한 입력 계약을 분리할 수 있기 때문이다. 실제 프로젝트에서 계층 확장 가능성이 없다면 PlayerBase를 생략할 수도 있다. 본 장에서는 다단계 상속, 추상 클래스와 공통 계약을 설명하기 위해 유지한다.

상속은 기존 클래스의 코드를 단순히 가져오는 기능이 아니다. 파생 클래스가 기반 클래스의 한 종류임을 타입 관계로 표현하고, 기반 클래스 포인터나 참조를 통해 서로 다른 구체 객체를 같은 방식으로 다루게 한다. 가상 함수를 사용하면 호출 코드가 실제 객체의 타입을 직접 확인하지 않아도 각 객체가 재정의한 동작을 실행할 수 있다.

```
Player player;
Monster monster;
BossMonster bossMonster;

std::vector<GameObject*> gameObjects {&player, &monster,
&bossMonster};
for (GameObject* obj : gameObjects)
{
    obj->Update();
}
```

Update()는 객체의 한 프레임 동작을 처리한다. 본 장에서는 프레임 시간과 게임 실행 상태를 GameSystem이 제공한다고 가정한다. 시간 값이 필요한 파생 클래스는 Update()의 매개변수로 전달받지 않고 GameSystem::GetDeltaTime()으로 조회한다. 따라서 중간 계층이 사용하지 않는 시간 값을 계속 전달할 필요가 없다.

Player, Monster, BossMonster가 GameObject의 계약을 구현하면 기반 클래스 참조를 통해 같은 형식으로 호출할 수 있다. 호출 형식은 같지만 실제로 실행되는 함수는 객체의 동적 타입에 따라 달라진다.

상속을 잘못 사용하면 기반 클래스의 변경이 여러 파생 클래스에 전파되고, 초기화와 소멸 순서가 복잡해지며, 실제 타입을 확인하는 형 변환 코드가 늘어날 수 있다. 단순한 기능 조합이나 코드 재사용이 목적이라면 구성과 위임이 더 적합할 수 있다.

본 장에서는 일반화와 상속 관계를 설계하는 기준부터 시작하여 상속 접근 지정자, 기반 클래스 초기화, 함수 재정의와 이름 숨김을 학습한다. 가상 함수와 동적 바인딩, 추상 클래스와 인터페이스, 다형 객체의 수명과 복제를 다룬 뒤 형 변환과 RTTI, 다중 상속과 가상 상속을 살펴본다. 마지막에는 상속과 구성을 비교하고 게임 객체 계층을 종합 예제로 구성한다.

8.1 일반화와 상속

여러 클래스에서 공통된 특성을 찾아 상위 개념으로 묶는 과정을 일반화(generalization)라고 한다. C++에서는 일반화한 상위 개념을 상위 클래스로 정의하고, 하위 클래스와의 관계를 상속으로 표현한다.

상속(inheritance)은 하위 클래스가 상위 클래스의 특성을 이어받아 더 구체적인 클래스를 정의하는 기능이다. 하위 클래스는 상위 클래스에 정의된 데이터와 기능을 기반으로 자신의 데이터와 기능을 추가하거나, 허용된 기능을 재정의한다. 상속은 상위 개념을 하위 개념으로 구체화하는 관계를 표현한다.

게임의 플레이어와 몹은 일반화와 상속을 설명하는 대표적인 예이다. 플레이어와 몹은 게임 세계에 존재하며 위치를 가진다. 두 객체는 모두 체력을 가지고 피해를 받을 수 있다. 플레이어 계열은 입력을 처리하고, 몹 계열은 인공지능에 따라 행동한다. 이 공통점과 차이점을 계층으로 정리하면 각 클래스의 책임이 분명해진다.

```
게임 객체
└─ 캐릭터
   ├─ 플레이어 계열
   │  └─ 플레이어
   └─ 몹 계열
      ├─ 일반 몬스터
      └─ 보스 몬스터
```

8.1.1 기반 클래스와 파생 클래스

상속 관계에서 공통 개념을 정의한 클래스를 기반 클래스(base class)라고 한다. 기반 클래스를 확장하거나 구체화한 클래스를 파생 클래스(derived class)라고 한다.

기반 클래스는 상위 클래스(superclass) 또는 부모 클래스(parent class), 파생 클래스는 하위 클래스(subclass) 또는 **자식 클래스(child class)**라고도 부른다.

게임과 같은 가상 세계는 여러 오브젝트를 기본 단위로 구성한다. 이 예제에서는 위치와 활성 상태를 가지는 기반 클래스를 GameObject로 정의하고, 플레이어와 몹, 보스 몹 등의 객체를 상속 관계로 구성한다.

GameObject는 위치와 활성 상태를 관리한다.

```
class GameObject
{
public:
    void SetPosition(float x, float y)
    {
        this->x = x;
        this->y = y;
    }

    float GetX() const
    {
        return x;
    }

    float GetY() const
    {
        return y;
    }

    bool IsActive() const
    {
        return active;
    }

protected:
    float x = 0.0f, y = 0.0f;
    bool active = true;
};
```

Character는 GameObject를 공개 상속하고 체력 상태를 추가한다.

```
class Character : public GameObject
{
public:
    void TakeDamage(int damage)
    {
        if (damage <= 0)
        {
            return;
```

```
        }
        if (hp -= damage; hp < 0)
        {
            hp = 0;
        }
    }

    int GetHp() const
    {
        return hp;
    }

protected:
    int hp = 100;
};
```

플레이어 계열과 몹 계열의 공통 기반을 분리한다.

```
class PlayerBase : public Character
{
public:
    void AddScore(int value)
    {
        if (value > 0)
        {
            score += value;
        }
    }

    int GetScore() const
    {
        return score;
    }

protected:
    int score = 0;
};

class MobBase : public Character
{
public:
    void SetAttackPower(int attackPower)
    {
        if (attackPower >= 0)
        {
            this->attackPower = attackPower;
        }
    }

    int GetAttackPower() const
    {
        return attackPower;
    }

protected:
    int attackPower = 10;
};
```

구체 클래스는 직접 기반 클래스의 상태와 기능을 물려받는다.

```
class Player : public PlayerBase
{
public:
    void AddItem(int itemId)
    {
        lastItemId = itemId;
    }
```

```
private:
    int lastItemId = 0;
};

class Monster : public MobBase
{
};

class BossMonster : public MobBase
{
public:
    void EnterRageMode()
    {
        rageMode = true;
    }

private:
    bool rageMode = false;
};
```

Player 객체에는 네 계층의 부분이 함께 존재한다.

```
Player 객체
├─ GameObject 부분
│  ├─ x, y
│  └─ active
├─ Character 부분
│  └─ hp
├─ PlayerBase 부분
│  └─ score
└─ Player 부분
   └─ lastItemId
```

BossMonster 객체도 네 계층으로 구성된다.

```
BossMonster 객체
├─ GameObject 부분
├─ Character 부분
├─ MobBase 부분
└─ BossMonster 부분
```

파생 클래스 객체는 상위 계층의 공개 멤버 함수를 사용할 수 있다.

```
Player player;

player.SetPosition(100.0f, 200.0f);
player.TakeDamage(20);
player.AddScore(500);
player.AddItem(101);

std::cout << player.GetX() << '\n';
std::cout << player.GetHp() << '\n';
std::cout << player.GetScore() << '\n';
```

기반 클래스의 private 멤버는 파생 클래스에서도 직접 접근할 수 없다. protected 멤버는 파생 클래스의 멤버 함수에서 접근할 수 있고, public 멤버는 외부 인터페이스를 통해 사용할 수 있다.

8.1.2 is-a 관계와 대체 가능성

공개 상속은 파생 클래스가 기반 클래스의 한 종류임을 나타낸다. 이를 is-a 관계라고 표현한다.

```
Player is a PlayerBase.
PlayerBase is a Character.
Monster is a MobBase.
BossMonster is a MobBase.
MobBase is a Character.
```

```
Character is a GameObject.
```

상속 관계는 전이된다.

```
Player is a Character.
Monster is a Character.
BossMonster is a GameObject.
```

Player가 Character의 한 종류라면 Character를 요구하는 위치에 사용할 수 있어야 한다.

```
void PrintCharacterState(const Character& character)
{
    std::cout << character.GetX() << ", "
              << character.GetY() << ", hp="
              << character.GetHp() << '\n';
}

Player player;
Monster monster;
BossMonster boss;

PrintCharacterState(player);
PrintCharacterState(monster);
PrintCharacterState(boss);
```

최상위 기반 클래스 인터페이스만 필요한 함수에는 GameObject 참조로 전달할 수 있다.

```
void PrintPosition(const GameObject& object)
{
    std::cout << object.GetX() << ", "
              << object.GetY() << '\n';
}
```

포인터도 같은 변환을 지원한다.

```
BossMonster boss;

MobBase* mob = &boss;
Character* character = &boss;
GameObject* object = &boss;
```

파생 클래스 포인터를 기반 클래스 포인터로 변환하는 것을 업캐스팅(upcasting)이라고 한다. 공개 상속에서는 일반적으로 암시적으로 수행할 수 있다.

대체 가능성은 문법상 변환이 된다는 의미보다 넓다. 파생 클래스는 기반 클래스가 약속한 의미와 규칙을 지켜야 한다.
Character::TakeDamage()가 양수 피해량만 처리하고 체력을 0 미만으로 만들지 않는다는 계약을 가진다면 PlayerBase와 MobBase 계열도 그 의미를 깨뜨리지 않아야 한다.

<공개 상속을 검토할 기준>

기준	확인 내용
개념 관계	파생 클래스가 기반 클래스의 한 종류인가
인터페이스	기반 클래스의 공개 연산이 파생 클래스에도 의미가 있는가
계약	파생 클래스가 기반 클래스의 규칙을 유지하는가
수명	기반 클래스 포인터로 다형 객체를 안전하게 관리할 수 있는가
변경 영향	기반 클래스 변경을 파생 클래스들이 수용할 수 있는가

8.1.3 상속을 사용하기 위한 조건

공통 코드가 있다는 이유만으로 상속을 선택하면 안 된다. 상속은 타입 관계와 인터페이스 계약을 함께 만든다.
상속이 자연스러운 구조는 여러 조건을 만족한다.

- 파생 클래스가 기반 클래스의 의미상 한 종류임

- 기반 클래스의 공개 인터페이스를 파생 클래스에도 적용할 수 있음
- 호출 코드가 구체 타입을 몰라도 공통 기능을 사용할 필요가 있음
- 동적 다형성이 실제로 필요함
- 기반 클래스가 안정적인 추상 개념을 제공함

Player는 PlayerBase의 한 종류이고, Monster는 MobBase의 한 종류이다. 반면 캐릭터와 무기는 is-a 관계가 아니다.

```
Character is a Weapon.    // 성립하지 않음
Character has a Weapon.   // 자연스러운 관계
```

무기는 구성으로 표현한다.

```
class Weapon
{
public:
    void Attack()
    {
        std::cout << "Weapon attack\n";
    }
};

class Player : public PlayerBase
{
public:
    void Attack()
    {
        weapon.Attack();
    }

private:
    Weapon weapon;
};
```

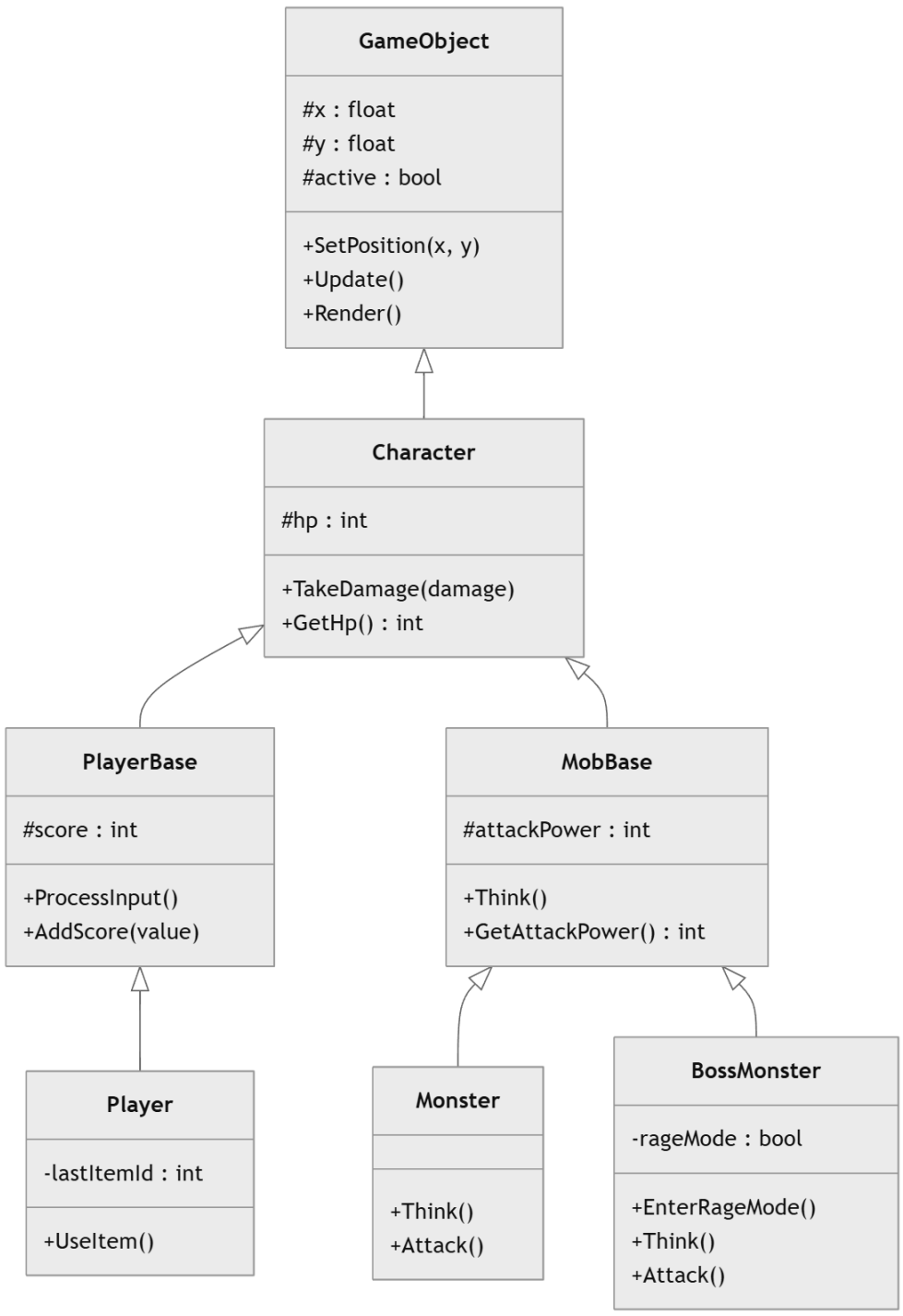
Player는 Weapon을 멤버로 가지며 공격 기능을 위임한다. Weapon의 구현을 바꾸더라도 Player의 타입 관계는 영향을 받지 않는다.

MobBase라는 이름도 목적에 맞게 제한해야 한다. 단순히 플레이어가 아닌 모든 객체를 MobBase 아래에 두면 계층의 의미가 흐려진다. 투사체, 아이템, 문과 같은 객체는 캐릭터가 아니므로 GameObject의 다른 파생 계층으로 구성해야 한다.

상속을 고려할 때 코드 재사용 가능성보다 기반 클래스 대신 파생 클래스를 사용해도 의미와 계약이 유지되는지를 먼저 확인한다.

8.1.4 UML로 표현하는 일반화

UML 클래스 다이어그램에서 일반화 관계는 파생 클래스에서 기반 클래스를 향하는 빈 삼각형 화살표로 표현한다.



UML은 설계 관계를 표현하지만 C++의 세부 구현을 모두 나타내지는 않는다. 상속 접근 지정자, 가상 소멸자 여부와 스마트 포인터 소유권은 주석이나 설명을 함께 사용해야 정확하게 전달할 수 있다.

8.2 상속의 기본 구조

파생 클래스 선언에서는 기반 클래스 이름 앞에 상속 접근 지정자를 작성한다.

```
class Derived : public Base
{
};
```

상속 접근 지정자는 기반 클래스의 public과 protected 멤버가 파생 클래스에서 어떤 접근 수준으로 보이는지 결정한다. 기반 클래스의 private 멤버는 어느 상속 방식에서도 파생 클래스가 직접 접근할 수 없다.

8.2.1 public 상속

public 상속은 공개적인 is-a 관계를 표현한다. 본 장의 게임 객체 계층은 모두 공개 상속을 사용한다.

```
class MobBase : public Character
{
};

class Monster : public MobBase
{
};
```

기반 클래스의 공개 멤버는 파생 클래스에서도 공개 멤버로 유지된다. protected 멤버도 protected로 유지된다.

```
Monster monster;
```

```
monster.SetPosition(50.0f, 30.0f);
monster.TakeDamage(10);
```

외부 코드는 Monster를 직접 기반 클래스인 MobBase로 변환할 수 있고, 더 위의 Character와 GameObject로도 변환할 수 있다.

```
MobBase& mob = monster;
Character& character = monster;
GameObject& object = monster;
```

<public 상속에서의 접근 변화>

기반 클래스 멤버	파생 클래스에서의 접근 수준
public	public
protected	protected
private	직접 접근 불가

공개 상속은 외부 코드가 파생 객체를 기반 객체로 다룰 수 있게 하므로 타입 계층을 표현할 때 사용하는 기본 방식이다.

8.2.2 protected 상속

protected 상속에서는 기반 클래스의 public과 protected 멤버가 파생 클래스에서 모두 protected가 된다.

```
class InternalCharacter : protected Character
{
public:
    void ResetPosition()
    {
        SetPosition(0.0f, 0.0f);
    }
};
```

InternalCharacter 내부에서는 Character와 GameObject의 공개·보호 멤버를 사용할 수 있다. 외부 코드에서는 기반 클래스의 공개 인터페이스가 보호 멤버로 바뀌므로 직접 사용할 수 없다.

```
InternalCharacter object;
object.ResetPosition();

// object.SetPosition(10.0f, 20.0f);    // 오류
```

외부에서 공개 업캐스팅도 허용되지 않는다.

```
// Character& character = object;    // 오류
```

<protected 상속에서의 접근 변화>

기반 클래스 멤버	파생 클래스에서의 접근 수준
public	protected
protected	protected
private	직접 접근 불가

protected 상속은 파생 계층 내부에서만 기반 기능을 공유하려는 경우 사용할 수 있지만, 일반적인 객체 모델에서는 구성이나 private 상속이 의도를 더 분명하게 표현하는 경우가 많다.

8.2.3 private 상속

private 상속에서는 기반 클래스의 public과 protected 멤버가 파생 클래스에서 모두 private이 된다.

```
class MovementService
{
```

```
public:
    void Move(float x, float y)
    {
        std::cout << "move: " << x << ", " << y << '\n';
    }
};

class PlayerController : private MovementService
{
public:
    void MovePlayer(float x, float y)
    {
        Move(x, y);
    }
};
```

외부 코드는 MovementService의 인터페이스를 직접 사용할 수 없다.

```
PlayerController controller;
controller.MovePlayer(10.0f, 20.0f);

// controller.Move(10.0f, 20.0f);    // 오류
```

<private 상속에서의 접근 변화>

기반 클래스 멤버	파생 클래스에서의 접근 수준
public	private
protected	private
private	직접 접근 불가

private 상속은 기반 구현을 내부적으로 재사용하지만 외부에 is-a 관계를 공개하지 않는다. 같은 목적을 멤버 구성으로 표현할 수 있다면 구성이 더 명확한 경우가 많다.

```
class PlayerController
{
public:
    void MovePlayer(float x, float y)
    {
        movement.Move(x, y);
    }

private:
    MovementService movement;
};
```

8.2.4 protected 멤버

protected 멤버는 클래스 외부에서는 접근할 수 없지만 파생 클래스의 멤버 함수에서는 접근할 수 있다.

```
class Character : public GameObject
{
protected:
    int hp = 100;
};

class MobBase : public Character
{
public:
    bool IsDefeated() const
    {
        return hp == 0;
    }
};
```

외부 코드는 hp에 직접 접근할 수 없다.

```
Monster monster;
// monster.hp = 0;    // 오류
```

protected 데이터는 파생 클래스 구현을 편리하게 만들지만 결합도를 높인다. Character가 체력 저장 방식을 바꾸면 모든 파생 클래스가 영향을 받을 수 있다.
접근 함수를 제공하면 내부 표현을 감출 수 있다.

```
class Character : public GameObject
{
public:
    int GetHp() const
    {
        return hp;
    }

protected:
    void SetHp(int hp)
    {
        if (hp < 0)
        {
            this->hp = 0;
        }
        else
        {
            this->hp = hp;
        }
    }

private:
    int hp = 100;
};
```

파생 클래스는 공개 또는 보호 멤버 함수를 통해 체력을 처리한다.

```
class BossMonster : public MobBase
{
public:
    void Recover(int amount)
    {
        if (amount > 0)
        {
            SetHp(GetHp() + amount);
        }
    }
};
```

상태를 단순히 노출하기보다 기반 클래스가 불변식과 변경 규칙을 관리하도록 설계하는 편이 안전하다.

8.2.5 상속 접근 지정자 선택

상속 접근 지정자는 코드 재사용 범위보다 타입 관계를 기준으로 선택한다.

상속 방식	외부에 공개되는 관계	대표 목적
public	is-a 관계 공개	객체 계층과 다형성
protected	파생 계층 내부에서만 기반 관계 사용	제한된 계층 구현 공유
private	기반 구현을 내부적으로 사용	구현 재사용

GameObject, Character, PlayerBase, MobBase 계층은 외부에서 대체 가능해야 하므로 public 상속이 적합하다.

```
class Character : public GameObject
{
};
```

```
class PlayerBase : public Character
{
};

class MobBase : public Character
{
};
```

구현을 재사용하려는 목적만 있다면 멤버 구성을 먼저 검토한다. 상속은 기반 클래스의 인터페이스와 변경에 파생 클래스를 결합시키기 때문이다.

8.3 기반 클래스와 파생 클래스의 초기화

파생 클래스 객체를 생성하면 객체 내부의 기반 클래스 부분도 함께 생성된다. 기반 부분이 먼저 유효한 상태가 되어야 파생 부분을 초기화할 수 있으므로 생성 과정은 계층의 위에서 아래로 진행된다.
Player 객체의 생성 계층은 네 단계이다.

```
GameObject → Character → PlayerBase → Player
```

BossMonster 객체도 같은 원리를 따른다.

```
GameObject → Character → MobBase → BossMonster
```

8.3.1 기반 클래스 생성자 호출

파생 클래스 생성자는 직접 기반 클래스의 생성자를 호출한다. 직접 기반 클래스의 생성자를 명시하지 않으면 컴파일러는 기본 생성자를 호출하려고 한다.

```
class GameObject
{
public:
    GameObject(int id, const std::string& name)
        : id(id), name(name)
    {
    }

private:
    int id;
    std::string name;
};
```

GameObject에는 기본 생성자가 없다. Character는 초기화 리스트에서 GameObject 생성자를 선택해야 한다.

```
class Character : public GameObject
{
public:
    Character(int id,
              const std::string& name,
              int hp)
        : GameObject(id, name), hp(hp)
    {
    }

private:
    int hp;
};
```

PlayerBase는 직접 기반 클래스인 Character를 초기화한다.

```
class PlayerBase : public Character
{
public:
    PlayerBase(int id,
               const std::string& name,
```

```
                int hp,
                int score)
        : Character(id, name, hp), score(score)
    {
    }

private:
    int score;
};
```

Player는 PlayerBase를 초기화한다.

```
class Player : public PlayerBase
{
public:
    Player(int id,
           const std::string& name,
           int hp,
           int score)
        : PlayerBase(id, name, hp, score)
    {
    }
};
```

Player가 GameObject나 Character를 건너뛰고 직접 초기화할 수는 없다.

```
class Player : public PlayerBase
{
public:
    Player(int id, const std::string& name)
        // : GameObject(id, name)    // 오류
        // : Character(id, name, 100) // 오류
        : PlayerBase(id, name, 100, 0)
    {
    }
};
```

각 클래스는 자신의 직접 기반 클래스만 선택한다. 더 위의 기반 클래스 초기화는 직접 기반 클래스 생성자가 담당한다.

8.3.2 멤버 초기화 리스트를 이용한 기반 클래스 초기화

기반 클래스 부분은 파생 클래스 생성자 본문이 실행되기 전에 초기화된다. 생성자 본문에서 기반 객체를 다시 대입하는 방식으로는 기반 부분의 초기화를 대신할 수 없다.

```
class MobBase : public Character
{
public:
    MobBase(int id,
            const std::string& name,
            int hp,
            int attackPower)
        : Character(id, name, hp),
          attackPower(attackPower)
    {
    }

private:
    int attackPower;
};
```

Monster는 MobBase의 생성자를 선택한다.

```
class Monster : public MobBase
{
public:
    Monster(int id,
```

```
            const std::string& name,
            int hp,
            int attackPower,
            int patrolRange)
        : MobBase(id, name, hp, attackPower),
          patrolRange(patrolRange)
    {
    }

private:
    int patrolRange;
};
```

BossMonster도 직접 기반 클래스인 MobBase를 초기화한다.

```
class BossMonster : public MobBase
{
public:
    BossMonster(int id,
                const std::string& name,
                int hp,
                int attackPower,
                int phaseCount)
        : MobBase(id, name, hp, attackPower),
          phaseCount(phaseCount)
    {
    }

private:
    int phaseCount;
};
```

기반 클래스 생성자 인수는 파생 클래스 생성자가 받은 값, 상수, 계산식으로 구성할 수 있다.

```
BossMonster(int id, const std::string& name)
    : MobBase(id, name, 1000, 80),
      phaseCount(3)
{
}
```

파생 클래스는 기반 클래스의 비공개 데이터 멤버를 직접 초기화하지 않는다. 기반 클래스 생성자에 필요한 값을 전달하고 기반 클래스가 자신의 상태를 책임지게 한다.

8.3.3 기반 클래스와 데이터 멤버의 초기화 순서

실제 초기화 순서는 초기화 리스트에 적은 순서가 아니라 클래스 선언 구조로 결정된다.

```
1. 가상 기반 클래스
2. 직접 기반 클래스
3. 데이터 멤버의 선언 순서
4. 생성자 본문
```

```
class Player : public PlayerBase
{
public:
    Player(int id,
           const std::string& name,
           int level,
           int gold)
        : gold(gold),
          PlayerBase(id, name, 100, 0),
          level(level)
    {
    }

private:
```

```
    int level;
    int gold;
};
```

초기화 리스트에서는 gold를 먼저 적었지만 데이터 멤버는 선언 순서에 따라 level, gold 순으로 초기화된다. 직접 기반 클래스 PlayerBase는 두 데이터 멤버보다 먼저 초기화된다.

혼동을 줄이려면 초기화 리스트도 실제 초기화 순서와 맞춘다.

```
Player(int id,
       const std::string& name,
       int level,
       int gold)
    : PlayerBase(id, name, 100, 0),
      level(level),
      gold(gold)
{
}
```

한 데이터 멤버가 다른 멤버에 의존하면 선언 순서가 더 중요하다.

```
class BossMonster : public MobBase
{
public:
    BossMonster(int id, int basePower)
        : MobBase(id, "Boss", 1000, basePower),
          doubledPower(basePower * 2),
          ragePower(doubledPower + 20)
    {
    }

private:
    int doubledPower;
    int ragePower;
};
```

doubledPower가 먼저 선언되어 있으므로 ragePower 초기화에 사용할 수 있다.

8.3.4 생성과 소멸 순서

생성자는 기반 클래스에서 파생 클래스로 호출된다. 소멸자는 그 반대 순서로 호출된다.

```
class GameObject
{
public:
    GameObject()
    {
        std::cout << "GameObject 생성\n";
    }

    virtual ~GameObject()
    {
        std::cout << "GameObject 소멸\n";
    }
};

class Character : public GameObject
{
public:
    Character()
    {
        std::cout << "Character 생성\n";
    }

    ~Character() override
    {
        std::cout << "Character 소멸\n";
```

```
    }
};

class MobBase : public Character
{
public:
    MobBase()
    {
        std::cout << "MobBase 생성\n";
    }

    ~MobBase() override
    {
        std::cout << "MobBase 소멸\n";
    }
};

class BossMonster : public MobBase
{
public:
    BossMonster()
    {
        std::cout << "BossMonster 생성\n";
    }

    ~BossMonster() override
    {
        std::cout << "BossMonster 소멸\n";
    }
};

{
    BossMonster boss;
}
```

실행 순서는 계층 구조를 반영한다.

```
GameObject 생성
Character 생성
MobBase 생성
BossMonster 생성
BossMonster 소멸
MobBase 소멸
Character 소멸
GameObject 소멸
```

BossMonster 생성자 본문이 실행될 때는 상위 세 부분이 이미 유효하다. BossMonster 소멸자 본문이 끝나면 파생 데이터 멤버가 역순으로 소멸하고 MobBase, Character, GameObject 소멸자가 차례로 호출된다.

<생성과 소멸의 방향>

과정	방향
생성	기반 클래스 → 파생 클래스
소멸	파생 클래스 → 기반 클래스

8.4 함수 재정의와 이름 숨김

파생 클래스는 기반 클래스에서 물려받은 함수와 같은 이름의 함수를 선언할 수 있다. 이때 오버로딩, 오버라이딩과 이름 숨김을 구분해야 한다.

8.4.1 오버로딩과 오버라이딩

오버로딩(overloading)은 같은 범위에서 이름은 같고 매개변수 목록이 다른 함수를 여러 개 선언하는 것이다.

```
class Character
```

```
{
public:
    void Move(float x, float y)
    {
        std::cout << "Move to " << x << ", " << y << '\n';
    }

    void Move(float distance)
    {
        std::cout << "Move distance " << distance << '\n';
    }
};
```

오버라이딩(overriding)은 파생 클래스가 기반 클래스의 가상 함수를 같은 시그니처로 재정의하는 것이다.

```
class GameObject
{
public:
    virtual void Update()
    {
        std::cout << "GameObject Update\n";
    }
};

class Character : public GameObject
{
};

class MobBase : public Character
{
};

class Monster : public MobBase
{
public:
    void Update() override
    {
        std::cout << "Monster AI Update\n";
    }
};
```

<오버로딩과 오버라이딩>

구분	오버로딩	오버라이딩
관계	같은 범위의 함수들	기반 클래스와 파생 클래스
매개변수	서로 달라야 함	같아야 함
virtual	필요 없음	기반 함수에 필요
선택 시점	주로 컴파일 시점	기반 포인터·참조에서는 실행 시점

8.4.2 이름 숨김

파생 클래스에서 기반 클래스 함수와 같은 이름의 함수를 선언하면 기반 클래스의 같은 이름을 가진 함수들이 이름 검색에서 가려질 수 있다. 매개변수 목록이 달라도 이름 숨김이 발생한다.

```
class Character : public GameObject
{
public:
    void Move(float x, float y)
    {
        std::cout << "Character Move\n";
    }
};
```

```
class PlayerBase : public Character
{
};

class Player : public PlayerBase
{
public:
    void Move(int direction)
    {
        std::cout << "Player input direction: "
                  << direction << '\n';
    }
};

Player player;

player.Move(1);
// player.Move(10.0f, 20.0f);    // 오류
```

Player::Move(int)가 존재하므로 이름 검색은 Player 범위에서 멈춘다. Character::Move(float, float)는 같은 이름 때문에 가려진다. 두 함수가 가상 함수인지 여부와는 관계가 없다.
기반 클래스 참조로 접근하면 기반 클래스 인터페이스를 사용할 수 있다.

```
Character& character = player;
character.Move(10.0f, 20.0f);
```

8.4.3 using 선언

파생 클래스 범위에서 기반 클래스의 오버로드 집합을 다시 보이게 하려면 using 선언을 사용한다.

```
class Player : public PlayerBase
{
public:
    using Character::Move;

    void Move(int direction)
    {
        std::cout << "Player input direction: "
                  << direction << '\n';
    }
};

Player player;

player.Move(1);
player.Move(10.0f, 20.0f);
```

using Character::Move;는 기반 클래스의 Move 오버로드 집합을 Player 범위에서 사용할 수 있게 한다.

생성자 상속에도 using을 사용할 수 있다.

```
class MobBase : public Character
{
public:
    using Character::Character;
};
```

생성자 상속은 간단한 계층에서 편리하지만 MobBase가 별도의 멤버를 초기화해야 하거나 생성 규칙을 추가해야 한다면 직접 생성자를 작성하는 편이 분명하다.

8.4.4 기반 클래스 함수 호출

파생 클래스가 기반 클래스의 구현을 확장하려면 범위 지정 연산자로 기반 함수를 직접 호출할 수 있다.

```
class Character : public GameObject
{
```

```
public:
    void Update() override
    {
        GameObject::Update();
        std::cout << "Character status update\n";
    }
};

class PlayerBase : public Character
{
public:
    void Update() override
    {
        Character::Update();
        std::cout << "PlayerBase input state update\n";
    }
};

class Player : public PlayerBase
{
public:
    void Update() override
    {
        PlayerBase::Update();
        std::cout << "Player animation update\n";
    }
};

Player player;
player.Update();
```

실행 과정은 상위 구현을 명시적으로 연결한다.

```
GameObject Update
Character status update
PlayerBase input state update
Player animation update
```

Character::Update()처럼 범위를 명시한 호출은 가상 디스패치를 사용하지 않고 해당 클래스의 구현을 직접 호출한다.
기반 구현 호출이 모든 파생 클래스에서 반드시 필요하다면 파생 클래스가 기억해서 호출하게 두기보다 비가상 인터페이스 패턴을 고려할 수 있다. 이 방식은 8.6.6에서 다룬다.

8.4.5 재정의 시그니처와 override

가상 함수를 재정의하려면 이름, 매개변수 타입, 매개변수 순서, 멤버 함수의 const와 참조 한정자 등이 일치해야 한다.

```
class MobBase : public Character
{
public:
    virtual void Think() const
    {
    }
};
```

const를 빠뜨리면 다른 함수가 선언된다.

```
class Monster : public MobBase
{
public:
    // void Think()    // 재정의 아님
    // {
    // }
};
```

override를 지정하면 컴파일러가 재정의 여부를 검사한다.

```
class Monster : public MobBase
{
public:
    void Think() const override
    {
        std::cout << "Monster patrol\n";
    }
};
```

잘못된 시그니처에는 컴파일 오류가 발생한다.

```
class BossMonster : public MobBase
{
public:
    // void Think() override; // 오류: const가 일치하지 않음
};
```

재정의하는 함수에는 override를 일관되게 붙이는 것이 좋다. 함수 선언만 보아도 기반 가상 함수의 재정의임을 알 수 있고, 기반 클래스 인터페이스가 변경되었을 때 오류를 조기에 발견할 수 있다.

8.5 가상 함수와 동적 바인딩

기반 클래스 포인터나 참조가 실제로 가리키는 객체에 따라 다른 함수를 호출하려면 가상 함수를 사용한다.

8.5.1 정적 바인딩과 동적 바인딩

함수 호출을 특정 구현에 연결하는 것을 바인딩(binding)이라고 한다. 컴파일 시점에 호출 함수를 결정하면 정적 바인딩이고, 실행 시점의 실제 객체 타입에 따라 결정하면 동적 바인딩이다.
가상 함수가 아닌 멤버 함수는 정적 타입을 기준으로 호출된다.

```
class GameObject
{
public:
    void PrintType() const
    {
        std::cout << "GameObject\n";
    }
};

class Character : public GameObject
{
};

class PlayerBase : public Character
{
};

class Player : public PlayerBase
{
public:
    void PrintType() const
    {
        std::cout << "Player\n";
    }
};

Player player;
GameObject& object = player;

object.PrintType();
```

호출 결과는 GameObject이다. 표현식 object의 정적 타입이 GameObject&이기 때문이다.

```
GameObject
```

함수를 가상 함수로 선언하면 실제 객체의 동적 타입을 사용한다.

```
class GameObject
{
public:
    virtual void PrintType() const
    {
        std::cout << "GameObject\n";
    }
};

class Player : public PlayerBase
{
public:
    void PrintType() const override
    {
        std::cout << "Player\n";
    }
};

Player player;
GameObject& object = player;

object.PrintType();
```

실행 결과는 Player이다.

```
Player
```

<정적 타입과 동적 타입>

구분	의미
정적 타입	변수나 표현식 선언에서 결정되는 타입
동적 타입	실행 중 실제로 존재하는 객체의 타입

object의 정적 타입은 GameObject&이고 동적 타입은 Player이다.

8.5.2 가상 함수

기반 클래스에서 virtual로 선언한 함수는 파생 클래스에서 재정의할 수 있다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual void Update()
    {
        std::cout << "GameObject Update\n";
    }

    virtual void Render() const
    {
        std::cout << "GameObject Render\n";
    }
};
```

계층의 중간 클래스와 구체 클래스가 각자의 역할을 구현한다.

```
class Character : public GameObject
{
public:
    void Update() override
    {
        std::cout << "Character state update\n";
    }
```

```
};

class PlayerBase : public Character
{
public:
    void Update() override
    {
        std::cout << "Player input update\n";
    }
};

class Player : public PlayerBase
{
public:
    void Update() override
    {
        std::cout << "Player Update\n";
    }

    void Render() const override
    {
        std::cout << "Player Render\n";
    }
};

class MobBase : public Character
{
public:
    void Update() override
    {
        std::cout << "Mob AI base update\n";
    }
};

class Monster : public MobBase
{
public:
    void Update() override
    {
        std::cout << "Monster patrol update\n";
    }

    void Render() const override
    {
        std::cout << "Monster Render\n";
    }
};

class BossMonster : public MobBase
{
public:
    void Update() override
    {
        std::cout << "Boss phase update\n";
    }

    void Render() const override
    {
        std::cout << "BossMonster Render\n";
    }
};
```

공통 함수는 구체 타입을 구분하지 않는다.

```
void Process(GameObject& object)
{
    object.Update();
    object.Render();
}
```

```
Player player;
Monster monster;
BossMonster boss;

Process(player);
Process(monster);
Process(boss);
```

각 객체의 재정의 함수가 실행된다.

```
Player Update
Player Render
Monster patrol update
Monster Render
Boss phase update
BossMonster Render
```

8.5.3 override와 final

override는 함수가 기반 클래스의 가상 함수를 재정의한다는 의도를 명시한다.

```
class Monster : public MobBase
{
public:
    void Update() override
    {
    }
};
```

final은 특정 가상 함수의 추가 재정의를 막는다. 기반 함수는 먼저 가상 함수로 선언되어 있어야 한다.

```
class MobBase : public Character
{
public:
    virtual void EnterRageMode()
    {
    }
};

class BossMonster : public MobBase
{
public:
    void EnterRageMode() final
    {
        std::cout << "Boss rage mode\n";
    }
};
```

BossMonster의 파생 클래스는 EnterRageMode()를 다시 재정의할 수 없다.

클래스 자체의 상속을 막을 수도 있다.

```
class FinalBossMonster final : public BossMonster
{
};

// class HiddenBoss : public FinalBossMonster // 오류
// {
// };
```

final은 확장을 막는 강한 제약이다. 보안이나 불변식 유지처럼 명확한 이유가 있을 때 사용하고, 단순히 현재 파생 클래스가 없다는 이유만으로 붙이지 않는 편이 좋다.

8.5.4 가상 함수 테이블의 개념적 모델

C++ 표준은 가상 함수 구현 방식으로 가상 함수 테이블을 요구하지 않는다. 그러나 대부분의 구현은 가상 함수 테이블(vtable)과 가상 함수 포인터(vptr)에 가까운 구조를 사용한다.

GameObject의 가상 함수 테이블을 개념적으로 표현할 수 있다.

```
GameObject 가상 함수 테이블
├─ Update  → GameObject::Update
├─ Render  → GameObject::Render
└─ 소멸자  → GameObject::~GameObject
```

Player 객체는 재정의한 함수 주소가 들어 있는 테이블을 가리킬 수 있다.

```
Player 가상 함수 테이블
├─ Update  → Player::Update
├─ Render  → Player::Render
└─ 소멸자  → Player::~Player

Player 객체
├─ vptr ──────▶ Player 가상 함수 테이블
├─ GameObject 부분
├─ Character 부분
├─ PlayerBase 부분
└─ Player 부분
```

기반 클래스 참조로 호출해도 객체 안의 가상 함수 정보가 실제 구현을 선택한다.

```
GameObject&
    │
    ▼
Player 객체의 vptr
    │
    ▼
Player::Update 호출
```

가상 함수는 일반적으로 간접 호출과 객체당 포인터 저장 비용을 동반한다. 비용은 대부분의 객체 지향 설계에서 작지만, 매우 많은 소형 객체나 성능이 민감한 내부 루프에서는 측정이 필요하다.

8.5.5 공변 반환형 (covariant return type)

가상 함수가 기반 클래스의 포인터나 참조를 반환하면 파생 클래스 재정의 함수는 더 구체적인 파생 클래스 포인터나 참조를 반환할 수 있다. 이를 공변 반환형(covariant return type)이라고 한다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual GameObject* CloneRaw() const
    {
        return new GameObject(*this);
    }
};

class Character : public GameObject
{
};

class PlayerBase : public Character
{
};

class Player : public PlayerBase
{
public:
    Player* CloneRaw() const override
    {
```

```
        return new Player(*this);
    }
};
```

Player*는 GameObject*보다 구체적인 반환 타입이므로 허용된다.
참조 반환도 공변 반환형을 사용할 수 있다.

```
class GameObject
{
public:
    virtual GameObject& Self()
    {
        return *this;
    }
};

class BossMonster : public MobBase
{
public:
    BossMonster& Self() override
    {
        return *this;
    }
};
```

스마트 포인터 템플릿 타입에는 공변 반환형 규칙이 적용되지 않는다.

```
// std::unique_ptr<Player>는
// std::unique_ptr<GameObject>의 공변 반환형이 아님
```

다형 복제에서는 모든 재정의 함수가 같은 반환 타입을 사용한다.

```
virtual std::unique_ptr<GameObject> Clone() const = 0;
```

8.5.6 생성자와 소멸자에서의 가상 함수 호출

생성자와 소멸자에서 가상 함수를 호출하면 현재 실행 중인 생성자나 소멸자의 클래스 범위를 기준으로 호출된다. 아직 생성되지 않은 파생 부분이나 이미 소멸한 파생 부분을 사용하지 않도록 하기 위한 규칙이다.

```
class GameObject
{
public:
    GameObject()
    {
        Initialize();
    }

    virtual ~GameObject()
    {
        Shutdown();
    }

    virtual void Initialize()
    {
        std::cout << "GameObject Initialize\n";
    }

    virtual void Shutdown()
    {
        std::cout << "GameObject Shutdown\n";
    }
};

class Player : public PlayerBase
{
public:
```

```
    void Initialize() override
    {
        std::cout << "Player Initialize\n";
    }

    void Shutdown() override
    {
        std::cout << "Player Shutdown\n";
    }
};
```

Player를 생성해도 GameObject 생성자 안의 Initialize()는 GameObject::Initialize()를 호출한다. 소멸 과정의 GameObject 소멸자에서도 GameObject::Shutdown()이 호출된다.

파생 클래스 초기화가 필요한 작업은 객체가 완전히 생성된 뒤 별도 함수에서 수행하거나 팩토리 함수가 생성 후 초기화를 완료하도록 구성한다.

8.5.7 가상 함수와 기본 인수

가상 함수의 구현은 동적 타입에 따라 선택되지만 기본 인수는 호출 표현식의 정적 타입에 따라 결정된다.

```
class GameObject
{
public:
    virtual void Draw(int layer = 0) const
    {
        std::cout << "GameObject layer: "
                  << layer << '\n';
    }
};

class Player : public PlayerBase
{
public:
    void Draw(int layer = 10) const override
    {
        std::cout << "Player layer: "
                  << layer << '\n';
    }
};

Player player;
GameObject& object = player;

player.Draw();
object.Draw();
```

실행 결과는 기본 인수 선택과 가상 함수 선택이 분리되어 있음을 보여 준다.

```
Player layer: 10
Player layer: 0
```

두 호출 모두 Player::Draw()를 실행하지만 object.Draw()의 기본 인수는 정적 타입 GameObject&의 선언에서 가져온다.
혼란을 줄이려면 가상 함수 재정의마다 다른 기본 인수를 지정하지 않는다. 기본 인수를 비가상 공개 함수에 두고 내부 가상 함수를 호출하는 방식도 사용할 수 있다.

8.6 추상 클래스와 인터페이스

모든 게임 객체가 갱신되고 렌더링되어야 하더라도 GameObject 자체의 구체적인 동작은 정의하기 어려울 수 있다. 이런 경우 기반 클래스는 공통 계약만 선언하고 실제 동작은 파생 클래스에 맡긴다.

8.6.1 순수 가상 함수

구현을 반드시 파생 클래스에 맡길 가상 함수는 = 0을 붙여 순수 가상 함수(pure virtual function)로 선언한다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual void Update() = 0;
    virtual void Render() const = 0;
};
```

순수 가상 함수를 하나 이상 가진 클래스는 직접 객체를 생성할 수 없다.

```
// GameObject object;    // 오류
```

파생 클래스가 모든 순수 가상 함수를 구현하면 구체 클래스가 된다.

```
class Player : public PlayerBase
{
public:
   void Update() override
   {
      ProcessInput();
   }

   void Render() const override
   {
      std::cout << "Render Player\n";
   }

   void ProcessInput() override
   {
      std::cout << "Process player input\n";
   }
};
```

순수 가상 함수도 별도의 정의를 가질 수 있지만, 호출하려면 범위를 명시해야 한다. 일반적인 인터페이스 계약에서는 구현을 두지 않는 편이 이해하기 쉽다.

8.6.2 추상 클래스

직접 객체를 만들 수 없고 파생 클래스의 공통 기반으로 사용하는 클래스를 추상 클래스(abstract class)라고 한다.
GameObject, PlayerBase, MobBase를 추상 클래스로 구성할 수 있다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual void Update() = 0;
    virtual void Render() const = 0;
};

class Character : public GameObject
{
public:
    explicit Character(int hp)
        : hp(hp)
    {
    }

    void TakeDamage(int damage)
    {
        if (damage > 0)
        {
            if (hp -= damage; hp < 0)
            {
                hp = 0;
            }
        }
    }
```

```
    int GetHp() const
    {
        return hp;
    }

private:
    int hp;
};
```

Character가 Update()와 Render()를 구현하지 않았다면 여전히 추상 클래스이다.
플레이어 계열은 입력 처리 계약을 추가한다.

```
class PlayerBase : public Character
{
public:
    using Character::Character;

    virtual void ProcessInput() = 0;
};
```

몹 계열은 판단과 공격 계약을 추가한다.

```
class MobBase : public Character
{
public:
    using Character::Character;

    virtual void Think() = 0;
    virtual void Attack(Character& target) = 0;
};

// PlayerBase playerBase(100); // 오류
// MobBase mobBase(100);       // 오류
```

추상 클래스는 공통 상태와 일부 구현을 가질 수 있다. 인터페이스만 가진 클래스와 달리 생성자, 데이터 멤버와 일반 멤버 함수를 포함할 수 있다.

8.6.3 인터페이스 역할의 클래스

C++에는 별도의 interface 키워드가 없다. 순수 가상 함수와 가상 소멸자만 제공하는 클래스를 인터페이스 역할로 사용할 수 있다.

```
class IDamageable
{
public:
    virtual ~IDamageable() = default;
    virtual void TakeDamage(int damage) = 0;
};
```

저장 기능을 계약으로 분리할 수도 있다.

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};
```

Character는 피해 처리 인터페이스를 구현할 수 있다.

```
class Character : public GameObject, public IDamageable
{
public:
    explicit Character(int hp)
        : hp(hp)
    {
```

```
    }

    void TakeDamage(int damage) override
    {
        if (damage > 0)
        {
            hp -= damage;

            if (hp < 0)
            {
                hp = 0;
            }
        }
    }

private:
    int hp;
};
```

인터페이스 이름의 I 접두어는 C++ 언어 규칙이 아니다. 프로젝트의 명명 규칙에 따라 사용할 수 있다.

8.6.4 구현 상속과 인터페이스 상속

상속은 구현 재사용과 인터페이스 계약이라는 두 목적을 가질 수 있다.

구현 상속은 기반 클래스의 상태와 동작을 물려받는다.

```
class MobBase : public Character
{
protected:
    int attackPower = 10;
};
```

인터페이스 상속은 구현보다 호출 계약에 초점을 둔다.

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};

BossMonster는 MobBase에서 캐릭터와 몹의 구현을 상속하고 ISavable의 계약을 구현할 수 있다.
class BossMonster : public MobBase, public ISavable
{
public:
    void Save(std::ostream& output) const override
    {
        output << "BossMonster\n";
    }
};
```

<구현 상속과 인터페이스 상속>

구분	구현 상속	인터페이스 상속
목적	상태와 구현 재사용	공통 호출 계약 제공
데이터 멤버	가질 수 있음	보통 두지 않음
일반 함수	포함 가능	순수 가상 함수 중심
결합도	기반 구현에 강하게 결합	계약에 상대적으로 약하게 결합

한 클래스가 여러 구현 기반을 상속하면 상태 중복과 이름 충돌이 생기기 쉽다. 여러 역할을 표현해야 할 때는 하나의 주 구현 계층과 여러 인터페이스를 결합하는 방식이 관리하기 쉽다.

8.6.5 최소 인터페이스와 계약

기반 클래스의 공개 인터페이스는 모든 파생 클래스에 의미가 있어야 한다. 특정 파생 클래스에서만 필요한 기능을 최상위 기반 클래스에 넣으면 다른 파생 클래스가 무의미한 함수를 구현하게 된다.

GameObject에 플레이어 입력 함수를 넣는 설계는 적절하지 않다.

```
class GameObject
{
public:
    virtual void ProcessInput() = 0;
};
```

Monster와 BossMonster는 플레이어 입력을 처리하지 않으므로 계약이 맞지 않는다.

입력 계약은 PlayerBase로 제한한다.

```
class PlayerBase : public Character
{
public:
    virtual void ProcessInput() = 0;
};
```

AI 판단 계약은 MobBase로 제한한다.

```
class MobBase : public Character
{
public:
    virtual void Think() = 0;
};
```

기반 인터페이스는 작고 안정적으로 유지하는 것이 좋다.

- 모든 파생 클래스에 의미가 있는 연산만 포함
- 구현 세부 사항보다 행동 계약을 표현
- 파생 클래스가 지켜야 할 사전 조건과 사후 조건을 명확히 정의
- 서로 독립적인 역할은 작은 인터페이스로 분리
- 변경 가능성이 큰 기능은 구성 객체로 이동

8.6.6 기본 구현과 비가상 인터페이스

가상 함수의 호출 전후에 공통 규칙을 반드시 적용해야 한다면 공개 함수는 비가상으로 두고 내부 가상 함수를 호출할 수 있다. 이를 비가상 인터페이스(non-virtual interface) 방식이라고 한다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    void Update()
    {
        if (!active)
        {
            return;
        }

        OnUpdate();
    }

    void Render() const
    {
        if (!active)
        {
            return;
        }
```

```
        OnRender();
    }

protected:
    virtual void OnUpdate() = 0;
    virtual void OnRender() const = 0;

private:
    bool active = true;
};
```

파생 클래스는 확장 지점만 구현한다.

```
class Monster : public MobBase
{
protected:
    void OnUpdate() override
    {
        Think();
    }

    void OnRender() const override
    {
        std::cout << "Render Monster\n";
    }
};
```

공개 Update()는 활성 상태를 검사하고, 파생 클래스는 실제 갱신 로직만 구현한다. 시간 값이 필요한 구현은 GameSystem::GetDeltaTime()으로 조회할 수 있다. 파생 클래스가 기반 검사를 빠뜨릴 가능성을 줄일 수 있다.

8.7 다형 객체 (Polymorphic object) 의 수명

다형 객체는 기반 클래스의 포인터나 참조를 사용해 파생 클래스 객체를 다루는 객체이다. 가상 함수를 사용하면 실제 객체의 타입에 맞는 함수가 호출된다.

다형 객체를 기반 클래스 포인터로 관리할 때는 소멸자, 복사와 소유권을 함께 설계해야 한다.

8.7.1 가상 소멸자

기반 클래스 포인터를 통해 파생 객체를 삭제할 가능성이 있다면 기반 클래스 소멸자는 가상이어야 한다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;
};
```

파생 클래스 소멸자는 자동으로 가상 함수가 된다.

```
class Character : public GameObject
{
public:
    ~Character() override = default;
};
```

override는 소멸자에도 사용할 수 있다.

다형 기반 클래스는 대부분 가상 함수를 이미 가지므로 가상 소멸자를 함께 제공하는 것이 자연스럽다. 다형 사용을 의도하지 않는 기반 클래스라면 소멸자를 protected 비가상으로 두어 외부의 기반 포인터 삭제를 막는 설계도 가능하다.

```
class NonPolymorphicBase
{
protected:
    ~NonPolymorphicBase() = default;
};
```

8.7.2 기반 클래스 포인터를 통한 삭제

가상 소멸자가 있는 기반 클래스 포인터로 파생 객체를 삭제하면 파생 소멸자부터 기반 소멸자까지 올바른 순서로 호출된다.

```
class GameObject
{
public:
    virtual ~GameObject()
    {
        std::cout << "GameObject 소멸\n";
    }
};

class Character : public GameObject
{
public:
    ~Character() override
    {
        std::cout << "Character 소멸\n";
    }
};

class MobBase : public Character
{
public:
    ~MobBase() override
    {
        std::cout << "MobBase 소멸\n";
    }
};

class BossMonster : public MobBase
{
public:
    ~BossMonster() override
    {
        std::cout << "BossMonster 소멸\n";
    }
};

GameObject* object = new BossMonster;
delete object;

// BossMonster 소멸
// MobBase 소멸
// Character 소멸
// GameObject 소멸
```

기반 클래스 소멸자가 비가상이면 기반 포인터를 통한 파생 객체 삭제는 정의되지 않은 동작이다. 파생 클래스가 소유한 자원이 해제되지 않거나 프로그램이 손상될 수 있다.
원시 포인터보다 스마트 포인터를 사용하더라도 기반 클래스의 가상 소멸자 설계는 중요하다.

8.7.3 객체 잘림 (Object slicing)

파생 객체를 기반 클래스 값으로 복사하면 기반 클래스 부분만 복사되고 파생 부분은 사라진다. 이를 객체 잘림(object slicing)이라고 한다.
설명을 위해 구체적인 Character 클래스를 사용한다.

```
class Character
{
public:
    virtual ~Character() = default;

    virtual void PrintType() const
    {
        std::cout << "Character\n";
    }

private:
    int hp = 100;
```

```
};

class PlayerBase : public Character
{
};

class Player : public PlayerBase
{
public:
    void PrintType() const override
    {
        std::cout << "Player\n";
    }

private:
    int score = 1000;
};

Player player;
Character copied = player;

copied.PrintType();
```

copied는 독립된 Character 객체이므로 Player 부분이 존재하지 않는다.

```
Character
```

함수 매개변수를 값으로 받으면 같은 문제가 생긴다.

```
void PrintCharacter(Character character)
{
    character.PrintType();
}
```

다형 객체는 포인터나 참조로 전달한다.

```
void PrintCharacter(const Character& character)
{
    character.PrintType();
}
```

컨테이너에도 기반 클래스 값을 직접 저장하지 않는다.

```
// 추상 클래스면 불가능하며 잘림 위험
// std::vector<GameObject> objects;
```

다형 객체는 std::unique_ptr<GameObject> 같은 스마트 포인터로 저장한다.

8.7.4 다형 객체의 복사와 Clone

기반 클래스 포인터가 실제로 가리키는 구체 타입을 유지하면서 복사하려면 가상 복제 함수를 제공할 수 있다.

```
class GameObject
{
public:
    virtual ~GameObject() = default;

    virtual std::unique_ptr<GameObject> Clone() const = 0;
};
```

각 구체 클래스는 자신의 타입으로 복사한 뒤 기반 스마트 포인터로 반환한다.

```
class Player : public PlayerBase
{
public:
```

```
    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<Player>(*this);
    }
};

class Monster : public MobBase
{
public:
    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<Monster>(*this);
    }
};

class BossMonster : public MobBase
{
public:
    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<BossMonster>(*this);
    }
};

std::unique_ptr<GameObject> original =
    std::make_unique<BossMonster>();

std::unique_ptr<GameObject> copied = original->Clone();
```

복제 결과는 BossMonster의 동적 타입을 유지한다.

Clone()은 복사 생성자의 다형적 대체 수단이다. 클래스가 복사를 지원하지 않아야 한다면 Clone()도 제공하지 않는다. 복제 의미가 필요한 상태와 외부 자원을 어떻게 처리할지도 클래스 계약으로 정해야 한다.

8.7.5 스마트 포인터와 다형 객체

고유 소유권을 가진 다형 객체는 std::unique_ptr<GameObject>로 관리할 수 있다.

```
std::unique_ptr<GameObject> player =
    std::make_unique<Player>();

std::unique_ptr<GameObject> monster =
    std::make_unique<Monster>();
```

std::unique_ptr<Player>는 이동을 통해 std::unique_ptr<GameObject>로 변환할 수 있다.

```
std::unique_ptr<Player> concrete =
    std::make_unique<Player>();

std::unique_ptr<GameObject> object =
    std::move(concrete);
```

다형 객체 컨테이너도 고유 소유권으로 구성할 수 있다.

```
std::vector<std::unique_ptr<GameObject>> objects;

objects.push_back(std::make_unique<Player>());
objects.push_back(std::make_unique<Monster>());
objects.push_back(std::make_unique<BossMonster>());

for (const auto& object : objects)
{
    object->Update();
    object->Render();
}
```

실제 공유 수명이 필요하면 std::shared_ptr<GameObject>를 사용할 수 있지만, 장면이 객체를 단독으로 소유하는 구조라면 unique_ptr가 수명과 책임을 더 명확하게 표현한다.

8.8 형 변환과 RTTI

상속 계층의 포인터와 참조는 업캐스팅과 다운캐스팅을 지원한다. 실행 중 실제 타입을 확인하는 기능을 RTTI(run-time type information)라고 한다.

8.8.1 업캐스팅

파생 클래스 포인터나 참조를 접근 가능한 기반 클래스 포인터나 참조로 변환하는 것을 업캐스팅이라고 한다.

```
Player player;

PlayerBase* playerBase = &player;
Character* character = &player;
GameObject* object = &player;
```

몹 계열도 상위 타입으로 변환할 수 있다.

```
BossMonster boss;

MobBase& mob = boss;
Character& character = boss;
GameObject& object = boss;
```

공개 상속의 업캐스팅은 안전하며 일반적으로 암시적으로 수행된다. 파생 객체 안에는 기반 클래스 부분이 실제로 존재하기 때문이다. 업캐스팅 후에는 정적 타입이 제공하는 인터페이스만 사용할 수 있다.

```
GameObject& object = boss;
object.Update();

// object.EnterRageMode(); // 오류
```

8.8.2 다운캐스팅

기반 클래스 포인터나 참조를 더 구체적인 파생 클래스 타입으로 변환하는 것을 다운캐스팅이라고 한다.

```
GameObject* object = new BossMonster;
```

object가 실제로 BossMonster를 가리킨다는 보장이 있다면 다운캐스팅할 수 있다. static_cast는 실행 중 타입을 검사하지 않는다.

```
BossMonster* boss = static_cast<BossMonster*>(object);
```

실제 객체가 BossMonster가 아니라면 잘못된 포인터가 만들어지고 이를 사용했을 때 정의되지 않은 동작이 발생할 수 있다.

```
GameObject* object = new Player;
BossMonster* boss = static_cast<BossMonster*>(object); // 위험
```

정적 지식으로 타입이 완전히 보장되는 제한된 내부 코드가 아니라면 dynamic_cast를 사용하거나 설계를 바꾸는 편이 안전하다.

8.8.3 dynamic_cast

dynamic_cast는 다형 기반 클래스의 RTTI를 사용하여 실행 중 타입을 검사한다. 기반 클래스에 가상 함수가 하나 이상 있어야 한다. 포인터 변환이 실패하면 nullptr를 반환한다.

```
void ActivateBossMode(GameObject* object)
{
    BossMonster* boss =
        dynamic_cast<BossMonster*>(object);
    if (boss != nullptr)
    {
        boss->EnterRageMode();
```

```
    }
}

Player player;
BossMonster boss;

ActivateBossMode(&player); // 변환 실패
ActivateBossMode(&boss);   // 변환 성공
```

중간 기반 타입으로 검사할 수도 있다.

```
void PrintControllerType(GameObject& object)
{
    if (dynamic_cast<PlayerBase*>(&object) != nullptr)
    {
        std::cout << "player controlled\n";
    }
    else if (dynamic_cast<MobBase*>(&object) != nullptr)
    {
        std::cout << "AI controlled\n";
    }
}
```

참조 변환이 실패하면 std::bad_cast 예외가 발생한다.

```
try
{
    BossMonster& boss =
        dynamic_cast<BossMonster&>(object);

    boss.EnterRageMode();
}
catch (const std::bad_cast& error)
{
    std::cout << error.what() << '\n';
}
```

실패를 정상적인 분기로 처리해야 한다면 포인터 형태가 편리하다.

8.8.4 typeid

typeid는 표현식이나 타입의 std::type_info 정보를 얻는다.

```
#include <typeinfo>

GameObject& object = boss;

std::cout << typeid(object).name() << '\n';
```

다형 클래스의 참조나 역참조된 포인터에 typeid를 적용하면 동적 타입을 확인한다.

```
if (typeid(object) == typeid(BossMonster))
{
    std::cout << "BossMonster object\n";
}
```

포인터 자체에 적용하면 포인터의 정적 타입 정보를 얻는다.

```
GameObject* pointer = &boss;
std::cout << typeid(pointer).name() << '\n';
```

name()의 문자열 형식은 구현에 따라 다르므로 프로그램 로직에서 문자열을 비교하지 않는다. 타입 비교가 필요하면 std::type_info 객체를 직접 비교한다.

8.8.5 다운캐스팅을 줄이는 설계

다운캐스팅이 반복되면 기반 인터페이스가 필요한 행동을 충분히 표현하지 못하고 있을 가능성이 있다.
구체 타입을 검사하여 공격 함수를 호출하는 코드는 확장에 취약하다.

```
void ExecuteAction(GameObject& object)
{
    if (auto* monster = dynamic_cast<Monster*>(&object))
    {
        monster->Attack();
    }
    else if (auto* boss = dynamic_cast<BossMonster*>(&object))
    {
        boss->Attack();
    }
}
```

MobBase에 공통 가상 함수를 두면 타입 분기를 제거할 수 있다.

```
class MobBase : public Character
{
public:
    virtual void Attack(Character& target) = 0;
};

void ExecuteAttack(MobBase& mob, Character& target)
{
    mob.Attack(target);
}
```

모든 게임 객체가 처리해야 하는 행동이면 GameObject의 가상 함수로 올릴 수 있다. 일부 역할에만 필요한 행동이면 작은 인터페이스를 도입할 수 있다.

```
class IInteractable
{
public:
    virtual ~IInteractable() = default;
    virtual void Interact(Player& player) = 0;
};
```

다운캐스팅이 적절한 경우도 있다.

- 편집기나 디버그 도구에서 타입별 정보를 표시
- 직렬화 시스템에서 등록된 구체 타입을 복원
- 외부 API가 기반 포인터만 제공
- 타입별 최적화 경로를 제한적으로 선택

게임 로직의 주요 분기를 RTTI에 의존하기보다 가상 함수, 인터페이스, 방문자 패턴이나 구성 객체로 표현하는 편이 확장에 유리하다.

8.9 다중 상속

한 파생 클래스가 둘 이상의 기반 클래스를 상속하는 것을 다중 상속이라고 한다. 여러 역할을 하나의 타입에 결합할 수 있지만 상태 중복, 이름 충돌과 다이아몬드 상속 문제가 생길 수 있다.

8.9.1 다중 상속의 구조

BossMonster가 몹 계층의 구현과 저장 인터페이스를 함께 상속할 수 있다.

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};

class BossMonster : public MobBase, public ISavable
{
```

```
public:
    void Save(std::ostream& output) const override
    {
        output << "BossMonster\n";
    }
};
```

이 구조는 주 구현 계층 하나와 인터페이스 하나를 결합한다. 인터페이스가 상태를 가지지 않으므로 객체 구조가 비교적 단순하다.
다중 상속의 문법 자체보다 각 기반 클래스가 어떤 역할을 담당하는지가 중요하다.

8.9.2 이름 충돌

둘 이상의 기반 클래스에 같은 이름의 멤버가 있으면 파생 클래스에서 이름 충돌이 발생할 수 있다.

```
class IRenderable
{
public:
    virtual ~IRenderable() = default;

    virtual void Reset()
    {
        std::cout << "IRenderable Reset\n";
    }
};

class ISavable
{
public:
    virtual ~ISavable() = default;

    virtual void Reset()
    {
        std::cout << "ISavable Reset\n";
    }
};

class BossMonster : public MobBase,
                    public IRenderable,
                    public ISavable
{
};

BossMonster boss;

// boss.Reset();    // 오류: 어느 기반 함수인지 모호함
```

범위를 명시하면 특정 기반 함수로 호출할 수 있다.

```
boss.IRenderable::Reset();
boss.ISavable::Reset();
```

파생 클래스에서 하나의 의미로 통합할 수도 있다.

```
class BossMonster : public MobBase,
                    public IRenderable,
                    public ISavable
{
public:
    void Reset() override
    {
        IRenderable::Reset();
        ISavable::Reset();
        std::cout << "BossMonster Reset\n";
    }
};
```

이름 충돌이 자주 발생한다면 인터페이스의 책임이 겹치거나 이름이 지나치게 일반적일 수 있다.

8.9.3 다이아몬드 상속

두 기반 클래스가 같은 상위 기반 클래스를 상속하고, 하나의 파생 클래스가 두 기반 클래스를 모두 상속하면 다이아몬드 구조가 만들어진다.

```
class GameObject
{
public:
    int GetId() const
    {
        return id;
    }

private:
    int id = 0;
};

class Character : public GameObject
{
};

class NetworkObject : public GameObject
{
};

class NetworkCharacter : public Character,
                         public NetworkObject
{
};
```

NetworkCharacter에는 Character 경로와 NetworkObject 경로를 통해 두 개의 GameObject 부분이 생긴다.

```
NetworkCharacter 객체
├─ Character 부분
│   └─ GameObject 부분
└─ NetworkObject 부분
    └─ GameObject 부분
```

어느 GameObject 부분을 사용할지 모호하다.

```
NetworkCharacter object;

// object.GetId();    // 오류: 모호함
```

범위를 지정하면 접근할 수 있지만 객체 안에 두 개의 기반 부분이 존재하는 문제는 남는다.

```
int first = object.Character::GetId();
int second = object.NetworkObject::GetId();
```

다이아몬드 구조는 동일한 기반 상태가 중복되고 업캐스팅 경로가 모호해질 수 있다.

8.9.4 가상 상속

다이아몬드 구조에서 최상위 기반 클래스 부분을 하나만 공유하려면 가상 상속(virtual inheritance)을 사용할 수 있다.

```
class Character : public virtual GameObject
{
};

class NetworkObject : public virtual GameObject
{
};

class NetworkCharacter : public Character,
                         public NetworkObject
{
```

```
};
```

```
NetworkCharacter 객체
├─ 공유 GameObject 부분
├─ Character 부분
├─ NetworkObject 부분
└─ NetworkCharacter 부분
```

```
NetworkCharacter object;
int id = object.GetId();
```

가상 상속은 기반 부분의 중복을 해결하지만 객체 배치, 초기화와 접근 비용을 복잡하게 만든다. 단순한 기능 조합을 위해 다이아몬드 구조를 만든 뒤 가상 상속으로 해결하기보다 계층 자체를 재검토하는 편이 좋다.

게임 객체의 네트워크 기능은 상속보다 구성으로 표현할 수 있다.

```
class Character : public GameObject
{
private:
    NetworkComponent network;
};
```

```
class Character : public GameObject
{
private:
    NetworkComponent network;
};
```

8.9.5 가상 기반 클래스 초기화

가상 기반 클래스는 가장 구체적인 파생 클래스가 직접 초기화한다.

```
class GameObject
{
public:
    explicit GameObject(int id)
        : id(id)
    {
    }

private:
    int id;
};
```

```
class Character : public virtual GameObject
{
public:
    Character()
        : GameObject(10)
    {
    }
};

class NetworkObject : public virtual GameObject
{
public:
    NetworkObject()
        : GameObject(20)
    {
    }
};

class NetworkCharacter : public Character,
                         public NetworkObject
{
```

```
public:
    NetworkCharacter()
        : GameObject(100),
          Character(),
          NetworkObject()
    {
    }
};
```

NetworkCharacter를 생성할 때는 가장 구체적인 클래스가 지정한 GameObject(100)이 사용된다. Character와 NetworkObject의 가상 기반 초기화 지정은 무시된다.

가상 기반 클래스는 일반 직접 기반 클래스보다 먼저 초기화된다.

<가상 상속 초기화 원칙>

항목	규칙
초기화 책임	가장 구체적인 파생 클래스
생성 순서	가상 기반 → 일반 기반 → 멤버 → 생성자 본문
중복 기반 부분	하나만 존재

8.9.6 인터페이스 다중 상속

상태와 구현을 가진 여러 클래스를 다중 상속하는 것보다 작은 인터페이스를 여러 개 구현하는 방식이 일반적으로 안전하다.

```
class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save(std::ostream& output) const = 0;
};

class ITargetable
{
public:
    virtual ~ITargetable() = default;
    virtual bool CanBeTargeted() const = 0;
};

class BossMonster : public MobBase,
                    public ISavable,
                    public ITargetable
{
public:
    void Save(std::ostream& output) const override
    {
        output << "BossMonster\n";
    }

    bool CanBeTargeted() const override
    {
        return true;
    }
};
```

호출 코드는 필요한 역할에만 의존한다.

```
void SaveObject(const ISavable& object,
                std::ostream& output)
{
    object.Save(output);
}

bool TryLockTarget(const ITargetable& target)
{
```

```
    return target.CanBeTargeted();
}
```

인터페이스 다중 상속은 한 객체가 여러 역할을 제공한다는 사실을 표현한다. 각 인터페이스는 작고 독립적인 계약으로 유지하는 편이 좋다.

8.10 상속과 구성

상속은 is-a 관계를 표현하고 구성은 has-a 관계를 표현한다. 두 방식은 코드 재사용 수단이 아니라 객체의 관계와 책임을 설계하는 선택이다.

8.10.1 has-a 관계와 구성

플레이어는 인벤토리와 무기를 가진다.

```
Player has an Inventory.
Player has a Weapon.

class Inventory
{
public:
    void AddItem(int itemId)
    {
        items.push_back(itemId);
    }

private:
    std::vector<int> items;
};

class Weapon
{
public:
    void Attack()
    {
        std::cout << "Weapon attack\n";
    }
};
```

Player는 두 객체를 멤버로 가진다.

```
class Player : public PlayerBase
{
public:
    void AddItem(int itemId)
    {
        inventory.AddItem(itemId);
    }

    void Attack()
    {
        weapon.Attack();
    }

private:
    Inventory inventory;
    Weapon weapon;
};
```

구성 객체는 자신의 상태와 규칙을 관리한다. Player는 외부에 필요한 기능만 제공한다.
보스의 페이즈 로직도 별도 객체로 분리할 수 있다.

```
class BossPhaseController
{
public:
    void EvaluatePhase(int hp)
    {
        if (hp < 300)
```

```
        {
            phase = 3;
        }
        else if (hp < 700)
        {
            phase = 2;
        }
    }

private:
    int phase = 1;
};

class BossMonster : public MobBase
{
private:
    BossPhaseController phaseController;
};
```

8.10.2 위임

객체가 자신이 가진 다른 객체에 작업을 맡기는 것을 위임(delegation)이라고 한다.

```
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp)
        : hp(maxHp), maxHp(maxHp)
    {
    }

    void TakeDamage(int damage)
    {
        if (damage > 0)
        {
            hp -= damage;

            if (hp < 0)
            {
                hp = 0;
            }
        }
    }

    int GetHp() const
    {
        return hp;
    }

private:
    int hp;
    int maxHp;
};
```

Character는 체력 처리를 HealthComponent에 위임할 수 있다.

```
class Character : public GameObject
{
public:
    explicit Character(int hp)
        : health(hp)
    {
    }

    void TakeDamage(int damage)
    {
        health.TakeDamage(damage);
    }
```

```
    int GetHp() const
    {
        return health.GetHp();
    }

private:
    HealthComponent health;
};
```

위임은 공개 인터페이스를 유지하면서 내부 구현을 바꿀 수 있게 한다. 체력 기능을 여러 종류의 객체에서 재사용해야 할 때 깊은 상속 계층을 만드는 것보다 구성 요소로 분리하는 편이 유연하다.

8.10.3 상속의 결합도

파생 클래스는 기반 클래스의 공개·보호 인터페이스, 초기화 규칙과 수명에 의존한다. protected 데이터까지 직접 사용하면 내부 표현에도 의존한다.

```
class MobBase : public Character
{
protected:
    void RecoverHalfHp()
    {
        hp += 50; // Character의 저장 방식에 직접 의존
    }
};
```

Character가 체력을 HealthComponent로 바꾸면 이 코드는 수정해야 한다.

```
class Character : public GameObject
{
public:
    int GetHp() const;
    void Recover(int amount);

private:
    HealthComponent health;
};
```

파생 클래스가 공개·보호 함수로만 상태를 처리하면 변경 영향이 줄어든다.

```
class MobBase : public Character
{
protected:
    void RecoverHalfHp()
    {
        Recover(50);
    }
};
```

<상속이 만드는 결합>

결합 대상	영향
기반 클래스 인터페이스	함수 변경이 파생 클래스에 전파
생성자	기반 초기화 인수가 파생 생성자에 전파
protected 데이터	내부 표현 변경에 취약
가상 함수 계약	모든 재정의 클래스가 의미를 유지해야 함
소멸 정책	기반 포인터 삭제와 소유권에 영향

8.10.4 상속보다 구성을 우선하는 이유

구성은 객체 사이의 관계를 멤버와 인터페이스로 표현한다. 구현 교체와 기능 조합이 상속보다 자유롭다.
AI 행동을 상속으로 세분화하면 계층이 빠르게 늘어날 수 있다.

```
MobBase
├─ PatrolMonster
├─ ChaseMonster
├─ RangedMonster
├─ PatrolRangedMonster
└─ ChaseRangedMonster
```

행동을 구성 객체로 분리하면 타입 조합을 줄일 수 있다.

```
class IMobBehavior
{
public:
    virtual ~IMobBehavior() = default;
    virtual void Think() = 0;
};

class PatrolBehavior : public IMobBehavior
{
public:
    void Think() override
    {
        std::cout << "Patrol\n";
    }
};

class ChaseBehavior : public IMobBehavior
{
public:
    void Think() override
    {
        std::cout << "Chase\n";
    }
};

class Monster : public MobBase
{
public:
    explicit Monster(std::unique_ptr<IMobBehavior> behavior)
        : behavior(std::move(behavior))
    {
    }

    void Think() override
    {
        behavior->Think();
    }

private:
    std::unique_ptr<IMobBehavior> behavior;
};
```

행동 객체를 교체하면 같은 Monster 타입에서 다른 AI를 사용할 수 있다.

구성을 우선하는 이유는 네 가지로 정리할 수 있다.

- 실행 중 구성 요소를 교체할 수 있음
- 기능 조합 때문에 파생 클래스 수가 늘어나는 것을 줄임
- 기반 구현 변경의 영향을 제한함
- 각 구성 요소를 독립적으로 테스트할 수 있음

8.10.5 상속이 필요한 경우

구성이 항상 상속보다 우월한 것은 아니다. 공개적인 타입 관계와 동적 대체가 필요하면 상속이 적합하다.

```
// Player, Monster, BossMonster → GameObject로 처리.
void ProcessObject(GameObject& object)
{
    object.Update();
    object.Render();
}
```

이 요구는 공개 상속과 가상 함수에 잘 맞는다.

```
상속이 적합한 경우:
파생 객체가 기반 객체의 한 종류임
공통 인터페이스로 여러 구체 타입을 처리해야 함
기반 클래스 계약이 안정적임
동적 바인딩이 필요함
프레임워크가 파생 클래스 확장 지점을 제공함
구성이 적합한 경우:
객체가 다른 기능 객체를 소유함
기능을 실행 중 교체해야 함
여러 기능의 조합이 필요함
타입 관계보다 구현 재사용이 목적임
기반 클래스 변경과의 결합을 줄여야 함
```

8.10.6 UML로 비교하는 상속과 구성

게임 객체의 종류는 상속으로, 객체가 가진 기능은 구성으로 표현할 수 있다.
빈 삼각형 화살표는 일반화를 나타낸다. 채워진 마름모는 강한 소유 관계인 합성을, 빈 마름모는 구성 요소의 수명이 외부와 공유될 수 있는 집계를 나타낼 수 있다. 실제 C++ 소유권은 값 멤버, unique_ptr, shared_ptr 등의 코드와 함께 확인해야 한다.

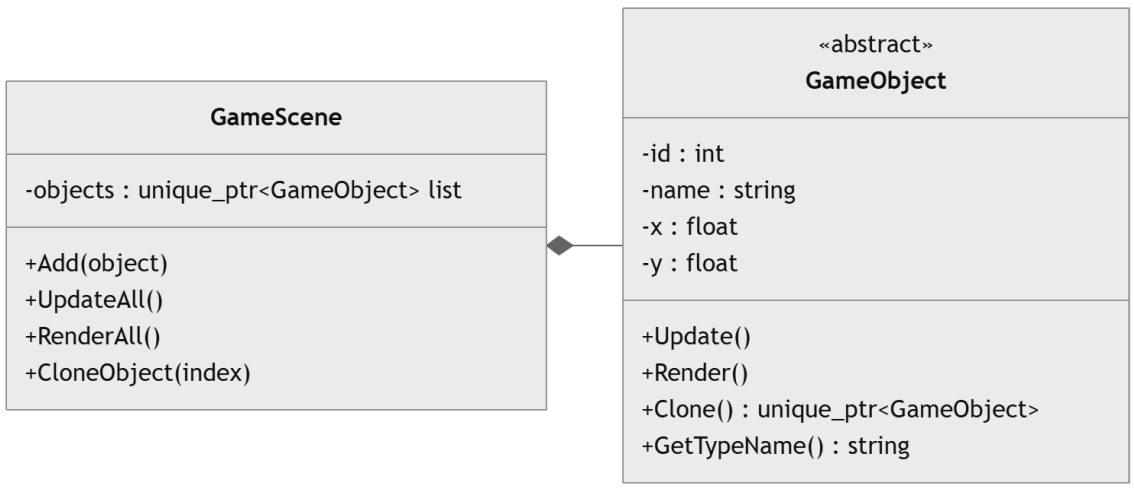
8.11 종합 예제: 게임 객체 계층과 구성

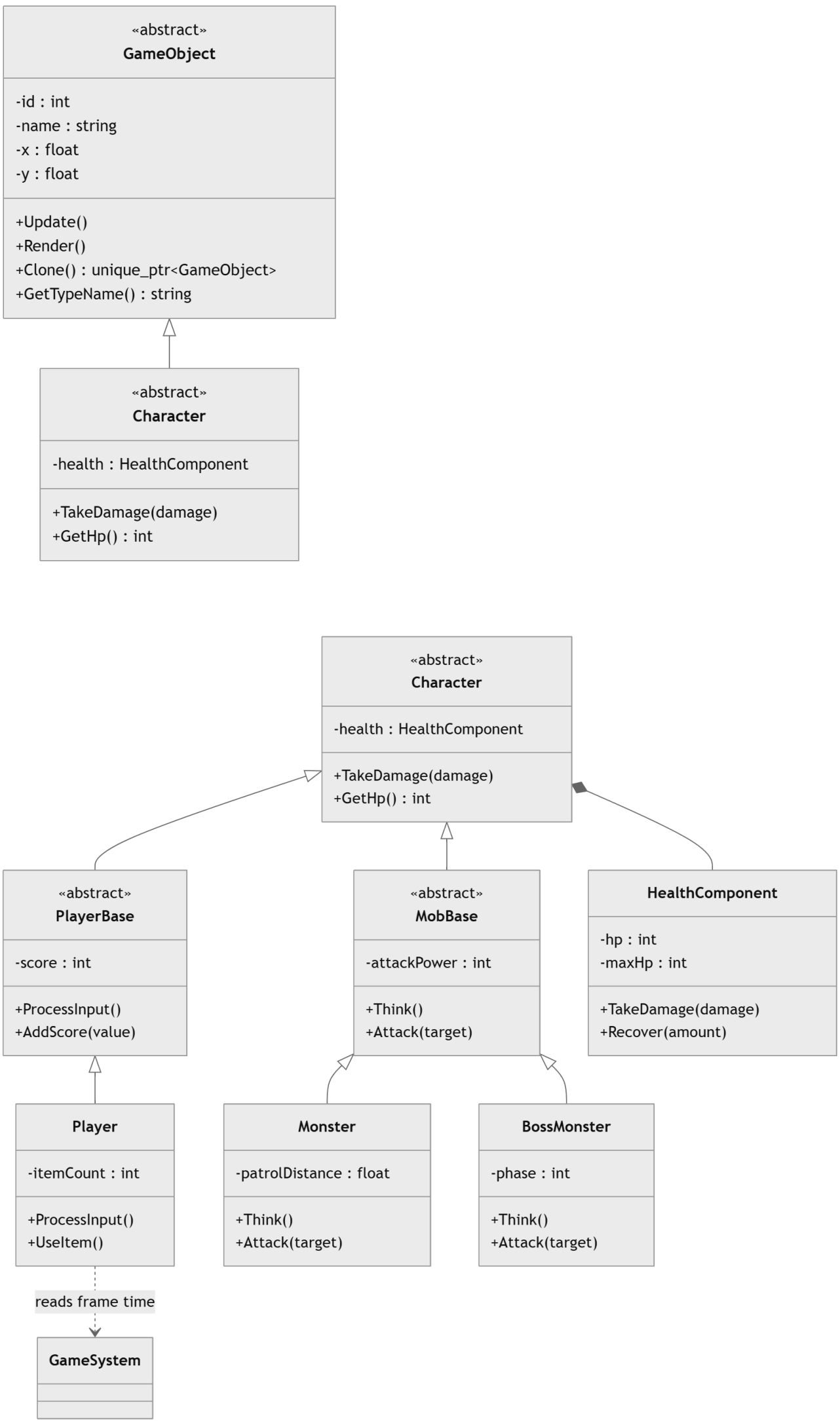
종합 예제는 장 전체에서 사용한 계층을 실제로 동작하는 코드로 구성한다.

```
GameObject
└─ Character
   ├─ PlayerBase
   │  └─ Player
   └─ MobBase
      ├─ Monster
      └─ BossMonster
```

GameScene은 std::unique_ptr<GameObject>를 저장하여 서로 다른 구체 객체를 고유 소유한다. 캐릭터의 체력은 HealthComponent가 관리하고, 구체 클래스는 자신의 갱신·렌더링·복제 동작을 구현한다. 프레임 시간은 GameSystem이 보관하며 시간 값이 필요한 객체만 이를 조회한다.

8.11.1 클래스 구조와 UML





8.11.2 GameSystem, HealthComponent와 GameObject

GameSystem은 프레임 시작 시 경과 시간을 저장하고 게임 객체에 읽기 기능을 제공한다. Update()의 공통 인터페이스는 인수 없이 유지하며, 실제로 시간 값이 필요한 구현만 GameSystem::GetDeltaTime()을 호출한다.

```
// GameHierarchy.h
#pragma once

#include <algorithm>
#include <cstddef>
#include <iostream>
```

```cpp
#include <memory>
#include <string>
#include <utility>
#include <vector>

class GameSystem
{
public:
    static void BeginFrame(float deltaTime)
    {
        GameSystem::deltaTime = std::max(deltaTime, 0.0f);
    }

    static float GetDeltaTime()
    {
        return deltaTime;
    }

private:
    inline static float deltaTime = 0.0f;
};
```

```cpp
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp)
        : hp(std::max(maxHp, 0)),
          maxHp(std::max(maxHp, 0))
    {
    }

    void TakeDamage(int damage)
    {
        if (damage <= 0)
        {
            return;
        }

        hp = std::max(hp - damage, 0);
    }

    void Recover(int amount)
    {
        if (amount <= 0)
        {
            return;
        }

        hp = std::min(hp + amount, maxHp);
    }

    int GetHp() const
    {
        return hp;
    }

    int GetMaxHp() const
    {
        return maxHp;
    }

    bool IsDead() const
    {
        return hp == 0;
    }

private:
    int hp;
    int maxHp;
```

```
};

class GameObject
{
public:
    GameObject(int id, std::string name)
        : id(id), name(std::move(name))
    {
    }

    virtual ~GameObject() = default;

    GameObject(const GameObject&) = default;
    GameObject& operator=(const GameObject&) = default;
    GameObject(GameObject&&) noexcept = default;
    GameObject& operator=(GameObject&&) noexcept = default;

    virtual void Update() = 0;
    virtual void Render() const = 0;
    virtual std::unique_ptr<GameObject> Clone() const = 0;
    virtual std::string GetTypeName() const = 0;

    int GetId() const
    {
        return id;
    }

    const std::string& GetName() const
    {
        return name;
    }

    void SetPosition(float x, float y)
    {
        this->x = x;
        this->y = y;
    }

    float GetX() const
    {
        return x;
    }

    float GetY() const
    {
        return y;
    }

private:
    int id;
    std::string name;
    float x = 0.0f, y = 0.0f;
};
```

GameObject는 추상 클래스이며 위치와 식별 정보를 제공한다. 복제 함수는 동적 타입을 유지하는 새 객체를 반환한다.

8.11.3 Character, PlayerBase와 Player

```
class Character : public GameObject
{
public:
    Character(int id, std::string name, int maxHp)
        : GameObject(id, std::move(name)),
          health(maxHp)
    {
    }

    void TakeDamage(int damage)
```

```
    {
        health.TakeDamage(damage);
    }

    void Recover(int amount)
    {
        health.Recover(amount);
    }

    int GetHp() const
    {
        return health.GetHp();
    }

    int GetMaxHp() const
    {
        return health.GetMaxHp();
    }

    bool IsDead() const
    {
        return health.IsDead();
    }

private:
    HealthComponent health;
};

class PlayerBase : public Character
{
public:
    PlayerBase(int id, std::string name, int maxHp)
        : Character(id, std::move(name), maxHp)
    {
    }

    virtual void ProcessInput() = 0;

    void AddScore(int value)
    {
        if (value > 0)
        {
            score += value;
        }
    }

    int GetScore() const
    {
        return score;
    }

private:
    int score = 0;
};

class Player : public PlayerBase
{
public:
    Player(int id, std::string name, int maxHp,
           int itemCount = 0)
        : PlayerBase(id, std::move(name), maxHp),
          itemCount(std::max(itemCount, 0))
    {
    }

    void ProcessInput() override
    {
        std::cout << GetName()
                  << " processes input\n";
```

```
    }

    void Update() override
    {
        ProcessInput();
        const float deltaTime = GameSystem::GetDeltaTime();

        std::cout << GetName()
                  << " updates for "
                  << deltaTime << " seconds\n";
    }

    void Render() const override
    {
        std::cout << "Render Player "
                  << GetName()
                  << " hp=" << GetHp()
                  << " score=" << GetScore()
                  << " items=" << itemCount
                  << '\n';
    }

    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<Player>(*this);
    }

    std::string GetTypeName() const override
    {
        return "Player";
    }

    void UseItem()
    {
        if (itemCount > 0)
        {
            --itemCount;
            Recover(10);
        }
    }

private:
    int itemCount;
};
```

8.11.4 MobBase, Monster와 BossMonster

```
class MobBase : public Character
{
public:
    MobBase(int id, std::string name, int maxHp, int attackPower)
        : Character(id, std::move(name), maxHp),
          attackPower(std::max(attackPower, 0))
    {
    }

    virtual void Think() = 0;
    virtual void Attack(Character& target) = 0;

protected:
    int GetAttackPower() const
    {
        return attackPower;
    }

private:
    int attackPower;
};
```

```
class Monster : public MobBase
{
public:
    Monster(int id, std::string name, int maxHp,
            int attackPower, float patrolDistance)
        : MobBase(id, std::move(name), maxHp, attackPower),
          patrolDistance(patrolDistance)
    {
    }

    void Think() override
    {
        std::cout << GetName()
                  << " patrols "
                  << patrolDistance
                  << " units\n";
    }

    void Attack(Character& target) override
    {
        target.TakeDamage(GetAttackPower());
    }

    void Update() override
    {
        Think();
    }

    void Render() const override
    {
        std::cout << "Render Monster "
                  << GetName()
                  << " hp=" << GetHp()
                  << '\n';
    }

    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<Monster>(*this);
    }

    std::string GetTypeName() const override
    {
        return "Monster";
    }

private:
    float patrolDistance;
};

class BossMonster : public MobBase
{
public:
    BossMonster(int id, std::string name, int maxHp, int attackPower)
        : MobBase(id, std::move(name), maxHp, attackPower)
    {
    }

    void Think() override
    {
        if (GetHp() <= GetMaxHp() / 3)
        {
            phase = 3;
        }
        else if (GetHp() <= GetMaxHp() * 2 / 3)
        {
            phase = 2;
        }
```

```
        std::cout << GetName()
                  << " thinks in phase "
                  << phase << '\n';
    }

    void Attack(Character& target) override
    {
        target.TakeDamage(GetAttackPower() * phase);
    }

    void Update() override
    {
        Think();
    }

    void Render() const override
    {
        std::cout << "Render BossMonster "
                  << GetName()
                  << " hp=" << GetHp()
                  << " phase=" << phase
                  << '\n';
    }

    std::unique_ptr<GameObject> Clone() const override
    {
        return std::make_unique<BossMonster>(*this);
    }

    std::string GetTypeName() const override
    {
        return "BossMonster";
    }

    void EnterRageMode()
    {
        phase = 3;
    }

private:
    int phase = 1;
};
```

8.11.5 GameScene의 다형 객체 소유

```
class GameScene
{
public:
    void Add(std::unique_ptr<GameObject> object)
    {
        if (object != nullptr)
        {
            objects.push_back(std::move(object));
        }
    }

    void UpdateAll()
    {
        for (const auto& object : objects)
        {
            object->Update();
        }
    }

    void RenderAll() const
    {
        for (const auto& object : objects)
```

```
        {
            object->Render();
        }
    }

    void CloneObject(std::size_t index)
    {
        if (index < objects.size())
        {
            objects.push_back(objects[index]->Clone());
        }
    }

    GameObject* FindById(int id)
    {
        for (const auto& object : objects)
        {
            if (object->GetId() == id)
            {
                return object.get();
            }
        }

        return nullptr;
    }

private:
    std::vector<std::unique_ptr<GameObject>> objects;
};
```

GameScene은 모든 객체를 고유 소유한다. 컨테이너에는 스마트 포인터가 저장되므로 객체 잘림이 발생하지 않는다. 장면이 소멸하면 각 구체 객체의 가상 소멸자가 호출된다.

8.11.6 실행 흐름과 다형 복제

```
// main.cpp
#include "GameHierarchy.h"

int main()
{
    GameScene scene;

    scene.Add(std::make_unique<Player>(1, "Knight", 150, 2));
    scene.Add(std::make_unique<Monster>(2, "Goblin", 60, 10, 5.0f));
    scene.Add(std::make_unique<BossMonster>(3, "Dragon", 900, 30));

    GameSystem::BeginFrame(0.016f);

    scene.UpdateAll();
    scene.RenderAll();

    scene.CloneObject(2);

    std::cout << "After clone\n";
    scene.RenderAll();

    GameObject* object = scene.FindById(3);

    if (auto* boss = dynamic_cast<BossMonster*>(object))
    {
        boss->EnterRageMode();
    }

    scene.RenderAll();
}
```

실행 결과는 각 객체의 동적 타입에 따라 다른 동작이 선택되고 복제된 객체가 구체 타입을 유지함을 보여 준다.

```
Knight processes input
Knight updates for 0.016 seconds
Goblin patrols 5 units
Dragon thinks in phase 1
Render Player Knight hp=150 score=0 items=2
Render Monster Goblin hp=60
Render BossMonster Dragon hp=900 phase=1
After clone
Render Player Knight hp=150 score=0 items=2
Render Monster Goblin hp=60
Render BossMonster Dragon hp=900 phase=1
Render BossMonster Dragon hp=900 phase=1
Render Player Knight hp=150 score=0 items=2
Render Monster Goblin hp=60
Render BossMonster Dragon hp=900 phase=3
Render BossMonster Dragon hp=900 phase=1
```

복제된 BossMonster는 원본과 독립된 객체이다. 원본만 분노 상태로 바뀌므로 마지막 두 보스의 phase 값이 서로 다르다.

8.12 학습 정리

상속은 공통 코드를 가져오는 기능이 아니라 타입 사이의 일반화 관계와 대체 가능성을 표현한다. 공개 상속을 사용한 파생 객체는 기반 클래스가 약속한 인터페이스와 의미를 유지해야 한다.

파생 클래스 생성자는 직접 기반 클래스 생성자를 초기화 리스트에서 선택한다. 객체는 기반 클래스에서 파생 클래스로 생성되고 파생 클래스에서 기반 클래스로 소멸한다. 실제 초기화 순서는 초기화 리스트의 작성 순서가 아니라 기반 클래스와 데이터 멤버의 선언 구조로 결정된다.

파생 클래스에서 같은 이름의 함수를 선언하면 기반 클래스의 오버로드 집합이 숨겨질 수 있다. using 선언은 기반 함수들을 파생 클래스 범위에 다시 노출한다. 가상 함수를 재정의할 때는 override를 사용하여 시그니처 오류를 컴파일러가 검사하게 한다.

가상 함수는 기반 클래스 포인터나 참조를 통해 실제 객체의 동적 타입에 맞는 함수를 호출한다. 생성자와 소멸자 안의 가상 호출은 현재 생성 또는 소멸 중인 클래스 범위를 기준으로 처리된다. 가상 함수의 기본 인수는 정적 타입에서 선택되므로 재정의마다 다른 기본값을 두지 않는 편이 좋다.

추상 클래스는 공통 상태와 일부 구현을 가지면서 순수 가상 함수로 파생 클래스의 계약을 정의할 수 있다. 작은 인터페이스는 서로 독립적인 역할을 표현한다. 기반 인터페이스에는 모든 파생 클래스에 의미가 있는 연산만 포함해야 한다.

다형 기반 클래스는 가상 소멸자를 제공해야 한다. 파생 객체를 기반 값으로 복사하면 객체 잘림이 발생하므로 다형 객체는 참조나 포인터로 전달하고 스마트 포인터 컨테이너에 저장한다. 가상 Clone()은 동적 타입을 유지하는 복제를 제공한다.
업캐스팅은 파생 객체를 기반 타입으로 안전하게 다룬다. 다운캐스팅은 실제 타입을 전제로 하므로 dynamic_cast로 검사할 수 있지만, 반복되는 타입 검사는 가상 함수나 인터페이스가 부족하다는 신호일 수 있다.
다중 상속은 여러 기반 역할을 결합하지만 이름 충돌과 다이아몬드 상속을 만들 수 있다. 여러 상태 기반을 상속하기보다 하나의 주 구현 계층과 작은 인터페이스들을 결합하는 방식이 관리하기 쉽다.
상속은 is-a 관계와 동적 대체가 필요할 때 사용한다. 무기, 인벤토리, 체력, AI 행동처럼 객체가 가진 기능은 구성과 위임으로 분리하는 편이 유연하다. 상속과 구성을 함께 사용하면 타입 계층은 단순하게 유지하면서 기능을 독립적으로 교체하고 테스트할 수 있다.

8.13 학습 과제

과제 **1.** 게임 객체 계층 구성
아래 조건에 맞는 클래스 계층을 작성한다.

```
GameObject
└─ Character
   ├─ PlayerBase
   │  └─ Player
   └─ MobBase
      ├─ Monster
      └─ BossMonster
```

- GameSystem은 프레임 시간을 제공하고 Update()는 인수 없는 공통 인터페이스로 유지
- GameObject는 Update()와 Render()를 순수 가상 함수로 선언
- Character는 체력과 피해 처리 기능 제공
- PlayerBase는 ProcessInput() 계약 제공
- MobBase는 Think()와 Attack() 계약 제공
- 구체 클래스는 모든 순수 가상 함수를 구현
- 모든 재정의 함수에 override 사용

- 기반 클래스 소멸자는 가상 소멸자로 선언

Mermaid 클래스 다이어그램도 함께 작성한다.

과제 **2.** 생성과 소멸 순서 추적
GameObject, Character, MobBase, BossMonster의 생성자와 소멸자에서 클래스 이름을 출력한다.

```
{
   BossMonster boss;
}
```

출력 결과를 기록하고 생성과 소멸 방향을 설명한다. 각 클래스에 데이터 멤버 객체를 하나씩 추가하여 기반 클래스와 멤버의 초기화 순서도 확인한다.

과제 **3.** 이름 숨김과 **using** 선언
Character에 두 개의 Move() 오버로드를 작성한다.

```
void Move(float x, float y);
void Move(float distance);
```

Player에 Move(int direction)을 추가한 뒤 기반 함수가 숨겨지는 현상을 확인한다. using Character::Move;를 적용하여 세 함수를 모두 호출할 수 있도록 수정한다.

과제 **4.** 가상 함수와 기본 인수
GameObject::Draw(int layer = 0)과 Player::Draw(int layer = 10)을 작성한다.

```
Player player;
GameObject& object = player;
player.Draw();
object.Draw();
```

두 호출에서 실행되는 함수와 전달되는 기본 인수가 어떻게 결정되는지 설명한다. 가상 함수마다 다른 기본 인수를 두지 않는 개선 코드도 작성한다.

과제 **5.** 다형 객체 복제
GameObject에 가상 Clone()을 추가하고 Player, Monster, BossMonster에서 구현한다.

```
virtual std::unique_ptr<GameObject> Clone() const = 0;
```

원본 객체를 복제한 뒤 원본의 상태만 변경하여 복사본이 독립적인지 확인한다. 복제 결과의 동적 타입이 유지되는지도 가상 함수 호출로 확인한다.

과제 **6. RTTI** 사용 코드 개선
구체 타입을 검사하는 코드를 작성한다.

```
void ExecuteAction(GameObject& object);
```

dynamic_cast로 Player, Monster, BossMonster를 구분하여 행동을 호출한 뒤, 공통 가상 함수나 작은 인터페이스를 추가하여 타입 분기를 제거한다. 두 구현의 확장성을 비교한다.

과제 **7.** 다이아몬드 상속
GameObject, Character, NetworkObject, NetworkCharacter로 다이아몬드 상속을 구성한다.

- 일반 상속에서 GameObject 부분이 두 개 생기는 현상 확인
- GetId() 호출의 모호성 확인
- 가상 상속으로 기반 부분을 하나로 통합
- 가장 구체적인 클래스가 가상 기반 클래스를 초기화하는 코드 작성

같은 기능을 NetworkComponent 구성으로 바꾼 설계도 제시한다.

과제 **8.** 상속에서 구성으로 변경
Monster의 행동을 파생 클래스로 구분한다.

```
Monster
├─ PatrolMonster
├─ ChaseMonster
└─ RangedMonster
```

이 구조를 IMobBehavior와 행동 객체 구성으로 변경한다.

- PatrolBehavior
- ChaseBehavior
- RangedAttackBehavior

실행 중 행동 객체를 교체할 수 있도록 구성하고 상속 방식과 클래스 수, 결합도, 테스트 방법을 비교한다.

과제 **9.** 게임 장면 다형 객체 관리

GameScene이 std::vector<std::unique_ptr<GameObject>>를 소유하도록 작성한다.
필수 기능:

- Player, Monster, BossMonster 추가
- 전체 객체 갱신과 렌더링
- 식별자로 객체 검색
- 인덱스로 객체 복제
- 장면 소멸 시 모든 객체 자동 해제
- 기반 클래스 포인터 삭제가 안전하도록 가상 소멸자 적용

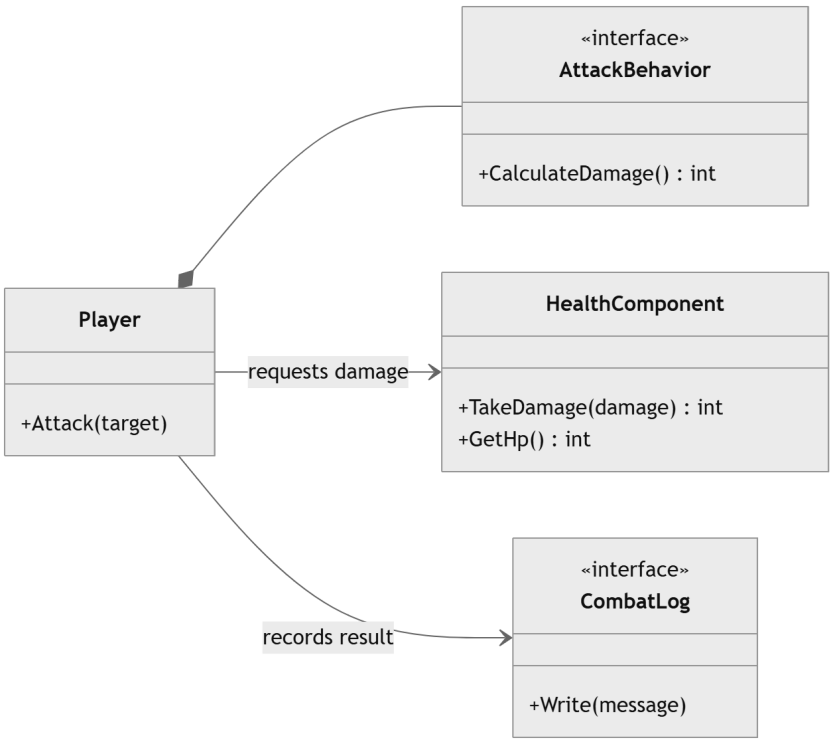
원시 포인터를 소유하지 않으며 객체 잘림이 발생하지 않아야 한다.

09. 객체지향 설계 원칙

이전 장에서 클래스를 정의하고 객체를 생성하며, 상속과 가상 함수를 이용해 여러 객체를 하나의 타입으로 다루는 방법을 학습했다. 그러나 클래스 문법을 사용했다고 해서 객체지향 설계가 자동으로 완성되는 것은 아니다. 하나의 클래스가 지나치게 많은 일을 맡을 수도 있고, 여러 객체가 서로의 내부 구조를 직접 알아야 동작하는 구조가 만들어질 수도 있다. 이런 코드는 처음에는 빠르게 작성할 수 있지만 요구 사항이 바뀔 때 수정 범위가 넓어지고 오류가 발생하기 쉽다.

객체지향 설계는 프로그램을 단순히 여러 클래스로 나누는 작업이 아니다. 프로그램이 수행해야 할 일을 객체의 책임으로 분배하고, 객체가 공개한 인터페이스를 통해 협력하도록 구조를 만드는 과정이다. 각 객체가 자신의 상태를 스스로 관리하고, 다른 객체는 필요한 동작만 요청하도록 구성하면 변경의 영향 범위를 줄일 수 있다.

게임에 플레이어의 공격 기능을 구현한다고 하자. 공격에는 공격력 계산, 치명타 판정, 대상의 체력 감소, 전투 기록 출력, 효과음 재생과 같은 여러 작업이 포함될 수 있다. 이 작업을 모두 Player에 넣으면 Player는 전투 규칙, 난수, 출력, 사운드 시스템까지 알아야 한다. 반대로 공격 계산, 체력 관리, 기록 출력을 서로 다른 객체가 맡고 공개된 인터페이스로 협력하면 각 규칙을 독립적으로 변경할 수 있다.



이 구조에서 Player는 공격을 시작하는 역할을 맡고, 공격 수치 계산은 AttackBehavior, 체력 변경은 HealthComponent, 기록 출력은 CombatLog가 담당한다. 각 객체는 자신의 책임에 필요한 상태와 동작을 함께 가진다.

이 장에서는 책임과 협력, 응집도와 결합도, 인터페이스 설계를 기준으로 객체 구조를 판단한다. 이어서 SOLID로 알려진 다섯 설계 원칙을 살펴보고, 한 클래스에 모여 있던 전투 기능을 여러 객체의 협력 구조로 개선한다. 설계 원칙은 반드시 지켜야 하는 문법 규칙이 아니라 변경 가능성과 복잡도를 판단하기 위한 기준이다. 작은 프로그램에 불필요한 계층과 인터페이스를 먼저 만들기보다 실제로 함께 바뀌는 부분과 독립적으로 바뀌는 부분을 구분하여 필요한 만큼 적용해야 한다.

9.1 책임과 협력

객체지향 프로그램은 여러 객체가 메시지를 주고받으며 작업을 완성하는 구조로 볼 수 있다. 여기서 메시지는 반드시 별도의 메시지 클래스를 뜻하지 않는다. 한 객체가 다른 객체의 공개 멤버 함수를 호출하여 작업을 요청하는 행위가 객체 사이의 메시지 전달에 해당한다.

전투 기능을 예로 들면 공격자는 피해량을 계산하고, 대상은 자신의 체력을 감소시키며, 기록 객체는 결과를 저장하거나 출력한다. 어느 객체가 어떤 작업을 맡을지 결정하는 것이 책임 분배이고, 객체들이 공개 인터페이스로 연결되어 하나의 기능을 완성하는 것이 협력이다.

9.1.1 객체의 책임

책임(responsibility)은 객체가 알고 있어야 하는 정보와 수행해야 하는 동작을 의미한다. 객체가 자신의 이름과 체력을 알고 있다면 상태에 관한 책임을 가진다. 피해를 적용하고 사망 여부를 판단한다면 동작에 관한 책임도 가진다.

체력 상태를 관리하는 객체를 구성해 보자.

```
#include <algorithm>
#include <stdexcept>

class HealthComponent
{
public:
    explicit HealthComponent(int maxHp) : hp(maxHp), maxHp(maxHp)
    {
        if (maxHp <= 0)
        {
```

```
            throw std::invalid_argument("maxHp must be positive");
        }
    }

    int TakeDamage(int damage)
    {
        if (damage <= 0 || IsDead())
        {
            return 0;
        }

        const int oldHp = hp;
        hp = std::max(hp - damage, 0);
        return oldHp - hp;
    }

    int Recover(int amount)
    {
        if (amount <= 0 || IsDead())
        {
            return 0;
        }

        const int oldHp = hp;
        hp = std::min(hp + amount, maxHp);
        return hp - oldHp;
    }

    int GetHp() const
    {
        return hp;
    }

    int GetMaxHp() const
    {
        return maxHp;
    }

    bool IsDead() const
    {
        return hp == 0;
    }

private:
    int hp;
    int maxHp;
};
```

HealthComponent는 현재 체력과 최대 체력을 알고 있으며, 피해와 회복 규칙을 수행한다. 체력을 변경하는 모든 규칙이 한 객체 안에 모여 있으므로 외부 객체가 체력 범위를 직접 보정할 필요가 없다.

객체의 책임은 클래스 이름과 공개 인터페이스를 통해 드러나야 한다. HealthComponent라는 이름을 가진 객체가 파일 저장, 사운드 재생, 입력 처리까지 맡는다면 이름과 책임이 일치하지 않는다. 클래스의 역할을 한 문장으로 설명하기 어렵거나 설명에 여러 개의 접속사가 반복되면 책임이 과도하게 모여 있을 가능성이 크다.

9.1.2 객체의 역할

역할(role)은 협력 안에서 객체가 수행하는 자리이다. 하나의 구체 클래스가 여러 역할을 수행할 수 있고, 같은 역할을 여러 구체 클래스가 수행할 수도 있다. C++에서는 작은 추상 클래스나 개념적인 인터페이스로 역할을 표현할 수 있다.
피해를 받을 수 있는 역할을 정의한다.

```
#include <string_view>

class IDamageable
{
public:
    virtual ~IDamageable() = default;
```

```
    virtual int TakeDamage(int damage) = 0;
    virtual bool IsDead() const = 0;
    virtual std::string_view GetName() const = 0;
};
```

IDamageable은 체력을 어떻게 저장하는지 규정하지 않는다. 피해를 적용할 수 있고 사망 여부와 이름을 확인할 수 있다는 역할만 표현한다.

```
#include <string>
#include <utility>

class Character : public IDamageable
{
public:
    Character(std::string name, int maxHp)
        : name(std::move(name)), health(maxHp)
    {
    }

    int TakeDamage(int damage) override
    {
        return health.TakeDamage(damage);
    }

    bool IsDead() const override
    {
        return health.IsDead();
    }

    std::string_view GetName() const override
    {
        return name;
    }

    int GetHp() const
    {
        return health.GetHp();
    }

private:
    std::string name;
    HealthComponent health;
};
```

Character는 IDamageable 역할을 수행하고 체력 처리는 HealthComponent에 위임한다. 협력하는 코드는 구체적인 Character가 아니라 IDamageable을 통해 피해를 요청할 수 있다.

```
void ApplyExplosionDamage(IDamageable& target, int damage)
{
    target.TakeDamage(damage);
}
```

이 함수는 대상이 Player, Monster, 파괴 가능한 상자 중 어느 클래스인지 알 필요가 없다. IDamageable 역할을 수행하는 객체라면 같은 방식으로 사용할 수 있다.

9.1.3 데이터와 행위의 배치

데이터를 가진 객체가 그 데이터를 해석하고 변경하는 행위도 함께 가지는 편이 일반적으로 자연스럽다. 외부 객체가 데이터를 꺼내 규칙을 적용한 뒤 다시 저장하면 객체의 내부 표현과 규칙이 외부로 퍼진다.

체력을 외부에서 직접 계산하는 구조를 살펴보자.

```
class Character
{
public:
    int GetHp() const
    {
```

```
        return hp;
    }

    void SetHp(int hp)
    {
        this->hp = hp;
    }

private:
    int hp = 100;
};

void ApplyDamage(Character& target, int damage)
{
    int hp = target.GetHp();
    hp -= damage;

    if (hp < 0)
    {
        hp = 0;
    }

    target.SetHp(hp);
}
```

ApplyDamage()는 Character의 체력이 0보다 작아서는 안 된다는 규칙을 알고 있다. 회복 함수, 독 피해 함수, 충돌 피해 함수가 각각 같은 규칙을 구현하면 보정 코드가 여러 곳에 반복된다.

행위를 상태가 있는 객체로 이동하면 규칙을 한곳에서 관리할 수 있다.

```
class Character
{
public:
    explicit Character(int maxHp)
        : health(maxHp)
    {
    }

    int TakeDamage(int damage)
    {
        return health.TakeDamage(damage);
    }

    int GetHp() const
    {
        return health.GetHp();
    }

private:
    HealthComponent health;
};
```

외부 코드는 체력을 계산하지 않고 피해 적용을 요청한다.

```
const int actualDamage = target.TakeDamage(attackPower);
```

객체에게 필요한 작업을 요청하고 내부 상태의 계산 과정을 맡기는 방식을 흔히 “묻지 말고 시켜라”라는 표현으로 설명한다. 이 원칙이 모든 조회 함수를 금지한다는 뜻은 아니다. 화면 표시, 기록, 조건 판단에는 상태 조회가 필요하다. 중요한 점은 객체의 핵심 규칙을 외부 코드가 대신 수행하지 않는 것이다.

9.1.4 책임이 많은 객체의 문제

기능을 빠르게 추가하다 보면 하나의 클래스가 여러 시스템을 직접 다루게 된다.

```
class Player
{
public:
```

```
    void Update();
    void ProcessInput();
    void Move();
    void Attack();
    void TakeDamage(int damage);
    void PlaySound();
    void SaveToFile();
    void LoadFromFile();
    void DrawHud();
    void WriteCombatLog();

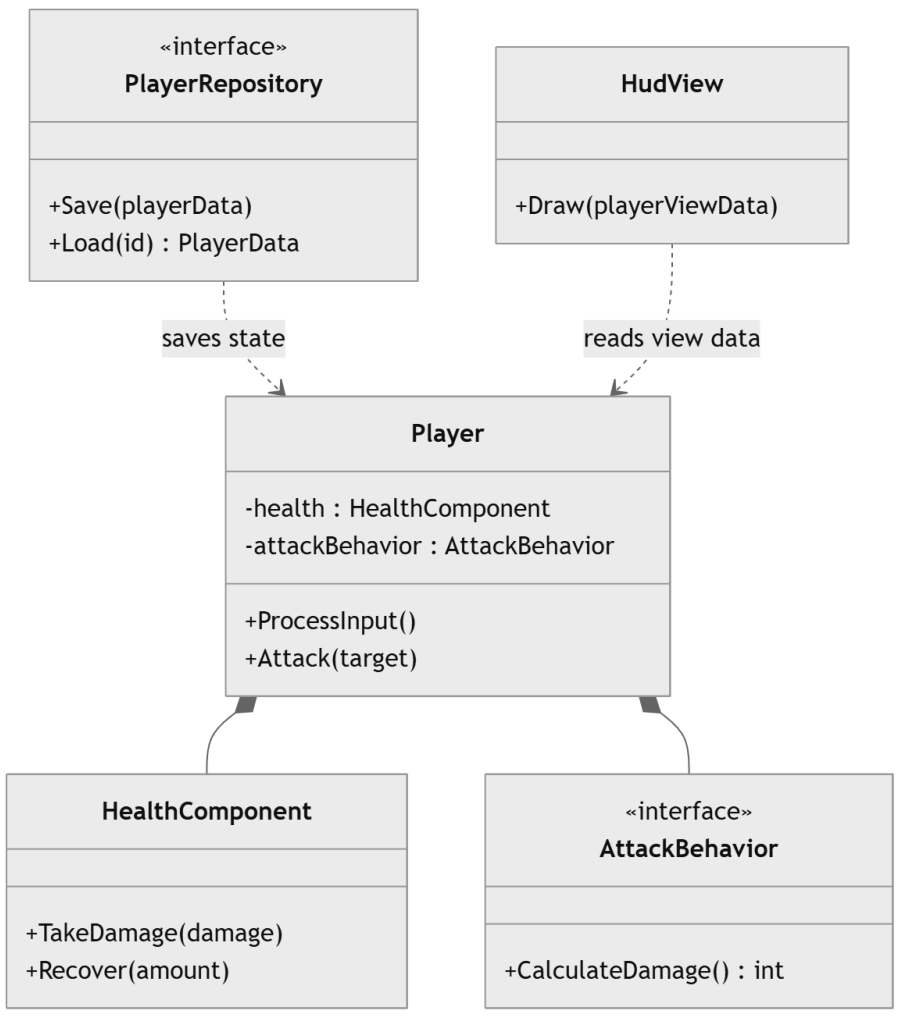
private:
    int hp = 100;
    int attackPower = 20;
    int score = 0;
};
```

이 클래스는 입력, 이동, 전투, 체력, 사운드, 저장, UI, 기록을 모두 담당한다. 입력 장치가 바뀌어도 Player를 수정하고 저장 형식이 바뀌어도 Player를 수정한다. 전투 규칙을 변경하는 작업과 UI를 변경하는 작업이 같은 파일에서 충돌할 수 있다.

책임이 많은 객체에는 여러 문제가 함께 나타난다.

- 클래스의 변경 이유가 많아진다.
- 많은 헤더 파일에 의존하므로 컴파일 영향 범위가 커진다.
- 일부 기능만 시험하려 해도 여러 시스템을 함께 준비해야 한다.
- 서로 관련 없는 데이터와 함수가 한 클래스 안에서 섞인다.
- 한 기능을 다른 객체에서 재사용하기 어렵다.

책임을 분리하면 객체 수와 연결 관계는 늘어난다. 객체 수가 적다는 이유만으로 좋은 설계가 되는 것은 아니며, 클래스가 많다는 이유만으로 좋은 설계가 되는 것도 아니다. 함께 변경되는 상태와 행위는 모으고 독립적으로 변경되는 책임은 분리하는 것이 핵심이다.



9.2 응집도와 결합도

응집도와 결합도는 클래스와 모듈의 구조를 평가하는 대표적인 기준이다. 응집도는 한 객체 안의 요소들이 얼마나 밀접하게 관련되어 있는지를 나타내고, 결합도는 객체들이 서로에게 얼마나 강하게 의존하는지를 나타낸다.

일반적으로 관련된 책임을 한곳에 모아 응집도를 높이고, 다른 객체의 구체적인 구현에 대한 의존을 줄여 결합도를 낮추는 방향이 변경에 유리하다. 그러나 수치가 정해진 절대 규칙은 아니다. 객체를 지나치게 잘게 분리하면 호출 관계와 수명 관리가 복잡해질 수 있으므로 변경 단위와 사용 맥락을 함께 판단해야 한다.

9.2.1 응집도

응집도(cohesion)가 높은 클래스는 데이터와 함수가 하나의 뚜렷한 목적을 중심으로 모여 있다. HealthComponent의 hp, maxHp, TakeDamage(), Recover(), IsDead()는 모두 체력 상태와 규칙에 관련된다.

```
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp);

    int TakeDamage(int damage);
    int Recover(int amount);
    int GetHp() const;
    int GetMaxHp() const;
    bool IsDead() const;

private:
    int hp;
    int maxHp;
};
```

반면 체력 클래스에 경험치 계산이나 파일 저장 기능이 포함되면 응집도가 낮아진다.

```
class HealthComponent
{
public:
    void TakeDamage(int damage);
    void Recover(int amount);

    void AddExperience(int value);  // 체력과 직접 관련되지 않음
    void SaveToFile();              // 저장 형식에 대한 책임
    void PlayHitSound();            // 사운드 시스템에 대한 책임
};
```

응집도를 판단할 때는 함수 이름만 보는 것이 아니라 데이터 사용 관계를 확인한다. 클래스의 함수들이 서로 전혀 다른 데이터 멤버만 사용한다면 사실상 여러 책임이 한 클래스 안에 들어 있을 가능성이 있다.

9.2.2 결합도

결합도(coupling)는 한 객체가 다른 객체의 구체적인 타입, 내부 데이터, 호출 순서와 전역 상태를 얼마나 많이 알아야 하는지를 나타낸다. 전투 기능이 구체적인 출력 장치에 직접 의존하는 코드를 살펴보자.

```
#include <iostream>
#include <string_view>

class CombatService
{
public:
    void Attack(Character& attacker, Character& target)
    {
        const int damage = 20;
        const int actualDamage = target.TakeDamage(damage);

        std::cout << attacker.GetName()
                  << " attacks "
                  << target.GetName()
                  << " for "
                  << actualDamage
                  << " damage\n";
    }
};
```

CombatService는 전투 규칙과 콘솔 출력에 함께 결합되어 있다. 전투 기록을 파일, 게임 화면, 네트워크 패킷으로 바꾸려면 이 클래스를 수정해야 한다.
출력 역할을 인터페이스로 분리할 수 있다.

```
class ICombatLog
{
```

```
public:
    virtual ~ICombatLog() = default;
    virtual void Write(std::string_view message) = 0;
};

#include <sstream>

class CombatService
{
public:
    explicit CombatService(ICombatLog& combatLog)
        : combatLog(combatLog)
    {
    }

    void Attack(Character& attacker, Character& target)
    {
        const int damage = 20;
        const int actualDamage = target.TakeDamage(damage);

        std::ostringstream output;
        output << attacker.GetName()
               << " attacks "
               << target.GetName()
               << " for "
               << actualDamage
               << " damage";

        combatLog.Write(output.str());
    }

private:
    ICombatLog& combatLog;
};
```

이제 CombatService는 구체적인 출력 방식이 아니라 전투 기록 역할에만 의존한다. 콘솔 출력, 파일 기록, 시험용 메모리 기록을 외부에서 선택할 수 있다.
결합도를 낮춘다는 것은 객체 사이의 연결을 모두 없앤다는 뜻이 아니다. 협력하려면 의존 관계가 필요하다. 중요한 점은 변하기 쉬운 구체 구현보다 안정적인 역할과 계약에 의존하도록 만드는 것이다.

9.2.3 높은 응집도와 낮은 결합도

높은 응집도와 낮은 결합도는 서로 독립된 목표가 아니다. 책임을 올바른 객체에 모으면 외부 객체가 내부 데이터를 자세히 알 필요가 줄어들어 결합도도 낮아지는 경우가 많다.

CombatService가 대상의 체력을 직접 읽고 계산하는 구조를 생각해 보자.

```
void CombatService::Attack(Character& target)
{
    int hp = target.GetHp();
    hp -= attackPower;

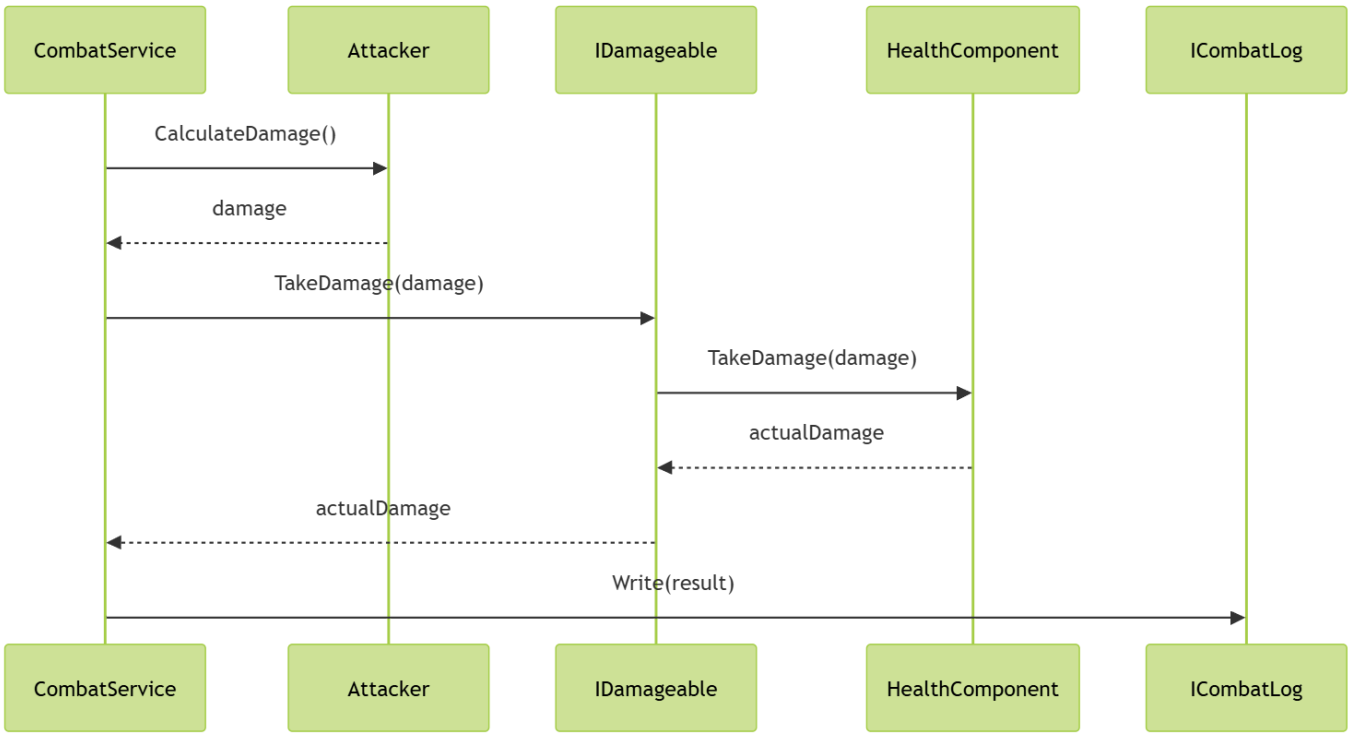
    if (hp < 0)
    {
        hp = 0;
    }

    target.SetHp(hp);
}
```

이 코드는 체력 규칙과 강하게 결합되어 있다. Character가 보호막, 피해 면역, 방어력 규칙을 추가하면 CombatService도 함께 수정해야 한다.

```
void CombatService::Attack(IDamageable& target)
{
    target.TakeDamage(attackPower);
}
```

피해 처리 책임을 대상에게 위임하면 CombatService는 공격 요청에 집중하고 대상은 자신의 상태를 관리한다. CombatService의 응집도는 높아지고 대상의 내부 구조와의 결합도는 낮아진다.



시퀀스 다이어그램은 객체가 어떤 순서로 메시지를 주고받는지 보여 준다. 전투 서비스는 공격 계산과 피해 대상의 내부 구현을 직접 처리하지 않고 각 역할에 작업을 요청한다.

9.2.4 변경의 영향 범위

설계의 품질은 기능이 정상적으로 동작할 때보다 요구 사항이 바뀔 때 더 분명하게 드러난다. 변경 하나가 여러 클래스와 함수에 퍼진다면 지식이나 책임이 여러 곳에 중복되어 있을 가능성이 크다.

공격력 계산 규칙이 여러 곳에 복사되어 있다고 하자.

```
int Player::CalculateDamage() const
{
    return attackPower + level * 2;
}

int Monster::CalculateDamage() const
{
    return attackPower + level * 2;
}

int BossMonster::CalculateDamage() const
{
    return attackPower + level * 2;
}
```

레벨 보정 공식이 바뀌면 세 클래스를 모두 수정해야 한다. 계산 규칙이 하나의 정책 객체에 모여 있으면 변경 지점도 한곳으로 줄어든다.

```
class LevelDamagePolicy
{
public:
    int Calculate(int attackPower, int level) const
    {
        return attackPower + level * 3;
    }
};
```

변경 영향 범위를 줄이는 대표적인 방법은 몇 가지로 정리할 수 있다.

- 핵심 규칙을 한곳에 둔다.
- 변하는 부분과 안정된 부분을 분리한다.
- 다른 객체의 내부 표현을 직접 사용하지 않는다.
- 구체 구현 대신 작은 역할에 의존한다.
- 전역 상태와 숨은 의존성을 줄인다.
- 소유권과 수명을 인터페이스에 드러낸다.

설계를 평가할 때 “클래스가 몇 개인가”보다 “한 요구 사항이 바뀌면 몇 곳을 수정해야 하는가”를 확인하는 편이 유용하다.

9.3 인터페이스 설계

인터페이스는 객체가 외부에 제공하는 사용 방법과 계약(객체간 약속)이다. C++에서는 공개 멤버 함수, 함수의 매개변수와 반환 타입, const, 예외 사양, 소유권을 나타내는 타입이 모두 인터페이스의 일부가 된다.

좋은 인터페이스는 객체가 무엇을 할 수 있는지 분명히 보여 주고, 내부 표현을 변경해도 사용자 코드가 영향을 적게 받도록 한다. 사용할 수 없는 기능이나 호출 순서를 외부가 추측해야 하는 구조는 피해야 한다.

9.3.1 최소 인터페이스

최소 인터페이스는 객체의 역할을 수행하는 데 필요한 기능만 공개한다는 원칙이다. 공개 함수가 많아질수록 외부 코드가 의존할 수 있는 지점도 많아지고 내부 구현을 변경하기 어려워진다.

체력 객체가 내부 값에 대한 자유로운 쓰기 접근을 제공하면 불변식을 보호할 수 없다.

```
class HealthComponent
{
public:
    int& Hp()
    {
        return hp;
    }

    int& MaxHp()
    {
        return maxHp;
    }

private:
    int hp = 100;
    int maxHp = 100;
};
```

외부 코드는 유효하지 않은 상태를 만들 수 있다.

```
HealthComponent health;
health.Hp() = -500;
health.MaxHp() = 0;
```

필요한 연산만 공개하면 상태 변경 경로를 통제할 수 있다.

```
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp);

    int TakeDamage(int damage);
    int Recover(int amount);
    int GetHp() const;
    int GetMaxHp() const;
    bool IsDead() const;

private:
    int hp;
    int maxHp;
};
```

최소 인터페이스는 함수 수를 무조건 줄이는 작업이 아니다. 사용자가 유효한 작업을 명확히 수행할 수 있어야 한다. 호출자가 여러 저수준 함수를 정해진 순서로 조합해야만 한 작업이 완성된다면 그 작업을 하나의 의미 있는 함수로 제공하는 편이 낫다.

```
// 호출 순서를 외부가 기억해야 하는 구조
weapon.ConsumeAmmo();
weapon.StartCooldown();
weapon.PlayAnimation();

// 의미 있는 작업으로 묶은 구조
weapon.Fire(target);
```

9.3.2 정보 은닉과 객체 불변식

정보 은닉(information hiding)은 데이터 멤버를 private으로 만드는 것만을 뜻하지 않는다. 바뀔 수 있는 구현 세부 사항과 규칙을 객체 내부에 감추고 외부에는 안정적인 사용 방법을 제공하는 것이다.

객체 불변식(invariant)은 객체가 유효한 동안 항상 만족해야 하는 조건이다. 체력 객체에는 이런 조건이 있다.

```
0 <= hp <= maxHp
maxHp > 0
```

생성자는 유효한 상태로 객체를 만들고, 모든 공개 멤버 함수는 연산이 끝난 뒤 불변식을 유지해야 한다.

```
#include <algorithm>
#include <stdexcept>

class HealthComponent
{
public:
    explicit HealthComponent(int maxHp)
        : hp(maxHp), maxHp(maxHp)
    {
        if (maxHp <= 0)
        {
            throw std::invalid_argument("maxHp must be
positive");
        }
    }

    int TakeDamage(int damage)
    {
        if (damage <= 0)
        {
            return 0;
        }

        const int oldHp = hp;
        hp = std::clamp(hp - damage, 0, maxHp);
        return oldHp - hp;
    }

    int Recover(int amount)
    {
        if (amount <= 0)
        {
            return 0;
        }

        const int oldHp = hp;
        hp = std::clamp(hp + amount, 0, maxHp);
        return hp - oldHp;
    }

    int GetHp() const
    {
        return hp;
    }

private:
    int hp;
    int maxHp;
};
```

외부 코드가 hp의 저장 타입이나 보정 방식에 의존하지 않으므로 내부 구현을 바꾸기 쉽다. 예를 들어 체력을 정수 대신 고정소수점 타입으로 바꾸거나 보호막을 먼저 소모하도록 변경해도 공개 계약을 유지할 수 있다.

9.3.3 명령과 질의

명령(command)은 객체의 상태를 변경하는 작업이고, 질의(query)는 상태를 관찰하여 값을 반환하는 작업이다. 명령과 질의를 구분하면 함수 호출의 부작용을 예측하기 쉬워진다.

```
class Inventory
{
public:
    bool AddItem(int itemId);      // 명령이며 성공 여부를 반환
    bool RemoveItem(int itemId);   // 명령이며 성공 여부를 반환

    bool Contains(int itemId) const; // 질의
    std::size_t GetItemCount() const; // 질의
};
```

C++에서 명령이 성공 여부나 실제 변경량을 반환하는 것은 자연스럽다. 명령과 질의 분리는 “상태를 변경하는 함수는 값을 반환하면 안 된다”는 절대 규칙이 아니다. 호출자가 함수 이름만 보고 상태 변경 여부를 예측할 수 있고, 조회 함수가 숨은 부작용을 만들지 않도록 설계하는 것이 목적이다.

조회 함수가 호출할 때마다 상태를 변경하면 사용자가 의도하지 않은 결과를 얻을 수 있다.

```
int GetScore()
{
    return ++score; // 조회처럼 보이지만 상태를 변경함
}
```

상태 변경은 명령으로 분리한다.

```
void AddScore(int value)
{
    if (value > 0)
    {
        score += value;
    }
}

int GetScore() const
{
    return score;
}
```

9.3.4 오류와 실패의 표현

인터페이스는 정상 결과뿐 아니라 실패를 어떻게 표현하는지도 알려야 한다. 실패 방식은 상황에 따라 달라진다.

- 값이 없을 수 있는 정상적인 상황은 std::optional로 표현할 수 있다.
- 여러 실패 이유를 구분해야 하면 결과 열거형이나 결과 객체를 사용할 수 있다.
- 객체를 유효하게 만들 수 없는 생성 오류는 예외로 표현할 수 있다.
- 호출자가 반드시 지켜야 하는 전제 조건은 문서화하고 필요하면 검증한다.

인벤토리에서 아이템 검색 실패는 정상적인 상황일 수 있다.

```
#include <optional>
#include <string>
#include <vector>

struct Item
{
    int id;
    std::string name;
};

class Inventory
{
public:
    std::optional<Item> FindById(int id) const
    {
        for (const Item& item : items)
```

```
        {
            if (item.id == id)
            {
                return item;
            }
        }

        return std::nullopt;
    }

private:
    std::vector<Item> items;
};
```

장착 실패 원인을 구분해야 하면 열거형을 사용할 수 있다.

```
enum class EquipResult
{
    Success,
    ItemNotFound,
    LevelTooLow,
    SlotUnavailable
};

EquipResult EquipItem(int itemId);
```

호출자는 각 결과를 명시적으로 처리할 수 있다.

```
const EquipResult result = inventory.EquipItem(1001);

switch (result)
{
case EquipResult::Success:
    break;

case EquipResult::ItemNotFound:
    // 아이템이 없음
    break;

case EquipResult::LevelTooLow:
    // 레벨 부족
    break;

case EquipResult::SlotUnavailable:
    // 장착 슬롯 부족
    break;
}
```

오류 표현을 선택할 때는 실패가 예외적인지, 호출자가 복구할 수 있는지, 실패 정보를 얼마나 제공해야 하는지를 고려한다. 같은 계층의 함수들이 서로 다른 방식으로 실패를 표현하면 사용하기 어려우므로 인터페이스 전체의 일관성도 중요하다.

9.3.5 소유권과 수명의 표현

C++ 인터페이스에서는 매개변수와 반환 타입으로 객체의 소유권과 수명 관계를 표현할 수 있다.

```
void Render(const GameObject& object);
void SetTarget(GameObject* target);
std::unique_ptr<GameObject> CreateObject();
void AddObject(std::unique_ptr<GameObject> object);
std::shared_ptr<Resource> LoadSharedResource();
```

각 선언은 서로 다른 의미를 가진다.

- const GameObject&는 호출 중 객체를 읽기만 하며 소유권을 넘겨받지 않는다.
- GameObject*는 nullptr이 가능하고 일반적으로 소유하지 않는 참조를 나타낸다.
- std::unique_ptr<GameObject> 반환은 새 객체의 고유 소유권을 호출자에게 전달한다.
- std::unique_ptr<GameObject> 매개변수는 소유권을 함수나 객체에 이전한다.
- std::shared_ptr<Resource>는 여러 소유자가 수명을 공유한다는 의미를 가진다.

소유하지 않는 포인터를 데이터 멤버로 저장할 때는 대상의 수명이 더 길다는 조건을 명확히 해야 한다.

```
class TargetTracker
{
public:
    void SetTarget(GameObject* target)
    {
        this->target = target;
    }

    GameObject* GetTarget() const
    {
        return target;
    }

private:
    GameObject* target = nullptr; // 비소유 포인터
};
```

수명 조건을 보장하기 어렵다면 식별자를 저장하고 관리 객체를 통해 다시 찾거나, 공유 소유권이 실제로 필요한 경우에만 shared_ptr와 weak_ptr를 고려한다. 소유권을 숨기지 않고 인터페이스에 드러내면 객체를 언제 파괴해야 하는지 추측하는 코드를 줄일 수 있다.

9.4 단일 책임 원칙 (SRP)

단일 책임 원칙(Single Responsibility Principle)은 클래스가 하나의 일만 해야 한다는 단순한 의미로 오해하기 쉽다. 여기서 책임은 함수 하나나 기능 하나가 아니라 하나의 변경 이유를 뜻한다. 서로 다른 이유로 변경되는 기능이 한 클래스에 들어 있으면 변경의 영향 범위가 넓어진다.

게임의 Player는 이동, 전투, 체력, 인벤토리처럼 플레이어 상태와 직접 관련된 기능을 가질 수 있다. 그러나 저장 파일 형식, 데이터베이스 연결, 화면 배치, 사운드 출력 규칙까지 직접 맡으면 서로 다른 변경 이유가 한 클래스에 모인다.

9.4.1 하나의 변경 이유

클래스가 누구의 요구에 의해 바뀌는지 생각하면 변경 이유를 구분하기 쉽다.

- 전투 기획이 바뀌면 공격 계산 규칙이 수정된다.
- UI 기획이 바뀌면 체력 표시 방식이 수정된다.
- 서버 규격이 바뀌면 저장 형식과 통신 코드가 수정된다.
- 사운드 리소스 정책이 바뀌면 효과음 재생 코드가 수정된다.

이 기능들이 모두 Player에 들어 있으면 여러 팀과 시스템의 변경이 한 클래스에 집중된다.

```
class Player
{
public:
    void Attack();
    void TakeDamage(int damage);
    void DrawStatusPanel();
    void SaveAsJson();
    void SendToServer();
    void PlayAttackSound();
};
```

Player라는 객체가 게임 규칙과 상태를 담당하는 것은 자연스럽지만, 화면 표현과 저장 매체의 세부 형식까지 알아야 할 이유는 없다.
단일 책임 원칙을 적용하면 변경 이유별로 책임을 분리한다.

```
class Player
{
public:
    void Attack(IDamageable& target);
    int TakeDamage(int damage);
    int GetHp() const;
};

class PlayerStatusView
{
public:
```

```
    void Draw(const Player& player) const;
};

class IPlayerRepository
{
public:
    virtual ~IPlayerRepository() = default;
    virtual void Save(const Player& player) = 0;
};

class IAudioService
{
public:
    virtual ~IAudioService() = default;
    virtual void Play(std::string_view soundId) = 0;
};
```

각 클래스와 인터페이스는 서로 다른 변경 이유를 가진다. UI가 변경되어도 Player의 전투 규칙은 영향을 받지 않고 저장 매체가 바뀌어도 화면 코드는 수정할 필요가 없다.

9.4.2 책임 분리

한 클래스에 섞인 책임을 분리할 때는 데이터와 행위의 관계를 먼저 확인한다. 어떤 함수들이 같은 데이터 멤버를 사용하고 같은 이유로 변경되는지 살펴보면 자연스러운 경계를 찾을 수 있다.
책임이 집중된 클래스를 구성해 보자.

```
#include <fstream>
#include <iostream>
#include <string>

class Player
{
public:
    Player(std::string name, int maxHp)
        : name(std::move(name)), hp(maxHp), maxHp(maxHp)
    {
    }

    void TakeDamage(int damage)
    {
        hp -= damage;

        if (hp < 0)
        {
            hp = 0;
        }

        std::cout << name << " hp=" << hp << '\n';
    }

    void Save(const std::string& fileName) const
    {
        std::ofstream output(fileName);
        output << name << ',' << hp << ',' << maxHp;
    }

private:
    std::string name;
    int hp;
    int maxHp;
};
```

이 클래스에는 세 책임이 섞여 있다.

- 플레이어의 이름과 상태 관리
- 체력 규칙 처리
- 텍스트 파일 저장
- 콘솔 출력

체력은 HealthComponent, 저장은 저장소 객체, 출력은 화면이나 기록 객체로 분리할 수 있다.

```
#include <fstream>
#include <stdexcept>
#include <string>
#include <string_view>
#include <utility>

struct PlayerData
{
    std::string name;
    int hp;
    int maxHp;
};

class Player
{
public:
    Player(std::string name, int maxHp)
        : name(std::move(name)), health(maxHp)
    {
    }

    int TakeDamage(int damage)
    {
        return health.TakeDamage(damage);
    }

    std::string_view GetName() const
    {
        return name;
    }

    int GetHp() const
    {
        return health.GetHp();
    }

    int GetMaxHp() const
    {
        return health.GetMaxHp();
    }

    PlayerData ExportData() const
    {
        return PlayerData
        {
            name,
            health.GetHp(),
            health.GetMaxHp()
        };
    }

private:
    std::string name;
    HealthComponent health;
};

class IPlayerRepository
{
public:
    virtual ~IPlayerRepository() = default;
    virtual void Save(const PlayerData& data) = 0;
};

class TextPlayerRepository : public IPlayerRepository
{
public:
    explicit TextPlayerRepository(std::string fileName)
```

```
        : fileName(std::move(fileName))
    {
    }

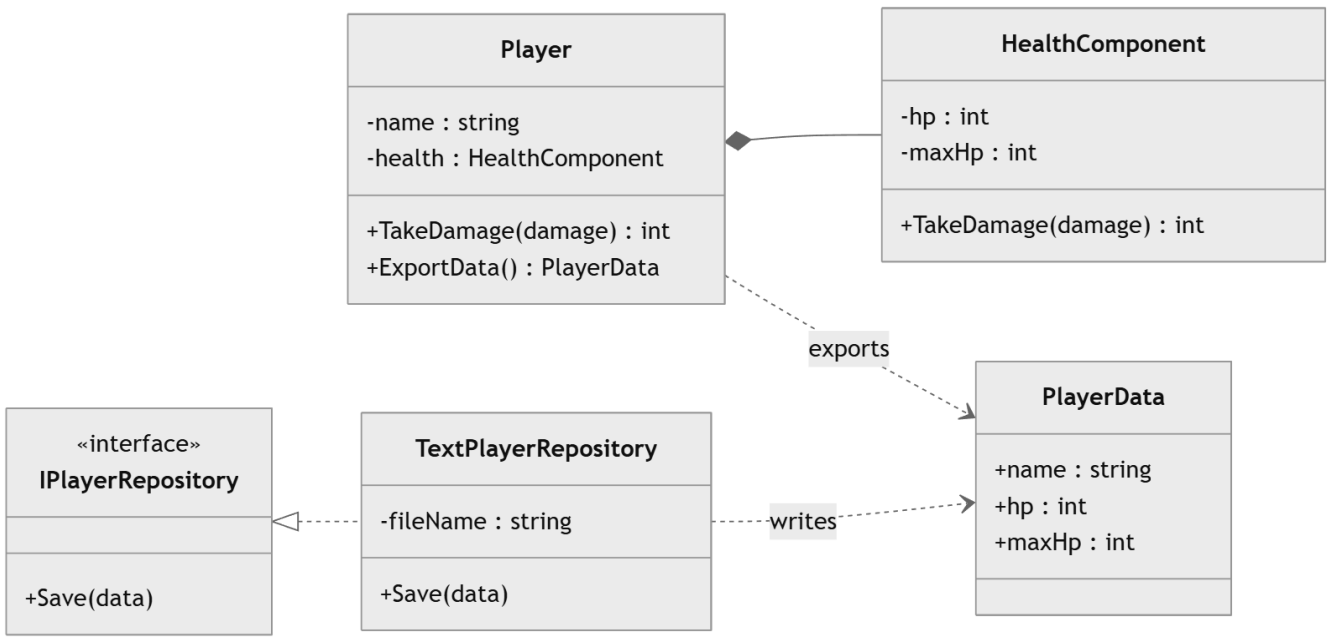
    void Save(const PlayerData& data) override
    {
        std::ofstream output(fileName);

        if (!output)
        {
            throw std::runtime_error("cannot open player
file");
        }

        output << data.name << ','
               << data.hp << ','
               << data.maxHp;
    }

private:
    std::string fileName;
};
```

Player는 자신의 상태를 관리하고 저장에 필요한 값 객체를 제공한다. TextPlayerRepository는 파일 형식과 입출력만 담당한다. 이후 JSON, 데이터베이스, 네트워크 저장 방식이 추가되어도 Player의 내부 규칙을 변경할 필요가 없다.



9.4.3 단일 책임의 범위

단일 책임 원칙을 지나치게 적용하면 한 줄짜리 함수마다 클래스를 만드는 구조가 될 수 있다. 책임의 범위는 프로그램의 크기, 변경 빈도, 재사용과 시험 요구에 따라 달라진다.

HealthComponent의 피해, 회복, 사망 판정은 모두 체력 규칙이라는 하나의 책임에 속한다. 이를 DamageCalculator, RecoveryCalculator, DeathChecker로 무조건 분리하면 객체 수와 연결 비용만 늘어날 수 있다. 반면 게임마다 피해 공식이 자주 교체되고 별도의 시험이 필요하다면 피해 계산 정책을 분리할 가치가 있다.

분리를 고려할 수 있는 징후에는 이런 것들이 있다.

- 클래스가 서로 관련 없는 여러 헤더에 의존한다.
- 클래스 이름을 설명할 때 “그리고”가 반복된다.
- 한 변경이 다른 기능의 시험을 깨뜨린다.
- 일부 멤버 함수만 특정 데이터 멤버를 사용한다.
- 클래스의 일부 기능을 다른 곳에서도 독립적으로 사용해야 한다.
- 서로 다른 개발자가 같은 클래스를 자주 수정한다.

반대로 항상 함께 변경되고 독립적으로 사용되지 않는 작은 기능까지 분리할 필요는 없다. 단일 책임 원칙은 가장 작은 클래스를 만드는 원칙이 아니라 변경 이유가 섞이지 않도록 경계를 정하는 원칙이다.

9.5 개방-폐쇄 원칙 (OCP)

개방-폐쇄 원칙(Open-Closed Principle)은 소프트웨어 요소가 확장에는 열려 있고 기존 코드의 변경에는 닫혀 있어야 한다는 원칙이다. 새로운 기능을 추가할 때 안정적으로 동작하던 코드를 계속 수정하기보다 새로운 구현을 추가하여 확장할 수 있는 구조를 지향한다.

기존 코드를 절대로 수정해서는 안 된다는 뜻은 아니다. 요구 사항이 바뀌면 코드는 수정된다. 핵심은 변화가 반복되는 지점을 찾아 안정적인 인터페이스와 교체 가능한 구현으로 분리하는 것이다.

9.5.1 확장에는 열려 있고 변경에는 닫혀 있는 구조

공격 방식이 근접 공격과 원거리 공격으로 나뉜다고 하자. 처음에는 열거형과 조건문으로 구현할 수 있다.

```
enum class AttackType
{
    Melee,
    Ranged
};

class Player
{
public:
    int CalculateDamage() const
    {
        switch (attackType)
        {
        case AttackType::Melee:
            return strength * 2;

        case AttackType::Ranged:
            return dexterity + arrowPower;
        }

        return 0;
    }

private:
    AttackType attackType = AttackType::Melee;
    int strength = 10;
    int dexterity = 8;
    int arrowPower = 5;
};
```

마법 공격, 범위 공격, 독 공격이 추가될 때마다 열거형과 switch문을 수정해야 한다. 같은 분기가 애니메이션, 사운드, UI 설명에도 존재한다면 공격 방식 하나를 추가하기 위해 여러 파일을 수정하게 된다.

변화하는 공격 계산을 역할로 분리할 수 있다.

```
class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;
    virtual int CalculateDamage() const = 0;
    virtual std::string_view GetName() const = 0;
};
```

새로운 공격은 인터페이스를 구현하는 클래스로 추가한다. Player는 구체 공격 종류를 분기하지 않는다.

9.5.2 조건문 증가의 문제

조건문 자체가 잘못된 것은 아니다. 입력 값에 따라 단순한 처리를 선택하는 코드는 조건문이 가장 명확할 수 있다. 문제는 같은 종류 분기가 프로그램 여러 곳에 반복되고, 새로운 종류가 추가될 때마다 모든 분기를 찾아 수정해야 하는 구조이다.

```
void Player::Attack()
{
    switch (attackType)
    {
    case AttackType::Melee:
        // 근접 피해 계산
        break;

    case AttackType::Ranged:
```

```
        // 원거리 피해 계산
        break;
    }
}

void Player::PlayAttackSound()
{
    switch (attackType)
    {
    case AttackType::Melee:
        // 검 소리
        break;

    case AttackType::Ranged:
        // 활 소리
        break;
    }
}

void Player::DrawAttackIcon()
{
    switch (attackType)
    {
    case AttackType::Melee:
        // 검 아이콘
        break;

    case AttackType::Ranged:
        // 활 아이콘
        break;
    }
}
```

공격 방식의 지식이 여러 함수에 흩어져 있다. 새 공격을 추가할 때 분기 하나를 빠뜨리면 일부 기능만 잘못 동작한다.
행동 객체에 관련 정보를 모으면 새 공격 방식의 변경 지점을 줄일 수 있다.

```
class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;

    virtual int CalculateDamage() const = 0;
    virtual std::string_view GetSoundId() const = 0;
    virtual std::string_view GetIconId() const = 0;
};
```

그러나 인터페이스가 공격 계산, 사운드, UI까지 모두 포함하면 서로 다른 책임이 다시 묶일 수 있다. 공격 종류별로 이 정보가 항상 함께 변하는지 확인해야 한다. 전투 규칙만 독립적으로 교체된다면 공격 계산 인터페이스만 두고 사운드와 UI는 별도의 데이터나 시스템에서 관리하는 편이 낫다.

9.5.3 추상화와 확장 지점

추상화는 모든 클래스 앞에 인터페이스를 만드는 작업이 아니다. 여러 구현이 필요하거나 변화 가능성이 높은 지점에 공통 계약을 정의하는 것이다.
근접 공격과 원거리 공격을 구현한다.

```
#include <algorithm>
#include <memory>
#include <string_view>
#include <utility>

class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;

    virtual int CalculateDamage() const = 0;
    virtual std::string_view GetName() const = 0;
```

```
};

class MeleeAttack : public IAttackBehavior
{
public:
    explicit MeleeAttack(int strength)
        : strength(std::max(strength, 0))
    {
    }

    int CalculateDamage() const override
    {
        return strength * 2;
    }

    std::string_view GetName() const override
    {
        return "Melee";
    }

private:
    int strength;
};
```

```
class RangedAttack : public IAttackBehavior
{
public:
    RangedAttack(int dexterity, int arrowPower)
        : dexterity(std::max(dexterity, 0)),
          arrowPower(std::max(arrowPower, 0))
    {
    }

    int CalculateDamage() const override
    {
        return dexterity + arrowPower;
    }

    std::string_view GetName() const override
    {
        return "Ranged";
    }

private:
    int dexterity;
    int arrowPower;
};
```

Player는 공격 행동을 고유 소유한다.

```
class Player
{
public:
    explicit Player(std::unique_ptr<IAttackBehavior> attackBehavior)
        : attackBehavior(std::move(attackBehavior))
    {
        if (this->attackBehavior == nullptr)
        {
            throw std::invalid_argument("attackBehavior is null");
        }
    }

    int CalculateDamage() const
    {
        return attackBehavior->CalculateDamage();
    }

    void ChangeAttack(
        std::unique_ptr<IAttackBehavior> newBehavior)
```

```
    {
        if (newBehavior == nullptr)
        {
            throw std::invalid_argument("newBehavior is null");
        }

        attackBehavior = std::move(newBehavior);
    }

private:
    std::unique_ptr<IAttackBehavior> attackBehavior;
};
```

새로운 마법 공격은 기존 Player, MeleeAttack, RangedAttack을 수정하지 않고 추가할 수 있다.

```
class MagicAttack : public IAttackBehavior
{
public:
    MagicAttack(int intelligence, int spellPower)
        : intelligence(std::max(intelligence, 0)),
          spellPower(std::max(spellPower, 0))
    {
    }

    int CalculateDamage() const override
    {
        return intelligence * 2 + spellPower;
    }

    std::string_view GetName() const override
    {
        return "Magic";
    }

private:
    int intelligence;
    int spellPower;
};

Player player(std::make_unique<MeleeAttack>(12));
std::cout << player.CalculateDamage() << '\n';

player.ChangeAttack(
    std::make_unique<MagicAttack>(15, 8));

std::cout << player.CalculateDamage() << '\n';
```

확장 지점은 요구 사항의 변화 방향과 일치해야 한다. 공격 계산만 바뀌는데 렌더링 계층 전체를 추상화할 필요는 없다. 실제로 변하는 축을 찾아 그 지점만 분리해야 한다.

9.5.4 상속과 구성을 통한 확장

상속은 기반 타입의 계약을 유지하면서 새로운 종류를 추가할 때 적합하다. 구성은 객체가 사용하는 기능이나 정책을 교체할 때 적합하다. 공격 방식마다 플레이어 클래스를 상속하면 클래스 수가 빠르게 늘어난다.

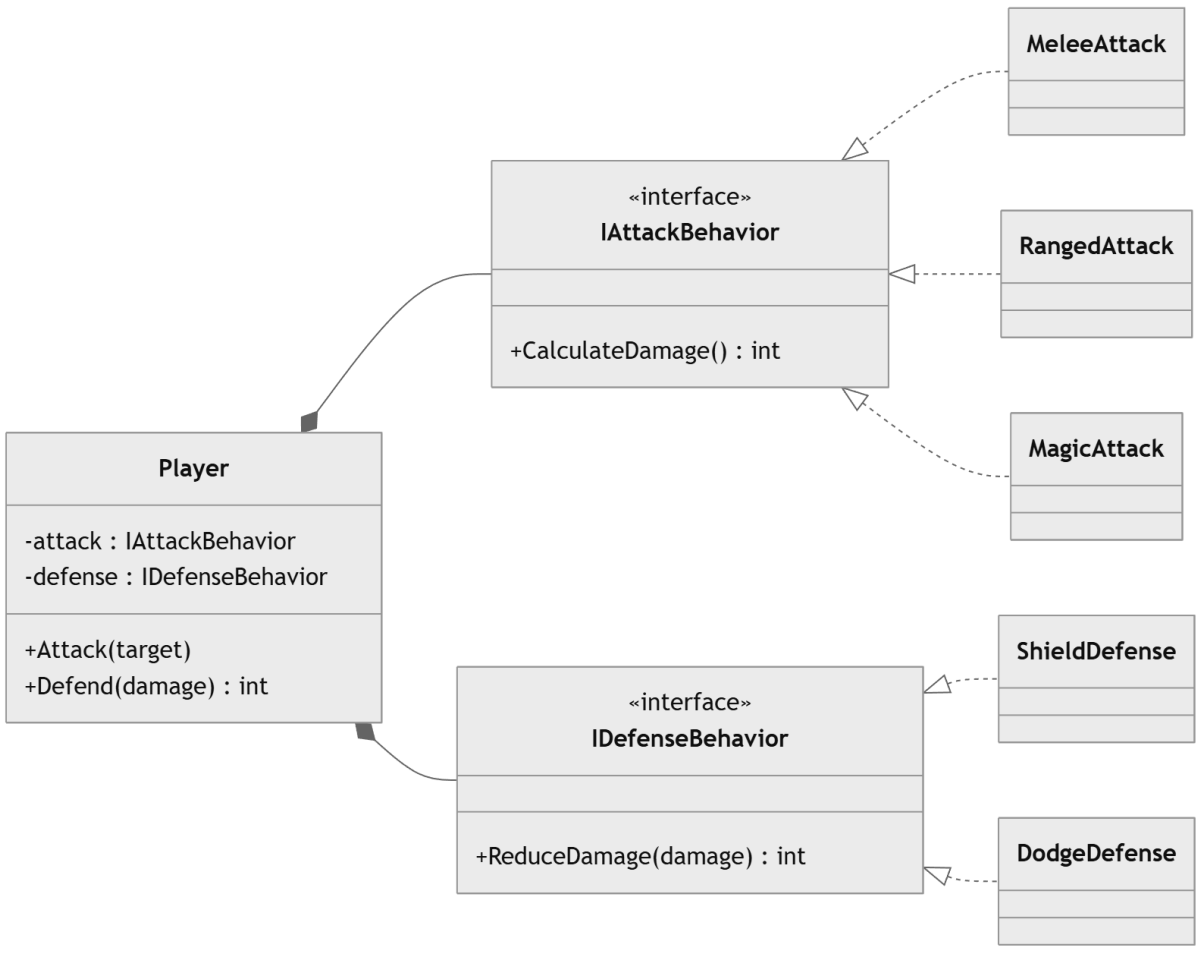
```
Player
├─ MeleePlayer
├─ RangedPlayer
├─ MagicPlayer
└─ PoisonPlayer
```

방어 방식까지 상속으로 표현하면 조합 수만큼 클래스가 필요하다.

```
MeleeShieldPlayer
MeleeDodgePlayer
RangedShieldPlayer
RangedDodgePlayer
```

```
MagicShieldPlayer
MagicDodgePlayer
```

공격과 방어를 구성으로 분리하면 독립적으로 조합할 수 있다.



Player가 캐릭터 계층의 한 종류라는 관계는 상속으로 표현할 수 있다. 공격과 방어처럼 플레이어가 사용하는 교체 가능한 기능은 구성으로 표현하는 편이 유연하다. 개방-폐쇄 원칙은 상속을 많이 사용하라는 원칙이 아니라 변화하는 동작을 안정적인 계약 뒤로 분리하라는 원칙이다.

9.6 리스코프 치환 원칙 (LSP)

리스코프 치환 원칙(Liskov Substitution Principle)은 기반 타입을 사용하는 코드가 파생 타입으로 교체되어도 올바르게 동작해야 한다는 원칙이다. 공개 상속은 문법적으로 멤버를 물려받는 것뿐 아니라 기반 타입의 의미와 계약을 이어받는 관계이다.

컴파일이 된다는 사실만으로 치환 가능성이 보장되지는 않는다. 파생 클래스가 기반 클래스의 함수 형식을 그대로 재정의해도 동작의 의미를 바꾸거나 호출 조건을 더 까다롭게 만들면 사용자 코드가 예상하지 못한 실패를 겪을 수 있다.

9.6.1 기반 타입과 파생 타입의 치환

이동할 수 있는 객체의 역할을 정의한다.

```
class IMovable
{
public:
    virtual ~IMovable() = default;

    virtual void MoveTo(float x, float y) = 0;
    virtual float GetX() const = 0;
    virtual float GetY() const = 0;
};
```

이 인터페이스를 사용하는 함수는 MoveTo() 호출 후 객체의 위치가 요청한 좌표로 바뀐다고 기대한다.

```
void MoveToOrigin(IMovable& object)
{
    object.MoveTo(0.0f, 0.0f);

    if (object.GetX() != 0.0f ||
        object.GetY() != 0.0f)
    {
        throw std::runtime_error("object did not move");
```

```
    }
}
```

Player와 Monster가 이 계약을 지키면 어느 객체를 전달해도 함수가 정상 동작한다.

```
class Player : public IMovable
{
public:
    void MoveTo(float x, float y) override
    {
        this->x = x;
        this->y = y;
    }

    float GetX() const override
    {
        return x;
    }

    float GetY() const override
    {
        return y;
    }

private:
    float x = 0.0f, y = 0.0f;
};
```

기반 타입을 사용하는 코드가 파생 타입의 세부 종류를 확인하거나 예외 처리 규칙을 추가해야 한다면 상속 계약이 올바른지 검토해야 한다.

9.6.2 계약 유지

계약에는 함수의 이름과 매개변수뿐 아니라 의미도 포함된다.

- 함수가 수행하는 작업
- 호출 전에 만족해야 하는 조건
- 호출 후 보장되는 결과
- 객체가 항상 유지해야 하는 불변식
- 실패를 표현하는 방식

IMovable::MoveTo()가 유효한 좌표를 받으면 위치를 변경한다고 약속했다면 파생 클래스도 그 의미를 유지해야 한다. 벽을 IMovable에서 파생시킨 뒤 MoveTo()를 아무 일도 하지 않는 함수로 만들면 문법은 맞지만 계약은 깨진다.

```
class Wall : public IMovable
{
public:
    void MoveTo(float, float) override
    {
        // 벽은 움직이지 않음
    }

    float GetX() const override
    {
        return x;
    }

    float GetY() const override
    {
        return y;
    }

private:
    float x = 10.0f, y = 20.0f;
};
```

MoveToOrigin()에 Wall을 전달하면 기반 계약이 깨진다. 벽이 게임 객체라는 이유만으로 이동 역할까지 상속할 필요는 없다. 공통 상태는 GameObject에 두고 실제로 움직일 수 있는 객체만 IMovable 역할을 구현하는 구조가 적합하다.

9.6.3 사전 조건과 사후 조건

사전 조건(precondition)은 함수를 호출하기 전에 호출자가 만족해야 하는 조건이다. 사후 조건(postcondition)은 함수가 정상적으로 끝난 뒤 제공하는 보장이다.
기반 클래스가 모든 0 이상의 피해 값을 허용한다고 가정한다.

```
class IDamageable
{
public:
    virtual ~IDamageable() = default;

    // damage >= 0이면 호출 가능
    // 반환값은 실제로 감소한 체력
    virtual int TakeDamage(int damage) = 0;
};
```

파생 클래스가 100 이하의 피해만 허용하도록 조건을 강화하면 기반 타입 사용자는 파생 클래스의 추가 조건을 알 수 없다.

```
class FragileObject : public IDamageable
{
public:
    int TakeDamage(int damage) override
    {
        if (damage > 100)
        {
            throw std::invalid_argument(
                "damage must not exceed 100");
        }

        // 피해 처리
        return damage;
    }
};
```

기반 계약상 유효한 damage == 200 호출이 파생 객체에서 실패하므로 사전 조건이 강화되었다.

사후 조건도 약화해서는 안 된다. 기반 함수가 성공 시 요청한 위치로 이동한다고 보장하는데 파생 함수가 절반만 이동하거나 아무 작업도 하지 않으면 사용자는 결과를 신뢰할 수 없다.

리스코프 치환 원칙을 계약 관점에서 정리하면 이런 기준을 사용할 수 있다.

- 파생 타입은 기반 타입보다 더 강한 사전 조건을 요구하지 않는다.
- 파생 타입은 기반 타입이 보장한 사후 조건을 약화하지 않는다.
- 기반 타입의 불변식을 유지한다.
- 기반 타입에서 허용한 작업을 임의로 금지하지 않는다.
- 사용자에게 새로운 예외 처리나 타입 검사를 강요하지 않는다.

9.6.4 잘못된 상속 관계

상속을 코드 재사용 수단으로만 사용하면 치환할 수 없는 계층이 만들어질 수 있다.

```
class MovableObject
{
public:
    virtual ~MovableObject() = default;
    virtual void MoveTo(float x, float y) = 0;
};

class StaticDecoration : public MovableObject
{
public:
    void MoveTo(float, float) override
    {
        throw std::logic_error("static decoration cannot
move");
    }
};
```

StaticDecoration은 일부 위치 관련 코드를 재사용하려고 MovableObject를 상속했지만 이동 계약을 지킬 수 없다. 공통 위치 데이터와 이동 역할을 분리하면 문제를 해결할 수 있다.

```
class GameObject
{
public:
    float GetX() const
    {
        return x;
    }

    float GetY() const
    {
        return y;
    }

protected:
    void SetPosition(float x, float y)
    {
        this->x = x;
        this->y = y;
    }

private:
    float x = 0.0f, y = 0.0f;
};
```

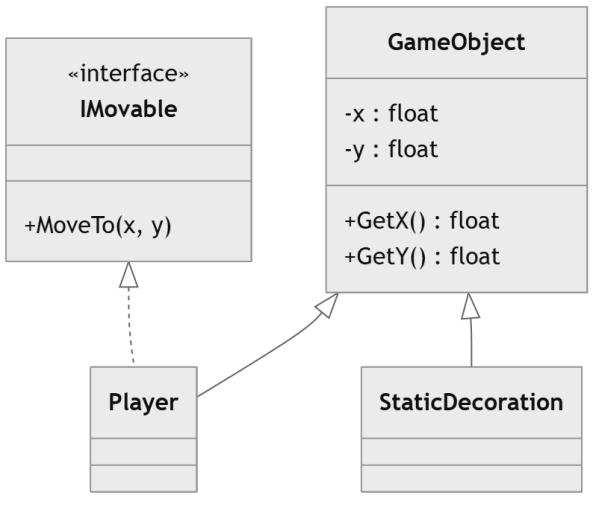
```
class IMovable
{
public:
    virtual ~IMovable() = default;
    virtual void MoveTo(float x, float y) = 0;
};

class Player : public GameObject, public IMovable
{
public:
    void MoveTo(float x, float y) override
    {
        SetPosition(x, y);
    }
};
```

```
class Player : public GameObject, public IMovable
{
public:
    void MoveTo(float x, float y) override
    {
        SetPosition(x, y);
    }
};

class StaticDecoration : public GameObject
{
};
```

이 구조에서 StaticDecoration은 게임 객체이지만 이동 역할은 수행하지 않는다. IMovable을 요구하는 코드에는 실제로 이동 가능한 객체만 전달된다.



9.7 인터페이스 분리 원칙 (ISP)

인터페이스 분리 원칙(Interface Segregation Principle)은 사용자가 필요하지 않은 기능에 의존하도록 강요해서는 안 된다는 원칙이다. 하나의 거대한 인터페이스보다 역할별로 분리된 작은 인터페이스가 서로 다른 객체와 시스템에 적용하기 쉽다.

9.7.1 큰 인터페이스의 문제

모든 게임 객체가 사용할 기능을 하나의 인터페이스에 넣었다고 하자.

```
class IGameEntity
{
public:
    virtual ~IGameEntity() = default;

    virtual void Update() = 0;
    virtual void Render() const = 0;
    virtual void ProcessInput() = 0;
    virtual int TakeDamage(int damage) = 0;
    virtual void Save() const = 0;
};
```

플레이어는 대부분의 기능을 사용할 수 있지만 벽, 배경 장식, 파티클 효과는 그렇지 않다.

```
class Wall : public IGameEntity
{
public:
    void Update() override
    {
    }

    void Render() const override
    {
        // 벽 렌더링
    }

    void ProcessInput() override
    {
        // 사용하지 않음
    }

    int TakeDamage(int) override
    {
        return 0;
    }

    void Save() const override
    {
        // 사용하지 않음
    }
};
```

사용하지 않는 함수에 빈 구현을 넣거나 예외를 던져야 한다면 인터페이스가 너무 많은 역할을 포함하고 있다는 신호이다. 인터페이스를 변경하면 실제로 그 기능을 사용하지 않는 클래스까지 다시 컴파일하고 수정해야 할 수 있다.

9.7.2 역할별 인터페이스

객체가 수행하는 역할을 기준으로 인터페이스를 분리한다.

```
class IUpdatable
{
public:
    virtual ~IUpdatable() = default;
    virtual void Update() = 0;
};

class IRenderable
{
public:
    virtual ~IRenderable() = default;
    virtual void Render() const = 0;
};

class IInputReceiver
{
public:
    virtual ~IInputReceiver() = default;
    virtual void ProcessInput() = 0;
};

class IDamageable
{
public:
    virtual ~IDamageable() = default;
    virtual int TakeDamage(int damage) = 0;
};

class ISavable
{
public:
    virtual ~ISavable() = default;
    virtual void Save() const = 0;
};
```

구체 클래스는 필요한 역할만 구현한다.

```
class Player : public IUpdatable,
               public IRenderable,
               public IInputReceiver,
               public IDamageable,
               public ISavable
{
public:
    void Update() override;
    void Render() const override;
    void ProcessInput() override;
    int TakeDamage(int damage) override;
    void Save() const override;
};

class Wall : public IRenderable,
             public IDamageable
{
public:
    void Render() const override;
    int TakeDamage(int damage) override;
};

class ParticleEffect : public IUpdatable,
                       public IRenderable
{
public:
    void Update() override;
    void Render() const override;
```

```
};
```

각 시스템도 자신이 사용하는 역할에만 의존한다.

```
void UpdateObject(IUpdatable& object)
{
    object.Update();
}

void RenderObject(const IRenderable& object)
{
    object.Render();
}

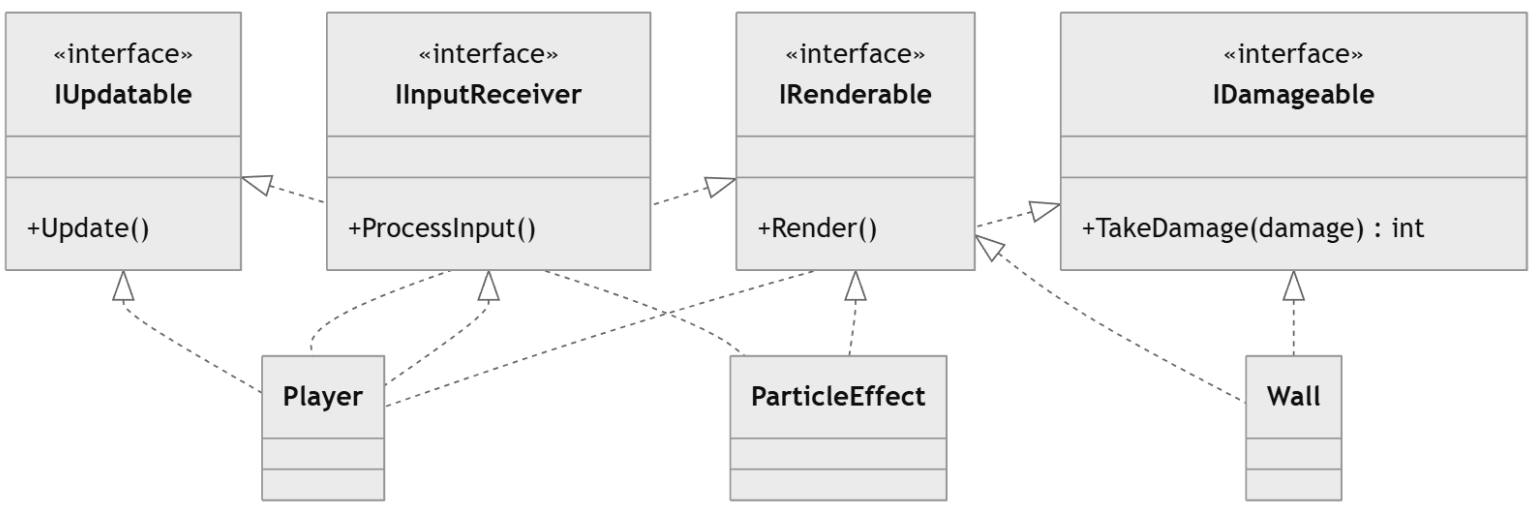
void ApplyDamage(IDamageable& object, int damage)
{
    object.TakeDamage(damage);
}
```

렌더링 시스템은 입력이나 저장 기능을 알 필요가 없고 피해 시스템은 렌더링 기능에 의존하지 않는다.

9.7.3 작은 인터페이스의 조합

작은 인터페이스는 다중 상속으로 역할을 조합할 수 있다. 상태를 가진 여러 기반 클래스를 복잡하게 상속하는 것과 달리, 순수 가상 함수만 가진 작은 역할 인터페이스를 조합하면 객체가 제공하는 능력을 명시적으로 표현할 수 있다.

인터페이스를 지나치게 작게 만들면 관련 기능을 사용하기 위해 여러 타입을 반복해서 전달해야 할 수 있다. 같은 사용자 집단이 항상 함께 사용하는 함수라면 하나의 역할로 묶는 것이 자연스럽다. 인터페이스 분리의 기준은 구현 클래스의 편의보다 인터페이스를 사용하는 코드가 실제로 필요로 하는 기능이다.



9.8 의존 역전 원칙 (DIP)

의존 역전 원칙(Dependency Inversion Principle)은 상위 수준의 정책 코드가 하위 수준의 구체 구현에 직접 의존하지 않고, 양쪽이 추상화에 의존하도록 만드는 원칙이다. 여기서 상위 수준은 프로그램의 핵심 규칙과 흐름을 의미하고, 하위 수준은 콘솔 출력, 파일 저장, 난수 생성, 네트워크 통신처럼 구체적인 기술을 의미한다.

9.8.1 구체 클래스에 대한 의존

전투 서비스가 난수 생성과 콘솔 출력을 내부에서 직접 만든다고 하자.

```
#include <iostream>
#include <random>

class CombatService
{
public:
    CombatService()
        : engine(std::random_device{}())
    {
    }

    void Attack(Character& attacker, Character& target)
    {
```

```
        std::uniform_int_distribution<int> distribution(1,
100);
        const bool critical = distribution(engine) <= 20;
        const int damage = critical ? 40 : 20;
        const int actualDamage = target.TakeDamage(damage);

        std::cout << attacker.GetName()
                  << " dealt "
                  << actualDamage
                  << " damage\n";
    }

private:
    std::mt19937 engine;
};
```

이 클래스는 전투 흐름, 난수 구현, 콘솔 출력에 동시에 의존한다. 시험할 때 치명타가 발생할지 제어하기 어렵고 기록 대상을 파일이나 화면으로 바꾸기도 어렵다.

9.8.2 추상 인터페이스에 대한 의존

변하는 기술을 역할로 분리한다.

```
class IRandomSource
{
public:
    virtual ~IRandomSource() = default;
    virtual int NextInt(int min, int max) = 0;
};

class ICombatLog
{
public:
    virtual ~ICombatLog() = default;
    virtual void Write(std::string_view message) = 0;
};
```

CombatService는 두 인터페이스에 의존한다.

```
class CombatService
{
public:
    CombatService(IRandomSource& randomSource,
                  ICombatLog& combatLog)
        : randomSource(randomSource),
          combatLog(combatLog)
    {
    }

    void Attack(Character& attacker, Character& target);

private:
    IRandomSource& randomSource;
    ICombatLog& combatLog;
};
```

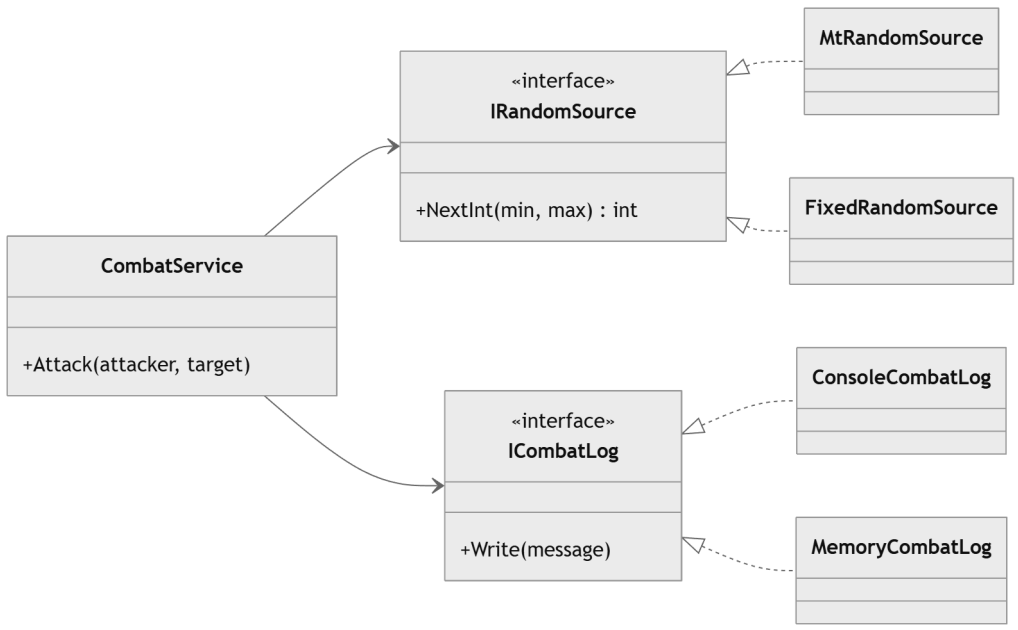
전투 규칙은 난수를 어떤 엔진으로 생성하는지, 기록을 어디에 출력하는지 알지 못한다. 하위 구현도 전투 서비스의 구체 클래스에 의존하지 않고 자신이 구현할 인터페이스만 안다.

9.8.3 의존성 방향

구체 구현에 직접 의존하는 구조에서는 핵심 정책이 기술 세부 사항을 향해 의존한다.

```
CombatService ──> MtRandomSource
CombatService ──> ConsoleCombatLog
```

추상화를 도입하면 핵심 정책과 세부 구현이 인터페이스를 향해 의존한다.



의존 방향이 역전되었다는 표현은 상위 정책이 하위 구현을 직접 가리키던 방향이 인터페이스를 중심으로 재구성되었다는 뜻이다. 실제 실행 시에는 구체 객체가 필요하지만 구체 객체를 선택하고 연결하는 책임을 프로그램의 바깥쪽으로 이동한다.

9.8.4 의존성 주입

의존성 주입(dependency injection)은 객체가 필요한 의존 객체를 내부에서 직접 생성하지 않고 외부에서 전달받는 방식이다. 생성자 주입은 객체가 유효하게 동작하는 데 반드시 필요한 의존성을 표현하기 좋다.

```
class CombatService
{
public:
    CombatService(IRandomSource& randomSource,
                  ICombatLog& combatLog)
        : randomSource(randomSource),
          combatLog(combatLog)
    {
    }

private:
    IRandomSource& randomSource;
    ICombatLog& combatLog;
};
```

참조 멤버는 두 의존 객체가 반드시 존재해야 하며 CombatService보다 오래 살아야 한다는 의미를 가진다. 선택적 의존성이 필요하면 포인터를 사용할 수 있지만 nullptr 처리 규칙을 명확히 해야 한다.

실제 구현을 구성하는 위치를 조립 지점(composition root)이라고 부를 수 있다. 일반적으로 main()이나 장면 초기화 코드처럼 프로그램 객체를 생성하는 바깥쪽에 둔다.

```
int main()
{
    MtRandomSource randomSource;
    ConsoleCombatLog combatLog;
    CombatService combatService(randomSource, combatLog);

    Player player("Knight", 150);
    Monster monster("Goblin", 80);

    combatService.Attack(player, monster);
}
```

CombatService는 어떤 구현이 주입되었는지 알지 못한다. 실행 환경에서는 실제 구현을 주입하고 시험 환경에서는 제어 가능한 가짜 구현을 주입할 수 있다.

9.8.5 테스트 가능한 구조

난수를 고정하는 시험용 구현을 작성한다.

```
class FixedRandomSource : public IRandomSource
{
```

```
public:
    explicit FixedRandomSource(int value)
        : value(value)
    {
    }

    int NextInt(int min, int max) override
    {
        return std::clamp(value, min, max);
    }

private:
    int value;
};
```

기록 내용을 메모리에 저장하는 구현도 작성한다.

```
#include <string>
#include <string_view>
#include <vector>

class MemoryCombatLog : public ICombatLog
{
public:
    void Write(std::string_view message) override
    {
        messages.emplace_back(message);
    }

    const std::vector<std::string>& GetMessages() const
    {
        return messages;
    }

private:
    std::vector<std::string> messages;
};
```

시험에서는 치명타가 반드시 발생하는 값을 주입하고 결과를 검증할 수 있다.

```
#include <cassert>

void TestCriticalAttack()
{
    FixedRandomSource randomSource(10);
    MemoryCombatLog combatLog;
    CombatService combatService(randomSource, combatLog);

    Player player("Knight", 100);
    Monster monster("Goblin", 100);

    combatService.Attack(player, monster);

    assert(monster.GetHp() == 60);
    assert(combatLog.GetMessages().size() == 1);
}
```

전역 난수 함수나 콘솔 출력을 직접 사용했다면 시험 결과를 고정하기 어렵다. 의존성 주입은 인터페이스 수를 늘리기 위한 기법이 아니라 실행 환경의 기술을 교체하고 핵심 규칙을 독립적으로 시험하기 위한 방법이다.

9.9 객체지향 설계의 적용

설계 원칙은 각각 독립된 규칙처럼 보이지만 실제 코드에서는 함께 작용한다. 책임을 분리하면 응집도가 높아지고, 작은 인터페이스를 사용하면 결합도와 변경 범위가 줄어든다. 구체 구현을 외부에서 주입하면 개방-폐쇄 원칙과 의존 역전 원칙을 적용하면서 시험 가능한 구조를 만들 수 있다.

전투 기능을 예로 들어 한 클래스와 전역 기능에 집중된 코드를 객체 협력 구조로 개선한다. 목표는 클래스 수를 최대한 늘리는 것이 아니라 변화하는 전투 계산, 난수, 기록 방식을 분리하고 안정적인 체력 상태와 전투 흐름을 명확하게 만드는 것이다.

9.9.1 설계 문제 분석

초기 구현에서는 Player가 공격 종류를 판정하고, 전역 난수 함수를 호출하며, 대상의 체력을 직접 변경하고, 콘솔에 결과를 출력한다고 가정한다.

```
#include <cstdlib>
#include <iostream>
#include <string>

class Monster
{
public:
    int hp = 100;
};

class Player
{
public:
    enum class WeaponType
    {
        Sword,
        Bow,
        Staff
    };

    void Attack(Monster& target)
    {
        int damage = 0;

        switch (weaponType)
        {
        case WeaponType::Sword:
            damage = strength * 2;
            break;

        case WeaponType::Bow:
            damage = dexterity + 5;
            break;

        case WeaponType::Staff:
            damage = intelligence * 2 + 8;
            break;
        }

        const bool critical = std::rand() % 100 < 20;

        if (critical)
        {
            damage *= 2;
        }

        target.hp -= damage;

        if (target.hp < 0)
        {
            target.hp = 0;
        }

        std::cout << "damage=" << damage
                  << " targetHp=" << target.hp
                  << '\n';
    }

private:
    WeaponType weaponType = WeaponType::Sword;
    int strength = 12;
```

```
    int dexterity = 10;
    int intelligence = 8;
};
```

코드는 짧지만 여러 설계 문제가 한곳에 모여 있다.

- Player가 모든 공격 방식의 계산식을 알고 있다.
- 공격 종류가 늘어날 때 switch문을 수정해야 한다.
- Monster::hp가 공개되어 있어 체력 불변식을 보호하지 못한다.
- 전역 상태를 사용하는 std::rand() 때문에 결과를 제어하기 어렵다.
- 콘솔 출력 방식이 전투 규칙에 포함되어 있다.
- 공격 계산만 시험하려 해도 몬스터와 콘솔 출력이 함께 필요하다.
- Monster 이외의 피해 대상에는 같은 코드를 재사용하기 어렵다.

설계 문제를 분석할 때는 현재 코드의 길이보다 예상되는 변경을 확인한다. 공격 방식 추가, 치명타 규칙 변경, 기록 출력 교체, 체력 규칙 변경이 서로 독립적으로 일어날 수 있다면 각 변경 축을 분리할 가치가 있다.

9.9.2 책임 분리

전투 기능의 책임을 구분한다.

```
HealthComponent
- 현재 체력과 최대 체력 유지
- 피해 적용과 사망 판정

IAttackBehavior 구현 객체
- 공격 피해량 계산
- 공격 방식별 확률 규칙

Character
- 이름과 체력 소유
- 공격 행동 객체 소유
- 공격 행동 교체

CombatService
- 공격자와 대상의 협력 순서 조정
- 전투 결과 기록 요청

IRandomSource 구현 객체
- 범위 안의 난수 제공

ICombatLog 구현 객체
- 전투 결과 출력 또는 저장
```

체력 규칙은 다른 시스템과 분리된 구체 클래스에 둔다.

```
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp)
        : hp(maxHp), maxHp(maxHp)
    {
        if (maxHp <= 0)
        {
            throw std::invalid_argument(
                "maxHp must be positive");
        }
    }

    int TakeDamage(int damage)
    {
        if (damage <= 0 || IsDead())
        {
            return 0;
        }

        const int oldHp = hp;
```

```
        hp = std::max(hp - damage, 0);
        return oldHp - hp;
    }

    int GetHp() const
    {
        return hp;
    }

    int GetMaxHp() const
    {
        return maxHp;
    }

    bool IsDead() const
    {
        return hp == 0;
    }

private:
    int hp;
    int maxHp;
};
```

HealthComponent는 체력이라는 하나의 책임에 집중한다. 전투 기록이나 공격 계산을 알지 못한다.

9.9.3 인터페이스 추출

변경 가능성이 있는 난수, 기록, 공격 계산에 인터페이스를 둔다.

```
class IRandomSource
{
public:
    virtual ~IRandomSource() = default;
    virtual int NextInt(int min, int max) = 0;
};

class ICombatLog
{
public:
    virtual ~ICombatLog() = default;
    virtual void Write(std::string_view message) = 0;
};

struct AttackRoll
{
    int damage;
    bool critical;
    std::string_view attackName;
};

class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;
    virtual AttackRoll Roll(IRandomSource& randomSource) const
= 0;
};
```

인터페이스는 각 사용자가 필요로 하는 기능만 포함한다. 난수 인터페이스는 특정 엔진 타입을 노출하지 않고 정수 범위 생성만 제공한다. 전투 기록 인터페이스는 출력 대상이나 파일 형식을 규정하지 않는다. 공격 행동은 피해 계산에 필요한 난수 역할에만 의존한다.

모든 클래스에 인터페이스를 만들지는 않는다. HealthComponent는 현재 하나의 안정적인 체력 규칙을 제공하고 독립적으로 교체할 요구가 없으므로 구체 클래스로 유지한다.

9.9.4 상속을 구성으로 변경

공격 방식별로 Player를 상속하면 공격 방식과 캐릭터 종류가 하나의 계층에 섞인다.

```
Character
└─ Player
   ├─ SwordPlayer
   ├─ BowPlayer
   └─ StaffPlayer
```

플레이어가 실행 중 무기를 바꾸려면 객체 타입을 교체하거나 공격 코드를 별도로 우회해야 한다. 몬스터도 같은 공격 방식을 사용하면 계층에 비슷한 클래스가 반복된다.
공격 행동을 구성으로 포함하면 Player와 Monster가 같은 행동 객체를 사용할 수 있고 실행 중 교체할 수도 있다.

```
class Character
{
public:
    Character(std::string name,
              int maxHp,
              std::unique_ptr<IAttackBehavior> attackBehavior)
        : name(std::move(name)),
          health(maxHp),
          attackBehavior(std::move(attackBehavior))
    {
        if (this->attackBehavior == nullptr)
        {
            throw std::invalid_argument(
                "attackBehavior is null");
        }
    }

    AttackRoll RollAttack(IRandomSource& randomSource) const
    {
        return attackBehavior->Roll(randomSource);
    }

    void ChangeAttack(
        std::unique_ptr<IAttackBehavior> newBehavior)
    {
        if (newBehavior == nullptr)
        {
            throw std::invalid_argument(
                "newBehavior is null");
        }

        attackBehavior = std::move(newBehavior);
    }

    int TakeDamage(int damage)
    {
        return health.TakeDamage(damage);
    }

    std::string_view GetName() const
    {
        return name;
    }

    int GetHp() const
    {
        return health.GetHp();
    }

    bool IsDead() const
    {
        return health.IsDead();
    }

private:
    std::string name;
    HealthComponent health;
    std::unique_ptr<IAttackBehavior> attackBehavior;
```

```
};
```

Character는 공격 계산의 세부 공식을 알지 않고 공격 행동 객체에 위임한다. 공격 행동의 수명은 Character가 고유 소유한다.

9.9.5 전역 상태 줄이기

전역 난수 객체와 전역 기록 객체는 함수의 매개변수와 클래스 멤버에 나타나지 않는 숨은 의존성을 만든다. 호출 결과가 외부 상태에 따라 달라지며 초기화와 종료 순서도 추적하기 어려워진다.

```
// 숨은 의존성을 만드는 형태
GlobalRandom::NextInt(1, 100);
GlobalCombatLog::Write(message);
```

전투 서비스가 필요한 의존성을 생성자에서 받도록 구성한다.

```
class CombatService
{
public:
    CombatService(IRandomSource& randomSource,
                  ICombatLog& combatLog)
        : randomSource(randomSource),
          combatLog(combatLog)
    {
    }

    void Attack(Character& attacker, Character& target)
    {
        if (attacker.IsDead() || target.IsDead())
        {
            return;
        }

        const AttackRoll roll =
            attacker.RollAttack(randomSource);

        const int actualDamage =
            target.TakeDamage(roll.damage);

        std::ostringstream output;
        output << attacker.GetName()
               << " uses " << roll.attackName
               << " on " << target.GetName()
               << " for " << actualDamage
               << " damage";

        if (roll.critical)
        {
            output << " critical";
        }

        output << " hp=" << target.GetHp();
        combatLog.Write(output.str());
    }

private:
    IRandomSource& randomSource;
    ICombatLog& combatLog;
};
```

의존 객체는 main()에서 생성하고 연결한다. 이 위치에서 실행용 구현과 시험용 구현을 선택할 수 있다.

9.9.6 과도한 추상화 피하기

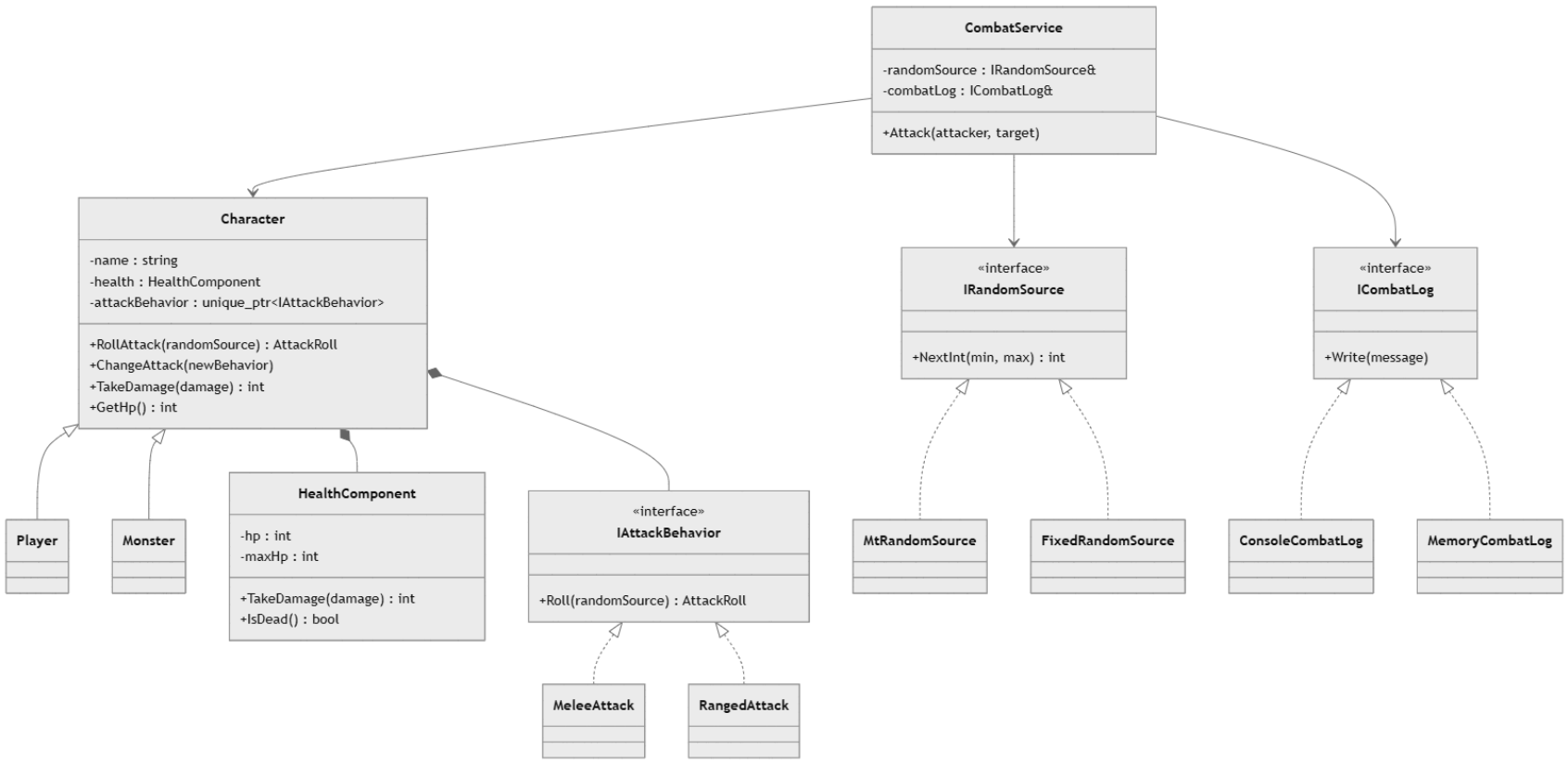
설계 원칙을 적용한다고 해서 모든 구체 클래스를 인터페이스로 감싸거나 모든 생성 과정을 별도 팩토리로 옮길 필요는 없다. 현재 예제에서 실제로 교체할 필요가 있는 부분은 공격 계산, 난수 공급, 기록 출력이다. 체력 규칙과 전투 결과 값은 단순한 구체 타입으로 유지한다.

과도한 추상화에는 이런 문제가 생길 수 있다.

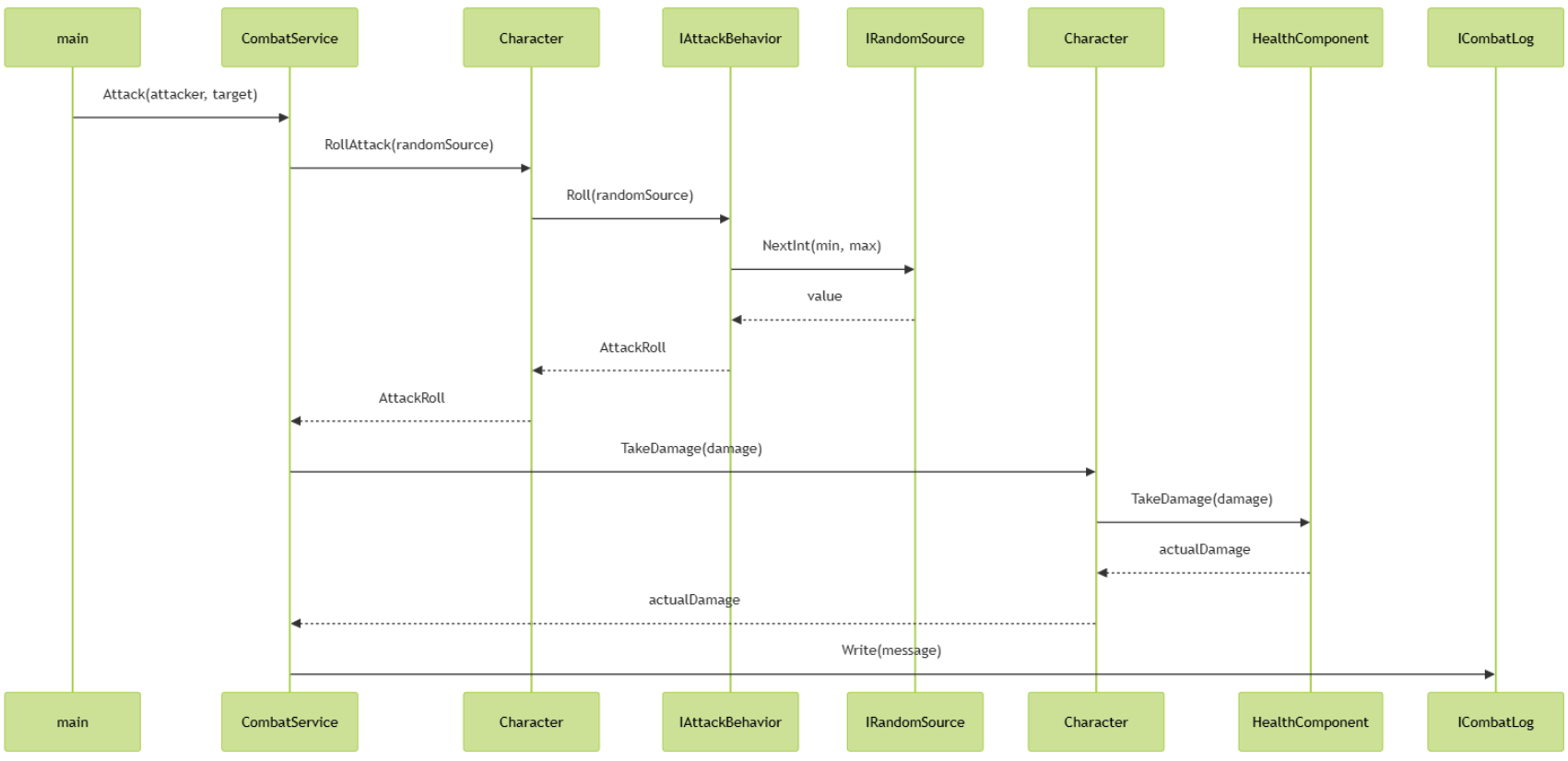
- 실제 구현이 하나뿐인데 인터페이스와 래퍼가 반복된다.
- 호출 흐름을 확인하기 위해 여러 파일을 이동해야 한다.
- 객체 소유권과 수명이 불필요하게 복잡해진다.
- 요구 사항이 확정되지 않았는데 잘못된 확장 지점을 먼저 고정한다.
- 작은 변경에도 여러 추상 계층을 함께 수정해야 한다.

추상화는 예상만으로 만들기보다 실제 변경과 중복이 드러나는 지점에 도입하는 편이 안전하다. 두 구현이 존재하거나, 시험에서 교체할 필요가 있거나, 외부 기술에 대한 의존을 격리해야 하는 경우에는 인터페이스의 가치가 분명하다.

종합 예제의 구조는 공격 행동에 구성을 사용하고, 전투 서비스가 난수와 기록 인터페이스에 의존하도록 구성한다.



공격 호출의 흐름은 시퀀스 다이어그램으로 확인할 수 있다.



전체 코드는 독립적으로 컴파일할 수 있다.

```
#include <algorithm>
#include <cassert>
#include <iostream>
#include <memory>
#include <random>
#include <sstream>
#include <stdexcept>
#include <string>
#include <string_view>
```

```cpp
#include <utility>
#include <vector>

class IRandomSource
{
public:
    virtual ~IRandomSource() = default;
    virtual int NextInt(int min, int max) = 0;
};

class MtRandomSource : public IRandomSource
{
public:
    MtRandomSource()
        : engine(std::random_device{}())
    {
    }

    int NextInt(int min, int max) override
    {
        std::uniform_int_distribution<int> distribution(min, max);
        return distribution(engine);
    }

private:
    std::mt19937 engine;
};

class FixedRandomSource : public IRandomSource
{
public:
    explicit FixedRandomSource(int value) : value(value)
    {
    }

    int NextInt(int min, int max) override
    {
        return std::clamp(value, min, max);
    }

private:
    int value;
};

class ICombatLog
{
public:
    virtual ~ICombatLog() = default;
    virtual void Write(std::string_view message) = 0;
};

class ConsoleCombatLog : public ICombatLog
{
public:
    void Write(std::string_view message) override
    {
        std::cout << message << '\n';
    }
};

class MemoryCombatLog : public ICombatLog
{
public:
    void Write(std::string_view message) override
    {
        messages.emplace_back(message);
    }

    const std::vector<std::string>& GetMessages() const
```

```
    {
        return messages;
    }

private:
    std::vector<std::string> messages;
};

class HealthComponent
{
public:
    explicit HealthComponent(int maxHp)
        : hp(maxHp), maxHp(maxHp)
    {
        if (maxHp <= 0)
        {
            throw std::invalid_argument("maxHp must be positive");
        }
    }

    int TakeDamage(int damage)
    {
        if (damage <= 0 || IsDead())
        {
            return 0;
        }

        const int oldHp = hp;
        hp = std::max(hp - damage, 0);
        return oldHp - hp;
    }

    int GetHp() const
    {
        return hp;
    }

    int GetMaxHp() const
    {
        return maxHp;
    }

    bool IsDead() const
    {
        return hp == 0;
    }

private:
    int hp;
    int maxHp;
};

struct AttackRoll
{
    int damage;
    bool critical;
    std::string_view attackName;
};

class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;
    virtual AttackRoll Roll(IRandomSource& randomSource) const = 0;
};

class MeleeAttack : public IAttackBehavior
{
public:
```

```
    MeleeAttack(int baseDamage, int criticalRate)
        : baseDamage(std::max(baseDamage, 0)),
          criticalRate(std::clamp(criticalRate, 0, 100))
    {
    }

    AttackRoll Roll(IRandomSource& randomSource) const override
    {
        const bool critical =
            randomSource.NextInt(1, 100) <= criticalRate;

        return AttackRoll
        {
            critical ? baseDamage * 2 : baseDamage,
            critical, "MeleeAttack"
        };
    }

private:
    int baseDamage;
    int criticalRate;
};

class RangedAttack : public IAttackBehavior
{
public:
    RangedAttack(int minDamage, int maxDamage)
        : minDamage(std::max(minDamage, 0)),
          maxDamage(std::max(maxDamage, this->minDamage))
    {
    }

    AttackRoll Roll(IRandomSource& randomSource) const override
    {
        return AttackRoll
        {
            randomSource.NextInt(minDamage, maxDamage),
            false, "RangedAttack"
        };
    }

private:
    int minDamage;
    int maxDamage;
};

class Character
{
public:
    Character(std::string name, int maxHp,
              std::unique_ptr<IAttackBehavior> attackBehavior)
        : name(std::move(name)), health(maxHp),
          attackBehavior(std::move(attackBehavior))
    {
        if (this->attackBehavior == nullptr)
        {
            throw std::invalid_argument("attackBehavior is null");
        }
    }

    virtual ~Character() = default;

    Character(const Character&) = delete;
    Character& operator=(const Character&) = delete;
    Character(Character&&) noexcept = default;
    Character& operator=(Character&&) noexcept = default;

    AttackRoll RollAttack(IRandomSource& randomSource) const
    {
```

```
        return attackBehavior->Roll(randomSource);
    }

    void ChangeAttack(std::unique_ptr<IAttackBehavior> newBehavior)
    {
        if (newBehavior == nullptr)
        {
            throw std::invalid_argument(
                "newBehavior is null");
        }

        attackBehavior = std::move(newBehavior);
    }

    int TakeDamage(int damage)
    {
        return health.TakeDamage(damage);
    }

    std::string_view GetName() const
    {
        return name;
    }

    int GetHp() const
    {
        return health.GetHp();
    }

    int GetMaxHp() const
    {
        return health.GetMaxHp();
    }

    bool IsDead() const
    {
        return health.IsDead();
    }

private:
    std::string name;
    HealthComponent health;
    std::unique_ptr<IAttackBehavior> attackBehavior;
};

class Player final : public Character
{
public:
    using Character::Character;
};

class Monster final : public Character
{
public:
    using Character::Character;
};

class CombatService
{
public:
    CombatService(IRandomSource& randomSource, ICombatLog& combatLog)
        : randomSource(randomSource),
          combatLog(combatLog)
    {
    }

    void Attack(Character& attacker, Character& target)
    {
        if (attacker.IsDead() || target.IsDead())
```

```
        {
            return;
        }

        const AttackRoll roll = attacker.RollAttack(randomSource);
        const int actualDamage = target.TakeDamage(roll.damage);

        std::ostringstream output;
        output << attacker.GetName()
               << " uses " << roll.attackName
               << " on " << target.GetName()
               << " for " << actualDamage
               << " damage";

        if (roll.critical)
        {
            output << " critical";
        }
        output << " hp=" << target.GetHp();
        combatLog.Write(output.str());
    }

private:
    IRandomSource& randomSource;
    ICombatLog& combatLog;
};

void TestCriticalAttack()
{
    FixedRandomSource randomSource(10);
    MemoryCombatLog combatLog;
    CombatService combatService(randomSource, combatLog);

    Player player(
        "Knight",
        150,
        std::make_unique<MeleeAttack>(20, 20));

    Monster monster(
        "Goblin",
        100,
        std::make_unique<MeleeAttack>(10, 0));

    combatService.Attack(player, monster);

    assert(monster.GetHp() == 60);
    assert(combatLog.GetMessages().size() == 1);
}

int main()
{
    TestCriticalAttack();

    MtRandomSource randomSource;
    ConsoleCombatLog combatLog;
    CombatService combatService(randomSource, combatLog);

    Player player("Knight", 150,
        std::make_unique<MeleeAttack>(20, 20));

    Monster monster("Goblin", 100,
        std::make_unique<RangedAttack>(8, 14));

    combatService.Attack(player, monster);
    combatService.Attack(monster, player);

    player.ChangeAttack(std::make_unique<RangedAttack>(15, 25));

    combatService.Attack(player, monster);
```

```
}
```

TestCriticalAttack()은 고정 난수와 메모리 기록 객체를 주입하여 치명타 결과를 반복해서 검증한다. 실행용 코드에서는 실제 난수와 콘솔 기록을 주입한다. Player는 공격 방식을 바꿀 수 있지만 전투 서비스와 체력 객체는 수정하지 않는다.
이 예제는 모든 설계 원칙을 기계적으로 적용한 완성형 구조가 아니다. 현재 요구에서 독립적으로 바뀌는 부분만 분리한 하나의 설계안이다. 게임 규모가 커져 공격 결과에 방어력, 상태 이상, 네트워크 동기화가 추가되면 책임과 협력 경계를 다시 검토해야 한다.

9.10 학습 정리

객체지향 설계는 프로그램을 많은 클래스로 나누는 작업이 아니라 기능을 책임으로 분배하고 객체들이 공개 인터페이스를 통해 협력하도록 만드는 과정이다. 객체는 자신이 가진 상태와 규칙을 함께 관리하고 외부 객체는 내부 데이터를 직접 조작하기보다 의미 있는 동작을 요청해야 한다.

역할은 협력 안에서 객체가 제공하는 능력을 나타낸다. 작은 인터페이스로 역할을 표현하면 서로 다른 구체 객체를 같은 방식으로 사용할 수 있다. 인터페이스는 구현 세부 사항을 숨기고 정상 동작, 실패 방식, 소유권과 수명에 관한 계약을 사용자에게 제공한다.

응집도가 높은 클래스는 하나의 목적과 관련된 상태와 행위를 모아 둔다. 결합도가 낮은 객체는 다른 객체의 구체 구현과 내부 표현을 적게 안다. 변경에 강한 구조는 객체 사이의 의존을 제거하는 것이 아니라 안정적인 역할을 통해 필요한 의존만 명시한다.

단일 책임 원칙은 클래스가 하나의 변경 이유를 가지도록 책임을 분리한다. 클래스가 아주 작아야 한다는 뜻은 아니며, 함께 변경되는 기능은 한 책임 안에 둘 수 있다. 서로 다른 팀이나 규칙에 의해 독립적으로 바뀌는 기능이 한 클래스에 섞이지 않도록 경계를 정하는 것이 중요하다.

개방-폐쇄 원칙은 반복해서 변하는 지점을 추상화하여 새 구현을 추가할 수 있도록 한다. 조건문은 단순한 선택에 적합하지만 종류별 분기가 여러 곳에 반복되면 행동 객체나 다형성으로 변경 지점을 모을 수 있다. 타입의 종류는 상속으로, 교체 가능한 기능과 정책은 구성으로 표현하는 방식을 함께 사용할 수 있다.

리스코프 치환 원칙은 파생 타입이 기반 타입의 계약을 유지하도록 요구한다. 파생 클래스는 기반 클래스보다 강한 호출 조건을 요구하거나 약한 결과를 제공해서는 안 된다. 기반 타입으로 사용할 때 특별한 타입 검사나 예외 처리가 필요하다면 상속 관계가 올바른지 검토해야 한다.

인터페이스 분리 원칙은 사용자가 필요하지 않은 함수에 의존하지 않도록 역할별 인터페이스를 구성한다. 큰 인터페이스를 여러 구현 클래스가 빈 함수로 채우는 구조보다 갱신, 렌더링, 입력, 피해 처리와 같은 역할을 필요한 객체가 선택하여 구현하는 구조가 유연하다.

의존 역전 원칙은 핵심 정책이 콘솔, 파일, 난수 엔진과 같은 세부 기술에 직접 의존하지 않도록 한다. 외부에서 의존 객체를 주입하면 실행 환경과 시험 환경에서 구현을 교체할 수 있고 전역 상태와 숨은 의존성을 줄일 수 있다.

설계 원칙은 미래의 모든 변화를 미리 추측하여 추상 계층을 만드는 규칙이 아니다. 실제로 함께 바뀌는 부분과 독립적으로 바뀌는 부분을 관찰하고, 변경 비용과 구현 복잡도의 균형을 고려하여 필요한 만큼 적용해야 한다.

9.11 학습 과제

과제 **1.** 책임이 집중된 클래스 분석
입력 처리, 이동, 공격, 체력, 저장, HUD 출력, 사운드 재생을 모두 담당하는 Player 클래스를 작성한다. 클래스의 데이터 멤버와 멤버 함수를 책임별로 분류하고 서로 다른 변경 이유를 최소 네 가지 이상 찾는다.

분석 결과를 바탕으로 클래스를 분리하고 Mermaid 클래스 다이어그램으로 관계를 표현한다. 각 클래스의 책임을 한 문장으로 설명할 수 있어야 한다.

과제 **2.** 데이터와 행위의 배치
Character가 GetHp()와 SetHp()만 제공하고 외부 함수가 피해와 회복을 계산하는 코드를 작성한다. 여러 외부 함수에서 체력 범위 보정이 반복되는 문제를 확인한다.

체력 상태와 규칙을 HealthComponent로 이동하여 구조를 개선한다. 아래 불변식을 모든 공개 함수가 유지해야 한다.

```
0 <= hp <= maxHp
maxHp > 0
```

피해량과 회복량이 0 이하일 때의 처리 규칙도 명확히 정한다.

과제 **3.** 응집도와 결합도 개선
CombatService가 대상의 체력을 직접 읽고 계산하며 콘솔에 결과를 출력하는 코드를 작성한다. 체력 처리와 기록 출력을 각각 IDamageable, ICombatLog 역할로 분리한다
.
개선 전후 코드에서 전투 공식, 체력 저장 방식, 출력 대상이 바뀔 때 수정해야 하는 클래스를 비교한다. 변경 영향 범위를 표로 정리한다.

과제 **4.** 최소 인터페이스와 불변식
인벤토리 클래스를 설계한다.

필수 기능:

- 아이템 추가
- 아이템 제거
- 식별자로 검색
- 보유 개수 확인
- 최대 슬롯 수 유지
- 중복 허용 여부 선택

내부 std::vector를 직접 반환하거나 수정 가능한 참조를 공개하지 않는다. 추가와 제거 실패는 열거형 또는 결과 객체로 표현한다. 클래스가 유지해야 할 불변식을 문서로 작성한다.

과제 **5.** 개방**-**폐쇄 원칙과 공격 행동
근접, 원거리, 마법 공격을 열거형과 switch문으로 구현한다. 독 공격을 추가하면서 수정한 위치를 기록한다.

공격 계산을 IAttackBehavior로 분리하고 각 공격을 구체 행동 객체로 구현한다. Player가 std::unique_ptr<IAttackBehavior>를 소유하고 실행 중 공격 방식을 교체할 수 있도록 만든다.

상속으로 MeleePlayer, RangedPlayer, MagicPlayer를 만드는 방식과 행동 객체를 구성하는 방식을 클래스 수, 실행 중 교체, 재사용 관점에서 비교한다.

과제 **6.** 리스코프 치환 원칙 검사
IMovable 인터페이스와 Player, Monster, StaticDecoration 클래스를 구성한다. 처음에는 세 클래스가 모두 IMovable을 구현하도록 작성하고 StaticDecoration::MoveTo()가 아무 작업도 하지 않도록 만든다.

기반 타입을 사용하는 시험 함수를 작성하여 치환 문제가 드러나게 한다. 위치 상태를 가진 GameObject와 이동 역할인 IMovable을 분리하여 계층을 개선한다. 사전 조건, 사후 조건, 불변식 관점에서 수정 이유를 설명한다.

과제 **7.** 인터페이스 분리
Update(), Render(), ProcessInput(), TakeDamage(), Save()를 포함하는 큰 IGameEntity를 작성한다. Player, Wall, ParticleEffect에서 구현하면서 사용하지 않는 함수가 생기는지 확인한다.

인터페이스를 역할별로 분리한다.

- IUpdatable
- IRenderable
- IInputReceiver
- IDamageable
- ISavable

각 구체 클래스가 필요한 역할만 구현하도록 수정하고 UML 클래스 다이어그램으로 표현한다.

과제 **8.** 의존성 주입과 시험
치명타 확률이 있는 전투 서비스를 작성한다. 처음에는 std::mt19937과 std::cout을 서비스 내부에서 직접 사용한다.

난수와 기록 기능을 IRandomSource, ICombatLog로 분리하고 생성자 주입을 적용한다. 시험용 FixedRandomSource와 MemoryCombatLog를 만들어 이런 항목을 검증한다.

- 치명타 발생 시 피해량
- 일반 공격 피해량
- 이미 사망한 대상에 대한 처리
- 기록 메시지 개수
- 실제 체력보다 큰 피해를 가했을 때의 실제 감소량

과제 **9.** 전역 상태 제거
전역 객체 또는 싱글턴으로 난수, 사운드, 기록 기능을 제공하는 작은 프로그램을 작성한다. 각 함수가 실제로 어떤 전역 기능에 의존하는지 목록으로 정리한다.

전역 접근을 제거하고 필요한 객체에 생성자 또는 함수 매개변수로 의존성을 전달한다. 객체 생성과 연결을 main()에 모으고 초기화 순서와 수명 관계가 코드에 어떻게 드러나는지 설명한다.

과제 **10.** 객체지향 설계 종합
플레이어와 몬스터가 서로 공격하는 전투 프로그램을 구성한다.

필수 요구 사항:

- 체력은 HealthComponent가 관리
- 공격 계산은 교체 가능한 행동 객체로 구성
- 근접 공격, 원거리 공격, 마법 공격 구현
- 난수 공급과 전투 기록은 인터페이스로 분리
- 전역 상태 사용 금지
- 소유권은 값, 참조, unique_ptr 중 의도에 맞는 타입으로 표현
- 시험용 난수와 기록 객체 제공
- 구체 타입을 검사하는 dynamic_cast 사용 금지
- Mermaid 클래스 다이어그램과 공격 시퀀스 다이어그램 작성

구현이 끝나면 각 클래스의 책임, 변경 이유, 의존 객체, 소유권을 표로 정리한다. 인터페이스를 추가하지 않아도 되는 구체 클래스가 무엇인지 설명하고, 과도한 추상화를 피하기 위해 선택한 부분도 함께 기술한다.